What we Learn…. (Module Structure)

1. Deep learning - What, Why, where
2. Artificial Neuron and Perceptron
3. Linear Neural Network for Regression
4. Linear Neural Network for Classification
5. Deep Feedforward Neural Network
6. Convolutional Neural Network
7. Recurrent Neural Networks
8. Attention Mechanism
9. Transformer
10. Optimization of Deep models
11. Regularization for Deep models

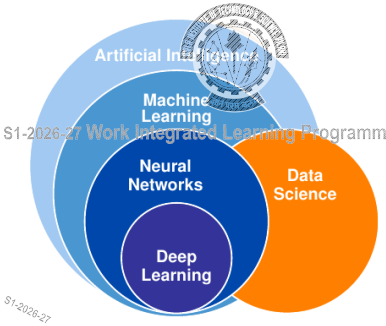
S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Definitions of Deep Learning

- Deep Learning is a type of **machine learning** based on **artificial neural networks** in which multiple layers of processing are used to extract progressively higher level features from data.
- Deep learning is a method in artificial intelligence (AI) that teaches computers to process data in a way that is **inspired by the human brain**.
- Deep learning is a machine learning technique that teaches computers to do what comes naturally to humans: **learn by example**.
- Deep learning is a subset of machine learning, which is essentially a neural network with **three or more layers**.
- Deep Learning gets its name from the fact that we add more "**Layers**" to learn from the data.

Where in AI sits DL?

AI is a general field that encompasses machine learning and deep learning, but that also includes many more approaches that don't involve any learning.



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

AI - ML - DL

AI : Artificial intelligence is the science of making things smart. The aim is make machines perform human tasks. Eg: Robot cleaning a room.

ML : Machine learning is an approach to AI. The machine learns or perform tasks through learning by experience.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

DL : Deep Learning is a technique for implementing machine learning to recognise patterns.

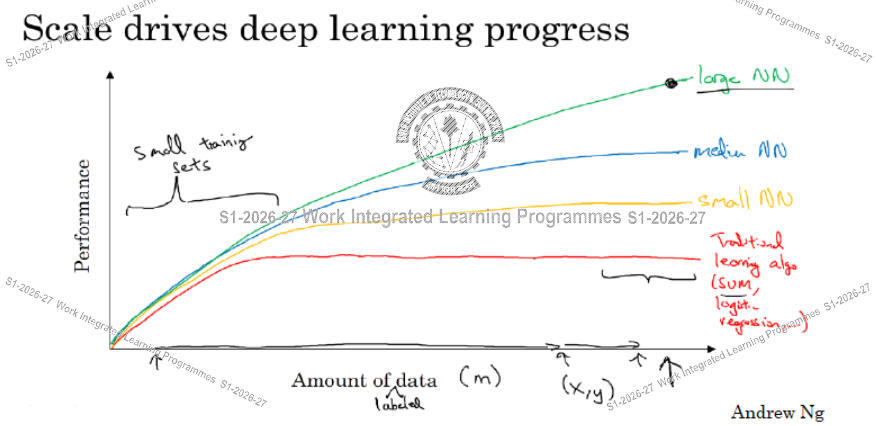
Deep (Machine) Learning

- Deep learning is a specific subfield of machine learning.
- Learning representations from data that puts an emphasis on learning successive layers of increasingly meaningful representations.
- The **deep** in deep learning stands for this idea of successive layers of representations.
- The number of layers that contribute to model the data is called the **depth** of the model.
- In deep learning, the layered representations are learned via models called **neural networks**, structured in literal layers stacked on top of each other.

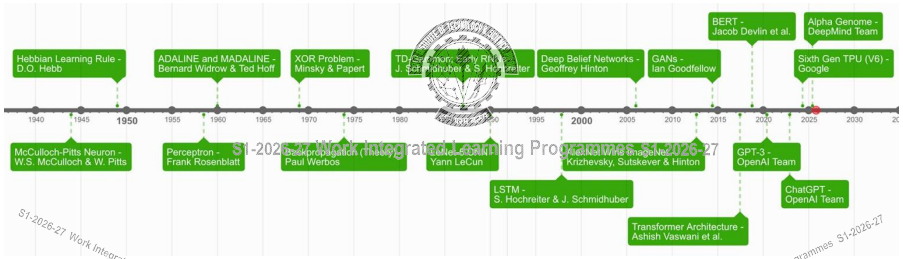
Why Deep Learning?

- Data and Storage
  - Large amounts of data
  - Lots and lots of unstructured data like images, text, audio, video
  - Cheap, high-quality sensors
  - Cheap data storage
- Computation
  - Cheap computation - CPU, GPU, Distributed clusters
- Algorithms
  - Learn by examples
  - Automated feature generation
  - Better learning capabilities
  - Scalability
  - Advance analytics can be applied

Why deep learning?

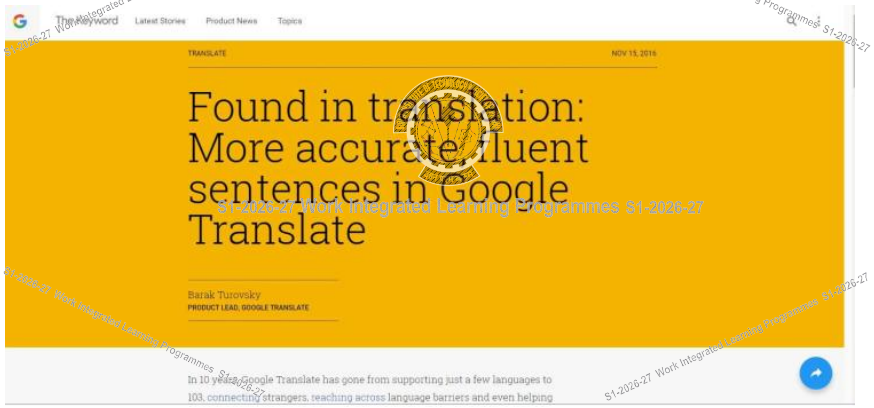


Deep Learning Timeline

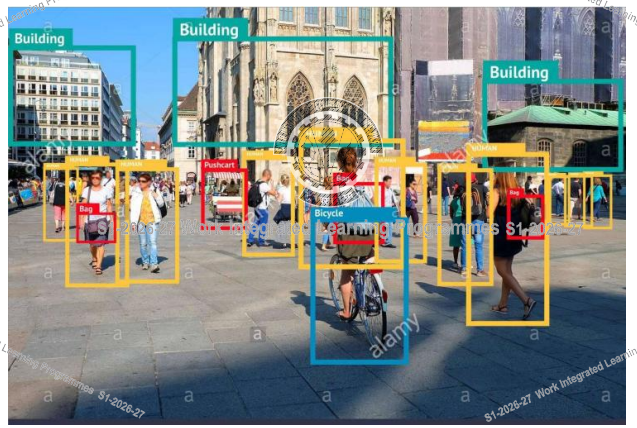


Breakthroughs with Neural Networks

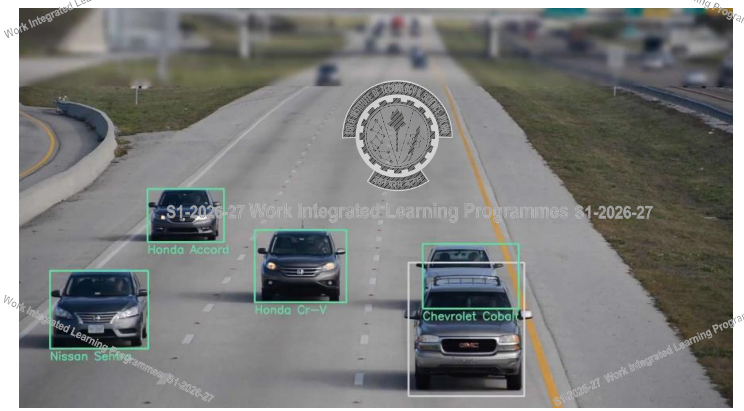
Breakthroughs with Neural Networks



Breakthroughs with Neural Networks



Breakthroughs with Neural Networks



Breakthroughs with Neural Networks

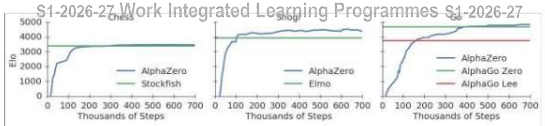
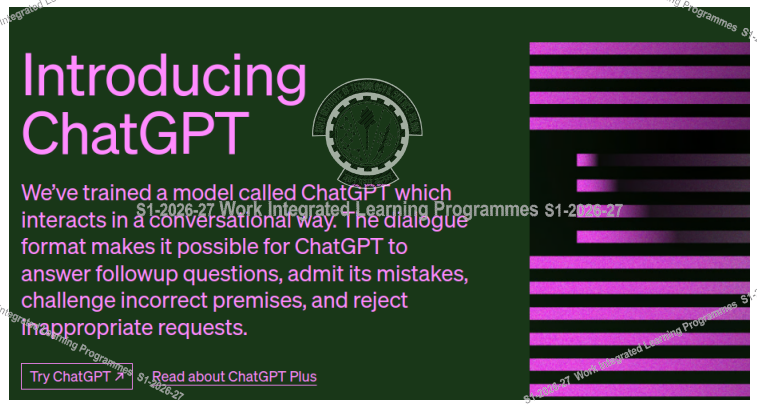


Figure 1: Training *AlphaZero* for 700,000 steps. Elo ratings were computed from evaluation games between different players when given one second per move. **a** Performance of *AlphaZero* in chess, compared to 2016 TCEC world-champion program *Stockfish*. **b** Performance of *AlphaZero* in shogi, compared to 2017 CSA world-champion program *Elmo*. **c** Performance of *AlphaZero* in Go, compared to *AlphaGo Lee* and *AlphaGo Zero* (20 block / 3 day) (*29*).

Breakthroughs with Neural Networks



Applications of Deep Learning

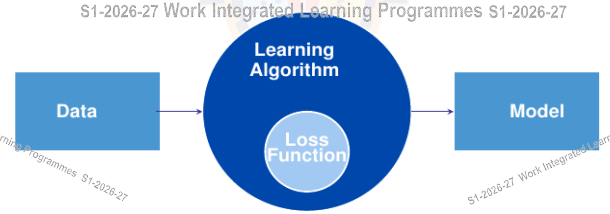
Application	Input	Output	Neural Network
Real Estate	House features	House Price	Std NN
Photo Tagging	Image	Text	CNN
Object detection	Image	Bounding box	CNN
Speech Recognition	Audio	Text transcript	RNN
Translation	English Text	French Text	RNN
Autonomous driving	Image, Sensors Radars	Position of other cars, objects, signals	Hybrid NN

Many more applications….

- Predict tomorrow's weather given geographic information, satellite images, and a trailing window of past weather.
- Takes in a question, expressed in free form text, and answers it correctly. (Q&A)
- Takes in an image and identify all the people it contains, drawing outlines around each. (Object detection)
- Presents users with products that they are likely to enjoy but unlikely, in the natural course of browsing, to encounter. (Recommendation systems)

Core components of DL problem

1. The data that we can learn from.
2. A model of how to transform the data.
3. An objective function that quantifies how well (or badly) the model is doing.
4. An algorithm to adjust the model's parameters to optimize the objective function.



Data

- Collection of examples.
- Data has to be converted to an useful and a suitable numerical **representation**.
- Each example (or data point, data instance, sample) typically consists of a set of attributes called **features** (or covariates), from which the model must make its predictions.
- In the supervised learning problems, the attribute to be predicted is designated as the **label** (or target).

$$Data = \mathcal{D} = \{X, t\}$$

Data

- Data = D = {X,t}
- Number of features = Number of Dimensions = d
- Number of examples = N
- As the number of features increase, we say the dimensionality increases.

Features

(Target)

Examples

ID	Bedrooms	Location	Sq Ft	Price
1	3	Urban	1500	350K
2	2	Suburban	1200	280K
3	4	Urban	2000	520K
4	3	Rural	1800	310K
5	2	Suburban	1100	245K
6	5	Urban	2500	680K
7	3	Rural	1600	275K
8	4	Suburban	1900	425K

2. Model

- Model denotes the **computational machinery** for ingesting data of one type, and spitting out predictions of a possibly different type.
- Deep learning models consist of many successive transformations of the data that are chained together top to bottom, thus the name deep learning.

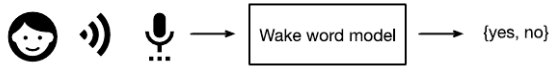


Fig. 1.1.1: Identify a wake word.

3. Objective Function

- Learning means improving at some task over time.
- **Objective functions are formal measures to denote how good (or bad) a mathematical model is.**
- By convention, objective functions are defined so that lower is better.
- Because lower is better, these functions are sometimes called **loss functions**.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

4. Learning Algorithms

- **Learning Algorithm is an algorithm capable of searching for the best possible parameters for minimizing the loss function.**
- Popular algorithm for deep learning – **gradient descent**.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Example of the Framework

➔ We have to tell a computer explicitly how to map from inputs to outputs.
➔ We have to define the problem precisely, pinning down the exact nature of the inputs and outputs, and choosing an appropriate model family.
➔ Collect a huge dataset containing examples of audio and label those that do and that do not contain the wake word.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

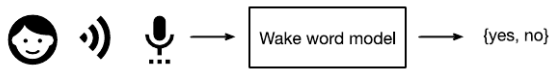


Fig. 1.1.1: Identify a wake word.

How to Create a Model?

➔ Start with a program whose behavior is determined by a number of parameters.
➔ How to determine the best possible set of parameters? Use data or examples.
➔ Parameters improve the performance of the program with respect to the loss function.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

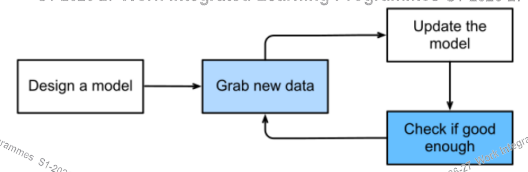


Fig. 1.1.2: A typical training process.

Train the Model

➔ Train the program with data to find the best set of parameters that improves the performance of the program with respect to the loss function.

➔ Once the parameters are finalised, we call the program a model.

- Eg: The model receives a snippet of audio as input, and the model generates a selection among yes / no as output.

➔ The set of all distinct programs (input-output mappings) that we can produce just by manipulating the parameters is called a family of models.

- Eg: Same model family should be suitable for ”Alexa” recognition and ”Hey Siri” recognition because they seem, intuitively, to be similar tasks.

Kinds of Deep Learning Problems

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Learning Problems

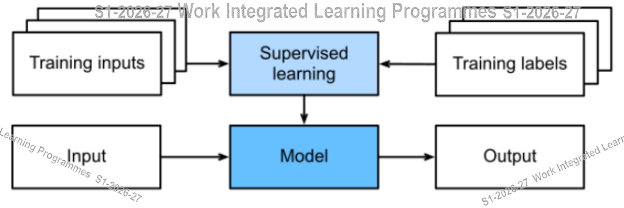
1. Supervised Learning
   - Training by examples
   - The model is given a dataset which has the data and its labels
   - Eg: Classification, Regression
   - You will learn this in this course and Machine Learning Course
2. Unsupervised Learning
   - Situations where we feed the model a giant dataset containing only the features
   - Eg: Clustering, Generate synthetic data (uses GAN)
   - You will learn this in Advanced Deep Learning Course and Machine Learning Course
3. Reinforcement Learning
   - Develop an agent that interacts with an environment and takes actions over a series of time steps.
   - You will learn this in Deep Reinforcement Learning Course

Supervised Learning

➔ Each **feature–label pair** is called an example.

➔ Goal is to produce **a model that maps any input to a label prediction**.

➔ The supervision comes into play because for choosing the parameters, we (the supervisors) provide the model with a dataset consisting of labeled examples, where each example is matched with conditional probability of a label given input features.



Supervised Learning Tasks

- Regression
  - how much or how many question
- Classification
  - Binary classification
  - Multi-class classification
  - Multi-label classification
- Recommender systems
- Sequence Learning
  - Tagging and Parsing
  - Automatic speech recognition
  - Text to speech
  - Machine translation

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

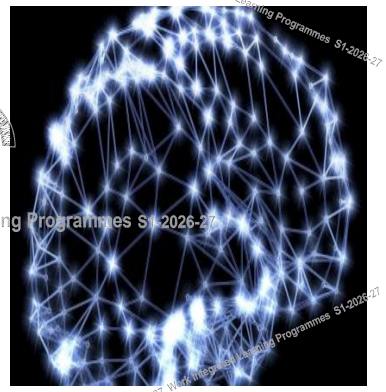
Human Brain

- It begins with human brain.
- Humans learn, solve problems, recognize patterns, create, think deeply about something, meditate and many, many more.....
- Humans learn through **association**. [Refer to Associationism for more details.]

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Human Brain

- The brain is a mass of interconnected neurons.
- Number of neurons is approximately 10^(10).
- Connections per neuron is approximately 10^(4 to 5).
- Neuron switching time is approximately 0.001 second.
- Scene recognition time is 1 second.
- 100 inference steps doesn't seem like enough. Lot of parallel computation.



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

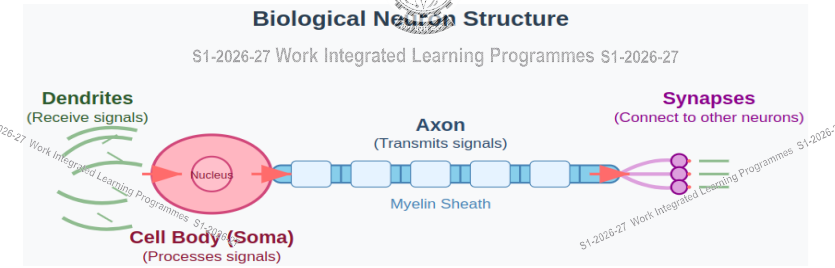
Human Brain

- Many neurons connect **in** to each neuron.
- Each neuron connects **out** to many neurons.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Key Components:

- **Dendrites:** Receive signals from other neurons
- **Cell Body (Soma):** Processes incoming signals
- **Axon:** Transmits output signal
- **Synapses:** Connection points between neurons

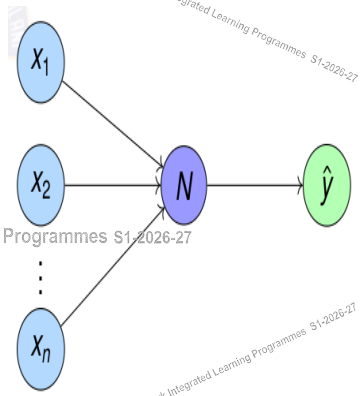


S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Artificial Neuron

- Neuron is a processing element inspired by how the brain works.
- Similar to biological neuron, each artificial neuron will be do some computation. Each neuron is interconnected to other neurons.

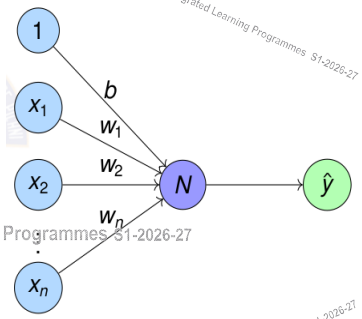
Similar to brain, the interconnections between neurons store the knowledge it learns. The knowledge is stored as parameters.



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Artificial Neuron

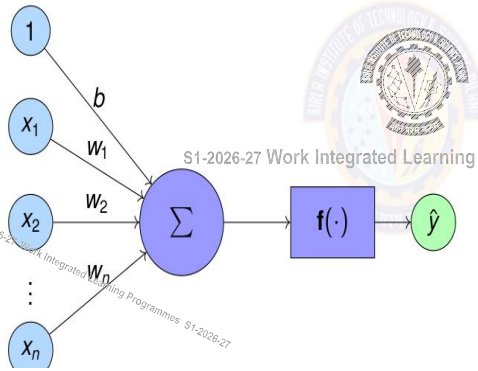
- Processing neuron N performs a very basic computation.
- For each feature x_i , weight w_i shows the importance to the feature
- N generates one numerical output y



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Artificial Neuron – Mathematical Computation

- Sum of weights w_i times corresponding features x_i
- Apply activation function f () on the sum.



$$z = \sum_{i=1}^{n} w_i x_i + b \qquad (1)$$

$$\hat{y} = \mathbf{f}\left(\sum_{i=1}^{n} w_i x_i + b\right) \qquad (2)$$

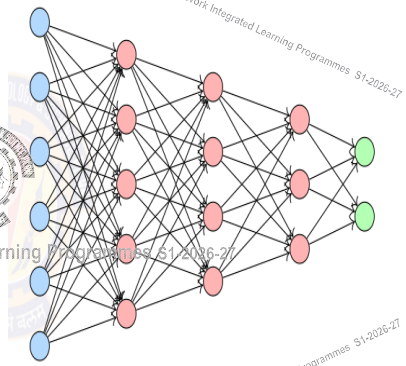
S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Comparison: Biological vs Artificial Neurons

Biological Neuron	Artificial Neuron
Complex biochemical processes	Simple mathematical model
Analog signal processing	Digital computation
Adaptive synaptic strengths	Adjustable weights
Parallel processing	Parallel computation possible
Learning through experience	Learning through algorithms
Fault tolerant	Deterministic behavior

Artificial Neural Network (ANN)

- Neural Network is a computational model inspired by human brain
- Interconnected neurons process information
- Interconnection can be forward, recursive or self loop
- Learn patterns from data through training
- Capable of approximating complex functions



Properties of Artificial Neural Network

- Many neuron-like threshold switching units.
- Many weighted interconnections among units.
- Highly parallel, distributed process.
- Emphasis on tuning parameters or weights automatically.

When to consider Artificial Neural Networks?

- Input is high-dimensional discrete or real-valued (e.g. raw sensor input).
- Possibly noisy data. Data has lots of errors.
- Output is discrete or real valued or a vector of values.
- Form of target function is unknown.
- Human readability, in other words, explainability , of result is unimportant.

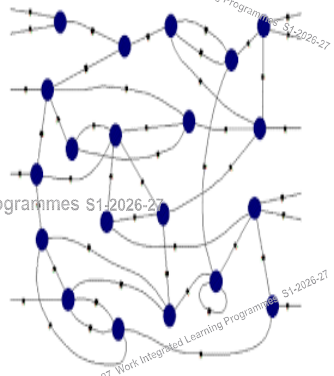
Examples:

- Speech phoneme recognition
- Image classification
- Financial prediction

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Connectionist Machines

- Network of processing elements, called artificial neurons
- The neurons are interconnected to form a network.
- Interconnection can be forward, recursive or self loop
- All world knowledge is stored in the connections between these elements.
- Neural networks are connectionist machines.

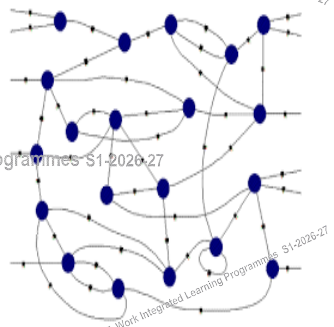


S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Connectionism Principles

Core Ideas:

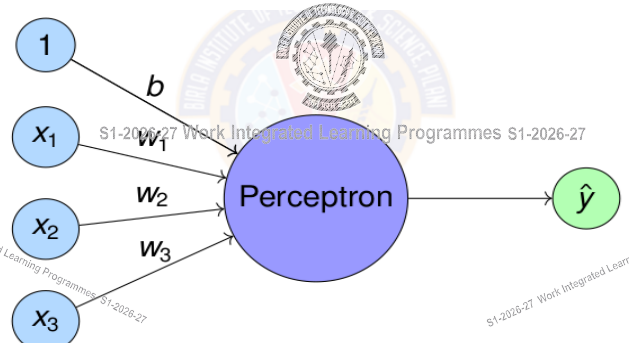
- Intelligence emerges from simple processing neurons
- Knowledge is stored in connection weights
- Learning modifies connection strengths
- Parallel distributed processing



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Perceptron

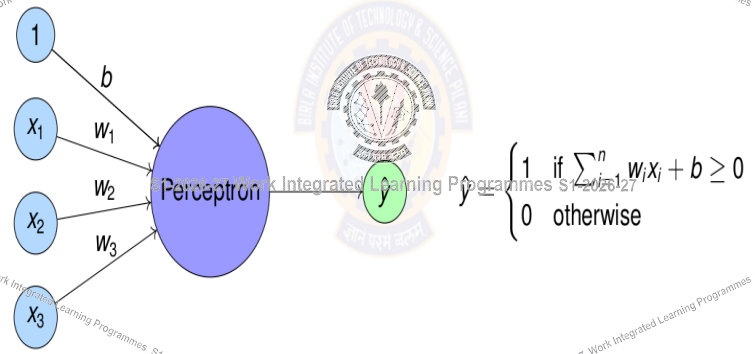
- Perceptron = Simplest form of artificial neuron
- Perceptron was introduced by Rosenblatt in 1958.



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Perceptron

- Perceptron = Simplest form of artificial neuron



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Perceptron

- Any linear decision can be represented by a Boolean equation.
- The quest is to find out how to represent the above using Perceptron.
- For this we test whether each of the logic gates can be represented by a Perceptron.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

AND Logic Gate

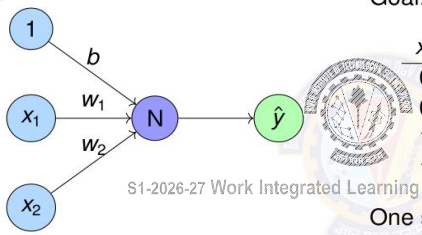
Question:

- How to represent AND gate using a perceptron?
- What are the parameters for the AND perceptron?
- Data is given below.

A	B	A and B
0	0	0
0	1	0
1	0	0
1	1	1

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Perceptron for AND gate



Goal: Find parameters w_0, w_1, w_2

x_1	x_2	t	h
0	0	0	$w_0 + 0 + 0 < 0$
0	1	0	$w_0 + 0 + w_2 < 0$
1	0	0	$w_0 + w_1 + 0 < 0$
1	1	1	$w_0 + w_1 + w_2 \geq 0$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

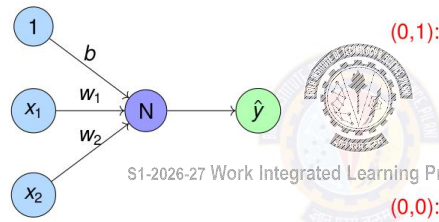
One solution is

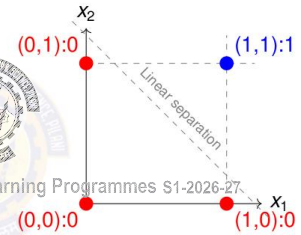
$w_0 = -1$
$w_1 = 0.75$
$w_2 = 0.75$

Perceptron equation is

$$\hat{y} = w_0 x_0 + w_1 x_1 + w_2 x_2$$

Perceptron for AND gate





S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Perceptron equation is

$$\hat{y} = w_0 x_0 + w_1 x_1 + w_2 x_2$$

One solution is

$w_0 = -1$
$w_1 = 0.75$
$w_2 = 0.75$

OR Logic Gate

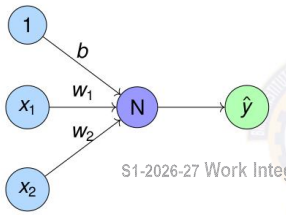
Question:

- How to represent OR gate using a perceptron?
- What are the parameters for the OR perceptron?
- Data is given below.

A	B	A or B
0	0	0
0	1	1
1	0	1
1	1	1

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Perceptron for OR gate



Goal: Find parameters w_0, w_1, w_2

x_1	x_2	t	h
0	0	0	$w_0 + 0 + 0 < 0$
0	1	1	$w_0 + 0 + w_2 \geq 0$
1	0	1	$w_0 + w_1 + 0 \geq 0$
1	1	1	$w_0 + w_1 + w_2 \geq 0$

One solution is

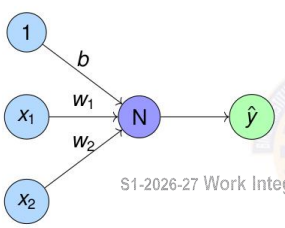
Perceptron equation is

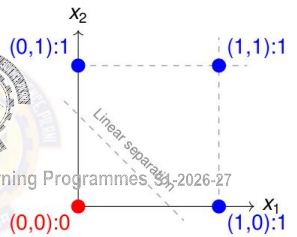
$$\hat{y} = w_0 x_0 + w_1 x_1 + w_2 x_2$$

$$w_0 = -1$$
$$w_1 = 2$$
$$w_2 = 2$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Perceptron for OR gate





Perceptron equation is

$$\hat{y} = w_0 x_0 + w_1 x_1 + w_2 x_2$$

One solution is $w_0 = -1$

$w_1 = 2$

$w_2 = 2$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Exercise

1. Represent OR gate using Perceptron. Compute the parameters of the perceptron.
2. Represent NOR gate using Perceptron. Compute the parameters of the perceptron.
3. Represent NAND gate using Perceptron. Compute the parameters of the perceptron.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Goal: Find weights that correctly classify training data (input, output) pair (x, t).

Algorithm:

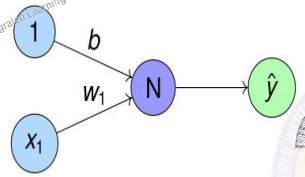
1. Initialize learning rate $\eta = 0.1$
2. Initialize weights randomly: w_i = random
3. For each training example (x, t):
   - Calculate output: $\hat{y} = sign(\sum_i w_i x_i - b)$
   - If $\hat{y} \neq t$, update weights:

$$w_i^{new} = w_i^{old} + \eta(t - \hat{y})x_i$$

$$b^{new} = b^{old} + \eta(t - \hat{y})$$

4. Repeat until convergence or max iterations

Convergence Theorem:

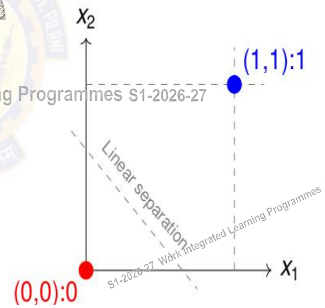


Example	x_1	t
Eg:1	-1	1
Eg:2	1	-1

Perceptron equation

$$h = sign(w_0 x_0 + w_1 x_1)$$

$$\hat{y} = \begin{cases} 1 & \text{if } h \geq 0 \\ -1 & \text{if } h < 0 \end{cases}$$



Assume $w_0 = w_1 = 0$. Let the learning rate$= \eta = 1$.

- Epoch 1 Example 1 pair (1,-1)

$$\hat{y} = sign(w_1 x_1 + w_0) = sign(0 \cdot -1 + 0) = -1$$

Is $\hat{y}$ equal to t? NO. Hence update weights.

$$w_i^{(new)} = w_i(old) + \eta(t - \hat{y})x_i$$

$$w_1^{(new)} = w_1(old) + \eta(t - \hat{y})x_1 = 0 + 1(1 - (-1)) \cdot (-1) = (-2)$$

$$w_0^{(new)} = w_0(old) + \eta(t - \hat{y})x_0 = 0 + 1(1 - (-1)) = (2)$$

- Epoch 1 Example 2 pair (1,-1)

$$\hat{y} = sign(w_1 x_1 + w_0) = sign((-2) \cdot 1 + 2) = sign(0) = -1$$

Is $\hat{y}$ equal to t? YES. NO update weights.

- Epoch 2 Example 1 pair (-1,1)

$$\hat{y} = sign(w_1 x_1 + w_0) = sign((-2) \cdot (-1) + 2) = sign(4) = 1$$

Is $\hat{y}$ equal to t? YES. NO update weights.

- Epoch 2 Example 2 pair (1,1)

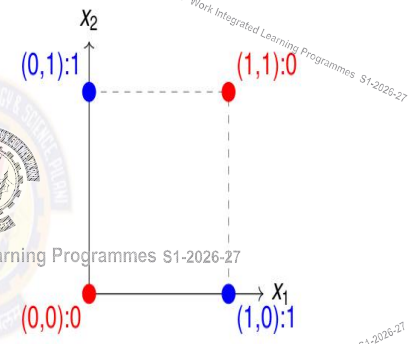
$$\hat{y} = sign(w_1 x_1 + w_0) = sign((-2) \cdot 1 + 2) = sign(0) = -1$$

Is $\hat{y}$ equal to t? YES. NO update weights.

The XOR Problem

Truth Table for XOR:

x_1	x_2	XOR Output
0	0	0
0	1	1
1	0	1
1	1	0



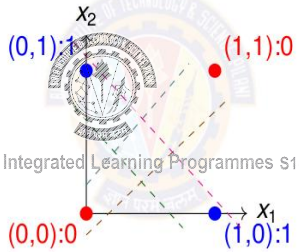
PROBLEM

XOR is not linearly separable! Single perceptron cannot solve XOR.

Why Single Perceptron Fails on XOR

Linear Decision Boundary Limitation:

Any line $w_1 x_1 + w_2 x_2 + b = 0$ cannot separate XOR classes.



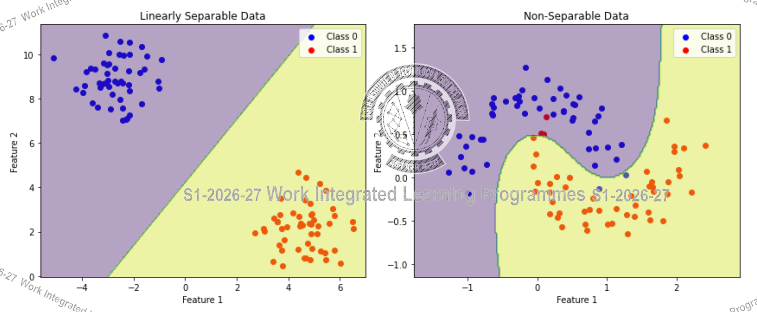
No single line works!

Solution: Need multiple layers (Multilayer Perceptron)

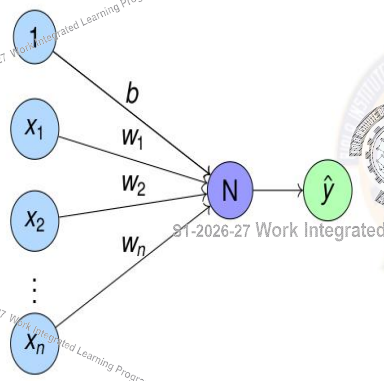
Linearly Separable Data

- **Two sets data points are separable in a n-dimensional space if they can be separated by an (n-1) dimensional hyperplane.**
- A straight line can be drawn to separate all the data examples belonging to class +1 from all the examples belonging to the class -1. Then the two-dimensional data are clearly linearly separable.
- An infinite number of straight lines can be drawn to separate the class +1 from the class -1.

Linearly vs Non-Linearly Separable Data



- Data
  - Truth tables or set of examples
- Model
  - Perceptron
  - Inputs x are the features or columns
  - Target output t is the required label
- Objective Function
  - Deviation of desired output or target t and the computed output ŷ
- Learning Algorithm
  - Perceptron Learning algorithm
- Challenge
  - How to learn these n + 1 parameters w_0 , w_1 . w_2 . . . w_n ?
- Solution
  - Use Perceptron learning algorithm.



$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \quad w = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix}$$

$$h = w_0 + w_1 x_1 + w_2 x_2 + \ldots + w_n x_n$$
$$= w^\top x$$

$$\hat{y} = \begin{cases} 1 & \text{if } h \geq 0 \\ -1 & \text{if } h < 0 \end{cases}$$

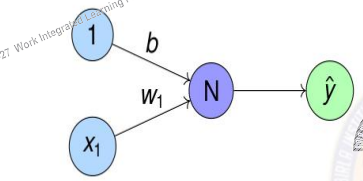
- Represent OR gate using Perceptron. Compute the parameters of the perceptron using perceptron learning algorithm.
- Represent AND gate using Perceptron. Compute the parameters of the perceptron using perceptron learning algorithm.
- Represent NOR gate using Perceptron. Compute the parameters of the perceptron using perceptron learning algorithm.
- Represent NAND gate using Perceptron. Compute the parameters of the perceptron using perceptron learning algorithm.
- Implement a perceptron for AND and OR gates
- Sketch decision boundaries for Single perceptron

- **Neural Network**: Biological inspiration and mathematical models
- **Perceptron**: Simplest neural unit, linear classification, learning algorithm
- **XOR Problem**: Limitation of linear models, need for hidden layers
- **Learning Paradigms**: Supervised, unsupervised, reinforcement learning

Session 3
(10th May,2026)

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

MODULE # 3 :
LINEAR NEURAL NETWORKS FOR REGRESSION

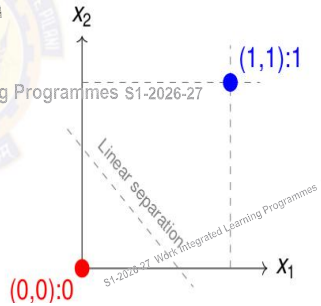


Example	x_1	t
Eg:1	-1	1
Eg:2	1	-1

Perceptron equation

$$h = sign(w_0 x_0 + w_1 x_1)$$

$$\hat{y} = \begin{cases} 1 & \text{if } h \geq 0 \\ -1 & \text{if } h < 0 \end{cases}$$



Assume $w_0 = w_1 = 0$. Let the learning rate$= \eta = 1$.

- Epoch 1 Example 1 pair (1,-1)

$$\hat{y} = sign(w_1 x_1 + w_0) = sign(0 \cdot -1 + 0) = -1$$

Is $\hat{y}$ equal to t? NO. Hence update weights.

$$w_i^{(new)} = w_i(old) + \eta(t - \hat{y})x_i$$

$$w_1^{(new)} = w_1(old) + \eta(t - \hat{y})x_1 = 0 + 1(1 - (-1)) \cdot (-1) = (-2)$$

$$w_0^{(new)} = w_0(old) + \eta(t - \hat{y})x_0 = 0 + 1(1 - (-1)) = (2)$$

- Epoch 1 Example 2 pair (1,-1)

$$\hat{y} = sign(w_1 x_1 + w_0) = sign((-2) \cdot 1 + 2) = sign(0) = -1$$

Is $\hat{y}$ equal to t? YES. NO update weights.

- Epoch 2 Example 1 pair (-1,1)

$$\hat{y} = sign(w_1 x_1 + w_0) = sign((-2) \cdot (-1) + 2) = sign(4) = 1$$

Is $\hat{y}$ equal to t? YES. NO update weights.

- Epoch 2 Example 2 pair (1,1)

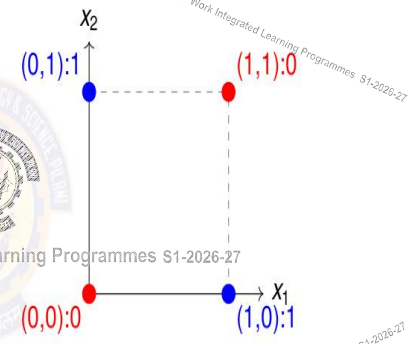
$$\hat{y} = sign(w_1 x_1 + w_0) = sign((-2) \cdot 1 + 2) = sign(0) = -1$$

Is $\hat{y}$ equal to t? YES. NO update weights.

The XOR Problem

Truth Table for XOR:

x_1	x_2	XOR Output
0	0	0
0	1	1
1	0	1
1	1	0



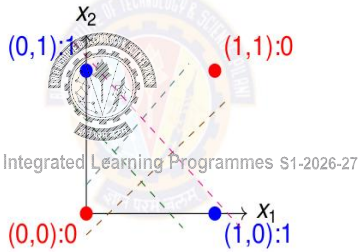
PROBLEM

XOR is not linearly separable! Single perceptron cannot solve XOR.

Why Single Perceptron Fails on XOR

Linear Decision Boundary Limitation:

Any line $w_1 x_1 + w_2 x_2 + b = 0$ cannot separate XOR classes.



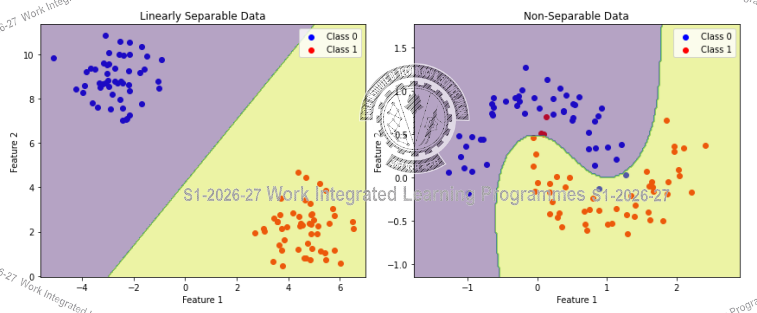
No single line works!

Solution: Need multiple layers (Multilayer Perceptron)

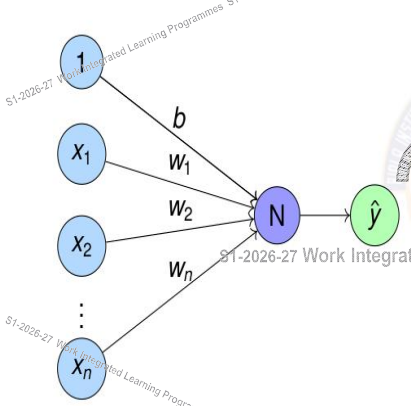
Linearly Separable Data

- **Two sets data points are separable in a n-dimensional space if they can be separated by an (n-1) dimensional hyperplane.**
- A straight line can be drawn to separate all the data examples belonging to class +1 from all the examples belonging to the class -1. Then the two-dimensional data are clearly linearly separable.
- An infinite number of straight lines can be drawn to separate the class +1 from the class -1.

Linearly vs Non-Linearly Separable Data



Perceptron for Linearly Separable Data



$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \quad w = \begin{bmatrix} w_0 \\ w_1 \\ w_2 \\ \vdots \\ w_n \end{bmatrix}$$

$$h = w_0 + w_1 x_1 + w_2 x_2 + \ldots + w_n x_n$$
$$= w^\top x$$

$$\hat{y} = \begin{cases} 1 & \text{if } h \geq 0 \\ -1 & \text{if } h < 0 \end{cases}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Table of Contents

- ✓ Regression
- ✓ Linear Regression
- ✓ The Linear Model: Single Neuron
- ✓ Worked Example
- ✓ Evaluation
- ✓ Python Implementation Implementation Tips
- ✓ Summary

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

What is Regression?

❑ **Definition**

Regression is a supervised learning task that predicts a continuous-valued output based on input features.

- Key Characteristics:
  - Input: Feature vector $\mathbf{x} \in \mathbb{R}^d$
  - Output: Real-valued target $y \in \mathbb{R}$
  - Goal: Learn a function $f : \mathbb{R}^d \rightarrow \mathbb{R}$ such that $y \approx f(\mathbf{x})$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Examples of Regression Problems

- **House Price Prediction**
  - Input: Size, location, number of rooms
  - Output: Price in dollars
- **Temperature Forecasting**
  - Input: Historical weather data, season, humidity
  - Output: Temperature in degrees
- **Stock Price Prediction**
  - Input: Historical prices, trading volume, market indicators
  - Output: Future stock price

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

WHAT IS LINEAR REGRESSION?

DEFINITION

- **Linear regression assumes the output is a linear combination of the input features.**

➢ **Mathematical Form:**

$$y = w_0 + w_1x_1 + w_2x_2 + \cdots + w_dx_d$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

where:

$\mathbf{x} = [x_1, x_2, \ldots, x_d]^T$ are the input features

$\mathbf{w} = [w_1, w_2, \ldots, w_d]^T$ are the weights

w_0 is the bias term

EXAMPLES OF LINEAR REGRESSION

➢ **Salary Prediction**

- Salary = $w_0 + w_1 \times$ Years of Experience

➢ **Crop Yield Prediction**

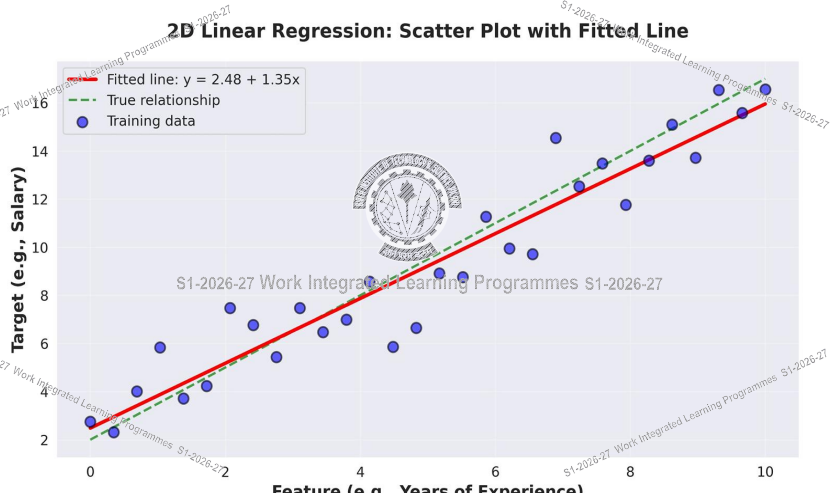
- Yield = $w_0 + w_1 \times$ Rainfall + $w_2 \times$ Fertilizer

➢ **Energy Consumption**

- Energy = $w_0 + w_1 \times$ Temperature + $w_2 \times$ Hour + $w_3 \times$ Day

The relationship between inputs and output is assumed to be linear.

VISUAL EXAMPLE: 2D LINEAR REGRESSION



THE FOUR COMPONENTS

- A complete machine learning system consists of:
  - Data: d -dimensional input vectors and target outputs
  - Model: Single neuron implementing linear function
  - Objective Function: Measures prediction error
  - Learning Algorithm: Gradient descent to optimize weights

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

MATRIX FORM OF DATA

For efficient computation, we organize data into matrices:

Input Data = Augmented Features:

Add bias term: $\tilde{\mathbf{x}}^{(i)} = [1, x_1^{(i)}, \ldots, x_d^{(i)}]^T$

Each row = one training example

Design Matrix: $\mathbf{X} \in \mathbb{R}^{N \times (d+1)}$ $\mathbf{X} = \begin{bmatrix} 1 & x_1^{(1)} & \cdots & x_d^{(1)} \\ 1 & x_1^{(2)} & \cdots & x_d^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(N)} & \cdots & x_d^{(N)} \end{bmatrix}$

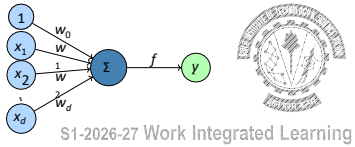
Target Vector: $\mathbf{y} \in \mathbb{R}^N$ $\mathbf{y} = \begin{bmatrix} y^{(1)} & y^{(2)} & \ldots & y^{(N)} \end{bmatrix}^\top$

Weight Vector: $\mathbf{w} \in \mathbb{R}^{d+1}$ $\mathbf{w} = \begin{bmatrix} w_0 & w_1 & w_2 & \ldots & w_d \end{bmatrix}^\top$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

LINEAR MODEL AS A SINGLE NEURON

- Model linear regression as a single artificial neuron:



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

- Input: d-dimensional feature vector $\mathbf{x} \in \mathbb{R}^d$
- Output: Single predicted value $\hat{y} \in \mathbb{R}$

LINEAR MODEL PREDICTION

Model: A single neuron computes a weighted sum and applied identity activation function.

Prediction for input **x**:

$$\hat{y} = f(\mathbf{w}^T\mathbf{x}) = f(w_0 + w_1x_1 + w_2x_2 + \cdots + w_dx_d) \quad (2)$$

where:

- $\mathbf{w} = [w_0, w_1, \ldots, w_d]^T \in R^{d+1}$ are the model parameters
- $\mathbf{x} = [1, x_1, \ldots, x_d]^T \in R^{d+1}$ is the input
- $f(\cdot)$ is the identity activation $f(z) = z$

Vectorized form: for all predictions

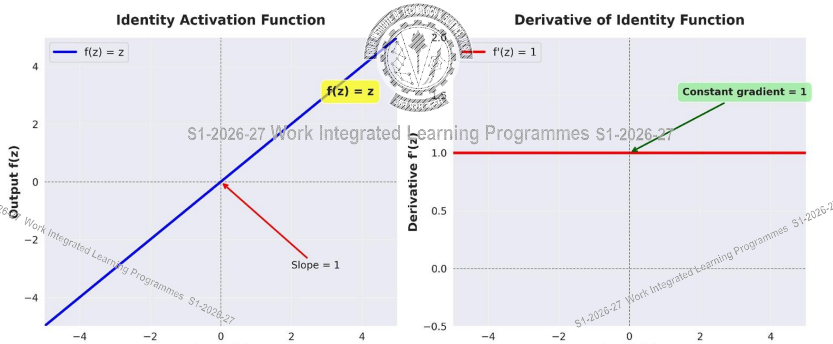
$$\hat{\mathbf{y}} = \mathbf{X}\mathbf{w} \quad (3)$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

IDENTITY ACTIVATION FUNCTION

Identity Activation Function

$$f(z) = z \quad (4)$$



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

IDENTITY ACTIVATION FUNCTION

❖ Properties of Identity Function

- Output: Continuous, (−∞, ∞)
- Differentiable everywhere: $f'(z) = 1$
- Allows output to take any real value
- Gradient flows unchanged through the neuron
- Perfect for predicting continuous targets

❖ Why identity for regression?

- Target values are continuous (prices, temperatures, etc.)
- Need to predict any real number, not just binary { 0,1 }
- Differentiability enables gradient-based optimization
- No information loss from input to output

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

OBJECTIVE FUNCTION: SQUARED ERROR LOSS

We need to measure how well our model fits the data.

Loss for single example:

$$\ell\left(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)}\right) = \frac{1}{2}\left(\hat{y}^{(i)} - y^{(i)}\right)^2 = \frac{1}{2}\left(\mathbf{w}^T\mathbf{x}^{(i)} - y^{(i)}\right)^2 \quad (5)$$

Total Loss (Mean Squared Error):

$$J(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N}\frac{1}{2}\left(\mathbf{w}^T\mathbf{x}^{(i)} - y^{(i)}\right)^2 \quad (6)$$

Goal: Find $\mathbf{w}^*$ that minimizes $J(\mathbf{w})$

$$\mathbf{w}^* = \arg\min_{\mathbf{w}} J(\mathbf{w}) \quad (7)$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

WITH SQUARED ERROR?

Advantages of squared error loss:

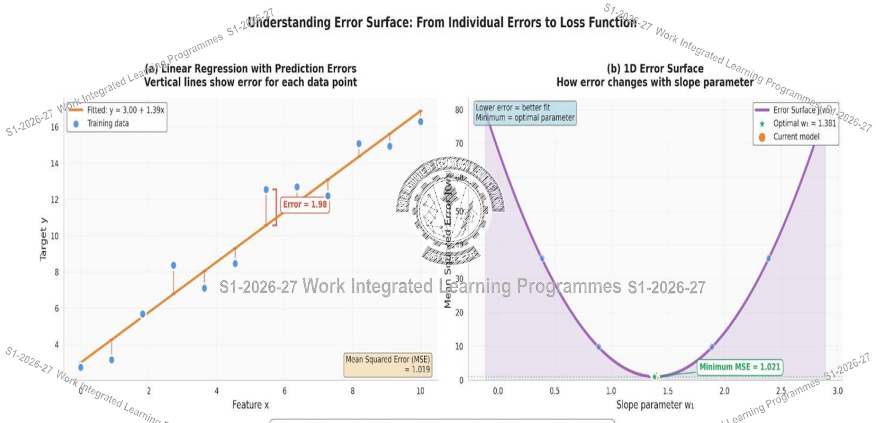
- **Differentiable**: Smooth everywhere, easy to optimize
- **Convex:** Single global minimum (for linear models)
- **Penalizes large errors**: Quadratic growth
- **Statistical interpretation**: Maximum likelihood under Gaussian noise

Vector Form:

$$J(\mathbf{w}) = \frac{1}{2N}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2 = \frac{1}{2N}(\mathbf{X}\mathbf{w} - \mathbf{y})^T(\mathbf{X}\mathbf{w} - \mathbf{y})$$

where $\mathbf{X} \in R^{N\times(d+1)}$ is the design matrix and $\mathbf{y} \in R^N$ is the target vector.

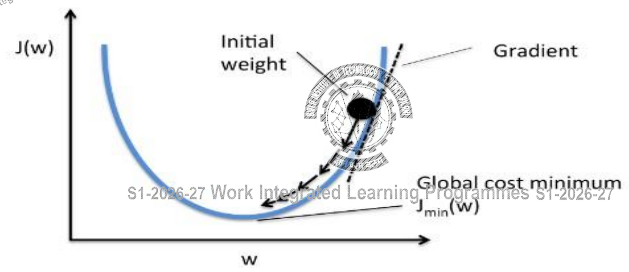
S1-2026-27 Work Integrated Learning Programmes S1-2026-27



- The error surface is convex with a single global minimum

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Idea: Iteratively update weights in the direction of steepest descent



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

where $\eta > 0$ is the learning rate (step size).

Intuition: Move downhill on the error surface to find the minimum.

Algorithm 1: Gradient Descent for Linear Regression

Input: Dataset $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{N}$, learning rate η, tolerance ϵ

Output: Learned weights $\mathbf{w}$

Initialize $\mathbf{w}^{(0)} = \mathbf{0}$ (or random);
$t \leftarrow 0$;
while *not converged* **do**
  Compute gradient: $\nabla J(\mathbf{w}^{(t)}) = \frac{1}{N}\mathbf{X}^T(\mathbf{X}\mathbf{w}^{(t)} - \mathbf{y})$;
  Update weights: $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla J(\mathbf{w}^{(t)})$;
  if $\|\nabla J(\mathbf{w}^{(t+1)})\| < \epsilon$ **then**
    break // Converged
  $t \leftarrow t + 1$;
return $\mathbf{w}^{(t+1)}$;

Gradient of squared error loss:

$$\nabla J(\mathbf{w}) = \frac{\partial J}{\partial \mathbf{w}} = \frac{1}{N}\sum_{i=1}^{N}\left(\mathbf{w}^T\mathbf{x}^{(i)} - y^{(i)}\right)\mathbf{x}^{(i)} \tag{9}$$

Matrix Form:

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

$$\nabla J(\mathbf{w}) = \frac{1}{N}\mathbf{X}^T(\mathbf{X}\mathbf{w} - \mathbf{y}) \tag{10}$$

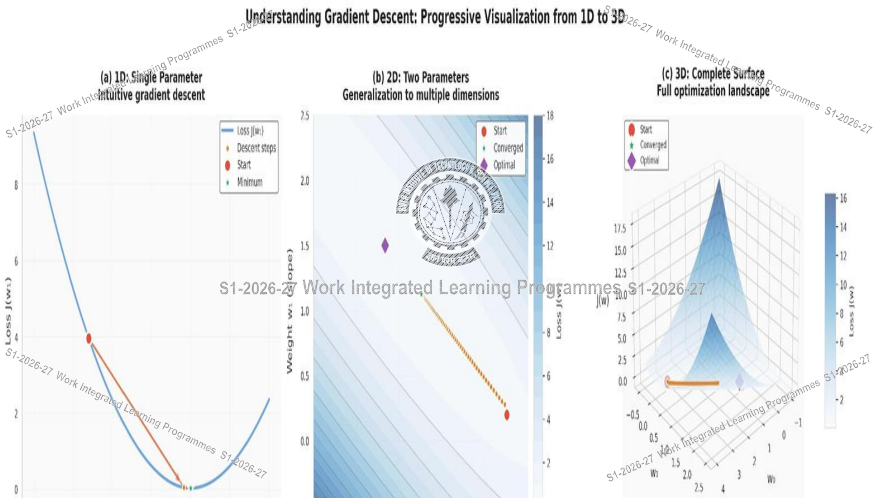
where $\mathbf{X}^T \in \mathbb{R}^{(d+1)\times N}$, $\mathbf{X}\mathbf{w} - \mathbf{y} \in \mathbb{R}^N$, so $\nabla J \in \mathbb{R}^{d+1}$

Update equation:

$$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \frac{\eta}{N}\mathbf{X}^T(\mathbf{X}\mathbf{w}^{(t)} - \mathbf{y}) \tag{11}$$

Key Parameters:

- Learning rate η: Controls step size
  - Too large: May overshoot minimum
  - Too small: Slow convergence
- Stopping criterion: $\|\nabla J(\mathbf{w})\| < \epsilon$ or max iterations

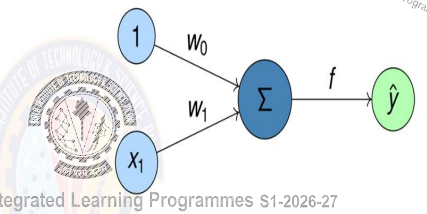


S1-2026-27 Work Integrated Learning Programmes S1-2026-27

NUMERICAL EXAMPLE: SETUP

Tiny Dataset: Predict house price from size

Size (x_1)	Price (y)
1	2
2	4
3	5



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Matrix Formulation:

$$\mathbf{X} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix} \in \mathbb{R}^{3\times 2}, \quad \mathbf{y} = \begin{bmatrix} 2 \\ 4 \\ 5 \end{bmatrix} \in \mathbb{R}^{3}, \quad \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix} \in \mathbb{R}^{2}$$

Hyperparameters: $\eta = 0.1 \quad N = 3$

Initialize weights: $\mathbf{w}^{(0)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$

Vectorized Prediction: $\hat{\mathbf{y}} = \mathbf{X}\mathbf{w}$

Initial predictions: $\hat{\mathbf{y}}^{(0)} = \mathbf{X}\mathbf{w}^{(0)} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Loss Function: $J(\mathbf{w}) = \frac{1}{2N}\|\mathbf{X}\mathbf{w} - \mathbf{y}\|^2$

Initial loss: $J(\mathbf{w}^{(0)}) = \frac{1}{6}\left\| \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 2 \\ 4 \\ 5 \end{bmatrix} \right\|^2 = \frac{1}{6}(4 + 16 + 25) = 7.5$

Error vector: $\mathbf{e}^{(0)} = \hat{\mathbf{y}}^{(0)} - \mathbf{y} = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} - \begin{bmatrix} 2 \\ 4 \\ 5 \end{bmatrix} = \begin{bmatrix} -2 \\ -4 \\ -5 \end{bmatrix}$

Gradient computation: $\nabla J(\mathbf{w}^{(0)}) = \frac{1}{N}\mathbf{X}^T\mathbf{e}^{(0)}$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

$$\nabla J(\mathbf{w}^{(0)}) = \frac{1}{3}\begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix}\begin{bmatrix} -2 \\ -4 \\ -5 \end{bmatrix}$$

$$= \frac{1}{3}\begin{bmatrix} -2-4-5 \\ -2-8-15 \end{bmatrix} = \begin{bmatrix} -3.67 \\ -8.33 \end{bmatrix}$$

Gradient descent update: $\mathbf{w}^{(1)} = \mathbf{w}^{(0)} - \eta \nabla J(\mathbf{w}^{(0)})$

Computing new weights: $\mathbf{w}^{(1)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} - 0.1 \begin{bmatrix} -3.67 \\ -8.33 \end{bmatrix} = \begin{bmatrix} 0.367 \\ 0.833 \end{bmatrix}$

New predictions: $\hat{\mathbf{y}}^{(1)} = \mathbf{X}\mathbf{w}^{(1)}$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

$$\hat{\mathbf{y}}^{(1)} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix} \begin{bmatrix} 0.367 \\ 0.833 \end{bmatrix} = \begin{bmatrix} 1.20 \\ 2.03 \\ 2.87 \end{bmatrix}$$

New error vector: $\mathbf{e}^{(1)} = \hat{\mathbf{y}}^{(1)} - \mathbf{y} = \begin{bmatrix} 1.20 \\ 2.03 \\ 2.87 \end{bmatrix} - \begin{bmatrix} 2 \\ 4 \\ 5 \end{bmatrix} = \begin{bmatrix} -0.80 \\ -1.97 \\ -2.13 \end{bmatrix}$

Loss after one iteration: $J(\mathbf{w}^{(1)}) = \frac{1}{6}\|\mathbf{e}^{(1)}\|^2 = \frac{1}{6}(0.64 + 3.88 + 4.54) \approx 1.51$

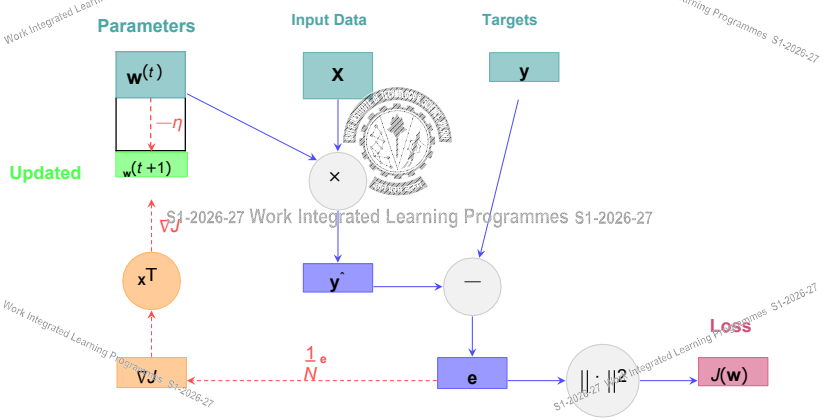
S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Progress:

- Initial loss: $J(\mathbf{w}^{(0)}) = 7.5$
- After 1 iteration: $J(\mathbf{w}^{(1)}) = 1.51$
- **80% reduction!** ✓

Continue until $\|\nabla J(\mathbf{w})\| < \epsilon$ or max iterations reached

COMPUTATIONAL GRAPH FOR ONE GRADIENT



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Key Principle: Evaluate on unseen data

Data Split:

- Training set: Used to learn weights $\mathbf{w}$ (typically 90-99%)
- Test set: Used to evaluate generalization (typically 1-10%)

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Training Error: $J_{\text{train}} = \frac{1}{N_{\text{train}}} \sum_{i \in \text{train}} \left(\hat{y}^{(i)} - y^{(i)}\right)^2$

Test Error: $J_{\text{test}} = \frac{1}{N_{\text{test}}} \sum_{i \in \text{test}} \left(\hat{y}^{(i)} - y^{(i)}\right)^2$

Goal: Minimize test error (generalization performance)

1. Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{N}\sum_{i=1}^{N}\left(\hat{y}^{(i)} - y^{(i)}\right)^2 \tag{12}$$

2. Root Mean Squared Error (RMSE):

$$\text{RMSE} = \sqrt{\text{MSE}} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}\left(\hat{y}^{(i)} - y^{(i)}\right)^2} \tag{13}$$

3. Mean Absolute Error (MAE):(Less sensitive to outliers than MSE)

$$\text{MAE} = \frac{1}{N}\sum_{i=1}^{N} \mid y^{(i)} - \hat{y}^{(i)} \mid \tag{14}$$

Definition: Proportion of variance explained by the model

$$R^2 = 1 - \frac{\sum_{i=1}^{N}\left(y^{(i)} - \hat{y}^{(i)}\right)^2}{\sum_{i=1}^{N}\left(y^{(i)} - \bar{y}\right)^2} = 1 - \frac{\text{SS}_{\text{residual}}}{\text{SS}_{\text{total}}} \tag{15}$$

where $\bar{y} = \frac{1}{N}\sum_{i=1}^{N} y^{(i)}$ is the mean.

Interpretation:

- $R^2 = 1$: Perfect fit (all variance explained)
- $R^2 = 0$: Model no better than predicting mean
- $R^2 < 0$: Model worse than baseline (possible on test set)

1. Choosing Learning Rate η:

- Start with $\eta \in \{0.001, 0.01, 0.1, 1.0\}$
- Plot loss vs iterations for each
- If loss increases or becomes NaN → decrease η
- If convergence too slow → increase η
- Use learning rate schedules for fine-tuning

2. When to Stop Training:

- Gradient magnitude: $\|\nabla J\| < \epsilon$ (e.g., $\epsilon = 10^{-4}$)
- Loss change: $|J^{(t)} - J^{(t-1)}| < \epsilon$
- Maximum iterations reached (e.g., $T = 1000$ or 10,000)

3. Weight Initialization:

- Zero initialization: $\mathbf{w}^{(0)} = \mathbf{0}$ (works well for linear regression)
- Avoid large initial values (can cause divergence or overflow)

4. Feature Engineering:

- Domain knowledge: Create meaningful derived features (e.g., price per sq ft)
- Feature selection: Remove irrelevant/redundant features

5. Feature Scaling and Normalization:

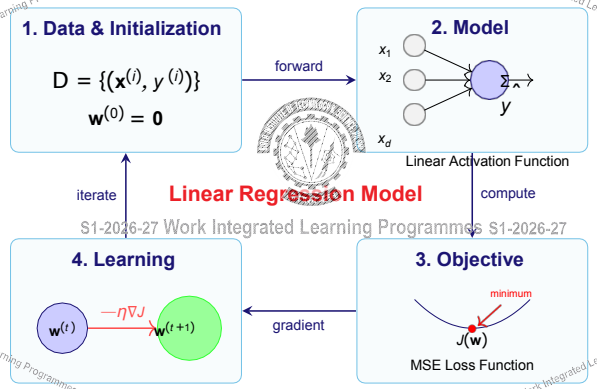
- Features with different scales can slow convergence
- 1. Min-Max Scaling: Scale to $[0, 1]$ $x_j' = \frac{x_j - \min(x_j)}{\max(x_j) - \min(x_j)}$
- 2. Standardization (Z-score): Zero mean, unit variance $x_j' = \frac{x_j - \mu_j}$

Common Issues and Solutions:

- **Loss is NaN/Inf:**
  - Learning rate too large
  - Numerical overflow (check feature scales)
  - Solution: Decrease η, scale features
- **Training works but test error high:**
  - Overfitting (too many features relative to data)
  - Solution: Regularization, more data, or simpler model
- **Loss oscillating:**
  - Learning rate too large
  - Solution: Decrease η or use learning rate decay
- **Loss not decreasing:**
  - Learning rate too small
  - Bug in gradient computation (check signs!)
  - Wrong sign in update rule (should be minus)
  - Solution: Increase η, verify code

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

SUMMARY: PUTTING IT ALL TOGETHER

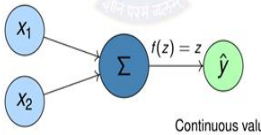


This framework extends to more complex models: neural networks, deep learning!

- Regression predicts continuous outputs from input features.
- Linear regression assumes a linear relationship: $y = \mathbf{w}^T\mathbf{x} + w_0$
- A single neuron with identity activation implements linear regression.
- Squared error loss provides a convex objective function.
- Gradient descent iteratively optimizes weights: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J$
- Feature scaling is critical for fast convergence.
- Evaluation on test set ensures generalization.

Key insight: Linear relationship between inputs and output, optimized by minimizing squared errors.

- Problem: Predict continuous values: $y \in \mathbb{R}$
- Model: Single neuron with identity activation
- Output: Predicted value $\hat{y} = \mathbf{w}^T\mathbf{x} \in (-\infty, \infty)$
- Loss: Mean Squared Error (MSE)
- Training: Batch Gradient Descent (GD)



WHAT IS CLASSIFICATION?

DEFINITION

Classification is a supervised learning task that predicts a discrete category or class label based on input features.

Key Characteristics:

- Input: Feature vector $\mathbf{x} \in \mathbb{R}^d$
- Output: Class label $y \in \{1, 2, \ldots, K\}$ (discrete)
- Goal: Learn a function $f : \mathbb{R}^d \rightarrow \{1, 2, \ldots, K\}$ that assigns inputs to classes

Difference from Regression:

- Regression: Predicts continuous values (e.g., price, temperature)
- Classification: Predicts discrete categories (e.g., spam/not spam, cat/dog)

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

EXAMPLES OF CLASSIFICATION PROBLEMS

Email Spam Detection

- Input: Email text, sender information, attachments
- Output: Spam or Not Spam

Medical Diagnosis

- Input: Patient symptoms, test results, medical history
- Output: Disease present or absent

Image Recognition

- Input: Pixel values of an image
- Output: Object category (cat, dog, car, etc.)

Sentiment Analysis

- Input: Customer review text

TYPES OF CLASSIFICATION

1.Binary Classification

- Two classes: $y \in \{0, 1\}$ or $\{-1, +1\}$
- Examples: Spam detection, disease diagnosis, fraud detection

2.Multi-class Classification

- Multiple classes: $y \in \{1, 2, \ldots, K\}$ where $K > 2$
- Examples: Digit recognition (0-9), animal classification
- Each example belongs to exactly one class

3.Multi-label Classification

- Each example can belong to multiple classes simultaneously
- Examples: Movie genres (action + comedy), document tagging

BINARY CLASSIFICATION PROBLEM

DEFINITION

Binary classification assigns each input to one of two classes.

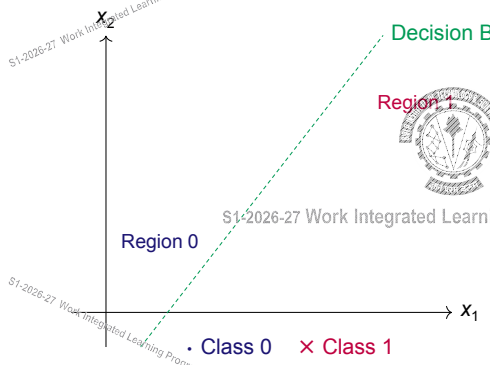
Standard Notation:

- Classes: $y \in \{0, 1\}$
  - $y = 1$: Positive class (e.g., spam, disease present)
  - $y = 0$: Negative class (e.g., not spam, disease absent)
- Dataset: $\mathcal{D} = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{N}$
- Goal: Learn $f(\mathbf{x})$ that outputs probability of class 1

Decision Boundary:

- Predict $\hat{y} = 1$ if $f(\mathbf{x}) \geq 0.5$
- Predict $\hat{y} = 0$ if $f(\mathbf{x}) < 0.5$

Binary Classification: Visual Example



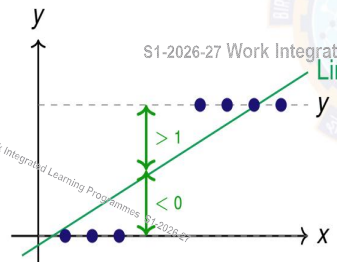
The decision boundary separates the two classes in feature space.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Why Not Use Linear Regression?

Problem with Linear Regression for Classification:

- Linear regression outputs: $\hat{y} \in (-\infty, \infty)$
- We need probabilities: $\hat{y} \in [0, 1]$
- Linear model can predict values < 0 or > 1



Solution:
Use a function that maps $(-\infty, \infty) \rightarrow [0, 1]$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

What is Logistic Regression?

Definition

Logistic regression is a linear model for binary classification that uses the sigmoid function to output probabilities.

Key Idea:

- Compute linear combination: $z = \mathbf{w}^T\mathbf{x} + w_0$
- Apply sigmoid function: $\hat{y} = \sigma(z)$
- Interpret output as probability: $\hat{y} = P(y = 1|\mathbf{x}) \in [0, 1]$

Mathematical Form:

$$\hat{y} = \sigma(\mathbf{w}^T\mathbf{x}) = \frac{1}{1 + e^{-\mathbf{w}^T\mathbf{x}}} \quad (1)$$

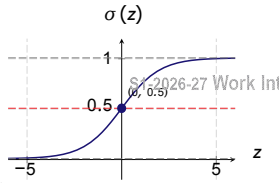
where $\sigma(\cdot)$ is the sigmoid (logistic) function.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Sigmoid (Logistic) Function: $\sigma(z) = \frac{1}{1+e^{-z}}$

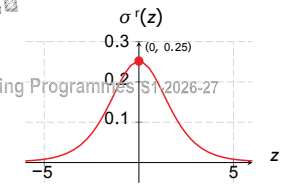
Derivative formula: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$

Function



At $z = 0$: $\sigma(0) = 0.5$

Derivative



$\sigma'(0) = 0.25$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Properties of Sigmoid Function

- **Key Properties:**
  - Range: $\sigma(z) \in (0, 1)$ → perfect for probabilities!
  - Monotonic: Always increasing
  - Smooth: Differentiable everywhere
  - Symmetry: $\sigma(-z) = 1 - \sigma(z)$
  - Derivative: $\sigma'(z) = \sigma(z)(1 - \sigma(z))$
- Behavior:
  - When $z \to +\infty$: $\sigma(z) \to 1$ (confident class 1)
  - When $z \to -\infty$: $\sigma(z) \to 0$ (confident class 0)
  - When $z = 0$: $\sigma(z) = 0.5$ (uncertain)

The derivative property makes sigmoid ideal for gradient-based learning!

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

The Four Components

A complete deep learning system for classification consists of:

- Data: d-dimensional input vectors and binary labels
- Model: Single neuron with sigmoid activation
- Objective Function: Binary cross-entropy loss
- Learning Algorithm: Stochastic gradient descent (SGD)

Note: We use **stochastic** gradient descent instead of batch gradient descent for efficiency. (This is for learning purposes only.)

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Matrix Form of Data

For efficient computation, we organize data into matrices:

Input Data = $\mathbf{x}^{(i)} = [1, x_1^{(i)}, \ldots, x_d^{(i)}]^T$

Design Matrix: $\mathbf{X} \in \mathbb{R}^{N \times (d+1)}$ $\mathbf{X} = \begin{bmatrix} 1 & x_1^{(1)} & \cdots & x_d^{(1)} \\ 1 & x_1^{(2)} & \cdots & x_d^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(N)} & \cdots & x_d^{(N)} \end{bmatrix}$

Label Vector: $\mathbf{y} \in \{0, 1\}^N$ $\mathbf{y} = \begin{bmatrix} y^{(1)} & y^{(2)} & \ldots & y^{(N)} \end{bmatrix}^\top$

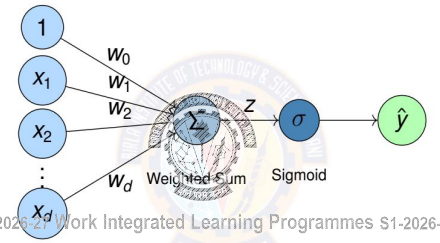
Weight Vector: $\mathbf{w} \in \mathbb{R}^{d+1}$ $\mathbf{w} = \begin{bmatrix} w_0 & w_1 & w_2 & \ldots & w_d \end{bmatrix}^\top$

Key difference from regression: $y^{(i)} \in \{0, 1\}$ (discrete), not continuous!

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Logistic Regression as a Single Neuron

Model binary classification as a single artificial neuron with sigmoid activation:



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

- Input: $\mathbf{x} \in \mathbb{R}^d$
- Linear combination: $z = \mathbf{w}^T\mathbf{x}$
- Activation: Sigmoid $\hat{y} = \sigma(z)$
- Output: Predicted probability $\hat{y} = P(y = 1|\mathbf{x}) \in [0, 1]$

Model Prediction

Model: A single neuron computes weighted sum and applies sigmoid activation.

Prediction for input $\mathbf{x}$:

$$z = \mathbf{w}^T\mathbf{x} = w_0 + w_1x_1 + w_2x_2 + \cdots + w_dx_d \tag{2}$$

$$\hat{y} = \sigma(z) = \frac{1}{1+e^{-z}} \tag{3}$$

$$\hat{y} = P(y = 1|\mathbf{x}) \in [0, 1] \tag{4}$$

where:

- z is the logit (pre-activation).
- $\hat{y}$ is the predicted probability.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Making Predictions

Decision Rule:

Given predicted probability $\hat{y} = P(y = 1|\mathbf{x})$:

$$\text{Predicted class} = \begin{cases} 1 & \text{if } \hat{y} \geq 0.5 \\ 0 & \text{if } \hat{y} < 0.5 \end{cases} \tag{5}$$

Equivalently, since $\sigma(z) = 0.5$ when $z = 0$:

$$\text{Predicted class} = \begin{cases} 1 & \text{if } \mathbf{w}^T\mathbf{x} \geq 0 \\ 0 & \text{if } \mathbf{w}^T\mathbf{x} < 0 \end{cases} \tag{6}$$

The decision boundary is: $\mathbf{w}^T\mathbf{x} = 0$ (a hyperplane in feature space)

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Binary Cross-Entropy Loss

We cannot use squared error for classification. Instead, we use cross-entropy loss.

Cross-entropy Loss for a single example:

$$\ell(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)}) = -\left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})\right] \tag{7}$$

where $\hat{y}^{(i)} = \sigma(\mathbf{w}^T\mathbf{x}^{(i)})$

Interpretation:

- If $y^{(i)} = 1$: Loss $= -\log(\hat{y}^{(i)})$
  - Penalizes if $\hat{y}^{(i)}$ is far from 1
- If $y^{(i)} = 0$: Loss $= -\log(1 - \hat{y}^{(i)})$
  - Penalizes if $\hat{y}^{(i)}$ is far from 0

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Total Loss Function

Average Binary Cross-Entropy Loss:

$$J(\mathbf{w}) = \frac{-1}{N}\sum_{i=1}^{N}\left[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})\right] \tag{8}$$

Goal:

Find $\mathbf{w}^*$ that minimizes $J(\mathbf{w})$

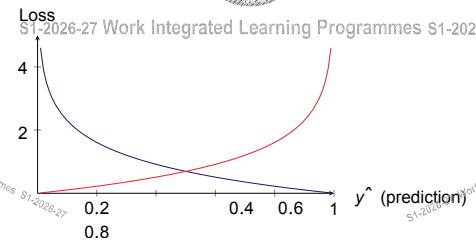
$$\mathbf{w}^* = \arg\min_{\mathbf{w}} J(\mathbf{w}) \tag{9}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Advantages:

- Probabilistic interpretation: Derived from maximum likelihood estimation
- Appropriate for probabilities: Heavily penalizes confident wrong predictions
- Convex: Has a single global minimum for logistic regression
- Well-behaved gradients: Works well with sigmoid activation

Loss behavior: Loss increases exponentially as prediction diverges from true label!



FROM BATCH TO STOCHASTIC GRADIENT DESCENT

Batch Gradient Descent (from Module 3):

- Uses **all** training examples in each update
- Computes gradient: $\nabla J(\mathbf{w}) = \frac{1}{N}\sum_{i=1}^{N} \nabla \ell(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)})$
- Update: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J(\mathbf{w})$

Problems with Batch GD:

- Slow for large datasets (must process all N examples per update)
- High memory requirements
- Redundant computations when examples are similar

Solution: Stochastic Gradient Descent (SGD)

- Update weights using **one random example** at a time
- Much faster iterations

STOCHASTIC GRADIENT DESCENT (SGD)

Key Idea:

Instead of computing gradient over entire dataset, use gradient from a **single random example**.

SGD Update Rule:

At iteration t :

- Randomly select one example $(\mathbf{x}^{(i)}, y^{(i)})$
- Compute gradient for this example: $\nabla l(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)})$ (noisy gradient estimate)
- Update weights: $\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} - \eta \nabla l(\mathbf{w}^{(t)}; \mathbf{x}^{(i)}, y^{(i)})$
- The noise in SGD actually helps escape local minima and often leads to better generalization!

COMPUTING THE GRADIENT FOR SGD

Gradient of cross-entropy loss for one example:

For example $(\mathbf{x}^{(i)}, y^{(i)})$:

$$\nabla_{\mathbf{w}} \ell(\mathbf{w}; \mathbf{x}^{(i)}, y^{(i)}) = \frac{\partial \ell}{\partial \mathbf{w}} = \left(\hat{y}^{(i)} - y^{(i)}\right) \mathbf{x}^{(i)} \quad (10)$$

where $\hat{y}^{(i)} = \sigma(\mathbf{w}^T \mathbf{x}^{(i)})$ for $j = 0, 1, \ldots, d$ (where $x_0^{(i)} = 1$ for bias)

Remarkably simple form!

- Error: $(\hat{y}^{(i)} - y^{(i)})$
- Direction: $\mathbf{x}^{(i)}$
- Same structure as linear regression gradient!

Stochastic Gradient Descent Algorithm

Algorithm 1: Stochastic Gradient Descent for Logistic Regression

Input: Dataset $D = \{(\mathbf{x}^{(i)}, y^{(i)})\}_{i=1}^{N}$, learning rate η, epochs T

Output: Learned weights $\mathbf{w}$

Initialize $\mathbf{w}^{(0)} = \mathbf{0}$ (or small random values);

for *epoch* $= 1$ *to* T **do**

  Shuffle dataset D;

  for *each example* $(\mathbf{x}^{(i)}, y^{(i)})$ *in* D **do**

    Compute prediction: $\hat{y}^{(i)} = \sigma(\mathbf{w}^T \mathbf{x}^{(i)})$;

    Compute gradient: $\nabla l = (\hat{y}^{(i)} - y^{(i)})\mathbf{x}^{(i)}$;

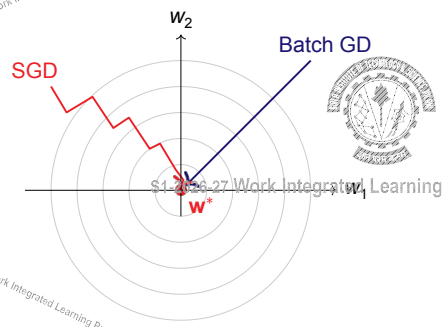
    Update weights: $\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla l$;

  Optional: Compute total loss $J(\mathbf{w})$ *for monitoring*;

return $\mathbf{w}$;

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

SGD vs Batch GD: Visual Comparison



Batch GD: Smooth, deterministic path

SGD: Noisy, stochastic path but faster iterations

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Benefits and Challenges of SGD

Benefits:

- Speed: Much faster per iteration (only one example)
- Memory: Can handle datasets that don't fit in memory
- Online learning: Can update model as new data arrives
- Generalization: Noise helps escape sharp minima
- Scalability: Works well with massive datasets

Challenges:

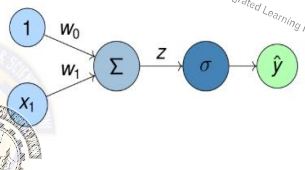
- Noisy updates: Path to minimum is not smooth
- Learning rate: More sensitive to η choice
- Convergence: May oscillate near minimum
- Hyperparameter tuning: Requires careful tuning

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Numerical Example: Setup

Tiny Dataset: Classify whether a student passes based on study hours

Hours (x_1)	Pass (y)
1	0
2	0
3	1
4	1



Matrix Formulation:

$$\mathbf{X} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{bmatrix}, \quad \mathbf{y} = \begin{bmatrix} 0 \\ 0 \\ 1 \\ 1 \end{bmatrix}, \quad \mathbf{w} = \begin{bmatrix} w_0 \\ w_1 \end{bmatrix}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Numerical Example: Initialization

Hyperparameters: $\eta = 0.5, \quad N = 4$

Initialize weights: $\mathbf{w}^{(0)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$

Sigmoid function: $\sigma(z) = \dfrac{1}{1+e^{-z}}$

Loss function: $\ell(\mathbf{w}; \mathbf{x}, y) = -[y \log(\hat{y}) + (1-y)\log(1-\hat{y})]$

Initial predictions with $\mathbf{w}^{(0)} = \begin{bmatrix} 0 & 0 \end{bmatrix}^T$: For all examples: $z = 0 \Rightarrow \hat{y} = \sigma(0) = 0.5$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

$$\hat{\mathbf{y}}^{(0)} = \begin{bmatrix} 0.5 \\ 0.5 \\ 0.5 \\ 0.5 \end{bmatrix}$$

Numerical Example: First SGD Iteration

Randomly select first example: $(\mathbf{x}^{(1)}, y^{(1)}) = (\begin{bmatrix} 1 & 1 \end{bmatrix}, 0)$

Step 1 - Forward pass: $z^{(1)} = \mathbf{x}^{(1)}\mathbf{w}^{(0)} = \begin{bmatrix} 1 & 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} = 0$

$\hat{y}^{(1)} = \sigma(0) = 0.5$

Step 2 - Compute error: $\text{error} = \hat{y}^{(1)} - y^{(1)} = 0.5 - 0 = 0.5$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Step 3 - Compute gradient: $\nabla \ell = (\hat{y}^{(1)} - y^{(1)})\mathbf{x}^{(1)} = 0.5 \begin{bmatrix} 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix}$

Step 4 - Update weights: $\mathbf{w}^{(1)} = \mathbf{w}^{(0)} - \eta \nabla \ell$

$$= \begin{bmatrix} 0 \\ 0 \end{bmatrix} - 0.5 \begin{bmatrix} 0.5 \\ 0.5 \end{bmatrix} = \begin{bmatrix} -0.25 \\ -0.25 \end{bmatrix}$$

Numerical Example: Second SGD Iteration

Randomly select second example: $(\mathbf{x}^{(3)}, y^{(3)}) = (\begin{bmatrix} 1 & 3 \end{bmatrix}, 1)$

Step 1: Forward pass: $z^{(3)} = \mathbf{x}^{(3)}\mathbf{w}^{(1)} = \begin{bmatrix} 1 & 3 \end{bmatrix} \begin{bmatrix} -0.25 \\ -0.25 \end{bmatrix} = -1.0$

$\hat{y}^{(3)} = \sigma(-1.0) = \dfrac{1}{1+e^{1.0}} \approx 0.269$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Step 2: Compute error: $\text{error} = 0.269 - 1 = -0.731$

Step 3: Compute gradient: $\nabla \ell = -0.731 \begin{bmatrix} 1 \\ 3 \end{bmatrix} = \begin{bmatrix} -0.731 \\ -2.193 \end{bmatrix}$

Step 4: Update weights: $\mathbf{w}^{(2)} = \begin{bmatrix} -0.25 \\ -0.25 \end{bmatrix} - 0.5 \begin{bmatrix} -0.731 \\ -2.193 \end{bmatrix} = \begin{bmatrix} 0.116 \\ 0.847 \end{bmatrix}$

Numerical Example: Checking Progress

After 2 SGD iterations, let's check predictions:

With $\mathbf{w}^{(2)} = \begin{bmatrix} 0.116 \\ 0.847 \end{bmatrix}$:

Example 1: $\hat{y}^{(1)} = \sigma(0.116 + 0.847 \cdot 1) \approx \sigma(0.963) \approx 0.724$

Example 2: $\hat{y}^{(2)} = \sigma(0.116 + 0.847 \cdot 2) \approx \sigma(1.810) \approx 0.859$

Example 3: $\hat{y}^{(3)} = \sigma(0.116 + 0.847 \cdot 3) \approx \sigma(2.657) \approx 0.934$

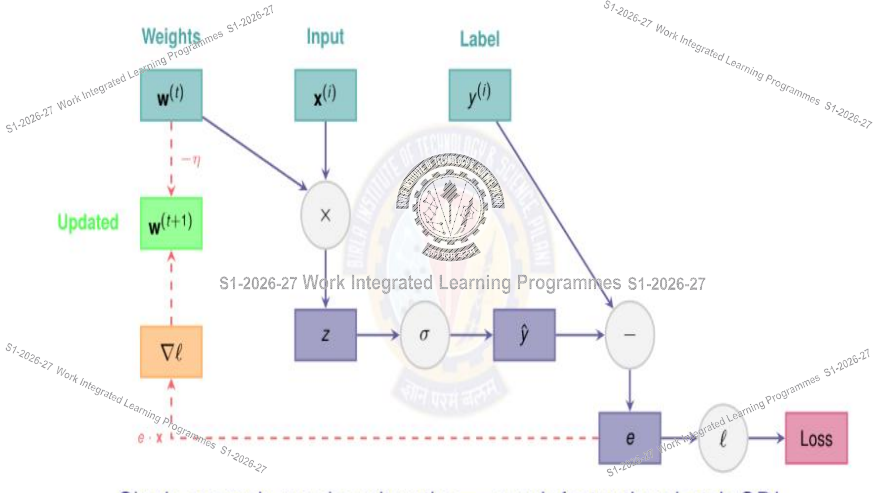
S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Example 4: $\hat{y}^{(4)} = \sigma(0.116 + 0.847 \cdot 4) \approx \sigma(3.504) \approx 0.971$

Progress:

- Examples 1, 2 (true label 0): predictions moving toward 0 ✓
- Examples 3, 4 (true label 1): predictions moving toward 1 ✓

Computational Graph for One SGD Step



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Evaluating Classification Models

Unlike regression, we need different metrics for classification. Key Questions:

- How often does the model predict correctly? (Accuracy)
- What types of errors does it make? (Confusion Matrix)
- How well does it identify positive cases? (Precision, Recall)

Data Split:

- Training set: Learn weights **w** (typically 90-99%)
- Validation set: Tune hyperparameters (typically 10-15%)
- Test set: Final evaluation (typically 1-10%)

Always evaluate on unseen test data to measure generalization!

Confusion Matrix

The confusion matrix shows all prediction outcomes:

		Predicted	
		Positive (1)	Negative (0)
Actual	Positive (1)	True Positive (TP)	False Negative (FN)
	Negative (0)	False Positive (FP)	True Negative (TN)

Definitions:

- TP: Correctly predicted positive
- TN: Correctly predicted negative
- FP: Incorrectly predicted positive (Type I error)
- FN: Incorrectly predicted negative (Type II error)

Classification Metrics I

1. Accuracy: Overall correctness

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{11}$$

2. Precision: Of predicted positives, how many are correct?

$$\text{Precision} = \frac{TP}{TP + FP} \tag{12}$$

Classification Metrics II

3. Recall (Sensitivity): Of actual positives, how many did we find?

$$\text{Recall} = \frac{TP}{TP + FN} \tag{13}$$

4. F1 Score: Harmonic mean of precision and recall.

$$F1 = 2 \cdot \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{14}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

When to use which metric?

Accuracy:

- Use when classes are balanced
- Misleading with imbalanced data! (e.g., 95% negative class)

Precision:

- Important when False Positives are costly
- Example: Spam detection (don't want to mark real emails as spam)

Recall:

- Important when False Negatives are costly
- Example: Disease detection (don't want to miss sick patients)

F1 Score:

- Use when you need balance between precision and recall
  Good for imbalanced datasets

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Example: Computing Metrics

Given predictions:

True	Pred	True	Pred
1	1 (TP)	0	0 (TN)
1	1 (TP)	0	0 (TN)
1	0 (FN)	0	1 (FP)
1	1 (TP)	0	0 (TN)

Counts: TP = 3, TN = 3, FP = 1, FN = 1

$$\text{Accuracy} = \frac{3+3}{3+3+1+1} = \frac{6}{8} = 0.75$$

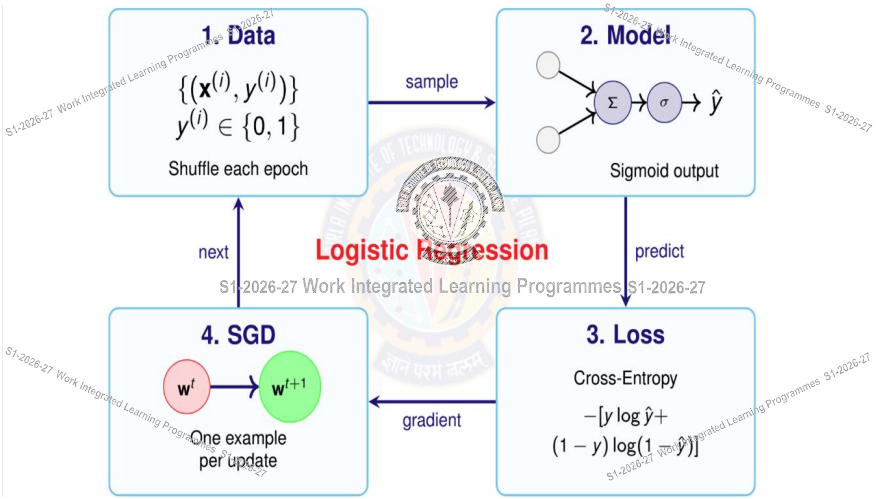
$$\text{Precision} = \frac{3}{3+1} = 0.75$$

$$\text{Recall} = \frac{3}{3+1} = 0.75$$

$$F1 = 2 \cdot \frac{0.75 \times 0.75}{0.75 + 0.75} = 0.75$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Summary: Binary Classification



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

TAKEAWAYS

- Binary classification predicts discrete categories using $y \in \{0, 1\}$.
- Sigmoid activation $\sigma(z)$ $\frac{1}{1+e^{-z}}$ maps to probabilities $[0, 1]$.
- Logistic regression = linear model + sigmoid activation.
- Cross-entropy loss is the appropriate objective for classification.
- Stochastic Gradient Descent (SGD) updates weights one example at a time — faster and more scalable.
- Feature scaling is essential for convergence and numerical stability.
- Evaluation metrics: Accuracy, Precision, Recall, F1 Score, Confusion Matrix.
- This framework extends naturally to multi-class classification and deep networks.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

IS MULTI-CLASS CLASSIFICATION

DEFINITION

Multi-class classification assigns each input to one of K classes, where $K > 2$.

Key Characteristics:

- Classes: $y \in \{1, 2, 3, \ldots, K\}$ (or $\{0, 1, 2, \ldots, K-1\}$)
- Each example belongs to **exactly one** class
- Output: Probability distribution over all K classes
- Probabilities sum to 1: $\sum_{k=1}^{K} P(y = k|\mathbf{x}) = 1$

Example:

- Input: Image of a handwritten digit
- Output: Probabilities for each of 10 classes (0, 1, 2, ..., 9)
- Prediction: Class with highest probability

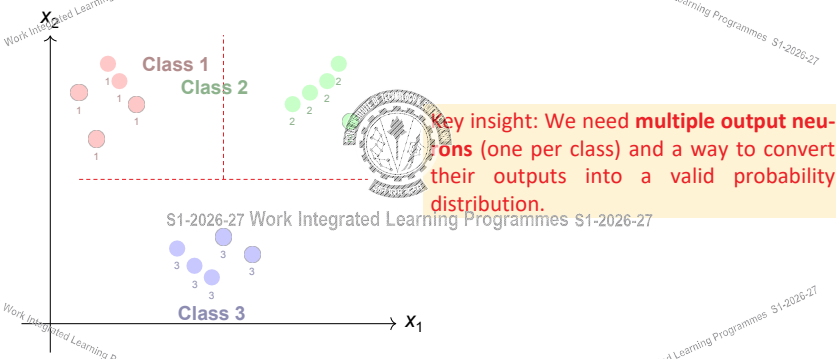
S1-2026-27 Work Integrated Learning Programmes S1-2026-27

EXAMPLES OF MULTI-CLASS CLASSIFICATION

- **Handwritten Digit Recognition (MNIST)**
  - Input: 28 × 28 pixel image
  - Output: One of 10 digits (0-9)
- **Image Classification (ImageNet)**
  - Input: Color image
  - Output: One of 1000 object categories (cat, dog, car, plane, ...)
- **Text Classification**
  - Input: News article text
  - Output: Topic category (politics, sports, technology, entertainment, business)
- **Speech Recognition**
  - Input: Audio features

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

MULTI-CLASS CLASSIFICATION: VISUAL EXAMPLE



Key insight: We need **multiple output neurons** (one per class) and a way to convert their outputs into a valid probability distribution.

- Decision boundaries separate $K = 3$ classes in feature space.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

MULTI-CLASS CLASSIFICATION

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

THE FOUR COMPONENTS FOR MULTI-CLASS

A complete multi-class classification system consists of:

- **Data:** Input features and class labels $y \in \{1, 2, \ldots, K\}$
- **Model**: Linear model with K output neurons (no hidden layers)
- **Activation**: Softmax function to produce probability distribution
- **Objective Function**: Categorical cross-entropy loss
- **Learning Algorithm**: Mini-batch stochastic gradient descent

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Note:

- We now use **mini-batch SGD** instead of single-example SGD for better efficiency and stability.

DATA REPRESENTATION

Input Features:

$$\mathbf{X} \in \mathbb{R}^{N \times (d+1)} \quad \mathbf{X} = \begin{bmatrix} 1 & x_1^{(1)} & \cdots & x_d^{(1)} \\ 1 & x_1^{(2)} & \cdots & x_d^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ 1 & x_1^{(N)} & \cdots & x_d^{(N)} \end{bmatrix}$$

Class Labels: Two representations

1. Integer encoding: $\mathbf{y} \in \{1, 2, \ldots, K\}^N$

$$\mathbf{y} = \begin{bmatrix} 3 & 1 & 2 & 1 & 3 \end{bmatrix}^T$$

2. One-hot encoding: $\mathbf{Y} \in \{0, 1\}^{N \times K}$ (preferred for computation)

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

ONE-HOT ENCODING

Definition: Convert class label to binary vector of length K

For class k : All zeros except 1 at position k

Example with $K = 4$ classes:

Class 1: $\mathbf{y} = [1, 0, 0, 0]^T$
Class 2: $\mathbf{y} = [0, 1, 0, 0]^T$
Class 3: $\mathbf{y} = [0, 0, 1, 0]^T$
Class 4: $\mathbf{y} = [0, 0, 0, 1]^T$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Why one-hot?

- Treats all classes equally (no implicit ordering)
- Matches the format of softmax output probabilities
- Simplifies loss computation
- Standard in deep learning frameworks

WEIGHT MATRIX FOR MULTI-CLASS

For K classes and d input features:

Weight Matrix: $\mathbf{W} \in \mathbb{R}^{(d+1)\times K}$

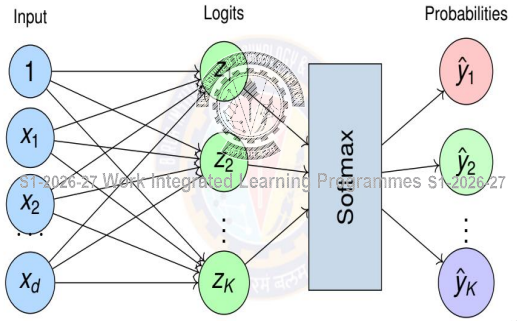
$$\mathbf{W} = \begin{bmatrix} w_{0,1} & w_{0,2} & \cdots & w_{0,K} \\ w_{1,1} & w_{1,2} & \cdots & w_{1,K} \\ w_{2,1} & w_{2,2} & \cdots & w_{2,K} \\ \vdots & \vdots & \ddots & \vdots \\ w_{d,1} & w_{d,2} & \cdots & w_{d,K} \end{bmatrix}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

- Each **column** $\mathbf{w}_k$ represents weights for class k
- Each class has its own linear model
- Row 0: Bias terms for all classes
- Total parameters: $(d+1) \times K$

MULTI-CLASS MODEL ARCHITECTURE

- Architecture: Multiple output neurons with softmax activation



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

LINEAR MODEL: COMPUTING LOGITS

Step 1: Compute logits (pre-activation values)

For a single input $\mathbf{x}$:

$$\mathbf{z} = \mathbf{W}^T\mathbf{x} = \begin{bmatrix} \mathbf{w}_1^T\mathbf{x} \\ \mathbf{w}_2^T\mathbf{x} \\ \vdots \\ \mathbf{w}_K^T\mathbf{x} \end{bmatrix} = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_K \end{bmatrix} \tag{15}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

where $z_k = \mathbf{w}_k^T\mathbf{x} = w_{0,k} + w_{1,k}x_1 + \cdots + w_{d,k}x_d$

For entire dataset (batch):

$$\mathbf{Z} = \mathbf{X}\mathbf{W} \in \mathbb{R}^{N\times K} \tag{16}$$

Each row of $\mathbf{Z}$ contains logits for one example.

THE SOFTMAX FUNCTION

Purpose: Convert K logits into a valid probability distribution

DEFINITION:

For logit vector $\mathbf{z} = [z_1, z_2, \ldots, z_K]^T$

$$\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}} \quad \text{for } k = 1, 2, \ldots, K \tag{17}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Output: Probability vector $\hat{\mathbf{y}} = [\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_K]^T$

$$\hat{y}_k = P(y = k|\mathbf{x}) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}} \tag{18}$$

Properties of Softmax

Key Properties:

1. Output range: $\hat{y}_k \in (0, 1)$ for all k
2. Probability distribution: $\sum_{k=1}^{K} \hat{y}_k = 1$
3. Preserves order: If $z_i > z_j$, then $\hat{y}_i > \hat{y}_j$
4. Translation invariant: $\text{softmax}(\mathbf{z}) = \text{softmax}(\mathbf{z} + c)$ for any constant c
5. Differentiable: Enables gradient-based learning
6. Reduces to sigmoid: When $K = 2$, softmax is equivalent to sigmoid

The exponential amplifies differences: larger logits get much larger probabilities!

Softmax Example: Numerical Computation

Given logits: $\mathbf{z} = [2.0, 1.0, 0.1]^T$ for $K = 3$ classes

Step 1: Compute exponentials

$$e^{z_1} = e^{2.0} \approx 7.39$$
$$e^{z_2} = e^{1.0} \approx 2.72$$
$$e^{z_3} = e^{0.1} \approx 1.11$$

Step 2: Compute sum

$$\sum_{j=1}^{3} e^{z_j} = 7.39 + 2.72 + 1.11 = 11.22$$

Step 3: Normalize

$$\hat{y}_1 = \frac{7.39}{11.22} \approx 0.659$$
$$\hat{y}_2 = \frac{2.72}{11.22} \approx 0.242$$
$$\hat{y}_3 = \frac{1.11}{11.22} \approx 0.099$$

Step 4: Predicted class:
$\arg\max_k \hat{y}_k = 1$ (highest probability)

Complete Forward Pass

For a batch of N examples $\mathbf{X}$:

$$\text{Step 1: Compute logits} \quad \mathbf{Z} = \mathbf{X}\mathbf{W} \in \mathbb{R}^{N \times K} \tag{19}$$

$$\text{Step 2: Apply softmax} \quad \hat{\mathbf{Y}} = \text{softmax}(\mathbf{Z}) \in [0, 1]^{N \times K} \tag{20}$$

(softmax applied row-wise to each example)

$$\text{Step 3: Predict class} \quad k = \arg\max_k \hat{y}_k \tag{21}$$

Each row of $\hat{\mathbf{Y}}$ is a probability distribution over K classes.

Categorical Cross-Entropy

Total loss over dataset:

$$J(\mathbf{W}) = -\frac{1}{N} \sum_{i=1}^{N} \sum_{k=1}^{K} y_k^{(i)} \log(\hat{y}_k^{(i)}) \tag{22}$$

Goal: Minimize $J(\mathbf{W})$ to find optimal weights $\mathbf{W}^*$

UNDERSTANDING CATEGORICAL

Intuition:

- Loss depends only on predicted probability for the **true class**
- Heavily penalizes **confident wrong predictions**
- Encourages high probability for correct class

Example: True class is 2 ($\mathbf{y} = [0, 1, 0]$)

Prediction	$\hat{y}_1$	$\hat{y}_2$	Loss $= -\log(\hat{y}_2)$
Good	0.1	**0.8**	0.22
Medium	0.3	**0.5**	0.69
Bad	0.7	**0.2**	1.61
Very bad	0.9	**0.05**	3.00

Loss increases exponentially as predicted probability for true class decreases!

WHY CATEGORICAL CROSS-ENTROPY?

Advantages:

- Probabilistic interpretation: Derived from maximum likelihood
- Appropriate for multi-class: Naturally handles K classes
- Works well with softmax: Numerically stable gradient
- Penalizes wrong confidence: Heavily penalizes confident mistakes
- Convex: For linear models (like ours)

Connection to binary cross-entropy:
When $K = 2$:

$$\text{Categorical: } -[y_1 \log(\hat{y}_1) + y_2 \log(\hat{y}_2)]$$
$$\text{Binary: } -[y \log(\hat{y}) + (1-y) \log(1-\hat{y})]$$

They are equivalent! (with $y_2 = y$ and $y_1 = 1 - y$)

FROM SGD TO MINI-BATCH SGD

Recap: Gradient Descent Variants

- **Batch Gradient Descent:**
  - Uses **all** N examples per update
  - Slow but stable, accurate gradient
- **Stochastic Gradient Descent (SGD):**
  - Uses **1** random example per update
  - Fast but noisy, unstable
- **Mini-batch SGD (Best of both worlds):**
  - Uses **small batch** of B examples per update ($1 < B << N$)
  - Typical batch sizes: $B \in \{32, 64, 128, 256, 512\}$
  - Good balance of speed and stability

Standard in modern deep learning!

WHY MINI-BATCH SGD?

Advantages of mini-batches:

Computational efficiency:
- Vectorized operations (matrix multiplication)
- Better GPU utilization
- Faster than processing examples one-by-one

Gradient estimation:
- Less noisy than single-example SGD
- More stable convergence
- Better approximation of true gradient

Generalization:
- Noise helps escape sharp minima
- Often leads to better test performance

Memory efficiency:
- Can handle datasets larger than memory

Mini-batch SGD: Key

- Batch size (B): Number of examples per update
  - Small: 32-64 (more noise, faster iterations)
  - Medium: 128-256 (balanced)
  - Large: 512-1024 (less noise, but slower convergence)
- Iteration: One weight update (processing one mini-batch)
- Epoch: One complete pass through the entire dataset

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

$$\text{Iterations per epoch} = \left\lceil \frac{N}{B} \right\rceil \tag{23}$$

Example: $N = 60{,}000$ samples, $B = 128$

- Iterations per epoch: $\lceil 60000/128 \rceil = 469$
- After 10 epochs: $469 \times 10 = 4690$ weight updates

Computing Gradients for Mini-batch

Gradient for a mini-batch of size B:

Let $\mathcal{B}$ be a mini-batch of B examples: $\mathcal{B} = \{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i \in \mathcal{I}}$

Average gradient over mini-batch:

$$\nabla J_{\mathcal{B}}(\mathbf{W}) = \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla \ell(\mathbf{W}; \mathbf{x}^{(i)}, \mathbf{y}^{(i)}) \tag{24}$$

For categorical cross-entropy with softmax:

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

$$\nabla_{\mathbf{W}} J_{\mathcal{B}} = \frac{1}{B} \mathbf{X}_{\mathcal{B}}^T (\hat{\mathbf{Y}}_{\mathcal{B}} - \mathbf{Y}_{\mathcal{B}}) \tag{25}$$

where:

- $\mathbf{X}_{\mathcal{B}} \in \mathbb{R}^{B \times (d+1)}$: Mini-batch inputs
- $\hat{\mathbf{Y}}_{\mathcal{B}} \in \mathbb{R}^{B \times K}$: Predicted probabilities

Mini-batch SGD Update Rule

Update at iteration t:

$$\mathbf{W}^{(t+1)} = \mathbf{W}^{(t)} - \eta \nabla J_{\mathcal{B}}(\mathbf{W}^{(t)}) \tag{26}$$

Expanded form:

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

$$\mathbf{W}^{(t+1)} = \mathbf{W}^{(t)} - \frac{\eta}{B} \mathbf{X}_{\mathcal{B}}^T (\hat{\mathbf{Y}}_{\mathcal{B}} - \mathbf{Y}_{\mathcal{B}}) \tag{27}$$

Mini-batch SGD Algorithm

Algorithm 2: Mini-batch SGD for Multi-class Classification

Input: Dataset $\mathcal{D} = \{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i=1}^N$, batch size B, learning rate η, epochs T
Output: Learned weight matrix $\mathbf{W}$

Initialize $\mathbf{W} \sim \mathcal{N}(0, 0.01)$ // Small random values
for *epoch* $= 1$ *to* T **do**
  Shuffle dataset $\mathcal{D}$;
  for *each mini-batch* $\mathcal{B}$ *of size* B **do**
    // Forward pass
    $\mathbf{Z}_{\mathcal{B}} = \mathbf{X}_{\mathcal{B}} \mathbf{W}$ // Compute logits
    $\hat{\mathbf{Y}}_{\mathcal{B}} = \text{softmax}(\mathbf{Z}_{\mathcal{B}})$ // Apply softmax
    $\nabla J = \frac{1}{B} \mathbf{X}_{\mathcal{B}}^T (\hat{\mathbf{Y}}_{\mathcal{B}} - \mathbf{Y}_{\mathcal{B}})$ // Compute gradient
    $\mathbf{W} \leftarrow \mathbf{W} - \eta \nabla J$ // Update weights
  Optional: Compute total loss for monitoring;
return $\mathbf{W}$;

Choosing Batch Size

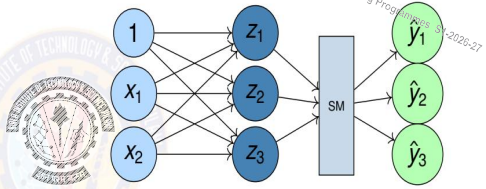
Batch Size	Advantages	Disadvantages
Small (32-64)	○ Fast iterations ○ More updates per epoch ○ Better generalization ○ Escapes sharp minima	○ Noisy gradients ○ Unstable training Poor ○ GPU utilization
Medium (128-256)	○ Good balance ○ Stable training ○ Good GPU usage ○ Standard choice	○ May need tuning
Large (512-1024)	○ Stable gradients ○ Better GPU utilization ○ Faster per-epoch time	○ Slow convergence ○ May generalize worse ○ More memory

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Numerical Example: Setup

Tiny Dataset: Classify 4 data points into 3 classes

x_1	x_2	Class
1	2	1
2	1	2
2	3	1
3	2	3



Parameters:

- $K = 3$ classes, $d = 2$ features, $N = 4$ examples
- Batch size: $B = 2$ (mini-batch)
- Learning rate: $\eta = 0.1$

Numerical Example: Data

Design Matrix: (with bias column) $\mathbf{X} = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 2 & 1 \\ 1 & 2 & 3 \\ 1 & 3 & 2 \end{bmatrix} \in \mathbb{R}^{4\times 3}$

One-hot Labels: $\mathbf{Y} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \in \mathbb{R}^{4\times 3}$ (class 1), (class 2), (class 1), (class 3)

Weight Matrix: Initialize randomly $\mathbf{W}^{(0)} = \begin{bmatrix} 0.1 & -0.1 & 0.2 \\ 0.2 & 0.1 & -0.1 \\ -0.1 & 0.2 & 0.1 \end{bmatrix} \in \mathbb{R}^{3\times 3}$

Numerical Example: First Mini-batch (examples

Mini-batch: First 2 examples after shuffling

$$\mathbf{X}_{\mathcal{B}} = \begin{bmatrix} 1 & 1 & 2 \\ 1 & 2 & 1 \end{bmatrix}, \quad \mathbf{Y}_{\mathcal{B}} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

Step 1: Compute logits

$$\mathbf{Z}_{\mathcal{B}} = \mathbf{X}_{\mathcal{B}}\mathbf{W}^{(0)}$$

$$= \begin{bmatrix} 1 & 1 & 2 \\ 1 & 2 & 1 \end{bmatrix} \begin{bmatrix} 0.1 & -0.1 & 0.2 \\ 0.2 & 0.1 & -0.1 \\ -0.1 & 0.2 & 0.1 \end{bmatrix}$$

$$= \begin{bmatrix} 0.1 & 0.4 & 0.3 \\ 0.4 & 0.2 & 0.1 \end{bmatrix}$$

Numerical Example: Softmax Computation

Step 2: Apply softmax to each row

Softmax for example 1: $\hat{\mathbf{y}}^{(1)} \approx \begin{bmatrix} 0.280 \\ 0.378 \\ 0.342 \end{bmatrix}$

Similarly for example 2: $\hat{\mathbf{y}}^{(2)} \approx [0.387, 0.315, 0.298]^T$

For the entire batch: $\hat{\mathbf{Y}}_{\mathcal{B}} = \begin{bmatrix} 0.280 & 0.378 & 0.342 \\ 0.387 & 0.315 & 0.298 \end{bmatrix}$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Numerical Example: Gradient and Update

Step 3: Compute gradient

$$\nabla J = \frac{1}{B}\mathbf{X}_{\mathcal{B}}^T(\hat{\mathbf{Y}}_{\mathcal{B}} - \mathbf{Y}_{\mathcal{B}})$$

$$\hat{\mathbf{Y}}_{\mathcal{B}} - \mathbf{Y}_{\mathcal{B}} = \begin{bmatrix} 0.280 & 0.378 & 0.342 \\ 0.387 & 0.315 & 0.298 \end{bmatrix} - \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

$$= \begin{bmatrix} -0.720 & 0.378 & 0.342 \\ 0.387 & -0.685 & 0.298 \end{bmatrix}$$

$$\nabla J = \frac{1}{2}\begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 2 & 1 \end{bmatrix}\begin{bmatrix} -0.720 & 0.378 & 0.342 \\ 0.387 & -0.685 & 0.298 \end{bmatrix}$$

$$\approx \begin{bmatrix} -0.167 & -0.154 & 0.320 \\ -0.024 & -0.496 & 0.469 \\ -0.527 & 0.036 & 0.491 \end{bmatrix}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Numerical Example: Weight Update

Step 4: Update weights

$$\mathbf{W}^{(1)} = \mathbf{W}^{(0)} - \eta\nabla J$$

$$= \begin{bmatrix} 0.1 & -0.1 & 0.2 \\ 0.2 & 0.1 & -0.1 \\ -0.1 & 0.2 & 0.1 \end{bmatrix} - 0.1 \begin{bmatrix} -0.167 & -0.154 & 0.320 \\ -0.024 & -0.496 & 0.469 \\ -0.527 & 0.036 & 0.491 \end{bmatrix}$$

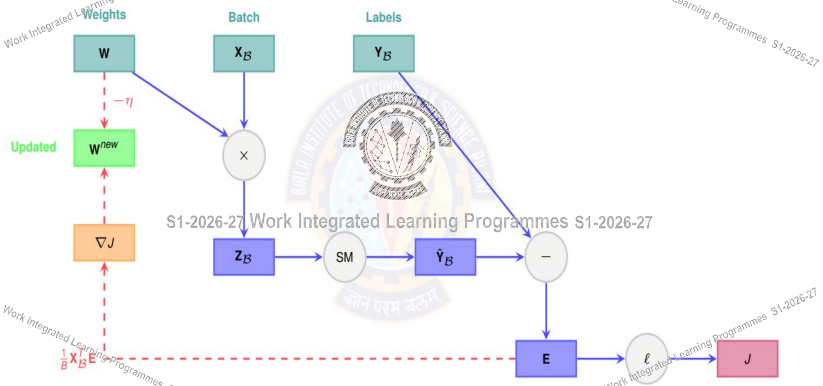
$$= \begin{bmatrix} 0.117 & -0.085 & 0.168 \\ 0.202 & 0.150 & -0.147 \\ -0.047 & 0.196 & 0.051 \end{bmatrix}$$

Progress:

- Weights adjusted based on mini-batch gradient
- Process next mini-batch (examples 3-4) with updated weights
- Continue for multiple epochs until convergence

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Computational Graph for Mini-batch Forward



Mini-batch processes B examples simultaneously — efficient and stable!

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Inference: Making Predictions

After training, use the model to classify new examples.

Inference process for input $\mathbf{x}_{\text{new}}$:

$$\text{Compute logits:} \quad \mathbf{z} = \mathbf{W}^T \mathbf{x}_{\text{new}} \tag{28}$$

$$\text{Apply softmax:} \quad \hat{\mathbf{y}} = \text{softmax}(\mathbf{z}) \tag{29}$$

$$\text{Predict class:} \quad k = \arg\max_{k=1,\ldots,K} \hat{y}_k \tag{30}$$

Optional: Report confidence scores (the probabilities $\hat{\mathbf{y}}$)

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Inference Example

Trained model with $K = 3$ classes, new input $\mathbf{x} = [1, 2.5, 1.8]^T$

Step 1: Compute logits $\mathbf{z} = \mathbf{W}^T \mathbf{x} = [1.2, 2.5, 0.8]^T$

Step 2: Apply softmax $e^{1.2} \approx 3.32, \quad e^{2.5} \approx 12.18, \quad e^{0.8} \approx 2.23$

$$\text{Sum} = 3.32 + 12.18 + 2.23 = 17.73$$

$$\hat{\mathbf{y}} = \begin{bmatrix} 3.32/17.73 \\ 12.18/17.73 \\ 2.23/17.73 \end{bmatrix} = \begin{bmatrix} 0.187 \\ 0.687 \\ 0.126 \end{bmatrix}$$

Step 3: Predict class $\hat{k} = \arg\max_k \hat{y}_k = 2$ (Class 2 with 68.7% confidence)

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Evaluation Metrics for Multi-Class

1. Accuracy: Overall correctness

$$\text{Accuracy} = \frac{\text{Number of correct predictions}}{N} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}[\hat{y}^{(i)} = y^{(i)}] \tag{31}$$

2. Confusion Matrix: $K \times K$ matrix
   - Row i, Column j: Number of class i examples predicted as class j
   - Diagonal: Correct predictions
   - Off-diagonal: Misclassifications

3. Per-class Precision and Recall:
   - Compute for each class separately (one-vs-all)

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Confusion Matrix for Multi-Class

Example: 3-class confusion matrix

Actual \ Predicted	Class 1	Class 2	Class 3
Class 1	45	3	2
Class 2	5	38	7
Class 3	2	4	44

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Interpretation:

- Diagonal (green): Correct predictions (45, 38, 44)
- Off-diagonal (red): Misclassifications
- Row sums: Total actual examples per class (50, 50, 50)
- Column sums: Total predicted examples per class

PER-CLASS METRICS

For each class k , compute:

Precision for class k : Of all predictions for class k , what fraction were correct?

$$\text{Precision}_k = \frac{TP_k}{TP_k + FP_k} \tag{32}$$

Recall for class k : Of all actual class k examples, what fraction did we identify?

$$\text{Recall}_k = \frac{TP_k}{TP_k + FN_k} \tag{33}$$

F1 Score for class k :

$$F_{1_k} = 2 \cdot \frac{\text{Precision}_k \times \text{Recall}_k}{\text{Precision}_k + \text{Recall}_k} \tag{34}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

AVERAGING STRATEGIES

How to combine per-class metrics into overall score?

1. Macro-average: Simple average across classes

$$\text{Precision}_{\text{macro}} = \frac{1}{K} \sum_{k=1}^{K} \text{Precision}_k \tag{35}$$

- Treats all classes equally
- Good when classes are equally important

2. Weighted average: Weight by class frequency

$$\text{Precision}_{\text{weighted}} = \sum_{k=1}^{K} w_k \cdot \text{Precision}_k \qquad w_k = N_k / N \tag{36}$$

- Accounts for class imbalance

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

TOP-K ACCURACY

For problems with many classes (e.g., ImageNet with 1000 classes):

Top-K Accuracy: Correct if true class is in top K predictions

$$\text{Top-K Accuracy} = \frac{1}{N} \sum_{i}^{N} \mathbb{1}[y^{(i)} \in \text{Top-K}(\hat{\mathbf{y}}^{(i)})] \tag{37}$$

Example: Image classification with 1000 classes

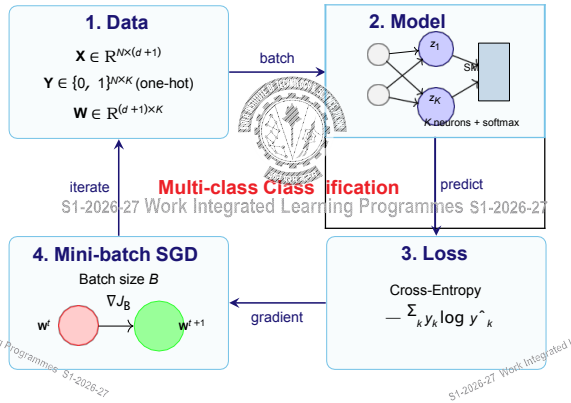
- Top-1 accuracy: Is the highest probability class correct?
- Top-5 accuracy: Is the true class in the top 5 predictions?

Why useful?

- More forgiving metric for large K
- Captures whether model has correct class among top candidates Standard in ImageNet challenge

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

SUMMARY: MULTICLASS CLASSIFICATION



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Implementation Tips

- 1. Learning Rate & Epochs:
  - Start: $\eta \in \{0.01, 0.1, 1.0\}$
  - Typical: 10-100 epochs
  - Use early stopping
- 2. Weight Initialization:
  - $w_j \sim \mathcal{N}(0, 0.01)$
  - Avoid zeros or large values
- 3. Batch Size:
  - Start: $B \in \{64, 128, 256\}$
  - Powers of 2 preferred
  - Larger: stable, slower
  - Smaller: noisy, faster
- 4. Data Handling:
  - **Always shuffle** each epoch
  - Check class balance
- 5. Feature Scaling: **Essential!**
  - Standardization: $x'_j = \frac{x_j - \mu_j}{\sigma_j}$
  - Prevents sigmoid saturation
- 6. Numerical Stability:
  - Use: $\log(\sigma(z))$
  - Stable softmax: $c = \max_k z_k$
  - $\mathrm{SM}(z)_k = \frac{e^{z_k - c}}{\sum_j e^{z_j - c}}$
  - Log-sum-exp trick
  - Prevents overflow/underflow

Debugging Checklist

- Loss is NaN/Inf:
  - η too large $\rightarrow$ decrease
  - Check $\log(0)$ in loss
  - Scale features
  - Use stable softmax
- Predicts same class:
  - Check class balance
  - Verify softmax/loss
  - η might be too small
  - Verify one-hot encoding
- Loss oscillates:
  - η too large
  - Use mini-batches
  - Apply learning rate decay
- Softmax outputs $\neq 1$:
  - Implementation bug
  - Numerical precision
- Loss not decreasing:
  - Check gradient signs
  - η too small/large
  - Verify feature scaling
  - Verify data shuffling
  - Check one-hot encoding
- High train, low test accuracy:
  - Overfitting
  - Add regularization
  - Get more training data
  - Simpler model
- Sanity checks:
  - Tiny dataset $\rightarrow$ 100% accuracy
  - Verify $\hat{y} \in [0, 1]$
  - Verify $\sum_k \hat{y}_k = 1$
  - Shuffling improves results

Activation Functions

Property	Identity	Sigmoid	Softmax
Formula	$f(z) = z$	$\sigma(z) = \frac{1}{1+e^{-z}}$	$\mathrm{SM}(z)_k = \frac{e^{z_k}}{\sum_j e^{z_j}}$
Output Range	$(-\infty, \infty)$	$[0, 1]$	$[0, 1]^K, \sum_k = 1$
Use Case	Regression	Binary classification	Multi-class
# Outputs	1	1	K (coupled)

Loss Functions

Aspect	MSE	Binary CE	Categorical CE
Formula	$\frac{1}{N}\sum_{i=1}^{N}(\hat{y}^{(i)} - y^{(i)})^2$	$\frac{-1}{N}\sum_{i=1}^{N}[y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$	$\frac{-1}{N}\sum_{i=1}^{N}\sum_{k=1}^{K} y_k^{(i)} \log \hat{y}_k^{(i)}$
Task	Regression	Binary classification	Multi-class classification
Output Type	Continuous value	Probability (2 classes)	Probability distribution (K classes)
Target y	Real number	$\{0, 1\}$	One-hot vector
Prediction $\hat{y}$	Any real number	Probability $[0, 1]$	Probability vector, $\sum_k \hat{y}_k = 1$
Activation	Identity	Sigmoid	Softmax
Penalizes	Distance from target	Confident wrong predictions	Wrong class probability
Properties	Convex, Quadratic	Convex, Probabilistic,	Convex, Extends binary

Gradient Descent Variants

Aspect	Batch GD	Stochastic GD	Mini-batch GD
Examples/update	All N	1	B (batch size)
Gradient	$\nabla J = \frac{1}{N}\sum_{i=1}^{N} \nabla \ell_i$	$\nabla \ell_i$ for random i	$\nabla J_{\mathcal{B}} = \frac{1}{B}\sum_{i \in \mathcal{B}} \nabla \ell_i$
Update rule	$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J$	$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla \ell_i$	$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J_{\mathcal{B}}$
Iterations/epoch	1	N	$\lceil N/B \rceil$
Speed per iteration	Slow	Fast	Medium
Convergence	Smooth, deterministic	Noisy, oscillating	Balanced
Memory	High (all data)	Low (1 example)	Medium (B examples)
Gradient quality	Exact	Noisy estimate	Good estimate
Advantages	Stable, Precise gradient, Reproducible	Fast iterations, Online learning, Escapes local minima	**Best balance**, GPU efficient, Stable & fast
Disadvantages	Slow for large N, High memory, NO online	Very noisy, Unstable, Hard to parallelize	Extra hyperparameter (B), Needs tuning
Best for	Small datasets	Online learning	**Deep learning (std)**

Comparison of Linear Models

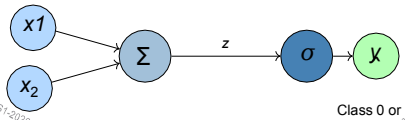
Aspect	Regression	Binary	Multi-class
Output	Continuous $y \in \mathbb{R}$	Discrete $y \in \{0,1\}$	Discrete $y \in \{1,..,K\}$
Activation	Identity: $f(z) = z$	Sigmoid: $\sigma(z)$	Softmax: SM($\mathbf{z}$)
Loss Function	Mean Squared Error	Binary Cross-Entropy	Categorical Cross-Entropy
Output Range	$(-\infty, \infty)$	$[0,1]$	$[0,1]^K, \sum_k = 1$
Interpretation	Predicted value	$P(y=1\|\mathbf{x})$	$P(y=k\|\mathbf{x})$ for all k
Output neurons	1	1	K
Weights	Vector $\mathbf{w} \in \mathbb{R}^{d+1}$	Vector $\mathbf{w} \in \mathbb{R}^{d+1}$	Matrix $\mathbf{W} \in \mathbb{R}^{(d+1)\times K}$
Evaluation	MSE, RMSE, R^2	Accuracy, Precision, Recall, F1	Confusion matrix, Top-K
Learning	Batch GD or SGD	SGD or Mini-batch	Mini-batch SGD

Recap: Binary Classification

Logistic Regression

Key insight: Sigmoid maps linear combination to probability; optimized by minimiz- ing cross-entropy loss.

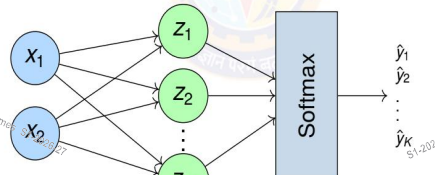
- Problem: Classify into two classes: $y \in \{0, 1\}$
- Model: Single neuron with sigmoid activation
- Output: Probability $\hat{y} = P(y = 1|\mathbf{x}) \in [0, 1]$
- Loss: Binary cross-entropy
- Training: Stochastic Gradient Descent (SGD)



Recap: Multi-class Classification

Key insight: Multiple outputs coupled through softmax create valid probability distribution over all classes.

- Problem: Classify into K classes: $y \in \{1, 2, \ldots, K\}$
- Model: K output neurons with softmax activation
- Output: Probability distribution $\hat{\mathbf{y}} = [\hat{y}_1, \ldots, \hat{y}_K]^T$, where $\sum_{k=1}^{K} \hat{y}_k = 1$
- Loss: Categorical cross-entropy
- Training: Mini-batch Stochastic Gradient Descent



REFERENCES

Zhang, A., Lipton, Z. C., Li, M., & Smola, A. J. (2023). *Dive into Deep Learning*. Cambridge University Press. Chapter 4 (Linear Neural Networks for Classification). https://d2l.ai/

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press. Chapter 6 (Deep Feedforward Networks). https://www.deeplearningbook.org/

Bishop, C. M. (2006). *Pattern Recognition and Machine Learning*. Springer. Chapter 4 (Linear Models for Classification).

Murphy, K. P. (2022). *Probabilistic Machine Learning: An Introduction*. MIT Press. Chapter 10 (Logistic Regression). https://probml.github.io/pml-book/

Linear Models Have Limitations

Recall: Linear models uses a single neuron with activation function f ()

$$y = f(\mathbf{w}^T\mathbf{x} + b)$$

Problem: Many real-world relationships are non-linear

- Image recognition (pixel intensities → object classes)
- Natural language understanding (words → sentiment)
- Medical diagnosis (symptoms → disease probability)
- Game playing (board state → optimal move)

Linear models cannot solve non-linearly separable problems!

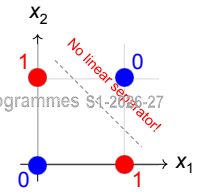
THE XOR PROBLEM

- XOR (Exclusive OR): A classic non-linearly separable problem

Truth Table:

x_1	x_2	y (XOR)
0	0	0
0	1	1
1	0	1
1	1	0

Visual Representation:



Challenge: No single straight line can separate the two classes!

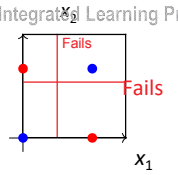
WITH SINGLE NEURON FAILS ON XOR

Single neuron model:

$$\hat{y} = \sigma(w_1x_1 + w_2x_2 + b) \quad (2)$$

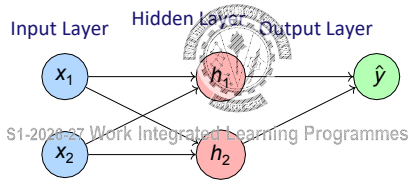
where σ is any activation function.

Decision boundary: $w_1x_1 + w_2x_2 + b = 0$ (a line/hyperplane)



SOLUTION: MULTI-LAYER PERCEPTRON (MLP)

Key Idea: Stack multiple layers of neurons with non-linear activations



Non-linear activation functions enable non-linear boundaries

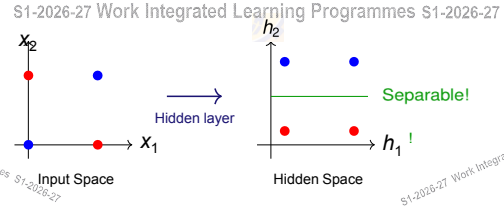
S1-2026-27 Work Integrated Learning Programmes S1-2026-27

MLP SOLVES XOR

Architecture: 2 inputs → 2 hidden units → 1 output

How it works:

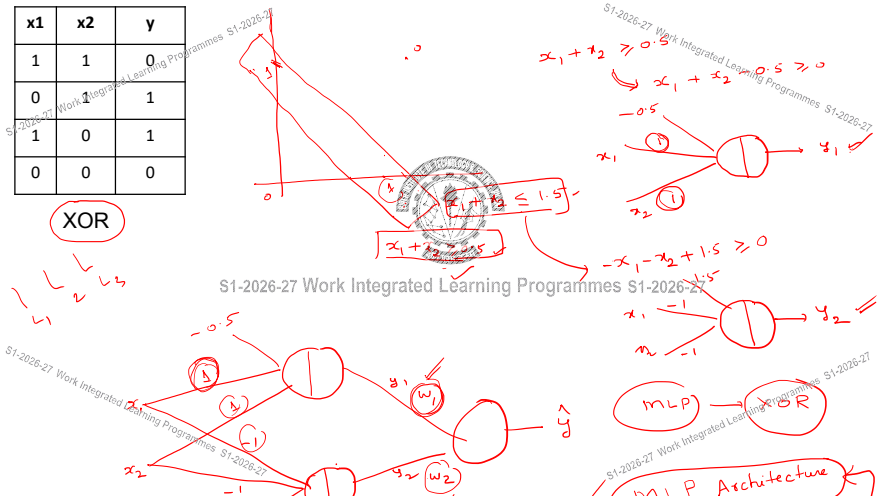
- Hidden layer creates new feature representations
- Each hidden unit learns a linear separator
- Output layer combines these separators
- Non-linear activation enables complex boundaries



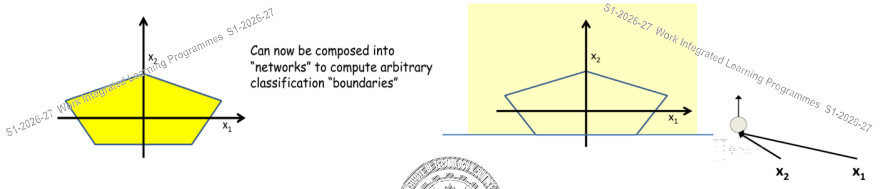
S1-2026-27 Work Integrated Learning Programmes S1-2026-27

x1	x2	y
1	1	0
0	1	1
1	0	1
0	0	0

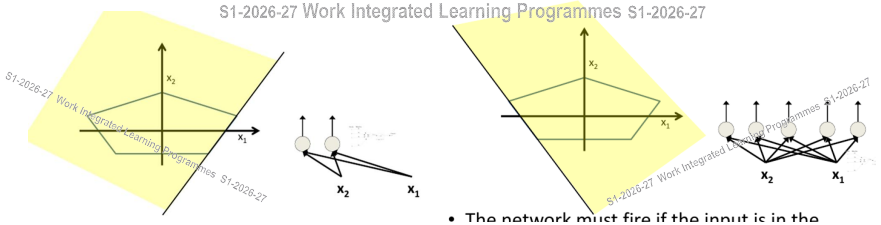
XOR



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

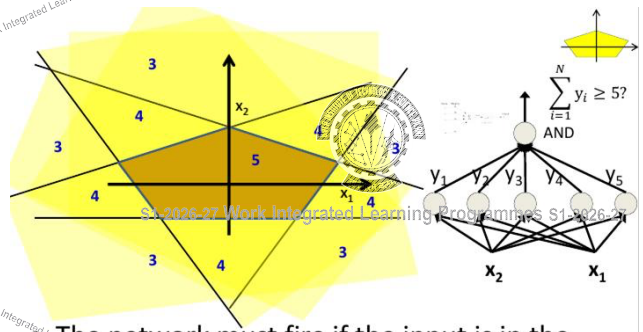


- Build a network of units with a single output that fires if the input is in the coloured area
- The network must fire if the input is in the coloured area



- The network must fire if the input is in the

S1-2026-27 Work Integrated Learning Programmes S1-2026-27



- The network must fire if the input is in the coloured area

What is non-linearity?

Definition

A function f is non-linear if it does not satisfy the linearity property:

$$f(ax_1 + bx_2) \neq a\,f(x_1) + b\,f(x_2)$$

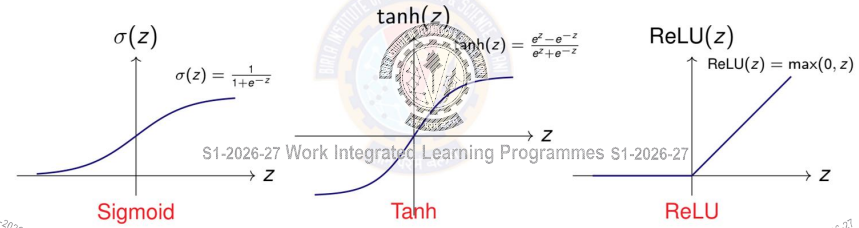
for all scalars a, b and inputs x_1, x_2. (3)

Why non-linearity matters:

- Without non-linearity, deep networks collapse to single-layer models
- Non-linear activations enable learning complex patterns

Common Activation Functions

Activation functions introduce non-linearity



Activation Function Properties

Property	Sigmoid	Tanh	ReLU	Leaky ReLU
Range	(0, 1)	(−1, 1)	[0, ∞)	(−∞, ∞)
Zero-centered	x	✓	x	✓
Differentiable	✓	✓	x at 0	x at 0
Vanishing gradient	High	Moderate	No	No
Computational cost	High	High	Low	Low
Typical use	Output layer	Hidden layer	Hidden layer	Hidden layer

- Modern preference: ReLU and variants for hidden layers
- Fast computation (simple thresholding)
- No vanishing gradient for positive values
- Sparse activation (biological plausibility)

ReLU: $f(z) = \max(0, z)$

Advantages:

1. **Computational efficiency**
   - Simple comparison and multiplication
   - Much faster than sigmoid/tanh (no exponentials)
2. **Avoids vanishing gradients**
   - Gradient is 1 for $z > 0$, flows unchanged through network
   - Enables training of very deep networks
3. **Sparse activation**
   - Many neurons output exactly 0
   - More efficient representations
   - Closer to biological neurons

Caveat: "Dying ReLU" problem when neurons always output 0

Deep Feed-Forward Neural Network (DFNN)

Definition

A DFNN is a neural network with one or more hidden layers between input and output layers, where information flows forward through non-linear transformations.

Complete system requires four components:

- Data: Input features and target outputs
- Model: Multi-layer architecture with non-linear activations
- Objective Function: Loss function to minimize
- Learning Algorithm: Backpropagation with gradient descent

This is the same framework as linear models, but with added depth and non-linearity!

Data Representation

$$\text{Dataset:} \quad \mathcal{D} = \{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i=1}^{N} \tag{4}$$

$$\text{Input Design Matrix:} \quad \mathbf{X} \in \mathbb{R}^{N \times d} = \begin{bmatrix} \mathbf{x}^{(1)T} \\ \mathbf{x}^{(2)T} \\ \vdots \\ \mathbf{x}^{(N)T} \end{bmatrix} = \begin{bmatrix} x_1^{(1)} & x_2^{(1)} & \cdots & x_d^{(1)} \\ x_1^{(2)} & x_2^{(2)} & \cdots & x_d^{(2)} \\ \vdots & \vdots & \ddots & \vdots \\ x_1^{(N)} & x_2^{(N)} & \cdots & x_d^{(N)} \end{bmatrix} \tag{5}$$

$$\text{Target Matrix:} \quad \mathbf{Y} \in \mathbb{R}^{N \times K} = \begin{bmatrix} \mathbf{y}^{(1)T} \\ \mathbf{y}^{(2)T} \\ \vdots \\ \mathbf{y}^{(N)T} \end{bmatrix} \tag{6}$$

where K is the output dimension (1 for regression, C for classification with C classes).

Mini-batch Representation

For computational efficiency, we process data in mini-batches

$$\text{Mini-batch of size } \mathcal{B}: \quad \mathcal{B} = \{(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})\}_{i \in \text{batch}} \tag{7}$$

$$\text{Mini-batch input matrix:} \quad \mathbf{X}_{\mathcal{B}} \in \mathbb{R}^{B \times d} \tag{8}$$

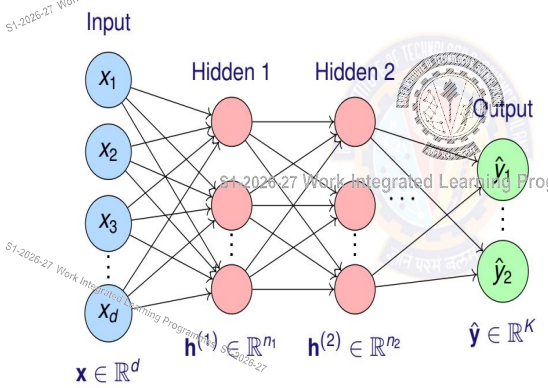
$$\text{Mini-batch target matrix:} \quad \mathbf{Y}_{\mathcal{B}} \in \mathbb{R}^{B \times K} \tag{9}$$

Common batch sizes: $B \in \{32, 64, 128, 256\}$

- Balances computational efficiency with gradient stability
- Enables parallelization on GPUs

DFNN Architecture

Network with L layers:



Notation:

- L: Number of layers (including output, excluding input)
- n_ℓ: Number of units in layer ℓ
- $d = n_0$: Input dimension
- $K = n_L$: Output dimension

Layer-wise Computation

For each layer $\ell = 1, 2, \ldots, L$:

$$\text{Parameters:} \quad \mathbf{W}^{(\ell)} \in \mathbb{R}^{n_{\ell-1} \times n_\ell} \quad \text{(weight matrix)} \tag{10}$$

$$\mathbf{b}^{(\ell)} \in \mathbb{R}^{n_\ell} \quad \text{(bias vector)} \tag{11}$$

$$\text{Initial condition:} \quad \mathbf{h}^{(0)} = \mathbf{x} \quad \text{(input)} \tag{12}$$

$$\text{Pre-activation (linear transformation):} \quad \mathbf{z}^{(\ell)} = \mathbf{h}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)} \in \mathbb{R}^{n_\ell} \tag{13}$$

$$\text{Activation (non-linear transformation):} \quad \mathbf{h}^{(\ell)} = \sigma^{(\ell)}(\mathbf{z}^{(\ell)}) \in \mathbb{R}^{n_\ell} \tag{14}$$

$$\text{Final output:} \quad \hat{\mathbf{y}} = \mathbf{h}^{(L)} \tag{15}$$

Vectorized Computation for Mini-Batch

For mini-batch $\mathbf{X}_\mathcal{B} \in \mathbb{R}^{B \times d}$:

$$\text{Initial:} \quad \mathbf{H}^{(0)} = \mathbf{X}_\mathcal{B} \tag{16}$$

$$\text{Layer } \ell \text{ computation:} \quad \mathbf{Z}^{(\ell)} = \mathbf{H}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{1}_B \mathbf{b}^{(\ell)T} \in \mathbb{R}^{B \times n_\ell} \tag{17}$$

$$\mathbf{H}^{(\ell)} = \sigma^{(\ell)}(\mathbf{Z}^{(\ell)}) \in \mathbb{R}^{B \times n_\ell} \tag{18}$$

$$\text{Final:} \quad \hat{\mathbf{Y}}_\mathcal{B} = \mathbf{H}^{(L)} \in \mathbb{R}^{B \times K} \tag{19}$$

where:

- $\mathbf{H}^{(\ell)} \in \mathbb{R}^{B \times n_\ell}$ contains activations for all B examples
- $\mathbf{1}_B \in \mathbb{R}^{B \times 1}$ is a vector of ones (for broadcasting bias)

Total Parameters in Network

Parameter count for layer ℓ:

$$\text{Parameters}^{(\ell)} = n_{\ell-1} \times n_\ell + n_\ell = n_\ell(n_{\ell-1} + 1) \tag{20}$$

Total parameters in network:

$$\text{Total} = \sum_{\ell=1}^{L} n_\ell(n_{\ell-1} + 1) \tag{21}$$

Example: Network with 3-layer architecture $784 \to 256 \to 128 \to 10$ neurons

- Layer 1: $784 \times 256 + 256 = 200{,}960$
- Layer 2: $256 \times 128 + 128 = 32{,}896$
- Layer 3: $128 \times 10 + 10 = 1{,}290$
- Total: 235,146 parameters

The loss function depends on the task:

1. Regression (continuous output):

$$\text{MSE:} \quad \ell(\hat{\mathbf{y}}, \mathbf{y}) = \frac{1}{2}\|\hat{\mathbf{y}} - \mathbf{y}\|^2 = \frac{1}{2}\sum_{k=1}^{K}(\hat{y}_k - y_k)^2 \tag{22}$$

2. Binary Classification ($K = 1$, output in $(0, 1)$):

$$\text{BCE:} \quad \ell(\hat{y}, y) = -[y \log(\hat{y}) + (1 - y)\log(1 - \hat{y})] \tag{23}$$

3. Multi-class Classification ($K = C$ classes):

$$\text{CCE:} \quad \ell(\hat{\mathbf{y}}, \mathbf{y}) = -\sum_{k=1}^{C} y_k \log(\hat{y}_k) \tag{24}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Loss for single example:

$$\ell^{(i)} = \ell(\hat{\mathbf{y}}^{(i)}, \mathbf{y}^{(i)}) \tag{25}$$

Average loss over mini-batch $\mathcal{B}$ of size B:

$$J(\mathbf{W}, \mathbf{b}) = \frac{1}{B}\sum_{i \in \mathcal{B}} \ell^{(i)} = \frac{1}{B}\sum_{i \in \mathcal{B}} \ell(\hat{\mathbf{y}}^{(i)}, \mathbf{y}^{(i)}) \tag{26}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

where $\mathbf{W} = \{\mathbf{W}^{(1)}, \mathbf{W}^{(2)}, \ldots, \mathbf{W}^{(L)}\}$ and $\mathbf{b} = \{\mathbf{b}^{(1)}, \mathbf{b}^{(2)}, \ldots, \mathbf{b}^{(L)}\}$ represent all network parameters.

Goal: Find parameters that minimize J:

$$\mathbf{W}^*, \mathbf{b}^* = \arg\min_{\mathbf{W}, \mathbf{b}} J(\mathbf{W}, \mathbf{b}) \tag{27}$$

- Core idea: Update parameters using gradients from mini-batches
- Update rule for layer l:

$$\mathbf{W}^{(\ell)} \leftarrow \mathbf{W}^{(\ell)} - \eta \frac{\partial J}{\partial \mathbf{W}^{(\ell)}} \tag{28}$$

$$\mathbf{b}^{(\ell)} \leftarrow \mathbf{b}^{(\ell)} - \eta \frac{\partial J}{\partial \mathbf{b}^{(\ell)}} \tag{29}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

where $\eta > 0$ is the learning rate.

Challenge: How to compute $\frac{\partial J}{\partial \mathbf{W}^{(\ell)}}$ and $\frac{\partial J}{\partial \mathbf{b}^{(\ell)}}$ efficiently?

Solution: **Backpropagation** algorithm!

- Efficiently computes gradients using chain rule
- Key innovation that enabled deep learning

Algorithm 1: DFNN Training with Mini-Batch SGD (Part 1: Forward Pass)

Input: Dataset $\mathcal{D}$, architecture $(n_0, n_1, \ldots, n_L)$, learning rate η, batch size B, epochs T
Output: Trained parameters $\{\mathbf{W}^{(\ell)}, \mathbf{b}^{(\ell)}\}_{\ell=1}^{L}$

for $\ell = 1$ **to** L **do**
  Initialize $\mathbf{W}^{(\ell)} \in \mathbb{R}^{n_{\ell-1} \times n_\ell}$ randomly;
  Initialize $\mathbf{b}^{(\ell)} \in \mathbb{R}^{n_\ell}$ to zeros;
for *epoch* $t = 1$ **to** T **do**
  Shuffle dataset $\mathcal{D}$;
  for *each mini-batch* $\mathcal{B}$ *of size* B **do**
    // Forward Pass
    $\mathbf{H}^{(0)} \leftarrow \mathbf{X}_{\mathcal{B}}$;
    for $\ell = 1$ **to** L **do**
      $\mathbf{Z}^{(\ell)} \leftarrow \mathbf{H}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{1}_B \mathbf{b}^{(\ell)T}$;
      $\mathbf{H}^{(\ell)} \leftarrow \sigma^{(\ell)}(\mathbf{Z}^{(\ell)})$;
    $\hat{\mathbf{Y}}_{\mathcal{B}} \leftarrow \mathbf{H}^{(L)}$;

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Algorithm 2: DFNN Training with Mini-Batch SGD (Part 2: BackProp & Update)

// Backward Pass (Backpropagation)

$\delta^{(L)} \leftarrow \frac{\partial J}{\partial \mathbf{Z}^{(L)}}$ // Output layer gradient Depends on loss & activation

for $\ell = L-1$ **to** 1 **do**

  $\delta^{(\ell)} \leftarrow \left(\delta^{(\ell+1)}\mathbf{W}^{(\ell+1)T}\right) \odot \sigma'^{(\ell)}(\mathbf{Z}^{(\ell)})$ // Backpropagate through hidden layers

for $\ell = 1$ **to** L **do**

  $\frac{\partial J}{\partial \mathbf{W}^{(\ell)}} \leftarrow \frac{1}{B}\mathbf{H}^{(\ell-1)T}\delta^{(\ell)}$ // Compute parameter gradients

  $\frac{\partial J}{\partial \mathbf{b}^{(\ell)}} \leftarrow \frac{1}{B}\mathbf{1}_B^T\delta^{(\ell)}$;

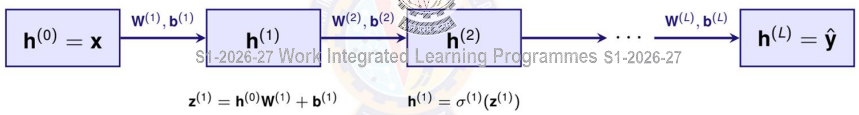
// Update parameters

for $\ell = 1$ **to** L **do**

  $\mathbf{W}^{(\ell)} \leftarrow \mathbf{W}^{(\ell)} - \eta\frac{\partial J}{\partial \mathbf{W}^{(\ell)}}$;

  $\mathbf{b}^{(\ell)} \leftarrow \mathbf{b}^{(\ell)} - \eta\frac{\partial J}{\partial \mathbf{b}^{(\ell)}}$;

Given input $\mathbf{x} \in \mathbb{R}^d$, compute output $\hat{\mathbf{y}} \in \mathbb{R}^K$



Algorithm 3: Forward Propagation

Input: Input $\mathbf{x} \in \mathbb{R}^d$, Parameters $\{\mathbf{W}^{(\ell)}, \mathbf{b}^{(\ell)}\}_{\ell=1}^{L}$

Output: Prediction $\hat{\mathbf{y}} \in \mathbb{R}^K$, All activations $\{\mathbf{h}^{(\ell)}\}_{\ell=0}^{L}$

// Initialize with input

$\mathbf{h}^{(0)} \leftarrow \mathbf{x}$;

// Forward through all layers

for $\ell = 1$ **to** L **do**

  // Linear transformation

  $\mathbf{z}^{(\ell)} \leftarrow \mathbf{h}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)}$;

  // Non-linear activation

  $\mathbf{h}^{(\ell)} \leftarrow \sigma^{(\ell)}(\mathbf{z}^{(\ell)})$;

  // Store activations for backpropagation

// Output is final layer activation

$\hat{\mathbf{y}} \leftarrow \mathbf{h}^{(L)}$;

For mini-batch $\mathbf{X}_{\mathcal{B}} \in \mathbb{R}^{B\times d}$:

Initialize: $\mathbf{H}^{(0)} = \mathbf{X}_{\mathcal{B}} \in \mathbb{R}^{B\times d}$ (30)

For each layer $\ell = 1, \ldots, L$:

$$\mathbf{Z}^{(\ell)} = \mathbf{H}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{1}_B\mathbf{b}^{(\ell)T} \in \mathbb{R}^{B\times n_\ell} \quad (31)$$

$$\mathbf{H}^{(\ell)} = \sigma^{(\ell)}(\mathbf{Z}^{(\ell)}) \in \mathbb{R}^{B\times n_\ell} \quad (32)$$

Output predictions :

$$\hat{\mathbf{Y}}_{\mathcal{B}} = \mathbf{H}^{(L)} \in \mathbb{R}^{B\times K} \quad (33)$$

Computational complexity : $O\left(B\sum_{\ell=1}^{L} n_{\ell-1}n_\ell\right)$ per forward pass (34)

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

After forward pass, compute loss for the mini-batch:

$$\text{Per-example loss:} \quad \ell^{(i)} = \ell(\hat{\mathbf{y}}^{(i)}, \mathbf{y}^{(i)}) \tag{35}$$

$$\text{Average loss over mini-batch:} \quad J = \frac{1}{B}\sum_{i=1}^{B} \ell^{(i)} \tag{36}$$

Common loss functions:

$$\text{Regression:} \quad MSE = \frac{1}{2}\|\hat{\mathbf{y}}^{(i)} - \mathbf{y}^{(i)}\|^2$$

$$\text{Binary Classification:} \quad BCE = -[y^{(i)} \log(\hat{y}^{(i)}) + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})]$$

$$\text{Multi-class:} \quad CCE = -\sum_{k=1}^{C} y_k^{(i)} \log(\hat{y}_k^{(i)})$$

First step of backpropagation: compute gradient w.r.t. output layer pre-activation

Output layer gradient:

$$\delta^{(L)} = \frac{\partial J}{\partial \mathbf{Z}^{(L)}} \in \mathbb{R}^{B \times K} \tag{37}$$

For common loss-activation pairs: (for all 3, the output gradient is same.)

1. MSE + Identity: (Regression)
2. BCE + Sigmoid: (Binary classification)
3. CCE + Softmax: (Multi-class classification)

$$\delta^{(L)} = \frac{1}{B}(\hat{\mathbf{Y}} - \mathbf{Y}) \tag{38}$$

PURPOSE

Backpropagation efficiently computes gradients of the loss w.r.t. all parameters by applying the chain rule backward through the network.

Key insight: Use chain rule recursively

Notation: Define error signal for layer l:

$$\delta^{(l)} = \frac{\partial J}{\partial \mathbf{Z}^{(l)}} \in \mathrm{R}^{B \times n_{\text{Æ}}} \tag{39}$$

Two main steps:

- Backpropagate error signals: $\delta^{(L)} \rightarrow \delta^{(L-1)} \rightarrow \cdots \rightarrow \delta^{(1)}$
- Compute parameter gradients using error signals

Given: $\delta^{(\ell+1)} = \frac{\partial J}{\partial \mathbf{Z}^{(\ell+1)}}$ Compute: $\delta^{(\ell)} = \frac{\partial J}{\partial \mathbf{Z}^{(\ell)}}$

$$\text{Chain rule:} \quad \delta^{(\ell)} = \frac{\partial J}{\partial \mathbf{Z}^{(\ell)}} = \frac{\partial J}{\partial \mathbf{H}^{(\ell)}} \frac{\partial \mathbf{H}^{(\ell)}}{\partial \mathbf{Z}^{(\ell)}} \tag{40}$$

$$\text{Derivation:} \quad \frac{\partial J}{\partial \mathbf{H}^{(\ell)}} = \frac{\partial J}{\partial \mathbf{Z}^{(\ell+1)}} \frac{\partial \mathbf{Z}^{(\ell+1)}}{\partial \mathbf{H}^{(\ell)}} = \delta^{(\ell+1)} \mathbf{W}^{(\ell+1)T} \tag{41}$$

$$\frac{\partial \mathbf{H}^{(\ell)}}{\partial \mathbf{Z}^{(\ell)}} = \sigma'^{(\ell)}(\mathbf{Z}^{(\ell)}) \quad \text{(element-wise derivative)} \tag{42}$$

Backpropagation equation:

$$\boxed{\delta^{(\ell)} = \left(\delta^{(\ell+1)} \mathbf{W}^{(\ell+1)T}\right) \odot \sigma'^{(\ell)}(\mathbf{Z}^{(\ell)})} \tag{43}$$

where $\odot$ denotes element-wise multiplication (Hadamard product)

Given error signal $\delta^{(\ell)}$, compute gradients w.r.t. parameters:

$$\text{Weight gradient: } \frac{\partial J}{\partial \mathbf{W}^{(\ell)}} = \frac{\partial J}{\partial \mathbf{Z}^{(\ell)}} \frac{\partial \mathbf{Z}^{(\ell)}}{\partial \mathbf{W}^{(\ell)}} = \frac{1}{B} \mathbf{H}^{(\ell-1)T} \delta^{(\ell)} \in \mathbb{R}^{n_{\ell-1} \times n_\ell} \tag{44}$$

$$\text{Bias gradient: } \frac{\partial J}{\partial \mathbf{b}^{(\ell)}} = \frac{\partial J}{\partial \mathbf{Z}^{(\ell)}} \frac{\partial \mathbf{Z}^{(\ell)}}{\partial \mathbf{b}^{(\ell)}} = \frac{1}{B} \mathbf{1}_B^T \delta^{(\ell)} \in \mathbb{R}^{n_\ell} \tag{45}$$

where $\mathbf{1}_B \in \mathbb{R}^{B \times 1}$ is a vector of ones.

Key: Error signal $\delta^{(\ell)}$ is shared for both weight and bias gradients!

Algorithm 4: Backpropagation

Input: Loss J, Activations $\{\mathbf{H}^{(\ell)}, \mathbf{Z}^{(\ell)}\}_{\ell=1}^{L}$, Weights $\{\mathbf{W}^{(\ell)}\}_{\ell=1}^{L}$
Output: Gradients $\{\frac{\partial J}{\partial \mathbf{W}^{(\ell)}}, \frac{\partial J}{\partial \mathbf{b}^{(\ell)}}\}_{\ell=1}^{L}$

// Compute output layer error signal
$\delta^{(L)} \leftarrow \frac{\partial J}{\partial \mathbf{Z}^{(L)}}$ // Depends on loss function
// Backpropagate error signals
for $\ell = L-1$ **to** 1 **do**
  $\delta^{(\ell)} \leftarrow \left(\delta^{(\ell+1)} \mathbf{W}^{(\ell+1)T}\right) \odot \sigma'^{(\ell)}(\mathbf{Z}^{(\ell)})$;
// Compute parameter gradients
for $\ell = 1$ **to** L **do**
  $\frac{\partial J}{\partial \mathbf{W}^{(\ell)}} \leftarrow \frac{1}{B} \mathbf{H}^{(\ell-1)T} \delta^{(\ell)}$;
  $\frac{\partial J}{\partial \mathbf{b}^{(\ell)}} \leftarrow \frac{1}{B} \mathbf{1}_B^T \delta^{(\ell)}$;

Common activation derivatives (for backpropagation):

1. Sigmoid:

$$\sigma(z) = \frac{1}{1+e^{-z}} \qquad \sigma'(z) = \sigma(z)(1-\sigma(z)) \tag{46}$$

2. Tanh:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \qquad \tanh'(z) = 1 - \tanh^2(z) \tag{47}$$

3. ReLU:

$$\text{ReLU}(z) = \max(0, z) \qquad \text{ReLU}'(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases} \tag{48}$$

Why backpropagation is efficient:

Naive approach: Compute each $\frac{\partial J}{\partial w_{ij}^{(\ell)}}$ independently

- Requires one forward pass per parameter
- Complexity: $O(P \times C_{\text{forward}})$ where P = total parameters
- For millions of parameters, this is intractable!

Backpropagation: Reuses intermediate computations

- One forward pass + one backward pass
- Complexity: $O(C_{\text{forward}} + C_{\text{backward}}) \approx 2 \times O(C_{\text{forward}})$
- Backward pass has same complexity as forward pass

Speedup: $\frac{P}{2}$ (e.g., $\sim 100{,}000\times$ for $P = 200{,}000$)

Key insight: Dynamic programming via chain rule!

PARAMETER UPDATE STEP

After computing gradients, update all parameters:

For each layer $l = 1, \ldots, L$: (Vector form for single example)

$$\mathbf{W}^{(l)} \leftarrow \mathbf{W}^{(l)} - \eta \mathbf{h}^{(l-1)T} \delta^{(l)} \quad (49)$$

$$\mathbf{b}^{(l)} \leftarrow \mathbf{b}^{(l)} - \eta \delta^{(l)} \quad (50)$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

where $\eta > 0$ is the learning rate.

This moves parameters in the direction that reduces the loss!

USING TRAINED DFNN FOR PREDICTION

After training, prediction is simply forward propagation:

Input: New example $\mathbf{x}_{new} \in \mathbb{R}^d$

Process:

1. Initialize: $\mathbf{h}^{(0)} = \mathbf{x}_{new}$
2. For $\ell = 1$ to L:

$$\mathbf{z}^{(\ell)} = \mathbf{h}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{b}^{(\ell)} \quad (51)$$

$$\mathbf{h}^{(\ell)} = \sigma^{(\ell)}(\mathbf{z}^{(\ell)}) \quad (52)$$

3. Output: $\hat{\mathbf{y}}_{new} = \mathbf{h}^{(L)}$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Interpretation depends on task:

- Regression: $\hat{\mathbf{y}}_{new}$ is the predicted continuous value
- Binary classification: $\hat{y}_{new} > 0.5 \rightarrow$ class 1, else class 0
- Multi-class: $\arg\max_k \hat{\mathbf{y}}_{new,k}$ is the predicted class

BATCH PREDICTION

Efficiently predict for multiple examples:

Input: Test set $\mathbf{X}_{test} \in \mathbb{R}^{N_{test} \times d}$

Vectorized forward pass:

$$\mathbf{H}^{(0)} = \mathbf{X}_{test} \quad (53)$$

$$\mathbf{Z}^{(\ell)} = \mathbf{H}^{(\ell-1)}\mathbf{W}^{(\ell)} + \mathbf{1}_{N_{test}}\mathbf{b}^{(\ell)T} \quad (54)$$

$$\mathbf{H}^{(\ell)} = \sigma^{(\ell)}(\mathbf{Z}^{(\ell)}) \quad (55)$$

$$\hat{\mathbf{Y}}_{test} = \mathbf{H}^{(L)} \in \mathbb{R}^{N_{test} \times K} \quad (56)$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

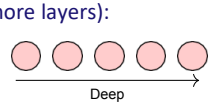
Advantages:

- Efficient matrix operations (GPU acceleration)
- All predictions in one forward pass
- Essential for large-scale deployment

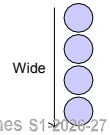
NETWORK DEPTH VS WIDTH

Two ways to increase model capacity:

Depth (more layers):



Width (more units per layer):



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Properties:

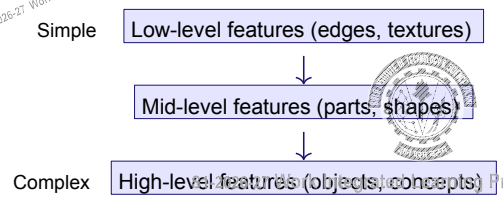
- Hierarchical features
- More expressive
- Better generalization

Properties:

- More parallel features
- Higher capacity
- Risk of overfitting

Why Depth Matters?

Deep networks learn hierarchical representations:



Deep networks can represent exponentially more functions than shallow ones with the same number of parameters!

Example: Image classification

Layer 1: Detect edges and colors
Layer 2: Combine into textures and simple shapes
Layer 3: Recognize object parts (eyes, wheels)
Layer 4+: Identify complete objects (faces, cars)

Universal Approximation Theorem

Theorem (Cybenko, 1989; Hornik et al., 1989)

A feed-forward network with a single hidden layer containing a finite number of neurons can approximate any continuous function on a compact subset of $\mathbb{R}^n$, given an appropriate activation function.

What this means:

Theoretically, even shallow networks are powerful
BUT: May require exponentially many hidden units
Deep networks are much more parameter-efficient

Practice:

Deep networks learn better representations
Achieve better performance with fewer parameters
Generalize better to new data

Depth enables efficient learning of hierarchical structure!

Challenges with Very Deep Networks

Problems that arise with increasing depth:

1. Vanishing Gradients:
   - Gradients become exponentially small in early layers
   - Network fails to learn useful features
   - Solution: ReLU activations, careful initialization, normalization
2. Exploding Gradients:
   - Gradients become exponentially large Causes numerical instability (NaN/Inf values)
   - Solution: Gradient clipping, proper initialization
3. Degradation Problem:
   - Very deep networks harder to optimize
   - Training error may increase with depth
   - Solution: Residual connections (ResNets), skip connections

Design Considerations

Key decisions when designing a DFNN:

1. Number of layers L
   - How deep should the network be?
2. Units per layer $n_1, n_2, \ldots, n_L$
   - How wide should each layer be?
3. Activation functions $\sigma^{(l)}$
   - Which non-linearity for each layer?
4. Output layer design
   - Depends on task (regression vs classification)
5. Regularization
   - How to prevent overfitting?

No universal answer - depends on problem, data, and computational resources!

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Output Layer Design

Output layer depends on the task:

1. Regression (continuous values):
   - Units: K = dimension of output
   - Activation: Identity (linear)
   - Loss: Mean Squared Error (MSE)
2. Binary Classification (2 classes):
   - Units: $K = 1$
   - Activation: Sigmoid $\sigma(z) = \frac{1}{1+e^{-z}}$
   - Loss: Binary Cross-Entropy (BCE)
3. Multi-class Classification (C classes):
   - Units: $K = C$
   - Activation: Softmax $\text{softmax}(\mathbf{z})_k = \frac{e^{z_k}}{\sum_{j=1}^{C} e^{z_j}}$
   - Loss: Categorical Cross-Entropy (CCE)

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

General Architecture Guidelines

1. Start Simple:
   - Begin with 2-3 hidden layers
   - 10-100 units per layer (depending on problem scale)
   - Gradually increase complexity if needed
2. Common Patterns:
   - Decreasing width: $n_1 > n_2 > \cdots > n_L$
     - Funnel architecture (compression)
   - Constant width: $n_1 = n_2 = \cdots = n_{L-1}$
     - Uniform representation
   - Hourglass: Wide → Narrow → Wide
     - Learns compressed representations
3. Scale with Data:
   More data → can support larger networks

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Hidden Layer Activation Functions

Activation	When to Use	Avoid When
ReLU	Default choice Fast, no vanishing gradient	Very deep networks (dying ReLU)
Leaky ReLU	Deep networks Fixes dying ReLU	Never (safe choice)
Tanh	Small networks Zero-centered	Very deep networks (vanishing gradient)
Sigmoid	Avoid for hidden layers (only for binary output)	Hidden layers (strong saturation)

Modern recommendation:

Default: ReLU for hidden layers

Alternative: Leaky ReLU, ELU for very deep networks

Avoid: Sigmoid/Tanh in hidden layers (unless specific reason)

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Regularization Techniques

Prevent overfitting with:

1. L2 Regularization (Weight Decay):
2. Dropout:
3. Early Stopping:
4. Batch Normalization:

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Note: You will learn about this in Module 10 and 11

PARAMETER INITIALIZATION

Proper initialization is critical for training success!

✗ Bad: Zero initialization

✓ Good: Random initialization

- Xavier/Glorot Initialization for Sigmoid / Tanh
- He Initialization for ReLU

Biases: Initialize to zero: $\mathbf{b}^{(l)} = \mathbf{0}$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Note: You will learn about this in Module 10 and 11

1. Learning Rate η:
   - Start with: $\eta \in \{0.001, 0.01, 0.1\}$
   - Consider adaptive methods (Adam with $\eta = 0.001$)
2. Batch Size B:
   - Common: $B \in \{32, 64, 128, 256\}$
   - Larger batch → faster but may hurt
   - generalization Smaller batch → better
   - generalization but slower Limited by GPU memory
3. Number of Epochs T :
   - Use early stopping based on validation loss
   - Typically 50-200 epochs for moderate datasets Monitor for overfitting

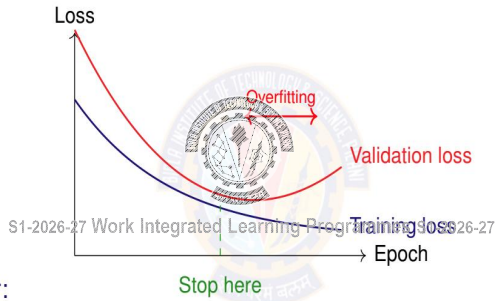
FEATURE PREPROCESSING

Preprocess features before training:

1. Normalization/Standardization:
   - Standardization (recommended)
   - Min-Max Scaling
   - Apply same transformation to validation/test sets
2. Handle Missing Values:
   - Impute with mean/median/mode
   - Or use indicator variables
3. Encode Categorical Variables:
   - One-hot encoding for nominal categories
   - Ordinal encoding for ordered categories

Monitoring Training Progress

Track metrics during training:



What to monitor:

- Training loss (should decrease steadily)
- Validation loss (should decrease then plateau/increase)
- Training/validation accuracy (for classification)
- Gradient norms (detect vanishing/exploding gradients)

Diagnosing Training Issues

Symptom	Diagnosis	Solution
Loss = NaN/Inf	Numerical instability	Decrease η, check gradients, normalize inputs
Loss not decreasing	Learning rate too small OR bug	Increase η, verify gradient computation
Loss oscillating	Learning rate too large	Decrease η, use learning rate schedule
Training loss ↓, validation loss ↑	Overfitting	Add regularization, dropout, more data
Both losses high	Underfitting	Increase capacity (wider/deeper), train longer
Slow convergence	Poor initialization OR bad features	Use proper init (He/Xavier), normalize features
Gradients → 0	Vanishing gradients	Use ReLU, better init, batch normalization
Gradients → ∞	Exploding gradients	Gradient clipping, decrease η

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Debugging Checklist

Before training:

- Features normalized/standardized Data
- split into train/val/test
- Output layer matches task (sigmoid for binary, softmax for multi-class)
- Loss function matches output activation
- Weights initialized properly (not all zeros!)

During training:

- Loss decreasing on training set
- Validation loss tracked separately
- Gradients neither vanishing nor exploding
- No NaN/Inf values in loss or activations
- Training accuracy improving

Sanity checks:

- Try overfitting a small batch (should reach 0 loss)

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Implementation Best Practices

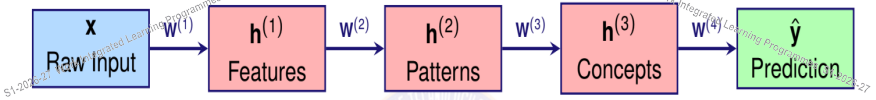
1. Vectorization:
   - Always use matrix operations, never loops over examples
   - Enables GPU acceleration
2. Modular Code:
   - Separate forward pass, loss, backward pass functions
   - Test each component independently
3. Numerical Stability:
   - Use stable implementations (e.g., log-sum-exp for softmax)
   - Clip gradients if needed: $\|\nabla\| > \text{threshold}$
   - Monitor for NaN/Inf values
4. Reproducibility:
   - Set random seeds
   - Save hyperparameters
   - Track experiments systematically

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Key Takeaways

- Non-linearity is essential for learning complex patterns; without it, deep networks collapse to linear models.
- DFNN = stacked layers with non-linear activations, enabling hierarchical feature learning.
- Four components: Data, Model (architecture), Objective (loss), Learning (backpropagation + SGD).
- Backpropagation efficiently computes gradients using chain rule; enables training deep networks.
- Depth enables learning hierarchical features and improves expressiveness.
- Width increases capacity within each layer but requires more data.
- Proper initialization, normalization, and regularization are critical for successful training.
- ReLU is the default activation for hidden layers; use task-specific activations for output.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27



Learns hierarchical representations automatically from data through gradient based optimization

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Deep learning has revolutionized:

- Computer vision (object detection, segmentation)
- Natural language processing (translation, generation)
- Speech recognition and synthesis
- Game playing (Go, Chess, StarCraft)

WHAT NEXT?

Specialized Architectures:

- Convolutional Neural Networks (CNNs): For images and spatial data
- Recurrent Neural Networks (RNNs): For sequences and time series
- Transformers: For attention-based models (GPT, BERT)
- Graph Neural Networks: For graph-structured data

Advanced Topics:

- Optimization algorithms (Adam, RMSprop, AdaGrad)
- Advanced regularization (Dropout, Batch Normalization)
- Transfer learning and pre-training
- Model interpretability and explainability
- Efficient inference and model compression

DENNs are the foundation – master them first!

1. Implement from scratch:
   - Write forward pass, backpropagation manually
   - Verify with gradient checking
   - Train on simple datasets (XOR, MNIST)
2. Use frameworks effectively:
   - PyTorch, TensorFlow, JAX handle backprop automatically
   - But understanding the math makes debugging easier
   - Focus on architecture design and hyperparameter tuning
3. Experiment and iterate:
   - Start simple, gradually increase complexity
   - Track experiments systematically
   - Learn from failures (most time spent debugging!)
4. Study successful architectures:
   - Read papers, examine code implementations

MODULE # 5 : DEEP FEED-FORWARD NEURAL NETWORKS (Numerical Examples)

Session 6
(31st May,2026)

Problem: Consider a simple 2-layer neural network with the following parameters. Calculate the forward propagation step by step.

Network Architecture:

- Input layer: 2 neurons
- Hidden layer: 3 neurons (ReLU activation)
- Output layer: 2 neurons (Sigmoid activation)

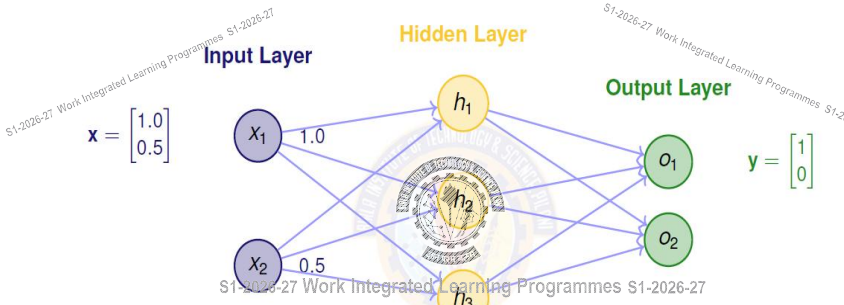
Given Parameters:

$$x = \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix} \quad W^{[1]} = \begin{bmatrix} 0.5 & 0.2 \\ -0.3 & 0.8 \\ 0.1 & -0.4 \end{bmatrix}, \quad b^{[1]} = \begin{bmatrix} 0.1 \\ -0.2 \\ 0.3 \end{bmatrix}$$

$$W^{[2]} = \begin{bmatrix} 0.4 & -0.1 & 0.6 \\ 0.2 & 0.7 & -0.3 \end{bmatrix}, \quad b^{[2]} = \begin{bmatrix} 0.2 \\ -0.1 \end{bmatrix}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

FORWARD PROP: STEP 0 - INITIAL NETWORK



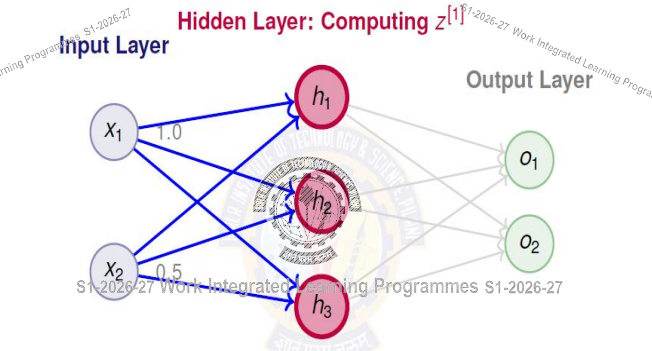
$$W^{[1]} = \begin{bmatrix} 0.5 & 0.2 \\ -0.3 & 0.8 \\ 0.1 & -0.4 \end{bmatrix}, \quad b^{[1]} = \begin{bmatrix} 0.1 \\ -0.2 \\ 0.3 \end{bmatrix}$$

$$W^{[2]} = \begin{bmatrix} 0.4 & -0.1 & 0.6 \\ 0.2 & 0.7 & -0.3 \end{bmatrix}, \quad b^{[2]} = \begin{bmatrix} 0.2 \\ -0.1 \end{bmatrix}$$

Ready to compute forward propagation!

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

FORWARD PROP: STEP 1 - COMPUTING $z^{[1]}$



$$z^{[1]} = W^{[1]}x + b^{[1]} = \begin{bmatrix} 0.5 & 0.2 \\ -0.3 & 0.8 \\ 0.1 & -0.4 \end{bmatrix} \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.2 \\ 0.3 \end{bmatrix} = \begin{bmatrix} 0.7 \\ -0.1 \\ 0.2 \end{bmatrix}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

SOLUTION: FORWARD PROP - LAYER 1

Step 1: Calculate $z^{[1]}$

$$z^{[1]} = W^{[1]}x + b^{[1]}$$

$$z^{[1]} = \begin{bmatrix} 0.5 & 0.2 \\ -0.3 & 0.8 \\ 0.1 & -0.4 \end{bmatrix} \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.2 \\ 0.3 \end{bmatrix}$$

$$= \begin{bmatrix} 0.5(1.0) + 0.2(0.5) \\ -0.3(1.0) + 0.8(0.5) \\ 0.1(1.0) + (-0.4)(0.5) \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.2 \\ 0.3 \end{bmatrix}$$

$$= \begin{bmatrix} 0.6 \\ 0.1 \\ -0.1 \end{bmatrix} + \begin{bmatrix} 0.1 \\ -0.2 \\ 0.3 \end{bmatrix} = \begin{bmatrix} 0.7 \\ -0.1 \\ 0.2 \end{bmatrix}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

SOLUTION: FORWARD PROP - LAYER 1 ACTIVATI

Step 2: Apply ReLU activation $a^{[1]} = \text{ReLU}(z^{[1]})$

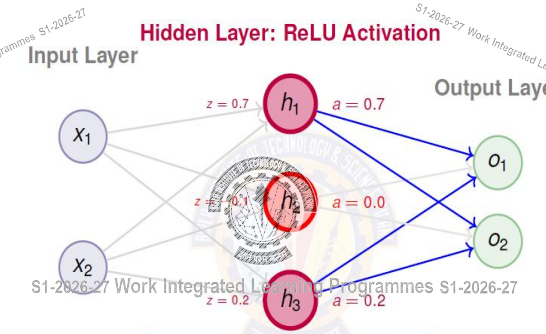
ReLU function: $\text{ReLU}(x) = \max(0, x)$

$$a^{[1]} = \text{ReLU}\begin{bmatrix} 0.7 \\ -0.1 \\ 0.2 \end{bmatrix} = \begin{bmatrix} \max(0, 0.7) \\ \max(0, -0.1) \\ \max(0, 0.2) \end{bmatrix} = \begin{bmatrix} 0.7 \\ 0.0 \\ 0.2 \end{bmatrix}$$

Hidden layer activation: $a^{[1]} = \begin{bmatrix} 0.7 \\ 0.0 \\ 0.2 \end{bmatrix}$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

FORWARD PROP: STEP 2 - APPLYING RELU



$$a^{[1]} = \text{ReLU}\begin{bmatrix} 0.7 \\ -0.1 \\ 0.2 \end{bmatrix} = \begin{bmatrix} 0.7 \\ 0.0 \\ 0.2 \end{bmatrix}$$

Neuron 2 is **dead** ($z < 0$)

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

SOLUTION: FORWARD PROP - LAYER 2

Step 3: Calculate $z^{[2]}$

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]}$$

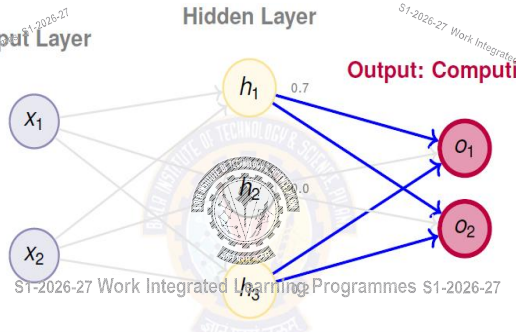
$$z^{[2]} = \begin{bmatrix} 0.4 & -0.1 & 0.6 \\ 0.2 & 0.7 & -0.3 \end{bmatrix}\begin{bmatrix} 0.7 \\ 0.0 \\ 0.2 \end{bmatrix} + \begin{bmatrix} 0.2 \\ -0.1 \end{bmatrix}$$

$$= \begin{bmatrix} 0.4(0.7) + (-0.1)(0.0) + 0.6(0.2) \\ 0.2(0.7) + 0.7(0.0) + (-0.3)(0.2) \end{bmatrix} + \begin{bmatrix} 0.2 \\ -0.1 \end{bmatrix}$$

$$= \begin{bmatrix} 0.28 + 0 + 0.12 \\ 0.14 + 0 - 0.06 \end{bmatrix} + \begin{bmatrix} 0.2 \\ -0.1 \end{bmatrix} = \begin{bmatrix} 0.4 + 0.2 \\ 0.08 - 0.1 \end{bmatrix} = \begin{bmatrix} 0.6 \\ -0.02 \end{bmatrix}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

FORWARD PROP: STEP 3 - COMPUTING $z^{[2]}$



$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]} = \begin{bmatrix} 0.4 & -0.1 & 0.6 \\ 0.2 & 0.7 & -0.3 \end{bmatrix}\begin{bmatrix} 0.7 \\ 0.0 \\ 0.2 \end{bmatrix} + \begin{bmatrix} 0.2 \\ -0.1 \end{bmatrix} = \begin{bmatrix} 0.6 \\ -0.02 \end{bmatrix}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Solution: Forward Prop - Output Layer

Step 4: Apply Sigmoid activation $a^{[2]} = \sigma(z^{[2]})$

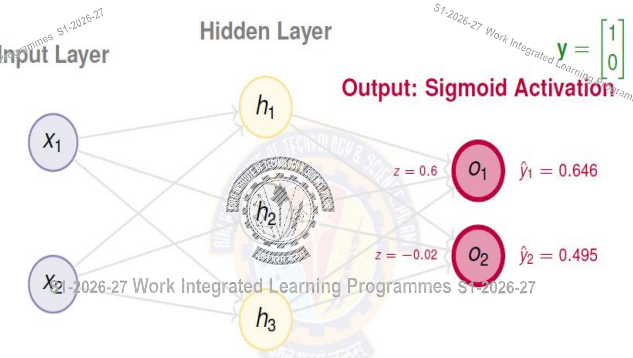
Sigmoid function: $\sigma(x) = \frac{1}{1+e^{-x}}$

$$a^{[2]} = \sigma \begin{bmatrix} 0.6 \\ -0.02 \end{bmatrix} = \begin{bmatrix} \frac{1}{1+e^{-0.6}} \\ \frac{1}{1+e^{0.02}} \end{bmatrix}$$

Output activation: $a^{[2]} = \begin{bmatrix} 0.646 \\ 0.495 \end{bmatrix}$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

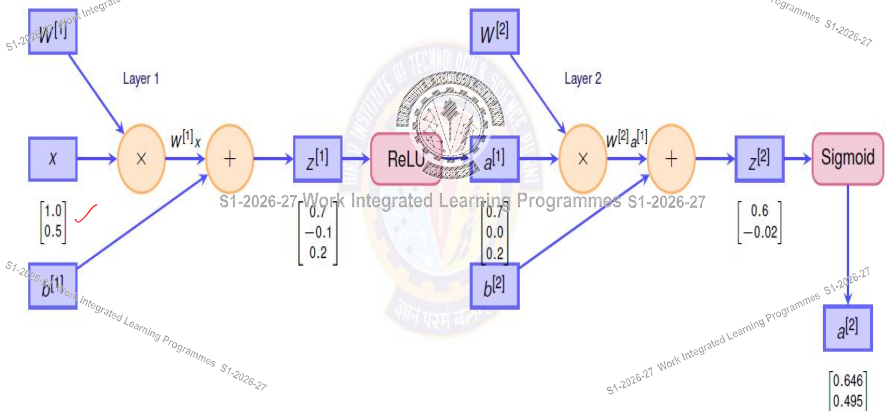
Forward Prop: Step 4 - Applying Sigmoid



$$a^{[2]} = \sigma \begin{bmatrix} 0.6 \\ -0.02 \end{bmatrix} = \begin{bmatrix} 0.646 \\ 0.495 \end{bmatrix}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Computational Graph: Forward Propagation



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

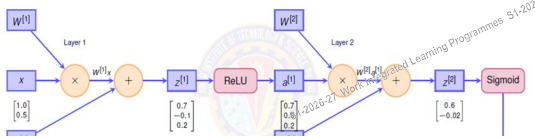
Numerical Problem: Backward Propagation

Problem: Using the previous network, calculate the backward propagation.

Given:

- Forward propagation results from previous problem
  - Target output: $y = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$
  - Loss function: Binary Cross-Entropy (BCE)

Calculate the gradients $\frac{\partial L}{\partial W^{[2]}}$, $\frac{\partial L}{\partial b^{[2]}}$, $\frac{\partial L}{\partial W^{[1]}}$, and $\frac{\partial L}{\partial b^{[1]}}$.



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Solution: Backward Propagation - Loss

Step 1: Calculate the Binary Cross-Entropy loss

For our outputs: $a^{[2]} = \begin{bmatrix} 0.646 \\ 0.495 \end{bmatrix} \quad y = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$

BCE Loss Formula: $L = \frac{-1}{n} \sum_{i=1}^{n} [y_i \log(a_i^{[2]}) + (1 - y_i) \log(1 - a_i^{[2]})]$

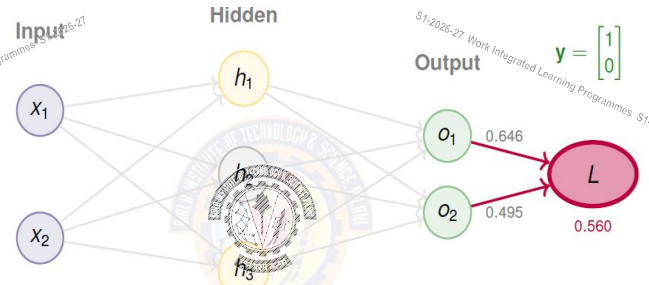
$$L = \frac{-1}{2} [1 \cdot \log(0.646) + (1 - 1) \log(1 - 0.646) + 0 \cdot \log(0.495) + (1 - 0) \cdot \log(1 - 0.495)]$$

$$= \frac{-1}{2} [\log(0.646) + \log(0.505)]$$

$$= \frac{-1}{2} [-0.437 + (-0.683)] = 0.560$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Computing Loss



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

$$L = -\frac{1}{2}[y_1 \log(\hat{y}_1) + (1 - y_1) \log(1 - \hat{y}_1) + y_2 \log(\hat{y}_2) + (1 - y_2) \log(1 - \hat{y}_2)]$$

$$L = -\frac{1}{2}[\log(0.646) + \log(0.505)] = -\frac{1}{2}[-0.437 - 0.683] = 0.560$$

Now we backpropagate to update weights!

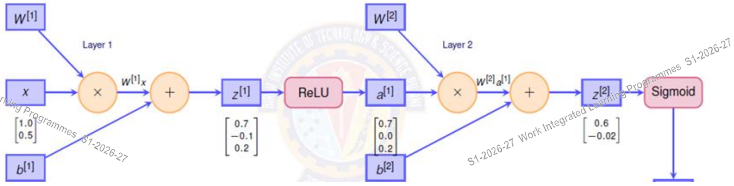
Solution: Backprop - Output Layer Error

Step 2: Calculate output layer error $\delta^{[2]}$

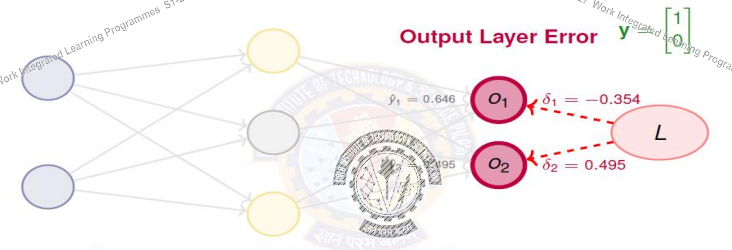
$$\frac{\partial L}{\partial z^{[2]}} = \delta^{[2]} = a^{[2]} - y$$

$$\delta^{[2]} = \begin{bmatrix} 0.646 \\ 0.495 \end{bmatrix} - \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} -0.354 \\ 0.495 \end{bmatrix}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27



Back-Prop Step 1 - Output Layer Error



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

$$\delta^{[2]} = \begin{bmatrix} 0.646 \\ 0.495 \end{bmatrix} - \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} -0.354 \\ 0.495 \end{bmatrix}$$

Solution: Backward Prop - Output Layer

Step 3: Calculate gradients for output layer

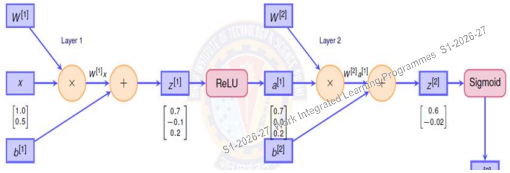
Recall: $a^{[1]} = \begin{bmatrix}0.7 & 0.0 & 0.2\end{bmatrix}^T$

$$\frac{\partial L}{\partial W^{[2]}} = \delta^{[2]}(a^{[1]})^T = \begin{bmatrix}-0.354\\0.495\end{bmatrix}\begin{bmatrix}0.7 & 0.0 & 0.2\end{bmatrix}$$

$$= \begin{bmatrix}-0.354\times 0.7 & -0.354\times 0.0 & -0.354\times 0.2\\0.495\times 0.7 & 0.495\times 0.0 & 0.495\times 0.2\end{bmatrix}$$

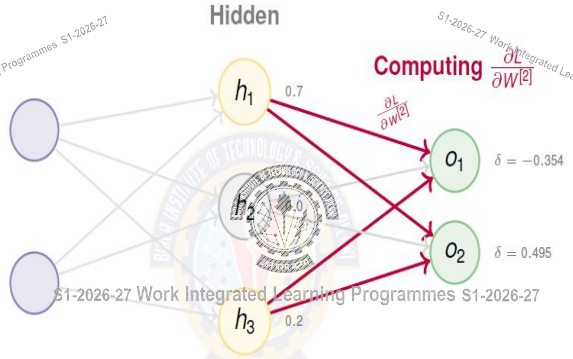
$$= \begin{bmatrix}-0.248 & 0.000 & -0.071\\0.347 & 0.000 & 0.099\end{bmatrix}$$

$$\frac{\partial L}{\partial b^{[2]}} = \delta^{[2]} = \begin{bmatrix}-0.354\\0.495\end{bmatrix}$$



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Back-Prop: Step 2 - Output Layer Gradien



$$\frac{\partial L}{\partial W^{[2]}} = \delta^{[2]}(a^{[1]})^T = \begin{bmatrix}-0.354\\0.495\end{bmatrix}\begin{bmatrix}0.7 & 0.0 & 0.2\end{bmatrix} = \begin{bmatrix}-0.248 & 0.000 & -0.071\\0.347 & 0.000 & 0.099\end{bmatrix}$$

$$\frac{\partial L}{\partial b^{[2]}} = \delta^{[2]} = \begin{bmatrix}-0.354\\0.495\end{bmatrix} \checkmark$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Solution: Backprop - Hidden Layer Error

Step 4: Calculate hidden layer error $\delta^{[1]}$

Backpropagate error to hidden layer: $\delta^{[1]} = (W^{[2]})^T\delta^{[2]} \odot g'^{[1]}(z^{[1]})$

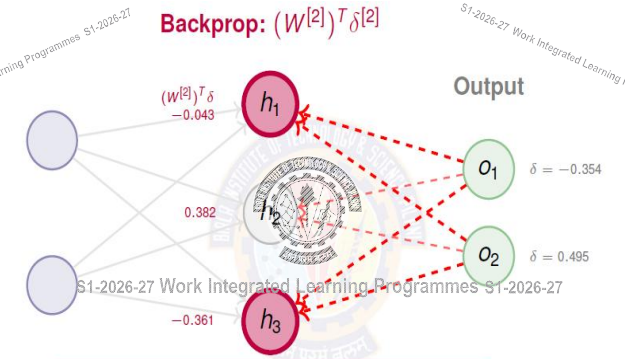
$$(W^{[2]})^T\delta^{[2]} = \begin{bmatrix}0.4 & 0.2\\-0.1 & 0.7\\0.6 & -0.3\end{bmatrix}\begin{bmatrix}-0.354\\0.495\end{bmatrix}$$

$$= \begin{bmatrix}0.4(-0.354)+0.2(0.495)\\-0.1(-0.354)+0.7(0.495)\\0.6(-0.354)+(-0.3)(0.495)\end{bmatrix}$$

$$= \begin{bmatrix}-0.142+0.099\\0.035+0.347\\-0.212-0.149\end{bmatrix} = \begin{bmatrix}-0.043\\0.382\\-0.361\end{bmatrix}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Back-Prop: Step 3 - Backprop to Hidden La



$$(W^{[2]})^T\delta^{[2]} = \begin{bmatrix}0.4 & 0.2\\-0.1 & 0.7\\0.6 & -0.3\end{bmatrix}\begin{bmatrix}-0.354\\0.495\end{bmatrix} = \begin{bmatrix}-0.043\\0.382\\-0.361\end{bmatrix}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Solution: Backprop - Hidden Layer Error

Step 4 (continued):

$$g'^{[1]}(z^{[1]}) = \begin{bmatrix} 1 & (\text{since } z_1^{[1]} = 0.7 > 0) \\ 0 & (\text{since } z_2^{[1]} = -0.1 \leq 0) \\ 1 & (\text{since } z_3^{[1]} = 0.2 > 0) \end{bmatrix}$$

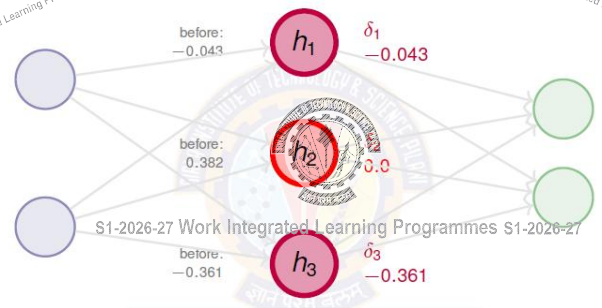
Error to Layer 1: $\delta^{[1]} = (W^{[2]})^T \delta^{[2]} \odot g'^{[1]}(z^{[1]})$

$$\delta^{[1]} = \begin{bmatrix} -0.043 \\ 0.382 \\ -0.361 \end{bmatrix} \odot \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -0.043 \\ 0.000 \\ -0.361 \end{bmatrix}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Back-Prop: Step 4 - Apply ReLU Derivative

Apply ReLU': $\delta^{[1]} = (W^{[2]})^T \delta^{[2]} \odot g'(z^{[1]})$



$$\delta^{[1]} = \begin{bmatrix} -0.043 \\ 0.382 \\ -0.361 \end{bmatrix} \odot \begin{bmatrix} 1 \\ 0 \\ 1 \end{bmatrix} = \begin{bmatrix} -0.043 \\ 0.0 \\ -0.361 \end{bmatrix}$$

Dead neuron gradient = 0 (no update!)

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Solution: Backprop - Hidden Layer Gradie

Step 5: Calculate gradients for hidden layer

$$\frac{\partial L}{\partial W^{[1]}} = \delta^{[1]} x^T = \begin{bmatrix} -0.043 \\ 0.000 \\ -0.361 \end{bmatrix} \begin{bmatrix} 1.0 & 0.5 \end{bmatrix}$$

$$\frac{\partial L}{\partial W^{[1]}} = \begin{bmatrix} -0.043 \times 1.0 & -0.043 \times 0.5 \\ 0.000 \times 1.0 & 0.000 \times 0.5 \\ -0.361 \times 1.0 & -0.361 \times 0.5 \end{bmatrix}$$

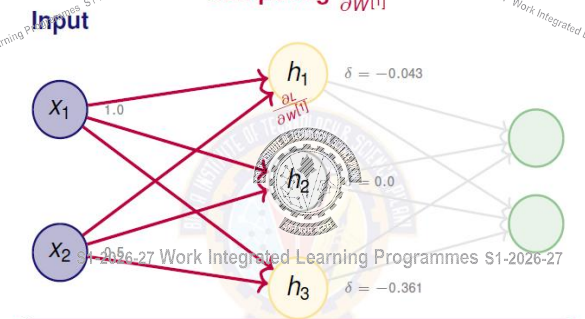
$$= \begin{bmatrix} -0.043 & -0.022 \\ 0.000 & 0.000 \\ -0.361 & -0.181 \end{bmatrix}$$

$$\frac{\partial L}{\partial b^{[1]}} = \delta^{[1]} = \begin{bmatrix} -0.043 \\ 0.000 \\ -0.361 \end{bmatrix}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Back-Prop: Step 5 - Hidden Layer Gradient

Computing $\frac{\partial L}{\partial W^{[1]}}$

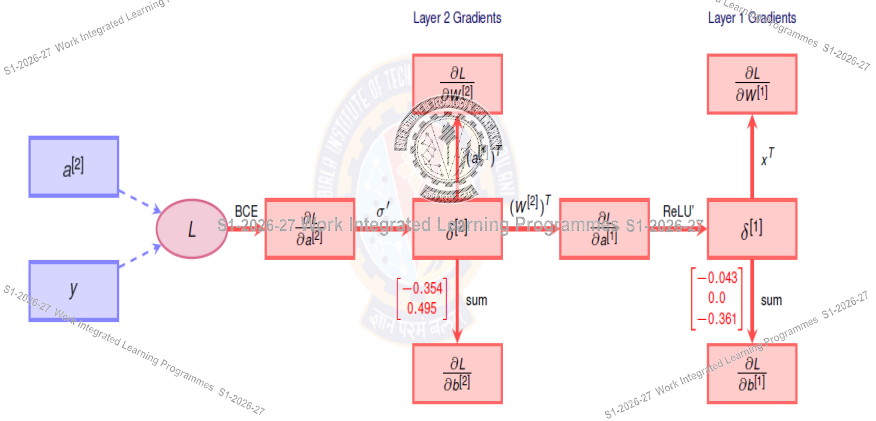


$$\frac{\partial L}{\partial W^{[1]}} = \delta^{[1]} x^T = \begin{bmatrix} -0.043 \\ 0.0 \\ -0.361 \end{bmatrix} \begin{bmatrix} 1.0 & 0.5 \end{bmatrix} = \begin{bmatrix} -0.043 & -0.022 \\ 0.0 & 0.0 \\ -0.361 & -0.181 \end{bmatrix}$$

$$\frac{\partial L}{\partial b^{[1]}} = \delta^{[1]} = \begin{bmatrix} -0.043 \\ 0.0 \end{bmatrix}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

COMPUTATIONAL GRAPH: BACKWARD PROPAGATI



NUMERICAL PROBLEM: WEIGHT UPDATION

Problem: Using the gradients computed in previous problem, update the network parameters with learning rate $\alpha = 0.1$.

Given gradients:

$$\frac{\partial L}{\partial W^{[2]}} = \begin{bmatrix} -0.248 & 0.000 & -0.071 \\ 0.347 & 0.000 & 0.099 \end{bmatrix} \quad \frac{\partial L}{\partial b^{[2]}} = \begin{bmatrix} -0.354 \\ 0.495 \end{bmatrix}$$

$$\frac{\partial L}{\partial W^{[1]}} = \begin{bmatrix} -0.043 & -0.022 \\ 0.000 & 0.000 \\ -0.361 & -0.181 \end{bmatrix} \quad \frac{\partial L}{\partial b^{[1]}} = \begin{bmatrix} -0.043 \\ 0.000 \\ -0.361 \end{bmatrix}$$

Calculate the updated parameters.

SOLUTION: PARAMETER UPDATES

Update rule: $\theta_{new} = \theta_{old} - \alpha \frac{\partial L}{\partial \theta}$

$$W^{[2]}_{new} = W^{[2]} - 0.1 \times \frac{\partial L}{\partial W^{[2]}}$$

$$W^{[2]}_{new} = \begin{bmatrix} 0.4 & -0.1 & 0.6 \\ 0.2 & 0.7 & -0.3 \end{bmatrix} - 0.1 \times \begin{bmatrix} -0.248 & 0.000 & -0.071 \\ 0.347 & 0.000 & 0.099 \end{bmatrix}$$

$$= \begin{bmatrix} 0.4 + 0.025 & -0.1 + 0.000 & 0.6 + 0.007 \\ 0.2 - 0.035 & 0.7 + 0.000 & -0.3 - 0.010 \end{bmatrix}$$

$$= \begin{bmatrix} 0.425 & -0.100 & 0.607 \\ 0.165 & 0.700 & -0.310 \end{bmatrix}$$

$$b^{[2]}_{new} = \begin{bmatrix} 0.2 \\ -0.1 \end{bmatrix} - 0.1 \times \begin{bmatrix} -0.354 \\ 0.495 \end{bmatrix} = \begin{bmatrix} 0.235 \\ -0.150 \end{bmatrix}$$

SOLUTION: PARAMETER UPDATES (CONTINUED)

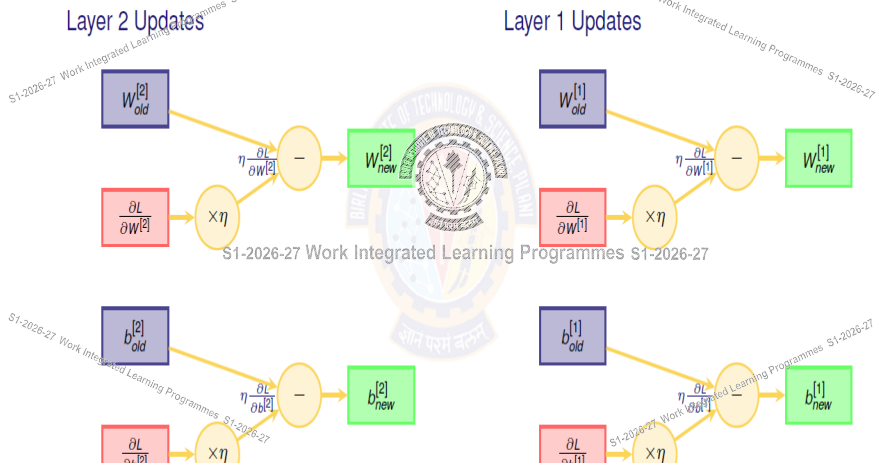
$$W^{[1]}_{new} = W^{[1]} - 0.1 \times \frac{\partial L}{\partial W^{[1]}}$$

$$W^{[1]}_{new} = \begin{bmatrix} 0.5 & 0.2 \\ -0.3 & 0.8 \\ 0.1 & -0.4 \end{bmatrix} - 0.1 \times \begin{bmatrix} -0.043 & -0.022 \\ 0.000 & 0.000 \\ -0.361 & -0.181 \end{bmatrix}$$

$$= \begin{bmatrix} 0.5 + 0.004 & 0.2 + 0.002 \\ -0.3 + 0.000 & 0.8 + 0.000 \\ 0.1 + 0.036 & -0.4 + 0.018 \end{bmatrix} = \begin{bmatrix} 0.504 & 0.202 \\ -0.300 & 0.800 \\ 0.136 & -0.382 \end{bmatrix}$$

$$b^{[1]}_{new} = \begin{bmatrix} 0.1 \\ -0.2 \\ 0.3 \end{bmatrix} - 0.1 \times \begin{bmatrix} -0.043 \\ 0.000 \\ -0.361 \end{bmatrix} = \begin{bmatrix} 0.104 \\ -0.200 \\ 0.336 \end{bmatrix}$$

Computational Graph: Weight Update Step



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Activation Functions

Sigmoid Function: Definition and Properties

The sigmoid function maps any real number to the interval $(0, 1)$.

Definition:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{1}$$

Properties:

- Output range: $(0, 1)$
- Smooth and differentiable everywhere
- $\sigma(0) = 0.5$
- $\sigma(-z) = 1 - \sigma(z)$ (symmetry)
- As $z \to \infty$: $\sigma(z) \to 1$

Use cases:

- Binary classification (output layer)
- Gate mechanisms in LSTMs

Key values:

- $\sigma(-2) \approx 0.12$
- $\sigma(0) = 0.5$
- $\sigma(2) \approx 0.88$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Sigmoid Function: Derivative Derivation

Goal: Find $\frac{d\sigma}{dz}$ where $\sigma(z) = \frac{1}{1+e^{-z}}$

Step 1: Rewrite and apply chain rule

$$\sigma(z) = (1 + e^{-z})^{-1} \quad \Rightarrow \quad \frac{d\sigma}{dz} = -(1 + e^{-z})^{-2} \cdot (-e^{-z}) = \frac{e^{-z}}{(1 + e^{-z})^2} \tag{2}$$

Step 2: Factor to express in terms of $\sigma(z)$

$$\frac{d\sigma}{dz} \quad \frac{e^{-z}}{(1 + e^{-z})^2} \quad \frac{1}{1 + e^{-z}} \quad \frac{e^{-z}}{1 + e^{-z}} \tag{3}$$

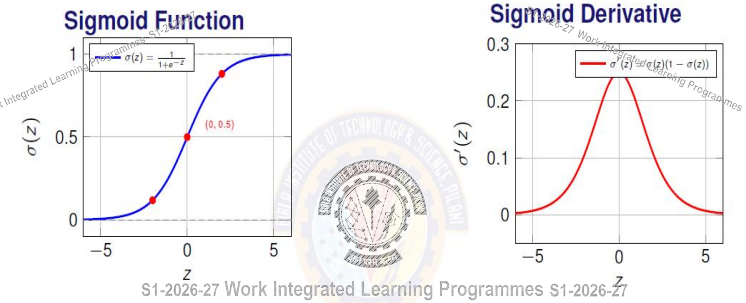
Step 3: Recognize that $\sigma(z) = \frac{1}{1+e^{-z}}$ and $1 - \sigma(z) = \frac{e^{-z}}{1+e^{-z}}$

$$\frac{d\sigma}{dz} = \sigma(z) \cdot (1 - \sigma(z)) \tag{4}$$

Beautiful property: Derivative computed using only the function value itself!

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Sigmoid Function: Visualization



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Key observations:

- Sigmoid squashes inputs to $(0, 1)$ - ideal for probabilities
- Derivative peaks at $z = 0$ with value 0.25
- Derivative vanishes for large $|z|$ (vanishing gradient problem!)

Softmax Function: Definition

The softmax function converts a vector of real numbers into a probability distribution.

For input vector $\mathbf{z} = [z_1, z_2, \ldots, z_C]^T$:

$$\text{softmax}(\mathbf{z})_j = \frac{e^{z_j}}{\sum_{k=1}^{C} e^{z_k}} \quad \text{for } j = 1, 2, \ldots, C \tag{5}$$

Properties:

- All outputs in $(0, 1)$: $0 < \text{softmax}(\mathbf{z})_j < 1$
- Outputs sum to 1: $\sum_{j=1}^{C} \text{softmax}(\mathbf{z})_j = 1$
- Monotonic: larger $z_j \rightarrow$ larger output
- Translation invariant: $\text{softmax}(\mathbf{z} + c) = \text{softmax}(\mathbf{z})$

Use case: Multi-class classification output layer

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Softmax Function: Notation

Let's denote the softmax outputs as:

$$\hat{y}_j = \frac{e^{z_j}}{\sum_{k=1}^{C} e^{z_k}} \quad \text{where} \quad \hat{\mathbf{y}} = [\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_C]^T \tag{6}$$

Example: $\mathbf{z} = [1.0, 2.0, 3.0]^T$

$$\hat{y}_1 = \frac{e^{1.0}}{e^{1.0} + e^{2.0} + e^{3.0}} = \frac{2.718}{2.718 + 7.389 + 20.086} \approx 0.090$$

$$\hat{y}_2 = \frac{e^{2.0}}{e^{1.0} + e^{2.0} + e^{3.0}} = \frac{7.389}{30.193} \approx 0.245$$

$$\hat{y}_3 = \frac{e^{3.0}}{e^{1.0} + e^{2.0} + e^{3.0}} = \frac{20.086}{30.193} \approx 0.665$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Softmax + Cross-Entropy: Beautiful

Cross-entropy Loss: $\ell = -\sum_c y_c \log(\hat{y}_c)$ where true class is k (7)

Chain rule: $$\frac{\partial \ell}{\partial z_j} = \sum_{i=1}^{C} \frac{\partial \ell}{\partial \hat{y}_i} \cdot \frac{\partial \hat{y}_i}{\partial z_j} \tag{8}$$

$$\frac{\partial \ell}{\partial z_j} = -\frac{1}{\hat{y}_k} \cdot \frac{\partial \hat{y}_k}{\partial z_j} \quad \text{where} \quad \frac{\partial \ell}{\partial \hat{y}_i} = -\frac{y_i}{\hat{y}_i} \quad y_k = 1 \tag{9}$$

$$= -\frac{1}{\hat{y}_k}[\hat{y}_k(\delta_{kj} - \hat{y}_j)] \tag{10}$$

$$= -(\delta_{kj} - \hat{y}_j) \tag{11}$$

$$= \hat{y}_j - \delta_{kj} = \hat{y}_j - y_j \tag{12}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

tanh Function: Definition and Properties

The hyperbolic tangent (tanh) maps real numbers to the interval $(-1, 1)$.

Definition:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \tag{13}$$

Properties:

- Output range: $(-1, 1)$
- Zero-centered: $\tanh(0) = 0$
- Odd function: $\tanh(-z) = -\tanh(z)$
- As $z \rightarrow \pm\infty$: $\tanh(z) \rightarrow \pm 1$

Advantages over sigmoid:

- Zero-centered
- Stronger gradients (max = 1)
- Faster convergence

Key values:

- $\tanh(-1) \approx -0.76$
- $\tanh(0) = 0$
- $\tanh(1) \approx 0.76$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Tanh Function: Derivative Derivation

Quotient Rule Method (where $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$)

Let $u = e^z - e^{-z}$, $v = e^z + e^{-z}$, so $u' = e^z + e^{-z}$, $v' = e^z - e^{-z}$

$$\frac{d\tanh}{dz} = \frac{u'v - uv'}{v^2} = \frac{(e^z + e^{-z})^2 - (e^z - e^{-z})^2}{(e^z + e^{-z})^2} = \frac{4}{(e^z + e^{-z})^2} \quad (14)$$

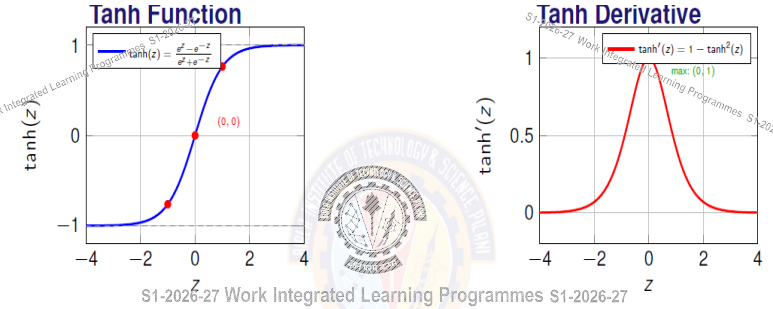
Note: $(e^z + e^{-z})^2 - (e^z - e^{-z})^2 = (e^{2z} + 2 + e^{-2z}) - (e^{2z} - 2 + e^{-2z}) = 4$

Tanh Derivative

$$\boxed{\tanh'(z) = 1 - \tanh^2(z) = \frac{4}{(e^z + e^{-z})^2}} \quad (15)$$

Beautiful property: Derivative computed using only the function value! Maximum at

Tanh Function: Visualization



Key observations:

- Tanh is zero-centered (unlike sigmoid) - better for hidden layers
- Derivative peaks at $z = 0$ with value 1 (4× larger than sigmoid!)
- Derivative vanishes for large $|z|$ but slower than sigmoid

Numerical Problem: Activation Functions

Problem: Given the input vector $z = \begin{bmatrix} 2.0 \\ -1.5 \\ 0.5 \\ -0.3 \end{bmatrix}$, calculate the output for each activation function. Also compute the derivatives for each function at the given input.

1. Sigmoid: $\sigma(z)$
2. Tanh: $\tanh(z)$
3. ReLU: $\mathrm{ReLU}(z)$

Solution: Sigmoid Activation

Sigmoid Function: $\sigma(z) = \frac{1}{1 + e^{-z}}$

$$\sigma(z) = \begin{bmatrix} \frac{1}{1+e^{-2.0}} \\ \frac{1}{1+e^{1.5}} \\ \frac{1}{1+e^{-0.5}} \\ \frac{1}{1+e^{0.3}} \end{bmatrix} = \begin{bmatrix} \frac{1}{1+0.135} = 0.881 \\ \frac{1}{1+4.482} = 0.182 \\ \frac{1}{1+0.449} = 0.690 \\ \frac{1}{1.350} = 0.426 \end{bmatrix} = \begin{bmatrix} 0.881 \\ 0.182 \\ 0.690 \\ 0.426 \end{bmatrix}$$

Sigmoid Derivative: $\sigma'(z) = \begin{bmatrix} 0.881(1 - 0.881) \\ 0.182(1 - 0.182) \\ 0.690(1 - 0.690) \\ 0.426(1 - 0.426) \end{bmatrix} = \begin{bmatrix} 0.105 \\ 0.149 \\ 0.214 \\ 0.245 \end{bmatrix}$

Solution: Tanh Activation

Tanh Function: $\tanh(z) = \dfrac{e^z - e^{-z}}{e^z + e^{-z}}$

$$\tanh(z) = \begin{bmatrix} \tanh(2.0) \\ \tanh(-1.5) \\ \tanh(0.8) \\ \tanh(-0.3) \end{bmatrix} = \begin{bmatrix} \frac{7.389-0.135}{7.389+0.135} = 0.964 \\ \frac{0.223-4.482}{0.223+4.482} = -0.905 \\ \frac{2.226-0.449}{2.226+0.449} = 0.664 \\ \frac{0.741-1.350}{0.741+1.350} = -0.291 \end{bmatrix} = \begin{bmatrix} 0.964 \\ -0.905 \\ 0.664 \\ -0.291 \end{bmatrix}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Tanh Derivative: $\tanh'(z) = \begin{bmatrix} 1 - 0.964^2 \\ 1 - (-0.905)^2 \\ 1 - 0.664^2 \\ 1 - (-0.291)^2 \end{bmatrix} = \begin{bmatrix} 0.070 \\ 0.181 \\ 0.559 \\ 0.915 \end{bmatrix}$

Solution: ReLU

ReLU Function: $\text{ReLU}(z) = \max(0, z)$

$$= \begin{bmatrix} \max(0, 2.0) \\ \max(0, -1.5) \\ \max(0, 0.8) \\ \max(0, -0.3) \end{bmatrix} = \begin{bmatrix} 2.0 \\ 0.0 \\ 0.8 \\ 0.0 \end{bmatrix}$$

ReLU Derivative: $\text{ReLU}'(z) = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z \leq 0 \end{cases}$

$$= \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Numerical Problem: Softmax Function

Problem: Given the logit vector z, compute:

1. The softmax output
2. The cross-entropy loss with true label y
3. The gradient of the loss with respect to the logits

$$z = \begin{bmatrix} 2.0 \\ 1.0 \\ 0.1 \\ -1.0 \end{bmatrix} \qquad y = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \end{bmatrix}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Cross-entropy Loss:

$$L = -\sum_{i=1}^{n} y_i \log(\text{softmax}(z_i))$$

Solution: Softmax Computation

Step 1: Compute softmax values First, for numerical stability, subtract the maximum:

$$z_{\text{stable}} = z - \max(z) = \begin{bmatrix} 2.0 \\ 1.0 \\ 0.1 \\ -1.0 \end{bmatrix} - 2.0 = \begin{bmatrix} 0.0 \\ -1.0 \\ -1.9 \\ -3.0 \end{bmatrix}$$

$$e^{z_{\text{stable}}} = \begin{bmatrix} e^{0.0} \\ e^{-1.0} \\ e^{-1.9} \\ e^{-3.0} \end{bmatrix} = \begin{bmatrix} 1.000 \\ 0.368 \\ 0.150 \\ 0.050 \end{bmatrix}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

$$\sum_{j=1}^{4} e^{z_j - \max(z)} = 1.000 + 0.368 + 0.150 + 0.050 = 1.568$$

Solution: Softmax Computation

$$\text{softmax}(z) = \frac{1}{1.568}\begin{bmatrix}1.000\\0.368\\0.150\\0.050\end{bmatrix} = \begin{bmatrix}0.638\\0.235\\0.096\\0.032\end{bmatrix}$$

Verification: $0.638 + 0.235 + 0.096 + 0.032 = 1.001 \approx 1$

Solution: Cross-Entropy Loss and Gradient

Step 2: Compute cross-entropy loss

$$L = -\sum_{i=1}^{4} y_i \log(\text{softmax}(z_i))$$

$$y = \begin{bmatrix}0\\1\\0\\0\end{bmatrix}$$ only the second term contributes

$$L = -y_2 \log(\text{softmax}(z_2)) = -1 \times \log(0.235) = -\log(0.235) = 1.447$$

Solution: Cross-Entropy Loss and Gradient

Step 3: Compute gradient For cross-entropy loss with softmax, the gradient is:

$$\frac{\partial L}{\partial z_i} = \text{softmax}(z_i) - y_i$$

$$\frac{\partial L}{\partial z} = \begin{bmatrix}0.638\\0.235\\0.096\\0.032\end{bmatrix} - \begin{bmatrix}0\\1\\0\\0\end{bmatrix} = \begin{bmatrix}0.638\\-0.765\\0.096\\0.032\end{bmatrix}$$

This result shows that the gradient is simply the difference between predicted and true probabilities!

Numerical Problem: Gradient Flow Analysis

Problem: Consider a 4-layer network with sigmoid activations. Analyze gradient flow through the network.

Given:

- All weight matrices have spectral norm $\|W^{[l]}\| = 0.8$
- Sigmoid derivatives at each layer: $g'^{[l]} = 0.25$ (maximum sigmoid derivative)
- Initial gradient magnitude at output: $|\delta^{[4]}| = 1.0$

Calculate the gradient magnitude at each layer and discuss the implications.

Gradient Propagation Formula:

$$|\delta^{[l-1]}| = |\delta^{[l]}| \cdot \|W^{[l]}\| \cdot |g'^{[l-1]}|$$

Solution: Gradient Flow Analysis

Layer-by-layer gradient magnitude:

Layer 4 (Output): $|\delta^{[4]}| = 1.0$ (given)

Layer 3: $|\delta^{[3]}| = |\delta^{[4]}| \cdot \|W^{[4]}\| \cdot |g'^{[3]}| = 1.0 \times 0.8 \times 0.25 = 0.2$

Layer 2: $|\delta^{[2]}| = |\delta^{[3]}| \cdot \|W^{[3]}\| \cdot |g'^{[2]}| = 0.2 \times 0.8 \times 0.25 = 0.04$

Layer 1: $|\delta^{[1]}| = |\delta^{[2]}| \cdot \|W^{[2]}\| \cdot |g'^{[1]}| = 0.04 \times 0.8 \times 0.25 = 0.008$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Analysis:

- Gradient magnitude decreases by factor of $0.8 \times 0.25 = 0.2$ per layer
- After 3 backward steps: gradient is reduced to 0.8% of original
- First layer receives extremely weak learning signal
- This demonstrates the vanishing gradient problem in deep sigmoid networks

Example 1: Small Tabular Dataset

Problem Setup: Compute the number of layers and number of neurons in each layer

- $d = 20$ features (customer data)
- $M = 800$ training samples
- $c = 3$ classes (customer segments)

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Example 1: Small Tabular Dataset

- Step 1: Apply constraints
  $L \leq \log_2(800/(20 \times 3)) + 1 = \log_2(13.33) + 1 \approx 4.7 \Rightarrow L \leq 4$
- Step 2: Choose starting architecture Since $M < 1000$, start with $L = 2$: $[20, 40, 3]$
- Step 3: Parameter count check Parameters $= 20 \times 40 + 40 + 40 \times 3 + 3 = 800 + 40 + 120 + 3 = 963$
  Constraint: $963 \leq 800/5 = 160$ ✗ Too many parameters!
- Step 4: Reduce complexity
  - Try $[20, 20, 3]$: Parameters = $20 \times 20 + 20 + 20 \times 3 + 3 = 483$ ✗ Still too many!
  - Try $[20, 10, 3]$: Parameters = $20 \times 10 + 10 + 10 \times 3 + 3 = 243$ ✗
  - Try $[20, 6, 3]$: Parameters = $20 \times 6 + 6 + 6 \times 3 + 3 = 147$ ✓
- Final recommendation: $[20, 6, 3]$ with regularization

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Example 2: Medium Image Dataset

Problem Setup: Compute the number of layers and number of neurons in each layer

- $d = 784$ features (28×28 grayscale images)
- $M = 5000$ training samples
- $c = 10$ classes (digit classification)

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Example 2: Medium Image Dataset

- **Step 1: Relaxed approach**
  For medium datasets, start with $L = 3$: $[784, 2352, 784, 10]$
- **Step 2: Parameter count check**
  Parameters $= 784 \times 2352 + 2352 + 2352 \times 784 + 784 + 784 \times 10 + 10 \approx 3.7M$
  Constraint $5000/5 = 1000$ ✗ Way too many!
- **Step 3: Practical architecture**
  - Try $[784, 128, 64, 10]$:
    Parameters $= 784 \times 128 + 128 + 128 \times 64 + 64 + 64 \times 10 + 10 = 109,000$ ✗
  - Try $[784, 64, 32, 10]$:
    Parameters $= 784 \times 64 + 64 + 64 \times 32 + 32 + 32 \times 10 + 10 = 52,458$ ✗
  - Try $[784, 32, 10]$: Parameters $= 784 \times 32 + 32 + 32 \times 10 + 10 = 25,450$ ✗
  - Try $[784, 16, 10]$: Parameters $= 784 \times 16 + 16 + 16 \times 10 + 10 = 12,714$ ✗

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Example 3: Large Text Dataset

Problem Setup: Compute the number of layers and number of neurons in each layer

- $d = 1000$ features (TF-IDF vectors)
- $M = 50000$ training samples
- $c = 5$ classes (sentiment categories)

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Example 3: Large Text Dataset

- **Step 1: Apply constraints**
  $L \leq \log_2(50000/(1000 \times 5)) + 1 = \log_2(10) + 1 \approx 4.3 \Rightarrow L \leq 4$
- **Step 2: Choose architecture**
  Since $M \geq 10000$, can use $L = 4$: $[1000, 5000, 2000, 1000, 5]$
- **Step 3: Parameter count check** Parameters
  $1000 \times 5000 + 5000 + 5000 \times 2000 + 2000 + 2000 \times 1000 + 1000 + 1000 \times 5 + 5 =$
  $5M + 10M + 2M + 5K = 17M$
  Constraint: $17M \leq 50000/5 = 10000$ ✗ Way too many!
- **Step 4: Reasonable architecture**
  Try $[1000, 512, 256, 128, 5]$:
  Parameters $= 1000 \times 512 + 512 + 512 \times 256 + 256 + 256 \times 128 + 128 + 128 \times 5 + 5$
  $= 512K + 131K + 32K + 640 = 675K$ parameters ✓

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Overview: Loss Functions

Loss functions measure the discrepancy between predicted and actual values.

We will include:

1. Mean Squared Error (MSE) Loss
2. Mean Absolute Error (MAE) Loss
3. Binary Cross-Entropy (BCE) Loss
4. Cross-Entropy Loss

Key Goal: Understand how to compute gradients for backpropagation

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

MSE Loss: Definition

Mean Squared Error (MSE) is commonly used for regression tasks.

$$\ell(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2 \tag{16}$$

Properties:

- Always non-negative: $\ell \geq 0$
- Convex function (for linear models)
- Differentiable everywhere
- Penalizes large errors quadratically
- Factor of $\frac{1}{2}$ simplifies derivative

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

MSE Loss: Derivative Derivation

Goal: Find $\frac{\partial \ell}{\partial \hat{y}}$ where $\ell = \frac{1}{2}(\hat{y} - y)^2$

Let $u = \hat{y} - y$, so $\ell = \frac{1}{2}u^2$

$$\frac{\partial \ell}{\partial \hat{y}} = \frac{\partial \ell}{\partial u} \cdot \frac{\partial u}{\partial \hat{y}} \quad \text{(chain rule)} \tag{17}$$

$$= \left(\frac{1}{2} \cdot 2u\right) \cdot (1) \quad \text{(compute derivatives)} \tag{18}$$

$$= u = \hat{y} - y \quad \text{(substitute back)} \tag{19}$$

MSE Loss Derivative:

$$\boxed{\frac{\partial \ell}{\partial \hat{y}} = \hat{y} - y} \tag{20}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

MSE Loss: Numerical Example

Example 1: $y = 3$, $\hat{y} = 5$

$$\ell = \frac{1}{2}(5-3)^2 = \frac{1}{2}(2)^2 = 2$$

$$\frac{\partial \ell}{\partial \hat{y}} = 5 - 3 = 2$$

Example 2: $y = 8$, $\hat{y} = 5$

$$\ell = \frac{1}{2}(5-8)^2 = \frac{1}{2}(-3)^2 = 4.5$$

$$\frac{\partial \ell}{\partial \hat{y}} = 5 - 8 = -3$$

Note: Gradient is negative, telling us to *increase* $\hat{y}$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

MAE Loss: Definition

Mean Absolute Error (MAE) is more robust to outliers than MSE.

$$\ell(\hat{y}, y) = |\hat{y} - y| \tag{21}$$

Properties:

- Always non-negative: $\ell \geq 0$
- Less sensitive to outliers than MSE
- Not differentiable at $\hat{y} = y$
- Linear penalty for errors

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

MAE Loss: Derivative Derivation

Goal: Find $\frac{\partial \ell}{\partial \hat{y}}$ where $\ell = |\hat{y} - y|$

Recall: Derivative of absolute value function

$$\frac{d|x|}{dx} = \begin{cases} 1 & \text{if } x > 0 \\ -1 & \text{if } x < 0 \\ \text{undefined} & \text{if } x = 0 \end{cases} \tag{22}$$

Apply to $\ell = |\hat{y} - y|$ using chain rule:

$$\frac{\partial \ell}{\partial \hat{y}} = \frac{\partial}{\partial \hat{y}}|\hat{y} - y| = \begin{cases} 1 & \text{if } \hat{y} > y \\ -1 & \text{if } \hat{y} < y \\ 0 & \text{if } \hat{y} = y \text{ (convention)} \end{cases} = \text{sign}(\hat{y} - y) \tag{23}$$

MAE Loss Derivative:

$$\boxed{\frac{\partial \ell}{\partial \hat{y}} = \text{sign}(\hat{y} - y)} \tag{24}$$

MAE Loss: Numerical Examples

Example 1: $y = 3$, $\hat{y} = 5$

$$\ell = |5 - 3| = 2$$

$$\frac{\partial \ell}{\partial \hat{y}} = \text{sign}(5 - 3) = \text{sign}(2) = +1$$

Example 2: $y = 8$, $\hat{y} = 5$

$$\ell = |5 - 8| = 3$$

$$\frac{\partial \ell}{\partial \hat{y}} = \text{sign}(5 - 8) = \text{sign}(-3) = -1$$

Comparison with MSE:

- MAE gradient is constant: ± 1
- MSE gradient varies with error magnitude: $\hat{y} - y$

BCE Loss: Definition

Binary Cross-Entropy (BCE) is used for binary classification where $y \in \{0, 1\}$ and $\hat{y} \in (0, 1)$.

For a single example:

$$\ell(\hat{y}, y) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})] \tag{25}$$

Properties:

- Measures divergence between predicted and true probability distributions
- Always non-negative: $\ell \geq 0$
- Heavily penalizes confident wrong predictions

BCE Loss: Understanding the Formula

Why this form?

Since $y \in \{0, 1\}$, the loss simplifies based on the true label:

Case 1: When $y = 1$

$$\ell = -[1 \cdot \log(\hat{y}) + (1 - 1) \log(1 - \hat{y})] \tag{26}$$

$$= -\log(\hat{y}) \tag{27}$$

Case 2: When $y = 0$

$$\ell = -[0 \cdot \log(\hat{y}) + (1 - 0) \log(1 - \hat{y})] \tag{28}$$

$$= -\log(1 - \hat{y}) \tag{29}$$

Intuition:

- If $y = 1$, want $\hat{y} \to 1$ to minimize $-\log(\hat{y})$
- If $y = 0$, want $\hat{y} \to 0$ to minimize $-\log(1 - \hat{y})$

Goal: Find $\frac{\partial \ell}{\partial \hat{y}}$ where $\ell = -[y \log(\hat{y}) + (1-y)\log(1-\hat{y})]$

Differentiate both terms (using $\frac{d}{dx}\log(x) = \frac{1}{x}$ and chain rule):

$$\frac{\partial \ell}{\partial \hat{y}} = -\frac{\partial}{\partial \hat{y}}[y \log(\hat{y})] - \frac{\partial}{\partial \hat{y}}[(1-y)\log(1-\hat{y})] \tag{30}$$

$$= -\frac{y}{\hat{y}} - (1-y) \cdot \frac{1}{1-\hat{y}} \cdot (-1) = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}} \tag{31}$$

Simplify with common denominator:

$$\frac{\partial \ell}{\partial \hat{y}} = \frac{-y(1-\hat{y}) + (1-y)\hat{y}}{\hat{y}(1-\hat{y})} = \frac{-y + y\hat{y} + \hat{y} - y\hat{y}}{\hat{y}(1-\hat{y})} = \frac{\hat{y} - y}{\hat{y}(1-\hat{y})} \tag{32}$$

BCE Loss Derivative:

$$\boxed{\frac{\partial \ell}{\partial \hat{y}} = \frac{\hat{y} - y}{\hat{y}(1-\hat{y})}} \tag{33}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Example 1: $y = 1$, $\hat{y} = 0.9$

$$\ell = -\log(0.9) \approx 0.105$$

$$\frac{\partial \ell}{\partial \hat{y}} = \frac{0.9 - 1}{0.9(1-0.9)} = \frac{-0.1}{0.09} \approx -1.11$$

Example 2: $y = 1$, $\hat{y} = 0.1$

$$\ell = -\log(0.1) \approx 2.303$$

$$\frac{\partial \ell}{\partial \hat{y}} = \frac{0.1 - 1}{0.1(1-0.1)} = \frac{-0.9}{0.09} = -10$$

Example 3: $y = 0$, $\hat{y} = 0.9$

$$\ell = -\log(0.1) \approx 2.303$$

$$\frac{\partial \ell}{\partial \hat{y}} = \frac{0.9 - 0}{0.9(1-0.9)} = \frac{0.9}{0.09} = 10$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Cross-Entropy Loss generalizes BCE to multi-class classification where y is one-hot encoded and $\hat{\mathbf{y}}$ is a probability distribution.

For C classes and single example:

$$\ell(\hat{\mathbf{y}}, \mathbf{y}) = -\sum_{c=1}^{C} y_c \log(\hat{y}_c) \tag{34}$$

where:

- $\mathbf{y} = [y_1, y_2, \ldots, y_C]^T$ is one-hot encoded (one $y_c = 1$, rest are 0)
- $\hat{\mathbf{y}} = [\hat{y}_1, \hat{y}_2, \ldots, \hat{y}_C]^T$ are predicted probabilities
- $\sum_{c=1}^{C} \hat{y}_c = 1$ (output of softmax)

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Goal: Find $\frac{\partial \ell}{\partial \hat{y}_j}$ for each class j, where $\ell = -\sum_{c=1}^{C} y_c \log(\hat{y}_c)$

Step 1: Take derivative with respect to $\hat{y}_j$

$$\frac{\partial \ell}{\partial \hat{y}_j} = -\frac{\partial}{\partial \hat{y}_j}\left[\sum_{c=1}^{C} y_c \log(\hat{y}_c)\right] \tag{35}$$

Step 2: Only the $c = j$ term depends on $\hat{y}_j$

$$\frac{\partial \ell}{\partial \hat{y}_j} = -\frac{\partial}{\partial \hat{y}_j}[y_j \log(\hat{y}_j)] = -y_j \cdot \frac{1}{\hat{y}_j} \tag{36}$$

Cross-Entropy Loss

$$\boxed{\frac{\partial \ell}{\partial \hat{y}_j} = -\frac{y_j}{\hat{y}_j}} \tag{37}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Cross-Entropy Loss: Gradient Vector

The gradient with respect to all outputs forms a vector:

$$\nabla_{\hat{\mathbf{y}}} \ell = \begin{bmatrix} \frac{\partial \ell}{\partial \hat{y}_1} \\ \frac{\partial \ell}{\partial \hat{y}_2} \\ \vdots \\ \frac{\partial \ell}{\partial \hat{y}_C} \end{bmatrix} = \begin{bmatrix} -\frac{y_1}{\hat{y}_1} \\ -\frac{y_2}{\hat{y}_2} \\ \vdots \\ -\frac{y_C}{\hat{y}_C} \end{bmatrix} \tag{38}$$

Since $\mathbf{y}$ is one-hot (only $y_k = 1$):

- For the true class k: $\frac{\partial \ell}{\partial \hat{y}_k} = -\frac{1}{\hat{y}_k}$
- For all other classes $j \neq k$: $\frac{\partial \ell}{\partial \hat{y}_j} = 0$

Important: This is the gradient w.r.t. softmax outputs. When combined with softmax derivative, we get a simpler form!

Activation Functions: Comparison

Property	Sigmoid	Tanh	Softmax
Formula	$\frac{1}{1+e^{-z}}$	$\frac{e^z - e^{-z}}{e^z + e^{-z}}$	$\frac{e^{z_i}}{\sum_k e^{z_k}}$
Range	$(0, 1)$	$(-1, 1)$	$(0, 1)$
Zero-centered?	No	Yes	No
Derivative	$\sigma(1-\sigma)$	$1 - \tanh^2$	Complex
Max Derivative	0.25	1.0	Varies
Output Sum	N/A	N/A	1.0
Use Case	Binary class	Hidden layers	Multi-class

Key Advantages:

- **Sigmoid:** Output is probability for binary classification
- **Tanh:** Zero-centered, stronger gradients

Loss Functions: Comparison

Loss	Formula	Derivative
MSE	$\frac{1}{2}(\hat{y} - y)^2$	$\hat{y} - y$
MAE	$\lvert\hat{y} - y\rvert$	$\text{sign}(\hat{y} - y)$
BCE	$-[y \log \hat{y} + (1-y) \log(1-\hat{y})]$	$\frac{\hat{y}-y}{\hat{y}(1-\hat{y})}$
Cross-Entropy	$-\sum_c y_c \log \hat{y}_c$	$-\frac{y_i}{\hat{y}_i}$

Use Cases:

- **MSE:** Regression tasks
- **MAE:** Regression with outliers
- **BCE:** Binary classification ($y \in \{0, 1\}$)
- **Cross-Entropy:** Multi-class classification

Special case: Softmax + Cross-Entropy gradient: $\hat{y}_j - y_j$ (very clean!)

Summary: Key Takeaways

Loss Functions:

- MSE penalizes errors quadratically, derivative is linear
- MAE is robust to outliers, derivative is constant ± 1
- BCE for binary classification, heavily penalizes confident errors
- Cross-Entropy for multi-class, only cares about correct class probability

Activation Functions:

- Sigmoid squashes to $(0, 1)$, useful for probabilities
- Tanh squashes to $(-1, 1)$, zero-centered is better for hidden layers
- Softmax converts logits to probability distribution

Practical Tips:

- Regression: Use MSE or MAE loss
- Binary classification: Use sigmoid + BCE
- Multi-class: Use softmax + cross-entropy

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial a^{(2)}} \cdot \frac{\partial a^{(2)}}{\partial z^{(2)}} \cdot \frac{\partial z^{(2)}}{\partial w_1}$$

$(\hat{y}-y)$ → (-0.4152) ; w_2 → 0.8 ; $a_1(1-a_1)$ → $0.7311(1-0.7311)$; x → 2

$= -0.1306$

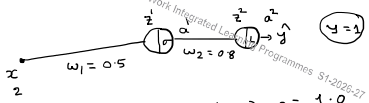
Weights update:

$w_1 = w_1 - \eta \left[\frac{\partial L}{\partial w_1}\right] = 0.5 - (0.1)(-0.1306)$

$= 0.5131$

$w_2 = w_2 - \eta \left[\frac{\partial L}{\partial w_2}\right] = 0.8 - (0.1)(-0.3035)$

$= 0.8304$



$z^{(1)} = w_1 x + b_1 = 2(0.5) + 0 = 1.0$

$a^{(1)} = \sigma(z^{(1)}) = \frac{1}{1+e^{-1.0}} = 0.7311$

$z^{(2)} = w_2 a^{(1)} + b_1 = (0.7311)(0.8)$

$= 0.5848$

$a^{(2)} = \hat{y} = 0.5848$

$L = \frac{1}{2}(\hat{y} - y)^2 = \frac{1}{2}(0.5848 - 1)^2$

$= 0.0862$

$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial w_2}$

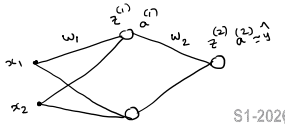
$(\hat{y}-y)$; a_1

$X = [2.0 \quad 3.0]$

$w_1 = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}$

$w_2 = \begin{bmatrix} 0.5 \\ 0.6 \end{bmatrix}$ output = 1

cross entropy



$z^{(1)} = xw_1 + b^{(1)} = [2.0 \quad 3.0]\begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}$

$= [1.1 \quad 1.6]$

$a^{(1)} = \sigma(z^{(1)}) = [\sigma(1.1), \sigma(1.6)]$

$= [0.7503 \quad 0.8320]$

$z^{(2)} = a^{(1)} w_2 + b_2 = [0.7503 \quad 0.8320]\begin{bmatrix} 0.5 \\ 0.6 \end{bmatrix}$

$a^{(2)} = \sigma(0.8743)$

$= 0.7056$

$= \hat{y}$

$\therefore$ Loss : $-[y \ln \hat{y} + (1-y)\ln(1-\hat{y})]$

$= -[1 \cdot \log(0.7056) + (1-1)\cdot\ ]$

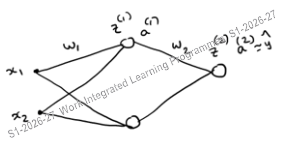
$= 0.3486$

Back propogation :-

we need to find

$\frac{\partial L}{\partial w_1}$, $\frac{\partial L}{\partial w_2}$ to

update weights



$z^{(1)} = xw_1 + b^{(1)} =$

$a^{(1)} = \sigma(z^{(1)}) = [0.7503 \quad 0.8320]$

$z^{(2)} = a^{(1)} w_2 + b_2 = [0.8743]$

$a^{(2)} = \sigma(0.8743) \because \sigma(z^{(2)})$

$= 0.7056$

$= \hat{y}$

$\therefore$ Loss : 0.3486

$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{\partial \hat{y}}{\partial z^{(2)}} \cdot \frac{\partial z^{(2)}}{\partial a^{(1)}} \cdot \frac{\partial a^{(1)}}{\partial z^{(1)}} \cdot \frac{\partial z^{(1)}}{\partial w_1}$

$(\hat{y}-y)$ → $(0.7056-1)$; w_2 → $[0.5 \quad 0.6]$; $\sigma_1(1-\sigma_1)$ → $[0.7503(1-0.7503) \quad 0.8320(1-0.8320)] = [0.1873 \quad 0.1398]$; x → $\begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}$

$\frac{\partial L}{\partial w_1} = \begin{bmatrix} -0.0552 & -0.0494 \\ -0.0827 & -0.0741 \end{bmatrix}$

$\therefore w_1^{(new)} = \begin{bmatrix} 0.1 & 0.2 \\ 0.3 & 0.4 \end{bmatrix} - (0.1)\begin{bmatrix} & \end{bmatrix}$

$= \begin{bmatrix} 0.1055 & 0.2049 \\ 0.3083 & 0.4074 \end{bmatrix}$

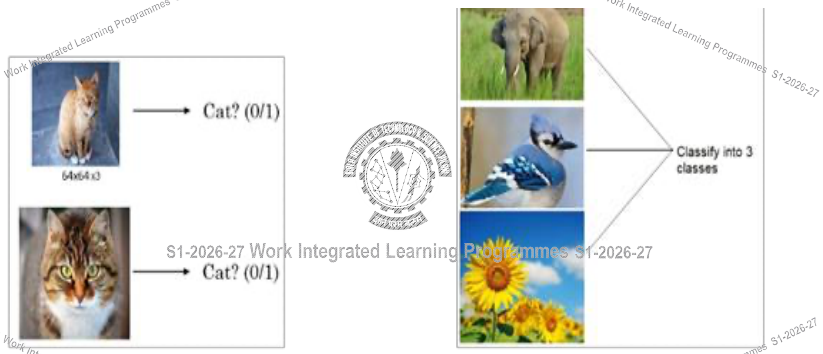
$w_2^{(new)} = \begin{bmatrix} 0.5 \end{bmatrix} - (0.1)\begin{bmatrix} -0.2208 \\ -0.2449 \end{bmatrix}$

CNN

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

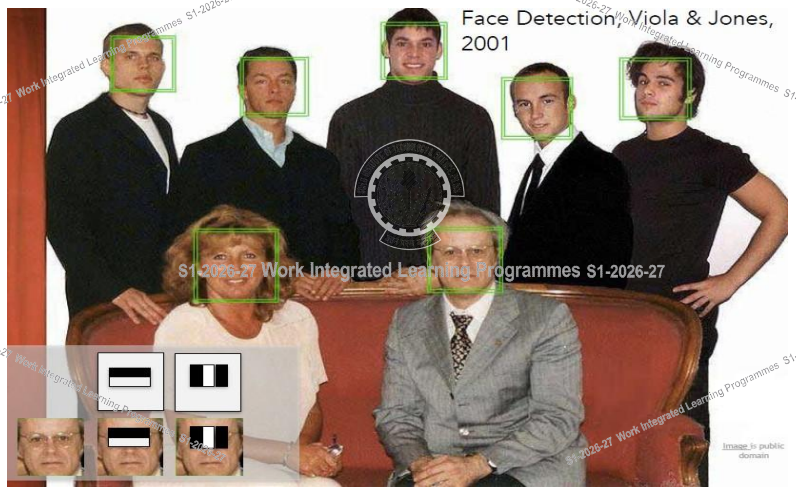
IMAGE CLASSIFICATION

329



FACE DETECTION

330



Fei-Fei Li & Justin Johnson & Serena Young

IMAGE CAPTIONING

331

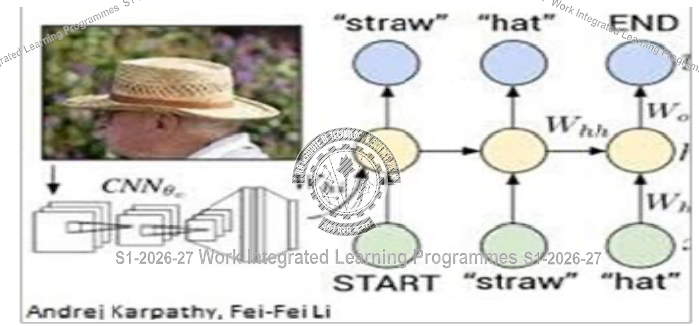
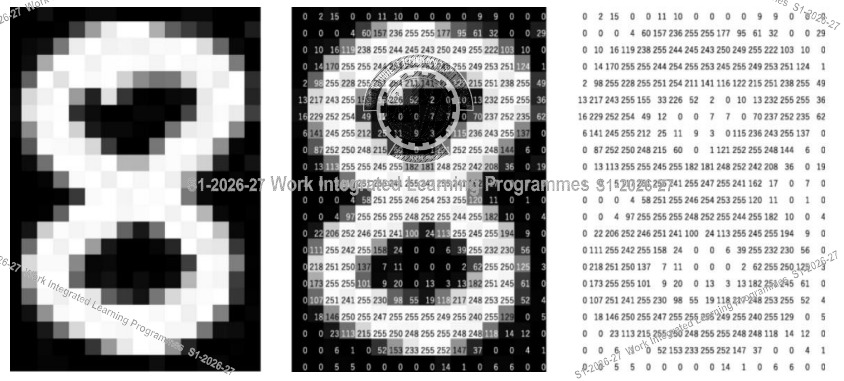


IMAGE BASICS AND IMAGE CLASSIFICATION CHALLENGES

332

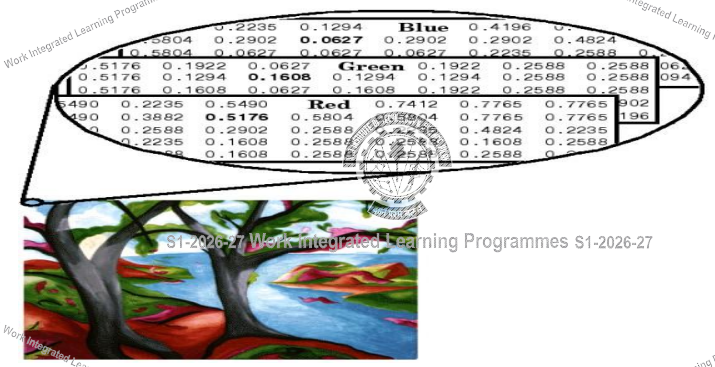
DIGITAL GRAYSCALE IMAGE

Pixel intensities = $I(x, y)$



Digital Color Image

333



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Image Classification - Challenges

334

Image Classification: A core task in Computer Vision

(assume given set of discrete labels)
{dog, cat, truck, plane, ...}

cat

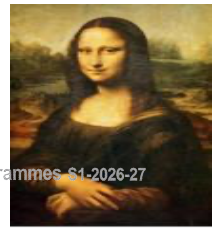
S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Image Classification - Challenges

335

Pixelated Images

textcolor Images



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

- A 50x50 Image requires 2500 input dimensions.
- A HD image 1920x1080 is about 2 million inputs. (2 Mega Pixels)

Image Invariance - Challenges

336

- Object can appear anywhere in the image.

Image A

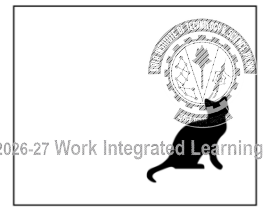
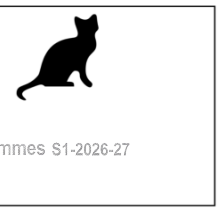


Image B



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

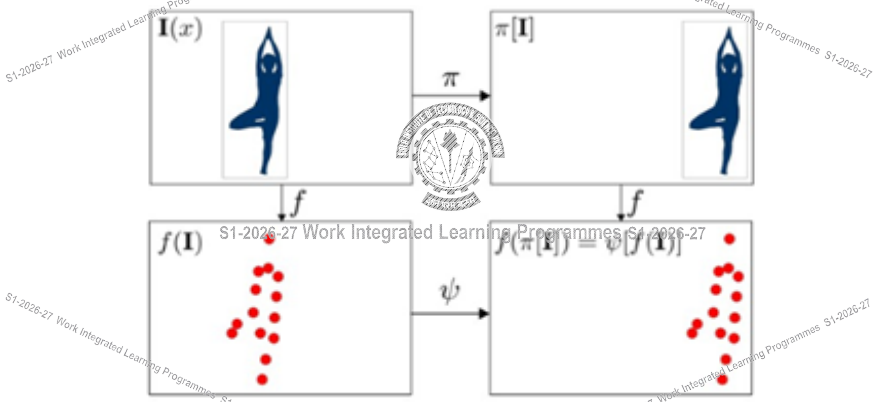
Model predicts “cat” for both images!

Source: https://penkovsky.com/neural-networks/day5/

Image Equivariance - Challenges

337

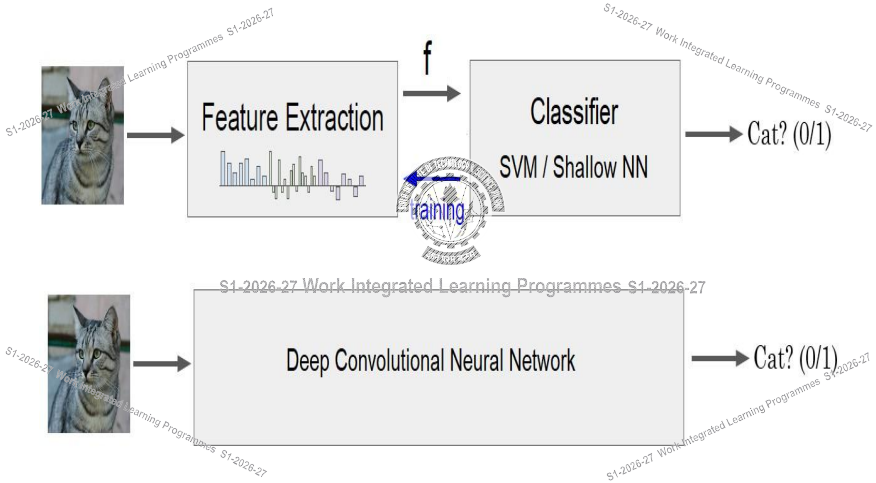
Changes to the input predictably (equivalently) should not affect the output.



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Image Classification - Solution

338



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

What is a Convolutional Neural Network?

339

Definition

A Convolutional Neural Network (CNN) is a specialized type of deep neural network designed to process data with a grid-like topology, such as images, where spatial relationships between data points are important.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Core Insight

CNNs mimic the visual cortex by detecting simple patterns (edges, corners) in early layers and combining them into complex objects (faces, cars) in deeper layers.

Why Convolutional Networks?

340

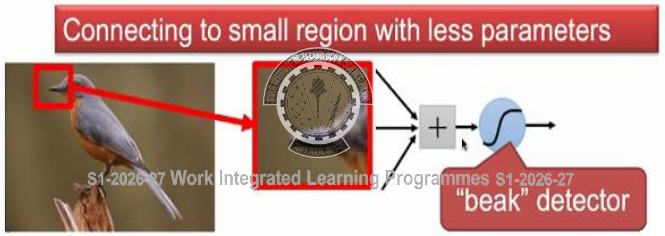
Why CNNs for Images?

- **Translation Invariance:** Object recognition regardless of position
- **Local Patterns:** Detect edges, textures, and shapes locally
- **Hierarchical Features:** Low-level → Mid-level → High-level features
- **Parameter Efficiency:** Fewer parameters than fully connected networks
- **Computational Efficiency:** Parallel processing of convolutions

Why CNN for Images - Locality

341

- Some patterns are much smaller than the whole image.



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Why CNN for Images - Locality

342

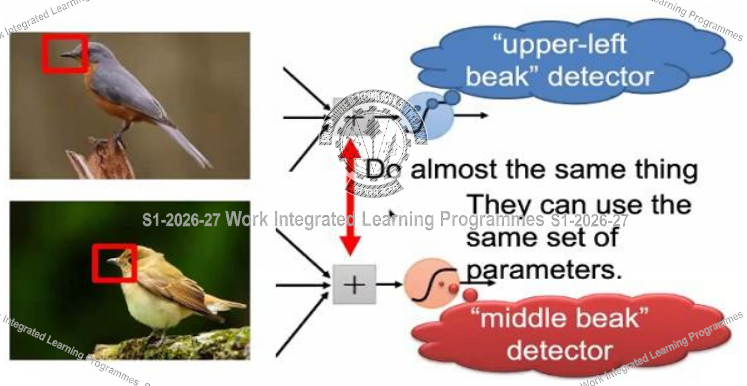
- Some patterns are much smaller than the whole image.
  - A neuron does not have to see the whole image to discover the pattern.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Why CNN for Images - Translation Invariance

343

Same pattern may appear in different regions.

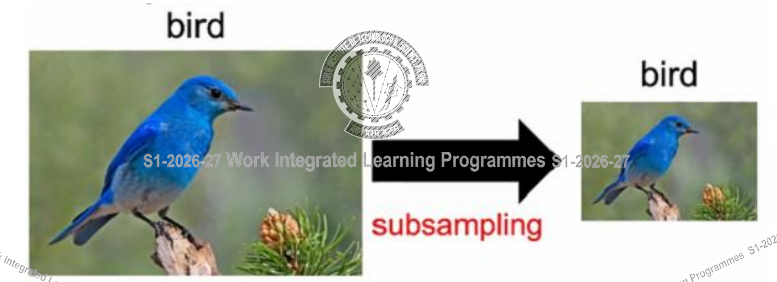


S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Why CNN for Images - Scale Invariance

344

- Sub-sampling the pixels will not change the object.

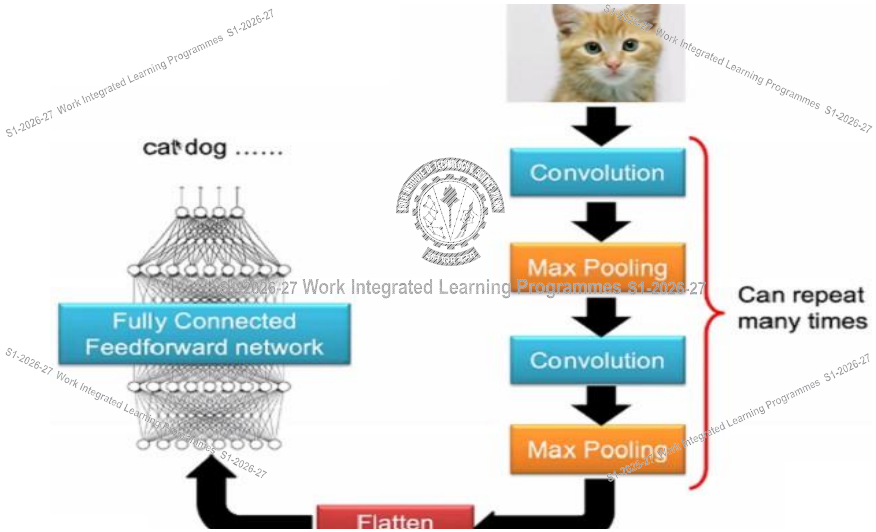


S1-2026-27 Work Integrated Learning Programmes S1-2026-27

- We can sub-sample the image to make it smaller.
- Less parameters for the network to process the image

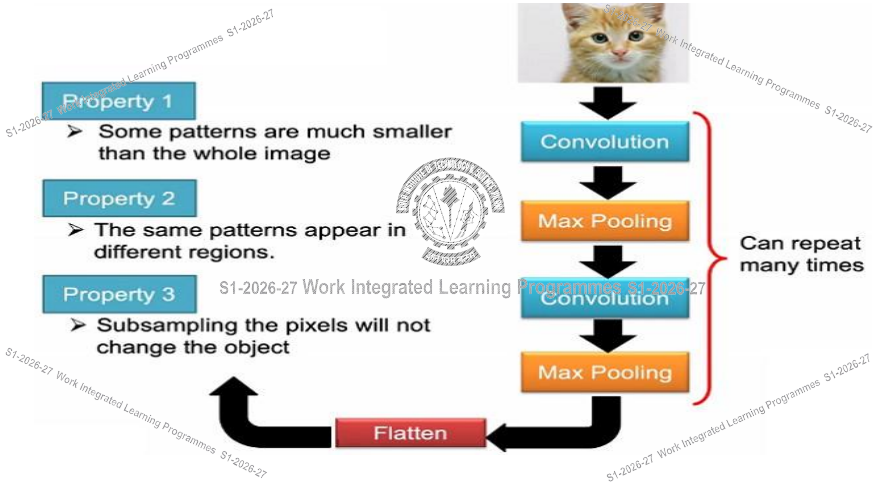
THE WHOLE CNN

345



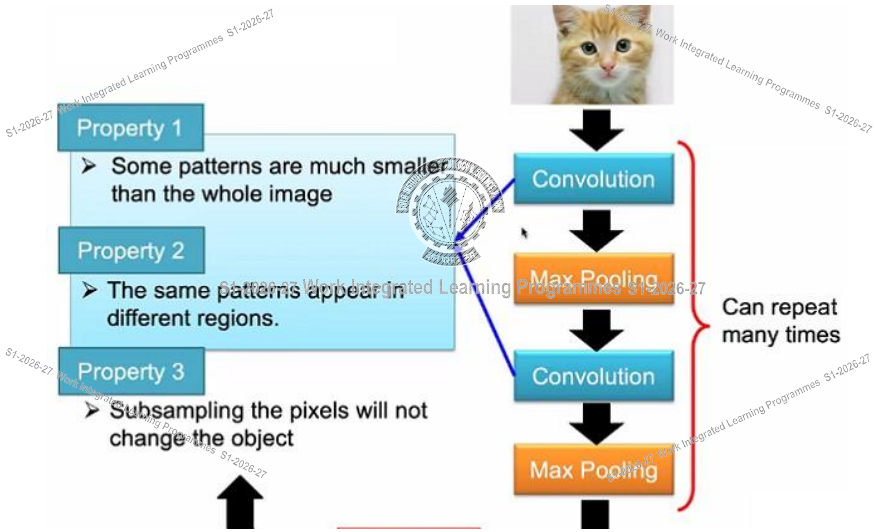
THE WHOLE CNN

346



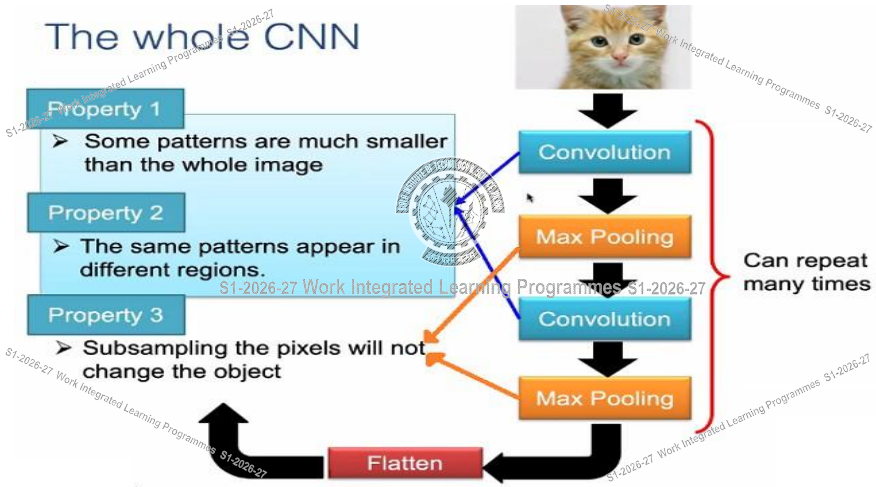
THE WHOLE CNN

347



THE WHOLE CNN

348



Key Principles of CNNs

Three Key Principles

- Local Connectivity (Sparse Interactions)
  - Each neuron connects only to a local region of the input
  - Receptive field concept
- Parameter Sharing
  - Same filter/kernel used across entire input
  - Dramatically reduces parameters
- Equivariant Representations
  - If input shifts, output shifts correspondingly
  - Built-in translation equivariance

CNN Key Characteristics

- **Convolutional Layers**: Apply learnable filters to detect local features
- **Pooling Layers**: Reduce spatial dimensions and provide translation invariance
- **Hierarchical Learning**: Build complex features from simple ones
- **Parameter Sharing**: Same filter applied across entire input
- **Sparse Connectivity**: Each neuron connects to local region only

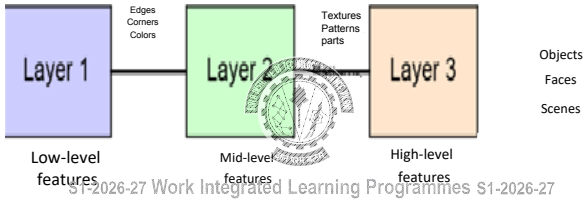
How CNN Achieves Image Classification

Hierarchical Feature Learning Process

CNNs achieve image classification through a hierarchical feature extraction process, where each layer builds increasingly complex representations from simpler ones.

Hierarchical Feature Learning

- Deep networks learn increasingly complex features:



Example: Face Detection

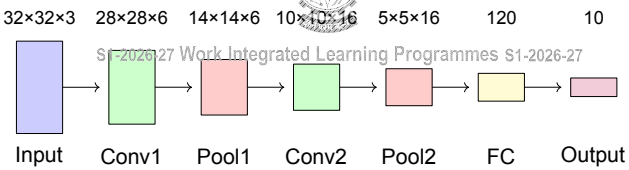
- Layer 1: Detects edges at various orientations
- Layer 2: Combines edges into facial features (eyes, nose, mouth)
- Layer 3: Combines features into complete face

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

CNN Architecture

Architecture Pattern

INPUT → [CONV → ACTIVATION → POOL] x N → FC → OUTPUT



CNN - Layer-by-Layer Breakdown

- Early Layers (1-2)
  - Purpose: Detect low-level features
  - Features: Edges, corners, lines Filters: Small (3×3, 5×5)
  - Receptive Field: Local, small regions
- Middle Layers (3-4)
  - Purpose: Combine low-level features
  - Features: Shapes, textures, patterns
  - Complexity: Medium-level abstractions
  - Receptive Field: textcolorBitsRedr regions

CNN - Layer-by-Layer Breakdown

Deep Layers (5+)

- Purpose: Detect object parts
- Features: Wheels, faces, eyes, ears
- Complexity: High-level semantics
- Receptive Field: textcolor Bits Red image regions

Fully Connected Layers

- Purpose: Final classification decision
- Input: Feature vector from conv layers
- Output: Class probability scores
- Function: Combine all learned features

The Classification Process

- Feature Extraction: Convolutional layers progressively extract features of increasing complexity
- Spatial Reduction: Pooling layers reduce spatial dimensions while preserving important information
- Feature Combination: Deeper layers combine simpler features to recognize complex patterns
- Decision Making: Fully connected layers use extracted features to make final classification decision
- Output: Softmax layer produces probability distribution over classes

Key Insight

Each layer acts as a feature detector that becomes increasingly sophisticated. Early layers detect simple patterns, while later layers detect complex objects by combining simpler patterns.

2D Convolution Operation

357

Cross-Correlation (Used in Deep Learning)

$$(I \star K)(i,j) = \sum_m \sum_n I(i+m, j+n) \cdot K(m,n)$$

In practice, deep learning frameworks implement cross-correlation but call it "convolution"

- Input: Feature map/image I
- Kernel/Filter: Learnable parameters K
- Output: Feature map showing detected patterns

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

2D Convolution Operation

358

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

Input Image 6 × 6

These values inside the matrix are to be learned by the network. We call these as parameters or weights.

1	-1	-1
-1	1	-1
-1	-1	1

Kernel / Filter 1

-1	1	-1
-1	1	-1
-1	1	-1

Kernel / Filter 2

Property 1: Each filter detects a small pattern (3 × 3)

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Convolution Operation

359

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

Input Image 6 × 6

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

Start at top left corner. Perform element wise product and add all the values.

Output

3

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Convolution Operation

360

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

Input Image 6 × 6

1	-1	-1
-1	1	-1
-1	-1	1

Filter 1

Slide by one pixel towards right. Perform element wise product and add all the values.

Output

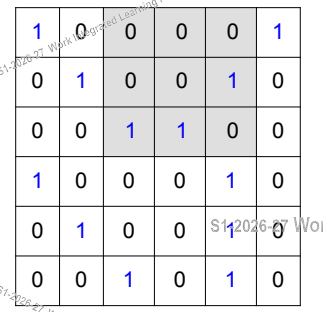
3 -1

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

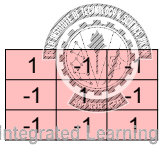
Convolution Operation

361

stride s = 1



Input Image 6 × 6



Filter 1

Slide by one pixel towards right.
Perform element wise product and add all the values.

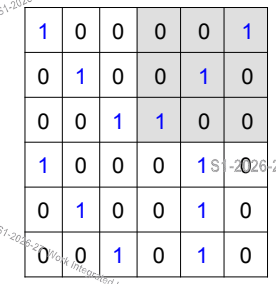
Output

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

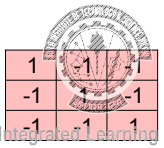
Convolution Operation

362

stride s = 1



Input Image 6 × 6



Filter 1

Slide by one pixel towards right.
Perform element wise product and add all the values.

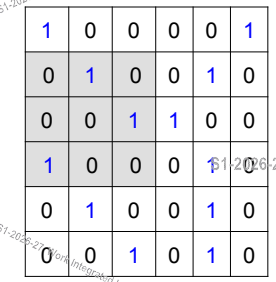
Output

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

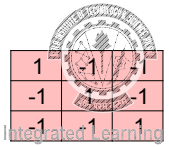
Convolution Operation

363

stride s = 1



Input Image 6 × 6



Filter 1

Jump to next row, left most pixel.
The filter has to completely superimpose the pixels of input.
NOT PARTIALLY.

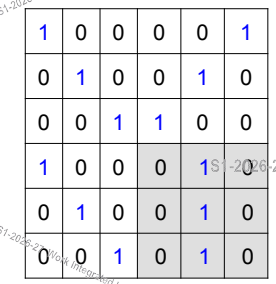
Output

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

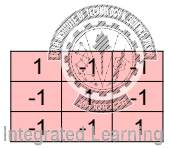
Convolution Operation

364

stride s = 1

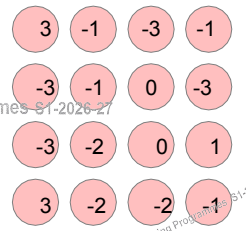


Input Image 6 × 6



Filter 1

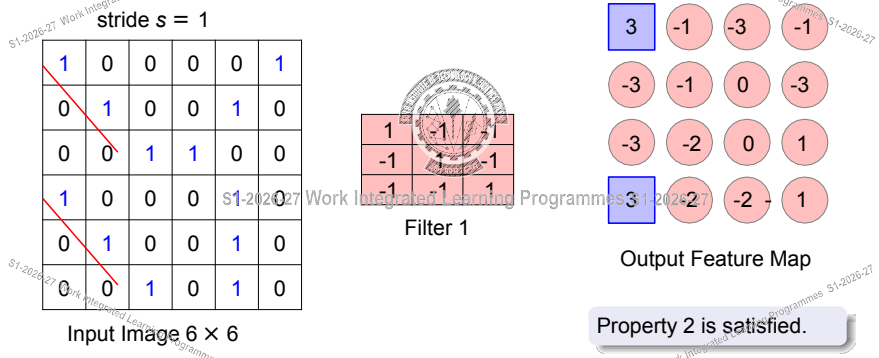
After sliding through the entire image.



Output Feature Map

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

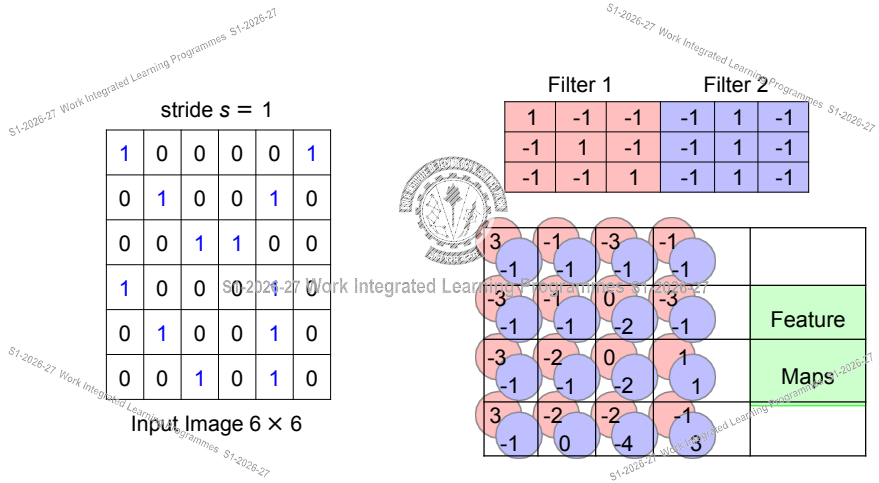
Convolution Operation



Property 2 is satisfied.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Convolution Operation



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

What is a Feature Map?

Definition

A feature map (or activation map) is the output produced when a convolutional filter is applied to an input image or previous layer's output.

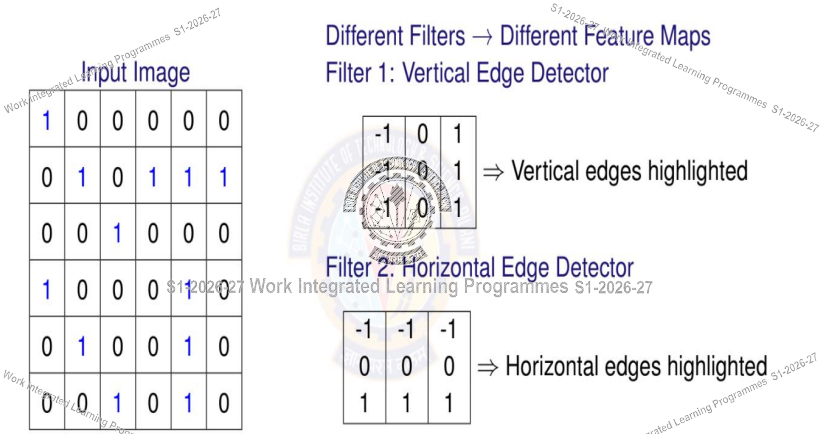
- Key Concepts:
  - Each filter detects a specific pattern (edges, textures, shapes)
  - Output of convolution = feature map showing where pattern was detected
  - High values in feature map = strong pattern presence
  - Low values = weak or absent pattern
- Mathematical Form:

$$\text{Feature Map}_k(i,j) = \sum_m \sum_n I(i+m, j+n) \cdot K_k(m,n) + b_k$$

where K_k is the k-th filter and b_k is its bias.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Feature Maps: Visual Example

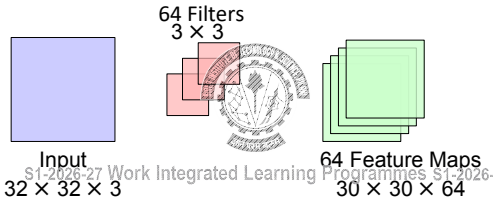


Key Insight: Different filters create different feature maps, each capturing different aspects of the input.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Multiple Feature Maps in CNNs

Single Convolutional Layer with Multiple Filters:



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Important:

- Number of filters = Number of output feature maps
- Each feature map = One filter's response across entire input
- Stack of feature maps = 3D volume (height × width × channels)

Receptive Field: Definition

Definition

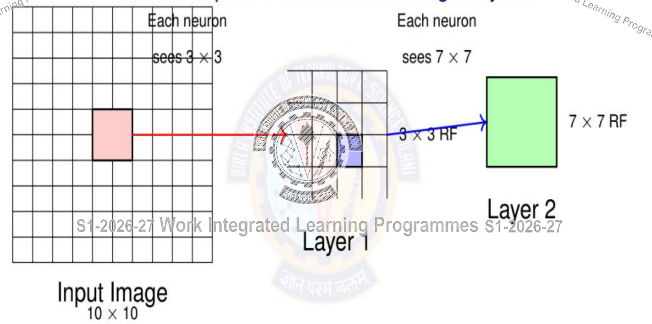
The receptive field of a neuron is the region in the input image that affects that neuron's activation value.

- **Key Concepts:**
  - Receptive field grows with network depth
  - Early layers: small receptive fields (local patterns)
  - Deep layers: large receptive fields (global context)
  - Determined by kernel size, stride, and pooling
- **Importance:**
  - Small receptive field: Cannot capture large objects
  - Large receptive field: Better context understanding
  - Trade-off: Computational cost vs. context

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Receptive Field: Visual Example

How Receptive Field Grows Through Layers



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Pattern: Receptive field size approximately doubles with each layer (depends on architecture).

Receptive Field: Calculation

Formula for receptive field in layer l:

$$RF_l = RF_{l-1} + (K_l - 1) \times \prod_{i=1}^{l-1} S_i$$

- RF_l = Receptive field at layer l
- K_l = Kernel size at layer l
- S_i = Stride at layer i

Example: Sequential 3×3 convolutions with stride 1

Layer 1: $RF_1 = 3$

Layer 2: $RF_2 = 3 + (3-1) \times 1 = 5$

Layer 3: $RF_3 = 5 + (3-1) \times 1 = 7$

Layer 4: $RF_4 = 7 + (3-1) \times 1 = 9$

Key Insight: Stacking small kernels (3×3) is more efficient than using large kernels (9×9) while achieving the same receptive field!

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Learning a Kernel: Overview

Key Question

How does the network learn what values to put in the convolutional kernels/filters?

Answer: Through Gradient Descent!

- Kernel weights are learnable parameters (like weights in fully connected layers)
- Initialized randomly (or with specific strategies)
- Updated via backpropagation and gradient descent
- Network learns kernels that minimize the loss function

Process:

Random kernels $\xrightarrow{\text{Training}}$ Meaningful edge/texture/pattern detectors

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Kernels as Parameters

- Convolutional layer parameters:
  For a filter of size $K \times K$ with C_{in} input channels:

$$\text{Parameters per filter} = K \times K \times C_{in} + 1$$

  (The $+1$ is for the bias term)
- Example: 3×3 filter, 64 input channels
  - Weights: $3 \times 3 \times 64 = 576$
  - Bias: 1
  - Total: 577 learnable parameters per filter
- Gradient Descent Update:

$$K_{new} = K_{old} - \eta \frac{\partial \mathcal{L}}{\partial K}$$

where $\mathcal{L}$ is the loss function and η is the learning rate.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

What Do Learned Kernels Look Like?

Layer 1: Low-level feature detectors

Vertical Edge

$$\begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix}$$

Horizontal Edge

$$\begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}$$

Diagonal Edge

$$\begin{bmatrix} 0 & -1 & -1 \\ 1 & 0 & -1 \\ 1 & 1 & 0 \end{bmatrix}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Characteristics of learned kernels:

- First layers: Edge detectors at various orientations and scales
- Middle layers: Texture and pattern detectors (blob detectors, corner detectors)
- Deep layers: Complex pattern detectors (harder to visualize)

Important: The network discovers these patterns automatically from data!

Learning Kernels: Backpropagation Through

- Forward Pass:

$$\text{Output}(i,j) = \sum_m \sum_n \text{Input}(i+m, j+n) \cdot K(m,n) + b$$

- Backward Pass (Gradient Calculation):
  Gradient w.r.t. kernel weights:

$$\frac{\partial \mathcal{L}}{\partial K(m,n)} = \sum_i \sum_j \frac{\partial \mathcal{L}}{\partial \text{Output}(i,j)} \cdot \text{Input}(i+m, j+n)$$

  Gradient w.r.t. bias:

$$\frac{\partial \mathcal{L}}{\partial b} = \sum_i \sum_j \frac{\partial \mathcal{L}}{\partial \text{Output}(i,j)}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Key Insight: Gradient computation is also a convolution-like operation!

Padding

Why Padding?

- Without padding: Output size = Input size - Kernel size + 1
- Information at borders gets less attention
- Deep networks would shrink feature maps rapidly

Types of Padding

- Valid Padding: No padding (output smaller than input)
- Same Padding: Pad to keep output size = input size
- Full Padding: Maximum padding

Same Padding Calculation

For kernel size k and stride $s = 1$: Padding = $(k - 1)/2$

Convolution with Padding

Input Image with padding 8 × 8

0	0	0	0	0	0	0	0
0	1	0	0	0	0	1	0
0	0	1	0	0	1	0	0
0	0	0	1	1	0	0	0
0	1	0	0	0	1	0	0
0	0	1	0	0	1	0	0
0	0	0	1	0	1	0	0
0	0	0	0	0	0	0	0

*

Filter1

1	-1	-1
-1	1	-1
-1	-1	1

Stride

Stride determines how much the kernel moves at each step.

- Stride = 1: Move kernel one pixel at a time (standard)
- Stride = 2: Move kernel two pixels at a time (downsampling)
- Stride > 1: Reduces output size, acts as downsampling

Output Size Calculation

For input size n, kernel size k, padding p, stride s:

$$\text{Output size} = \frac{n + 2p - k}{s} + 1$$

Result must be an integer for valid convolution. So take floor of the result.

Convolution Operation with Stride

stride $s = 3$

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

Input Image 6 × 6

1	-1	-1
-1	1	-1
-1	-1	1

Filter1

3	-1
3	-1

Feature Map 2x2

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Strided Convolution

- Standard Stride ($s = 1$): Move filter one pixel at a time
- Strided Convolution ($s > 1$): Skip pixels to downsample
- Output size formula:

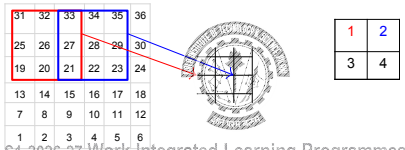
$$O = \left\lfloor \frac{n + 2p - k}{s} \right\rfloor + 1$$

where n = input size, p = padding, k = kernel size, s = stride

- Purpose of Strided Convolutions:
  - Spatial downsampling: Reduce feature map size
  - Replace pooling: Learn downsampling instead of fixed operation
  - Computational efficiency: Fewer computations in subsequent layers
  - Increase receptive field: Larger context per neuron

Strided Convolution: Example

Stride = 2 Example:



Input: 6 × 6 3 × 3 kernel Output: 2 × 2 (stride=2)

With stride = 2: Output size = $\frac{6-3}{2} + 1 = 2$

Trade-off: Lose spatial resolution but gain computational efficiency

Dilated (Atrous) Convolution

Definition

Dilated convolution (also called atrous convolution) inserts spaces (holes) between kernel elements to increase receptive field without increasing parameters.

Dilation rate r : Number of pixels between kernel elements

- $r = 1$: Standard convolution (no dilation)
- $r = 2$: One pixel gap between kernel elements
- $r = 3$: Two pixel gaps between elements

Effective receptive field:

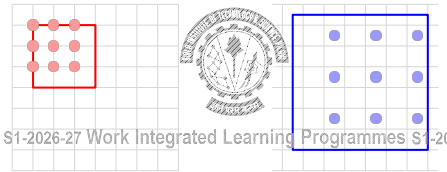
$$k_{eff} = k + (k - 1)(r - 1)$$

where k = kernel size, r = dilation rate

Example: 3 × 3 kernel with $r = 2$ effective 5 × 5 receptive field

Dilated Convolution: Visual Example

Standard vs. Dilated Convolution



Standard ($r = 1$)
Receptive field: 3 × 3

Dilated ($r = 2$)
Receptive field: 5 × 5

Key Advantage: Same number of parameters (9), but larger receptive field!

Dilated Convolution: Applications and Benefits

- When to Use Dilated Convolutions:
  - Semantic segmentation: Need large receptive field while maintaining resolution
  - Dense prediction tasks: Object detection, depth estimation
  - Audio processing: WaveNet uses dilated convolutions for sequence modeling
- Benefits:
  - Larger receptive field without pooling or striding
  - Preserve spatial resolution (no downsampling)
  - Same parameter count as standard convolution
  - Multi-scale context by using different dilation rates
- Trade-offs:
  - May lose fine-grained local information
  - Gridding artifacts possible (mitigated with varying dilation rates)
  - Slightly more memory usage during forward pass

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Strided vs. Dilated Convolution: Comparison

Property	Strided Conv	Dilated Conv
Output size	Reduced	Same as input
Receptive field	Increased	Increased
Parameters	Same	Same
Downsampling	Yes	No
Resolution loss	Yes	No
Typical use	Classification	Segmentation

Summary:

- Strided convolution: Reduce spatial dimensions, typically for classification
- Dilated convolution: Increase receptive field while maintaining resolution, for dense predictions
- Both are alternatives to pooling for different purposes

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Multiple Channels: Introduction

Reality Check

Real images have multiple channels:

- RGB images: 3 channels (Red, Green, Blue)
- Grayscale: 1 channel
- Feature maps in CNNs: Typically c4, 128, 256, or 512 channels

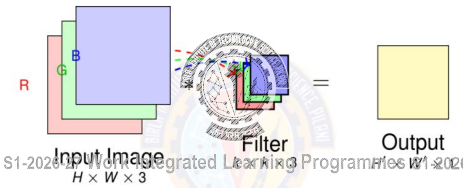
Questions:

- How does convolution work with multi-channel inputs?
- How do we get multi-channel outputs?
- How many parameters do we need?

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Multi-Channel Input: Single Filter

Convolution with 3-channel input (e.g., RGB image):



Key Point: Filter has same depth as input!

$$\text{Output}(i,j) = \sum_{c=1}^{3} \sum_{m} \sum_{n} I(i+m, j+n, c) \cdot K(m,n,c) + b$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Multi-Channel Convolution: Mathematical

- For input $I \in \mathbb{R}^{H \times W \times C_{in}}$ and kernel $K \in \mathbb{R}^{k \times k \times C_{in}}$:
- Step 1: For each channel c, compute 2D convolution:

$$O_c(i,j) = \sum_m \sum_n I(i+m, j+n, c) \cdot K(m,n,c)$$

- Step 2: Sum across all input channels:

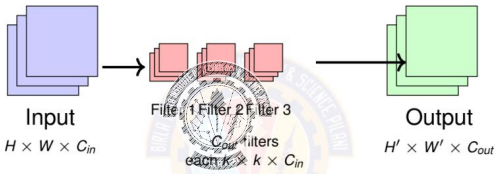
$$O(i,j) = \sum_{c=1}^{C_{in}} O_c(i,j) + b = \sum_{c=1}^{C_{in}} \sum_m \sum_n I(i+m, j+n, c) \cdot K(m,n,c) + b$$

- Result: One filter produces one output channel (feature map)
- Parameters per filter: $k \times k \times C_{in} + 1$ (bias)
- Example: 3×3 filter, 64 input channels = $3 \times 3 \times 64 + 1 = 577$ parameters

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Multi-Channel Input and Output

To get multiple output channels: Use multiple filters!



General Rule:

Input	$: H \times W \times C_{in}$
Filters	$: C_{out}$ filters of size $k \times k \times C_{in}$
Output	$: H' \times W' \times C_{out}$
Total parameters	$: (k \times k \times C_{in} + 1) \times C_{out}$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Multi-Channel Example: Numerical

Setup:

- Input: $4 \times 4 \times 2$ (2 channels)
- Filter: $3 \times 3 \times 2$ (must match input depth)
- Output: $2 \times 2 \times 1$ (one filter, stride=1, no padding)

Input Channel 1: $\begin{bmatrix} 1 & 2 & 3 & 0 \\ 4 & 5 & 6 & 1 \\ 7 & 8 & 9 & 2 \\ 0 & 1 & 2 & 3 \end{bmatrix}$ Input Channel 2: $\begin{bmatrix} 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$

Filter Channel 1: $\begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$ Filter Channel 2: $\begin{bmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{bmatrix}$

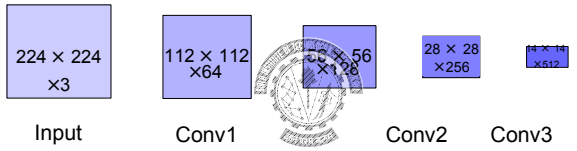
Computation at position (0,0):

Sum from Ch1 $= 1(1) + 2(0) + 3(-1) + 4(1) + 5(0) + 6(-1) + 7(1) + 8(0) + 9(-1)$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Channel Evolution in Deep CNNs

Typical pattern: Increase channels while decreasing spatial dimensions



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Why this pattern?

- Early layers: High resolution, few channels (local patterns)
- Deep layers: Low resolution, many channels (semantic features)
- Maintains roughly constant computational cost per layer
- Total information capacity: $H \times W \times C$ approximately constant

Multi-Channel Convolution: Key Takeaway

- Input channels: Filter depth = Input depth (must match)
- Output channels: Number of filters = Number of output channels
- Parameters: $(k \times k \times C_{in} + 1) \times C_{out}$
- Computation: Each filter operates on all input channels simultaneously

Important Relationships:

- One filter → One output channel (feature map)
- More filters → More output channels. Network can learn more diverse features
- Typical progression: Channels increase with depth (3 64 128 256 512)

Memory consideration: Deep networks trade spatial resolution for channel depth.

$$\text{Memory} \propto H \times W \times C$$

Pooling Operations

What Pooling Layers Do

- Spatial Downsampling: Reduce height and width dimensions but preserve channel depth.
- Translation Invariance: Make features less sensitive to small shifts.
- Computational Efficiency: Reduce number of computations in subsequent layers.
- Overfitting Reduction: Act as a form of regularization.
- Pooling layers are parameter-free but still important for network performance
- Global Average Pooling (GAP) is used to replace fully connected layers.

Two types

- Max pooling
- Average pooling

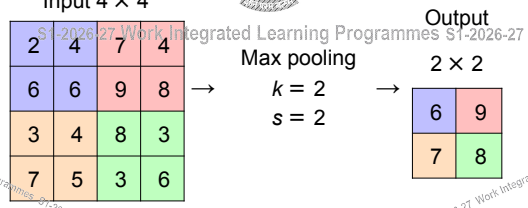
Max Pooling

Takes maximum value in each region

Most common pooling operation

Preserves strongest activations

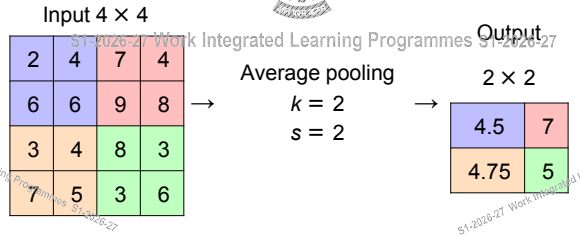
$$output = \max(\text{inputs in window})$$



Average Pooling

- Takes average of values in region
- Smoother downsampling
- Used in final layers sometimes

$$output = \frac{1}{k^2}(\text{inputs in window})$$



1 1 Convolution: Introduction

Definition

A 1×1 convolution uses a kernel of size $1 \times 1 \times C_{in}$ to perform a weighted sum across channels at each spatial location.

At First Glance:

- Seems pointless: No spatial information, just a single pixel!
- But actually very powerful: "Network in Network"

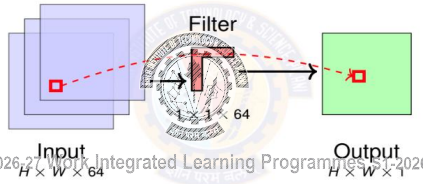
What it does:

- Combines information across channels
- Acts as a learned linear combination
- Does NOT look at neighboring pixels
- Applied independently at each spatial position

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

1 1 Convolution: How It Works

Input: $H \times W \times C_{in}$
Kernel: $1 \times 1 \times C_{in}$
Output: $H \times W \times 1$ (for one filter)



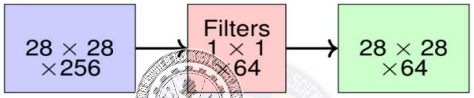
At each spatial position (i, j):

$$\text{Output}(i,j) = \sum_{c=1}^{C_{in}} \text{Input}(i,j,c) \cdot w_c + b$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

1 x 1 Convolution: Key Applications

1. Dimensionality Reduction (Channel Reduction)



Reduces channels from 256 → 64 while preserving spatial dimensions.

2. Adding Non-linearity

Input $\xrightarrow{1\times1\ \text{Conv}}$ Linear combination $\xrightarrow{\text{ReLU}}$ Non-linear features

3. Computational Efficiency

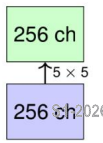
- Reduce channels before expensive 3×3 or 5×5 convolutions
- Used extensively in Inception and ResNet architectures

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

1 x 1 Convolution: Bottleneck Architecture

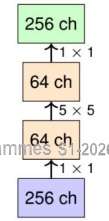
Example: Reducing computation in deep networks

Without 1×1 Conv:



Parameters:
$= 5 \times 5 \times 256 \times 256 = 1,638,400$

With 1×1 Conv:



Parameters:
$= 256 \times 64 + 5^2 \times 64^2 + 64 \times 256 = 147,456$

11× reduction in parameters! This is the "bottleneck" design pattern.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

ReLU Activation Function

Definition

ReLU (Rectified Linear Unit) is a simple and powerful activation function that outputs the input directly if positive, otherwise outputs zero.

Key Properties

- Non-linear: Enables learning complex patterns
- Sparse: Many outputs are exactly zero
- Computationally efficient: Simple max operation
- Non-saturating: No upper bound on positive values

Usage in CNNs ReLU is the default choice for activation functions in CNNs and is typically applied after each convolutional layer:

Conv Layer Output → ReLU → Next Layer

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

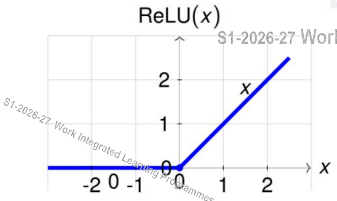
ReLU Activation Function

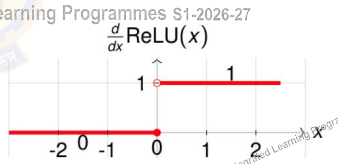
ReLU Function Graph

$$\text{ReLU}(x) = \max(0, x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$

ReLU Derivative Graph

$$\frac{d}{dx}\text{ReLU}(x) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{if } x < 0 \\ \text{undefined} & \text{if } x = 0 \end{cases}$$





S1-2026-27 Work Integrated Learning Programmes S1-2026-27

ReLU Activation Function

Advantages of ReLU

- Solves Vanishing Gradient Problem: Gradient is either 0 or 1 (no saturation for positive values)
- Computational Efficiency: No expensive operations (exp, log, etc.)
- Sparse Representation: Naturally produces sparse networks
- Biological Plausibility: Similar to neuron firing patterns
- Scale Invariant: $\text{ReLU}(\alpha x) = \alpha \cdot \text{ReLU}(x)$ for $\alpha > 0$

Disadvantages of ReLU

- Dying ReLU Problem: Neurons can become permanently inactive (output always 0)
- Not Zero-Centered: Output is always non-negative
- Non-Differentiable at Zero: Can cause issues in some optimization scenarios

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Convolution + ReLu + Max Pooling

Input Image 6 × 6

1	0	0	0	0	1
0	1	0	0	1	0
0	0	1	1	0	0
1	0	0	0	1	0
0	1	0	0	1	0
0	0	1	0	1	0

Filter

-1	1	-1
-1	1	-1
-1	1	-1

Convolve, s=1

-1	-1	-1	-1
-1	-1	-2	-1
-1	-1	-2	1
-1	0	-4	3

Apply ReLU

0	0	0	0
0	0	0	0
0	0	0	1
0	0	0	3

Maxpool k=2, s=2

0	0
0	3

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Fully Connected (Dense) Layer

Definition

A Fully Connected (FC) Layer or Dense Layer is a layer where every neuron is connected to every neuron in the previous layer, performing a linear transformation followed by an activation function.

Role in CNN Architecture

- Feature Flattening: Convert spatial features to vector: $(H \times W \times C) \rightarrow (H \cdot W \cdot C)$
- Classification Head: Map features to class scores
- Dimensionality Control: Adjust feature vector size
- Non-linear Mapping: Learn complex decision boundaries
- Typical CNN Flow:

Conv Layers $\rightarrow$ Flatten $\rightarrow$ FC Layer(s) $\rightarrow$ Output

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Fully Connected (Dense) Layer

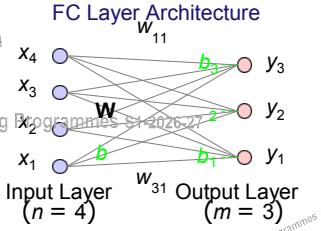
$$\mathbf{y} = f(\mathbf{W}\mathbf{x} + \mathbf{b})$$

- $\mathbf{x} \in \mathbb{R}^n$: Input Vector
- $\mathbf{W} \in \mathbb{R}^{m \times n}$: Weight matrix
- $\mathbf{b} \in \mathbb{R}^m$: Bias vector f
- $(\cdot)$: Activation function
- $\mathbf{y} \in \mathbb{R}^m$: Output vector

Parameter Count

- Weights: $n \times m$ parameters
- Biases: m parameters
- Total: $n \times m + m = m(n + 1)$

FC Layer Architecture



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

FC Layer - Numerical Example

Given: Input $\mathbf{x} = [2, -1, 3]$, weights and biases for 2 output neurons

Weight Matrix: $\mathbf{W} = \begin{bmatrix} 0.5 & -0.2 & 0.1 \\ -0.3 & 0.4 & 0.2 \end{bmatrix}$, Bias: $\mathbf{b} = \begin{bmatrix} 0.1 \\ -0.2 \end{bmatrix}$

Step 1: Linear transformation

$$z_1 = 0.5(2) + (-0.2)(-1) + 0.1(3) + 0.1 = 1.0 + 0.2 + 0.3 + 0.1 = 1.6$$
$$z_2 = (-0.3)(2) + 0.4(-1) + 0.2(3) + (-0.2) = -0.6 - 0.4 + 0.6 - 0.2 = -0.6$$

Step 2: Apply activation (ReLU)

$$y_1 = \text{ReLU}(1.6) = 1.6 \quad (1)$$
$$y_2 = \text{ReLU}(-0.6) = 0.0 \quad (2)$$

Result: $\mathbf{y} = [1.6, 0.0]$

Parameters: $3 \times 2 + 2 = 8$ total parameters

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

FC - Advantages and Disadvantages

Advantages:

- Universal Approximation: Can learn any function
- Flexible Output Size: Any input any output dimension
- Non-linear Transformations: Complex mappings
- Feature Integration: Combines all spatial information

Disadvantages:

- Parameter Heavy: Many parameters $\rightarrow$ overfitting risk
- Spatial Information Loss: No spatial structure preservation
- Fixed Input Size: Requires consistent input dimensions
- Computational Cost: Matrix multiplication overhead

Modern Alternatives

- Global Average Pooling (GAP): Replace FC with average pooling
- 1×1 Convolutions: Maintain spatial structure while reducing channels

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

DEFINITION

Softmax is an activation function that converts a vector of real numbers into a probability distribution, commonly used in the output layer for multi-class classification.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Usage in CNNs Softmax is typically used in the final layer for multi-class classification:

Final Conv/FC Layer → Softmax → Class Probabilities

For input vector $\mathbf{z} = [z_1, z_2, ..., z_K]$:

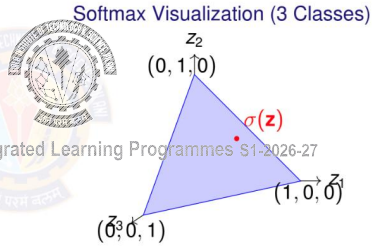
$$\text{Softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$$

Where:

- z_i is the i-th element of input vector
- K is the number of classes
- Output: $\sigma(\mathbf{z}) = [\sigma_1, \sigma_2, ..., \sigma_K]$

Derivative

$$\frac{\partial \sigma_i}{\partial z_j} = \begin{cases} \sigma_i(1-\sigma_i) & \text{if } i = j \\ -\sigma_i \sigma_j & \text{if } i \neq j \end{cases}$$



- Given logits: $\mathbf{z} = [2.0, 1.0, 0.1]$ for 3 classes
- Step 1: Calculate exponentials

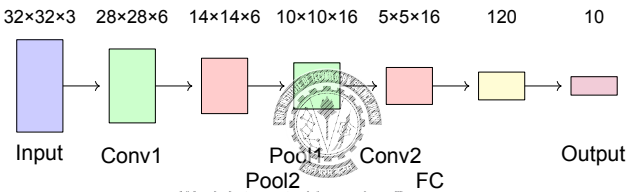
$$\begin{bmatrix} e^{z_1} \\ e^{z_2} \\ e^{z_3} \end{bmatrix} \begin{bmatrix} e^{2.0} \\ e^{1.0} \\ e^{0.1} \end{bmatrix} = \begin{bmatrix} 7.389 \\ 2.718 \\ 1.105 \end{bmatrix}$$

- Step 2: Calculate sum

$$\sum_{j=1}^{3} e^{z_j} = 7.389 + 2.718 + 1.105 = 11.212$$

- Step 3: Calculate softmax probabilities

$$\begin{bmatrix} \sigma_1 \\ \sigma_2 \\ \sigma_3 \end{bmatrix} = \begin{bmatrix} \frac{7.389}{11.212} \\ \frac{2.718}{11.212} \\ \frac{1.105}{11.212} \end{bmatrix} = \begin{bmatrix} 0.659 \\ 0.242 \\ 0.099 \end{bmatrix}$$



Pattern: CONV → POOL → CONV → POOL → FC → OUTPUT

- Early layers: Detect low-level features (edges, corners)
- Middle layers: Combine features (shapes, textures)
- Later layers: High-level concepts (objects, faces)
- Spatial size decreases, depth increases

For a convolutional layer with:

- Kernel size: $K \times K$
- Input channels: C_{in}
- Output channels (number of filters): C_{out}
- Bias: Yes/No, Where bias = 1 if bias is used, 0 otherwise

$$\text{Total Parameters} = (K \times K \times C_{in} + \text{bias}) \times C_{out}$$

- Parameters are independent of input spatial dimensions (height, width)
- Each filter has the same depth as input channels
- Number of parameters grows quadratically with kernel size
- Adding more filters increases parameters linearly

- Example 1: First Convolutional Layer
  - Input: RGB image $(224 \times 224 \times 3)$
  - Kernel size: 5×5
  - Number of filters: 64
  - Bias: Yes
- Calculation:
  - Parameters per filter $= 5 \times 5 \times 3 + 1 = 76$
  - Total parameters $= 76 \times 64 = \mathbf{4,\ 864}$

- Example 2: Deeper Convolutional Layer
  - Input: Feature map $(56 \times 56 \times 128)$
  - Kernel size: 3×3
  - Number of filters: 256
  - Bias: Yes
- Calculation:
  - Parameters per filter $= 3 \times 3 \times 128 + 1 = 1{,}153$
  - Total parameters $= 1{,}153 \times 256 = \mathbf{295{,}168}$

Why Zero Parameters?

Pooling operations are fixed mathematical functions: max or average

- No weights or biases to learn
- Only hyperparameters: pool size k and stride s

Max Pooling Example

- Input: $(56 \times 56 \times 128)$
- Pool size: 2×2
- Stride: 2
- Output: $(28 \times 28 \times 128)$

Parameters: **0**

Average Pooling Example

- Input: $(56 \times 56 \times 128)$
- Pool size: 2×2
- Stride: 2
- Output: $(28 \times 28 \times 128)$

Parameters: **0**

How do we design effective deep CNN architectures?

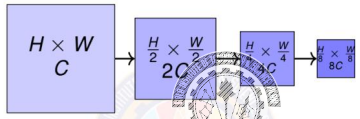
Key Design Patterns (Common to successful architectures):

1. Spatial reduction + Channel expansion
2. Repetitive building blocks Skip/residual
3. connections
4. Bottleneck designs
5. Multi-scale feature extraction

Goal: Balance between:

- Model capacity (expressiveness)
- Computational efficiency Memory
- requirements
- Training stability

Core Principle: As we go deeper, reduce spatial dimensions and increase channels



Rationale:

- Early: High resolution for capturing fine details
- Later: More channels for complex semantic features
- Maintains computational balance: $H \times W \times C \approx$ constant

Implementation:

- Pooling layers (max/average)
- Strided convolutions (learnable downsampling)
- Typically double channels when halving spatial dimensions

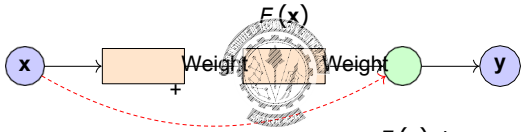
Instead of arbitrary layers, use repeated modular blocks

VGG Block:
Multiple 3×3 convs + pooling

ResNet Block:
Convs + skip connection

Benefits: Modularity, easier to scale, proven designs

Problem: Deep networks suffer from vanishing gradients
Solution: Add identity shortcuts (skip connections)

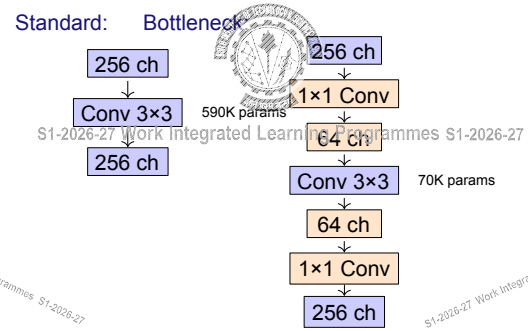


Key Benefits:

- Easier gradient flow during backpropagation
- Network can learn identity mapping if needed
- Enables training of very deep networks (100+ layers)
- Used in ResNet, DenseNet, U-Net

Pattern 4: Bottleneck Designs

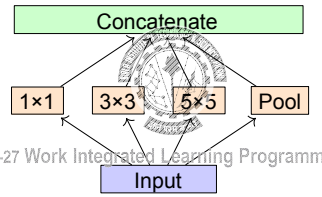
Goal: Reduce computational cost without sacrificing performance
Strategy: Use 1×1 convolutions to reduce dimensions before expensive operations
8× parameter reduction! Used in ResNet-50/101/152, Inception, MobileNet



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Pattern 5: Multi-Scale Feature Extraction

Idea: Process input at multiple scales simultaneously
Inception Module Example:



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Benefits:

- Capture features at different scales
- More robust to object size variations
- Network decides which scale is important

How to Choose Architecture Depth

Key Considerations:

1. Task complexity
   - Simple: MNIST 5-10 layers sufficient
   - Complex: ImageNet 50-152 layers
   - Very complex: Medical imaging 100+ layers
2. Dataset size
   - Small dataset (< 10K): Shallow network (avoid overfitting)
   - Medium (10K-100K): Moderate depth (20-50 layers)
   - Large (> 1M): Very deep (50-200 layers)
3. Computational budget
   - Mobile/edge: MobileNet, EfficientNet (10-20 layers)
   - Desktop/server: ResNet-50, VGG-16 (16-50 layers)
   - Data center: ResNet-152, EfficientNet-B7 (100+ layers)

Start shallow, add depth if underfitting and you have enough data

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Architecture Depth: Practical Guidelines

Empirical observations:

Network	Layers	Params	Use Case
LeNet-5	5	60K	MNIST, simple tasks
AlexNet	8	60M	ImageNet (2012)
VGG-16	16	138M	High accuracy, features
ResNet-50	50	25M	General purpose
ResNet-152	152	60M	State-of-art accuracy
EfficientNet-B0	18	5M	Mobile, efficient

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Trends:

- Modern architectures: 50-200 layers with skip connections
- Efficient architectures: 10-30 layers with smart design Depth
- alone is not enough: Need proper training techniques

Warning: Very deep networks require:

- Batch normalization

Trade-offs: Depth vs. Width

Depth: Number of layers (vertical stacking)
Width: Number of channels per layer (horizontal expansion)

Aspect	Deeper (More Layers)	Wider (More Channels)
Feature complexity	Higher	Moderate
Receptive field	Larger	Same
Parameters	Sub-linear growth	Quadratic growth
Training difficulty	Harder (vanishing grad)	Easier
Overfitting risk	Lower (if regularized)	Higher
Inference speed	Slower (sequential)	Faster (parallel)

Example:

- ResNet-50: Deep (50 layers), moderate width (256-2048 channels)
- Wide ResNet: Fewer layers (16-28), very wide (up to 10× channels)

Current consensus: Depth + Width both matter; optimal ratio depends on task

Depth vs. Width: Practical Recommendations

When to go deeper:

- Large, diverse datasets (ImageNet, COCO)
- Complex tasks requiring hierarchical features
- When you can use skip connections
- Sufficient computational resources

When to go wider:

- Smaller, more homogeneous datasets
- Need faster training/inference
- Limited to shallow networks (no residual connections)
- Ensemble-like behavior (different filters learn different patterns)

Hybrid approach (EfficientNet):

- Scale depth, width, AND resolution together
- Use compound scaling: depth^d × width^w × resolution^r = 2
- Achieves better accuracy-efficiency trade-off

Design Decision Framework

1. Start with baseline
   - Use proven architecture (ResNet, EfficientNet)
   - Transfer learning if possible
2. Estimate requirements
   - Dataset size → depth constraint
   - Computational budget → width/depth trade-off
   - Input resolution → receptive field needs
3. Iterate systematically
   - Underfitting? → Add depth or width
   - Overfitting? → Regularize (dropout, data aug) before reducing capacity
   - Too slow? → Reduce width or use efficient blocks
4. Apply design patterns
   - Always: Spatial reduction + channel expansion
   - Deep networks: Skip connections mandatory
   - Tight budget: Bottleneck designs

ImageNet Data

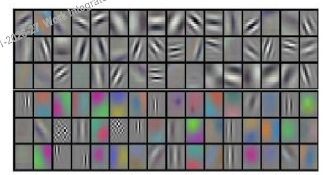


Figure 3: 96 convolutional kernels of size 11*11*3 learned by the first convolutional layer on the 224*224*3 input images. The top 48 kernels were learned on GPU 1 while the bottom 48 kernels were learned on GPU

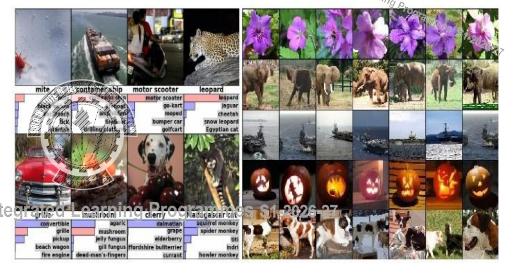
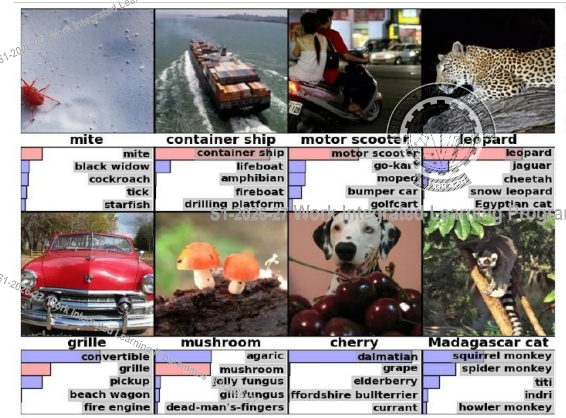


Figure 4: **(Left)** Eight ILSVRC-2010 test images and the five labels considered most probable by our model. The correct label is written under each image, and the probability assigned to the correct label is also shown with a red bar (if it happens to be in the top 5). **(Right)** Five ILSVRC-2010 test images in the first column. The remaining columns show the six training images that produce feature vectors in the last hidden layer with the smallest Euclidean distance from the feature vector for the test image.

IMAGENET DATA



The ImageNet set that was used has ~1.2 million images and 1000 classes

Accuracy is measured as "top-5" performance: Correct prediction if the true label matches one of the top 5 predictions of the model

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

ALEXNET

- AlexNet competed in the ImageNet Large Scale Visual Recognition Challenge on September 30, 2012.[3] The network achieved a top-5 error of 15.3%, more than 10.8 percentage points lower than that of the runner up.
- Details/Retrospectives:
  - first use of ReLU
  - used Local Response Normalisation (LRN) layers (not common anymore)
  - Approx. 60 million parameters are learned.
  - heavy data augmentation
  - dropout 0.5
  - batch size 128
  - SGD Momentum 0.9
  - Learning rate 1e −2, reduced by 10 manually
  - L2 weight decay 5e −4
  - 7 CNN ensemble: 18.2%−>15.4%

Fei-Fei Li, Jiajun Wu, Ruohan Gao

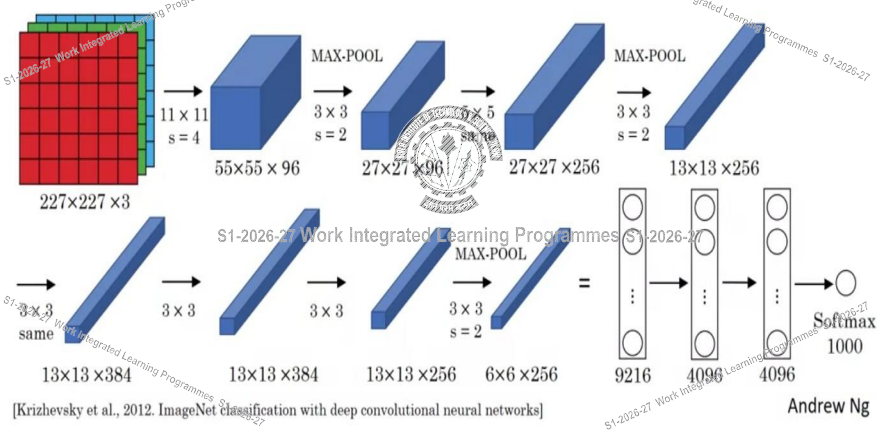
S1-2026-27 Work Integrated Learning Programmes S1-2026-27

ALEXNET ARCHITECTURE

- 227x227x3 INPUT
- 55x55x96 CONV1 : 96 11x11 filters at stride 4, pad 0
  27x27x96
- MAX POOL1 : 3x3 filters at stride 2 27x27x96
- NORM1 : Normalization layer 27x27x256
- CONV2 : 256 5x5 filters at stride 1, pad 2 13x13x256
- MAX POOL2 : 3x3 filters at stride 2 13x13x256
- NORM2 : Normalization layer 13x13x384
- CONV3 : 384 3x3 filters at stride 1, pad 1 13x13x384
- CONV4 : 384 3x3 filters at stride 1, pad 1 13x13x256
- CONV5 : 256 3x3 filters at stride 1, pad 1 6x6x256
- MAX POOL3 : 3x3 filters at stride 2
- 4096 FC6 : 4096 neurons
- 4096 FC7 : 4096 neurons
- 1000 FC8 : 1000 neurons (class scores)

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

ALEXNET



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

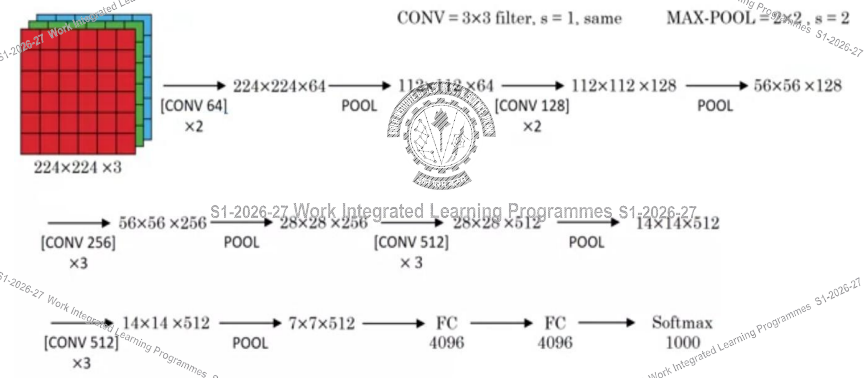
VGG 16

- VGG stands for Visual Geometry Group with 16 Layers
- Plug and play in Caffe
- Deeper the better
- Details
  - ILSVRC'14 2nd in classification, 1st in localization
  - Similar training procedure as Krizhevsky 2012
  - Approx. 138 million parameters are learned.
  - No Local Response Normalisation (LRN)
  - Use VGG16 or VGG19 (VGG19 only slightly better, more memory)
  - Use ensembles for best results
  - All convolutions with a 3 ×3 kernel
  - All max-pooling layers with a 2 ×2 kernel
  - FC7 features generalize well to other tasks

Fei-Fei Li, Jiajun Wu, Ruohan Gao

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

VGG 16



[Simonyan & Zisserman 2015. Very deep convolutional networks for large-scale image recognition]

Andrew Ng

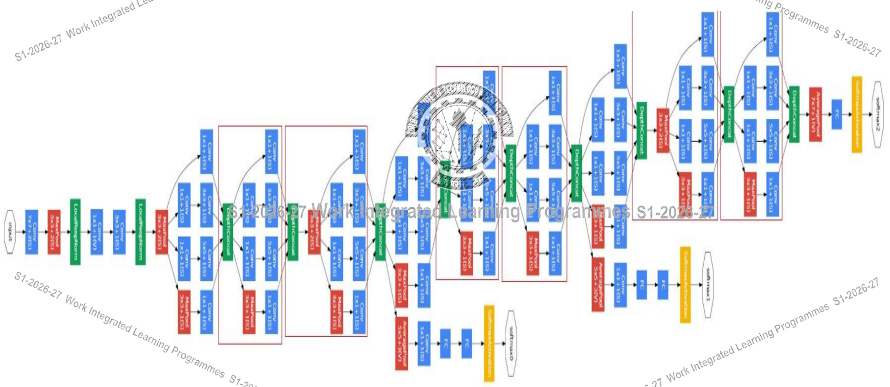
S1-2026-27 Work Integrated Learning Programmes S1-2026-27

GOOGLENET / INCEPTION NET

- ILSVRC'14 classification winner (6.7% top 5 error) 22 layers with weights
- Only 5 million parameters (12x less than AlexNet and 27x less than VGG-16)
- Inception Module - convolutional "blocks" – efficient
  - Design a good local network topology (network within a network) and then stack these modules on top of each other.
- Linear layers at the end
- Max pooling in between, multiple Conv layers between pooling
- Great ideas for data augmentation
- Deeper networks, with computational efficiency
- No FC layers.
- After the last convolutional layer, a global average pooling layer is used that spatially averages across each feature map, before final FC layer.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

INCEPTION NET

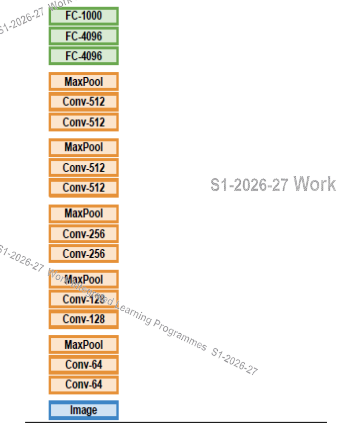


S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Transfer Learning with CNNs

Donahue et al, "DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition", ICML 2014
Razavian et al, "CNN Features Off-the-Shelf: An Astounding Baseline for Recognition", CVPR Workshops 2014

1. Train on Imagenet

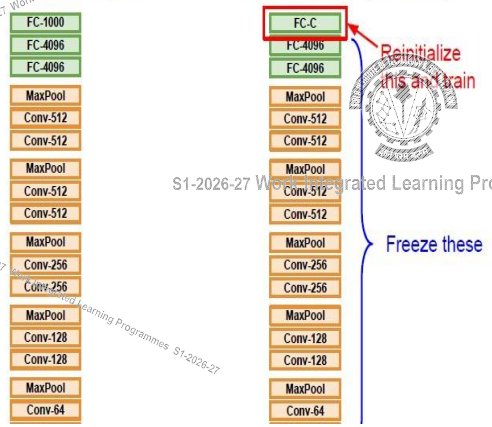


S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Transfer Learning with CNNs

Donahue et al, "DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition", ICML 2014
Razavian et al, "CNN Features Off-the-Shelf: An Astounding Baseline for Recognition", CVPR Workshops 2014

1. Train on Imagenet
2. Small Dataset (C classes)

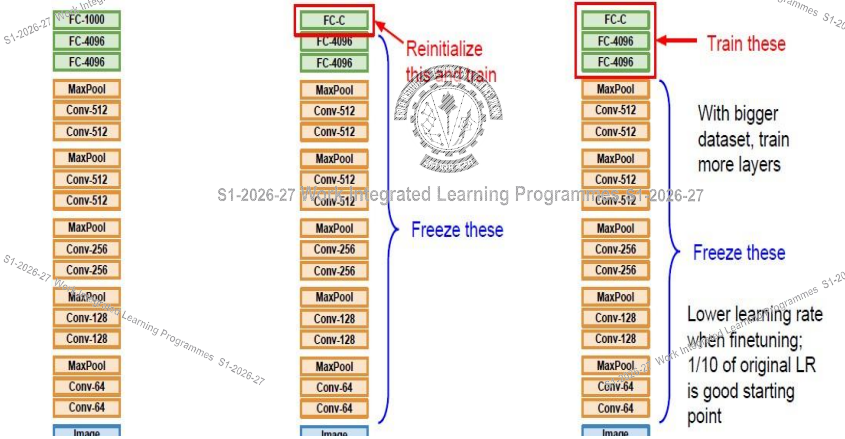


S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Transfer Learning with CNNs

Donahue et al, "DeCAF: A Deep Convolutional Activation Feature for Generic Visual Recognition", ICML 2014
Razavian et al, "CNN Features Off-the-Shelf: An Astounding Baseline for Recognition", CVPR Workshops 2014

1. Train on Imagenet
2. Small Dataset (C classes)
3. Bigger dataset



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

- Use a deep learning model that is pre-trained on large dataset like ImageNet or MS Coco.
- Oxford VGG Model
- Google Inception
- Model Microsoft
- ResNet Model

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

TRANSFER LEARNING FOR TEXT DATA

- Embedding is the mapping of words to a high-dimensional continuous vector space where different words with similar meanings have similar vector representations.
- Google's word2vec Model
- Stanford's GloVe Model
- FastText
- Gensim


```
model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
model.add(layers.MaxPooling2D((2, 2)))
 model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.Flatten()) model.add(layers.Dense(64, activation='relu'))
model.add(layers.Dense(10))
```

```
model.compile(optimizer='adam',
es.SparseCategoricalCrossentropy(from_logits=True), metrics=['accuracy'])
```

TRANSFER LEARNING - WHEN TO USE?

- You need a lot of a data if you want to train/use CNNs / RNNs.
- Task A and Task B have the same type of input. Eg: Input is images for both tasks.
- We have lot of data for training Task A and relatively low data for training in Task B.
- Low level features obtained from Task A could be more helpful for learning Task B.

MODULE # 7 :
RNN

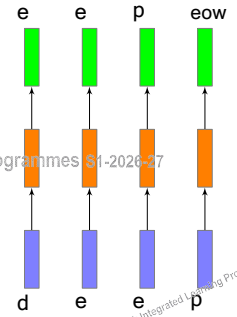
S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Example: Auto-completion

This is not true in many applications.

- The size of the input is not always fixed.
- Successive inputs may not be independent of each other.
- Each network (blue – orange – green structure) is performing the same task –
  - input : character
  - output : character.

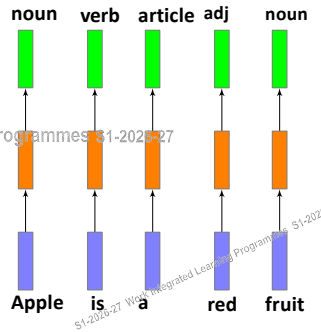
Example: Auto-completion.



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

PART OF SPEECH TAGGING

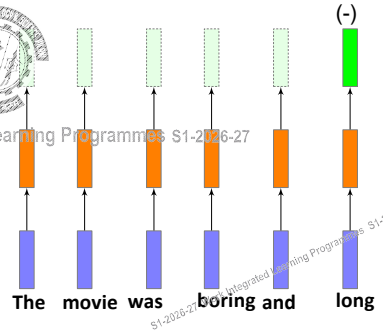
- Task is predicting the part of speech tag (noun, adverb, adjective, verb) of each word in a sentence.
- When we see an adjective we are almost sure, the next word should be a noun.
- The current output depends on the current input as well as the previous input.
- The size of the input is not fixed. Sentences have any number of words.
- An output is produced at end of each time step.
- Each network is performing the same task – input : word, output : tag.



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

SENTIMENT ANALYSIS

- Task is predicting the sentiment of a whole sentence.
- Input is the entire sequence of inputs.
- An output is **not** produced at end of each time step.
- Each network is performing the same task –
  - input : word,
  - output : polarity +/−.



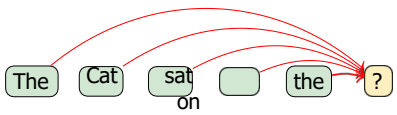
S1-2026-27 Work Integrated Learning Programmes S1-2026-27

MOTIVATION: WHY SEQUENCE MODELS?

- Many real-world problems involve sequential data:
  - Natural Language: "The cat sat on the _"
  - Time Series Forecasting: Stock prices, temperature prediction, sensor data
  - Speech: Audio signals over time
  - Video: Frame sequences
  - Genomics: DNA/protein sequences

Key characteristic:

- Order matters Dependencies exist across time steps



Dependencies

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

SEQUENCE LEARNING PROBLEMS

To model a sequence we need

- Process an input or sequence of inputs.
- The inputs may have be dependent.
- We may have to maintain the sequence order.
- Each input corresponds to one time step.
- Keep track of long term dependencies.
- Produce an output or sequence of outputs.
- Supervised Learning.
- Share parameters across the sequences.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

WHY FEEDFORWARD NETWORKS FAIL?

Key issues:

- Fixed input size: Cannot handle variable-length sequences
- No memory: Each input processed independently
- No parameter sharing: Different weights for each position

Example Problem: Predict next word in a sentence

- Input: ”I grew up in France, so I speak fluent _”
- Need context from 7 words back (”France”) to predict ”French”
- Feedforward network treats each position independently

We need: A model that can remember past information

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

TEMPORAL DEPENDENCIES

Short-term dependencies: Information needed is recent

- ”The clouds are in the __” → ”sky”

Long-term dependencies: Information needed is far back

- ”I grew up in France... I speak fluent __” → ”French”

Challenge: Model must learn to remember relevant information and forget irrelevant information.

Different sequence lengths require flexible architectures

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

AUTOREGRESSIVE MODELS

- Goal: Model the probability of a sequence
  For a sequence $x_1, x_2, \ldots, x_T$:

$$P(x_1, x_2, \ldots, x_T) = \prod_{t=1}^{T} P(x_t | x_1, \ldots, x_{t-1}) \quad (1)$$

- Autoregressive property: Current value depends on previous values

$$P(x_t | x_1, \ldots, x_{t-1}) = f(x_{t-1}, x_{t-2}, \ldots, x_1) \quad (2)$$

- Challenge: How much history to consider? (t grows over time)

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Autoregressive Models: Example

Sequence: ”The cat sat on the”

Predict: Next word (e.g., ”mat”)

Autoregressive decomposition:

$$P(\text{The, cat, sat, on, the, mat}) \approx P(\text{The}) \times P(\text{cat}|\text{The}) \times P(\text{sat}|\text{The, cat}) \times P(\text{on}|\text{The, cat, sat}) \times P(\text{the}|\text{The, cat, sat, on}) \times P(\text{mat}|\text{The, cat, sat, on, the})$$

- Each word depends on ALL previous words
- History grows with sequence length

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Markov Assumption

- Simplification: Limit the context window
- First-order Markov: Only immediate past matters

$$P(x_t|x_1, \ldots, x_{t-1}) \approx P(x_t|x_{t-1}) \quad (3)$$

- k -th order Markov: Last k steps matter

$$P(x_t|x_1, \ldots, x_{t-1}) \approx P(x_t|x_{t-k}, \ldots, x_{t-1}) \quad (4)$$

- Advantage: Fixed-size context, tractable computation
- Disadvantage: May lose important long-term dependencies

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Markov Assumption: Example

- Sequence: ”The cat sat on the”
- Predict: Next word (e.g., ”mat”)
- First-order Markov (bigram):

$$P(\text{mat}|\text{The, cat, sat, on, the}) \approx P(\text{mat}|\text{the})$$

- Second-order Markov (trigram):

$$P(\text{mat}|\text{The, cat, sat, on, the}) \approx P(\text{mat}|\text{on, the})$$

- Key insight:
  - Fixed context window (1 or 2 words)
  - Ignores earlier words like ”cat” and ”sat”
  - Computationally efficient but limited expressiveness

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Latent Variable Models

- Alternative approach: Summarize past with a hidden state

$$h_t = f(x_t, h_{t-1}) \quad (5)$$

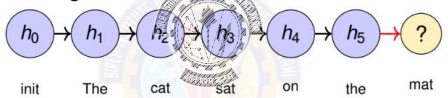
$$P(x_t|x_1, \ldots, x_{t-1}) = P(x_t|h_t) \quad (6)$$

- Key insight:
  - h_t is a *latent* (hidden) variable
  - Contains compressed summary of history
  - Fixed-size representation regardless of sequence length
  - Updated recursively at each time step
- This is the foundation of RNNs!

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Latent Variable Models: Example

- Sequence: "The cat sat on the"
- Predict: Next word using hidden state



- Key insight:
  - h_5 summarizes entire history: "The cat sat on the"
  - Predict: $P(\text{mat} \mid h_5)$
  - Fixed-size h_t regardless of sequence length

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Numerical Example: Temperature Forecasting - Setup

- Task: Predict tomorrow's temperature (regression)
- Observed sequence: 20°C, 22°C, 25°C, 24°C, 21°C
- Approach 1: Autoregressive (First-order)
  - Linear relationship learned from data:

$$T_{t+1} = \alpha \cdot T_t + \beta$$

  - From historical data, we learned:

$$\alpha = 0.8, \qquad \beta = 4.0$$

  - Example: If today is 25°C:

$$T_{\text{tomorrow}} = 0.8 \times 25 + 4.0 = 24.0^\circ C$$

Temperature Forecasting - Autoregressive Prediction

- Sequence: 20°C, 22°C, 25°C, 24°C, 21°C
- Model: $T_{t+1} = 0.8 \cdot T_t + 4.0$
- Predict next day (day 6):
  - Last observation: $T_5 = 21^\circ C$
  - Prediction: $T_6 = 0.8 \times 21 + 4.0 = 20.8^\circ C$
- Multi-step ahead (day 7, using predicted T_6):
  - $T_7 = 0.8 \times 20.8 + 4.0 = 20.64^\circ C$
- Limitation: Only uses immediate past
  - Cannot capture weekly/seasonal patterns
  - Error accumulates in multi-step predictions

Temperature Forecasting - Latent Variable Model

- Approach 2: Latent Variable (Hidden State)
- Hidden state $h_t \in \mathbb{R}^d$ captures weather pattern/trend:
  - Warming trend, cooling trend, stable period
  - Seasonal effects, time of year
- Model equations:

$$\hat{T}_{t+1} = g(h_t) \quad \text{where} \quad h_t = f(T_t, h_{t-1})$$

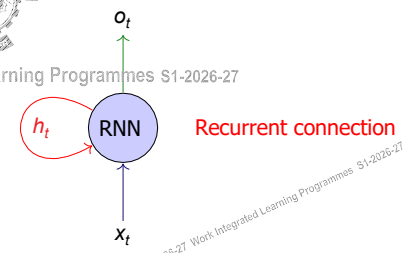
- Sequence: $20^\circ C, 22^\circ C, 25^\circ C, 24^\circ C, 21^\circ C$
  - h_1 processes $20^\circ C$: captures "starting point"
  - h_2 processes $22^\circ C$: captures "warming trend" $(20 \rightarrow 22)$
  - h_3 processes $25^\circ C$: captures "strong warming continues" $(20 \rightarrow 22 \rightarrow 25)$
  - h_4 processes $24^\circ C$: captures "peak reached at 25℃, cooling starts" $(25 \rightarrow 24)$
  - h_5 processes $21^\circ C$: captures "cooling trend continues" $(25 \rightarrow 24 \rightarrow 21)$
  - h_5 encodes entire temperature trajectory, not just last value $(21^\circ C)$
  - Prediction uses full context: $\hat{T}_6 = g(h_5)$ might predict $19 - 20^\circ C$ (continue cooling)

Recurrent Neural Network (RNN)

Definition

Neural network with loops, allowing information to persist

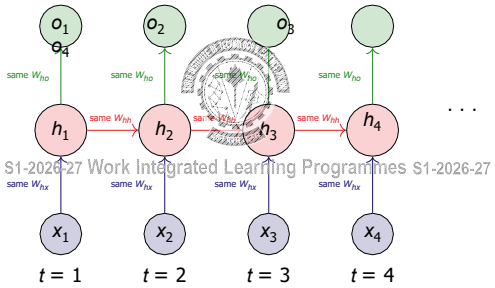
- Key idea:
  - Process sequential data one step at a time
  - Maintain hidden state that summarizes past information
  - Same parameters shared across all time steps
- The loop: Hidden state h_t feeds back into itself at next time step



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

RNN: Unrolled Computation Graph

Unrolling: Visualize RNN across time steps



Key insight: Same parameters (W_{hx}, W_{hh}, W_{ho}) applied at each time step

RNN Mathematical Formulation I

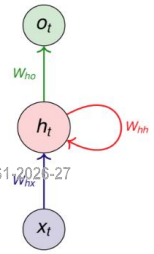
- Hidden state update:

$$h_t = \phi(W_{hh}h_{t-1} + W_{hx}x_t + b_h) \quad (7)$$

where:

- $x_t \in \mathbb{R}^d$: input at time t
- $h_t \in \mathbb{R}^h$: hidden state at time t
- $W_{hx} \in \mathbb{R}^{h \times d}$: input-to-hidden weights
- $W_{hh} \in \mathbb{R}^{h \times h}$: hidden-to-hidden (recurrent) weights
- $b_h \in \mathbb{R}^h$: hidden bias
- ϕ: activation function (typically tanh or ReLU)

- Initial hidden state: $h_0 = \mathbf{0}$ or learned parameter



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

RNN Mathematical Formulation II

- Output at each time step:

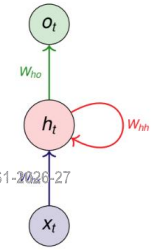
$$o_t = W_{ho}h_t + b_o \quad (8)$$

where:

- $o_t \in \mathbb{R}^V$: output logits at time t
- $W_{ho} \in \mathbb{R}^{V \times h}$: hidden-to-output weights
- $b_o \in \mathbb{R}^V$: output bias
- V: vocabulary size (for language modeling)

- Predicted probabilities (using softmax):

$$P(x_t = k | x_1, \ldots, x_{t-1}) = \frac{\exp(o_t^{(k)})}{\sum_{j=1}^{V} \exp(o_t^{(j)})} \quad (9)$$



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Parameter Sharing Across Time

- Key insight: Same parameters used at every time step
  - W_{hx}, W_{hh}, b_h: Shared across all time steps
  - W_{ho}, b_o: Shared across all time steps
- Advantages:
  - Fixed number of parameters regardless of sequence length
  - Can process sequences of any length
  - Generalization: patterns learned at one position transfer to others

Contrast with feedforward: Different weights for each position

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

RNN Parameter Counting

- Given:
  - Input dimension: d
  - Hidden dimension: h
  - Output/vocabulary size: V
- Parameter count:

$$W_{hx}: h \times d$$
$$W_{hh}: h \times h$$
$$b_h: h$$
$$W_{ho}: V \times h$$
$$b_o: V$$
$$\text{Total}: hd + h^2 + h + Vh + V = h(d + h + V + 1) + V$$

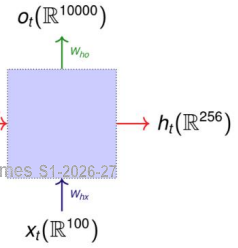
S1-2026-27 Work Integrated Learning Programmes S1-2026-27

RNN Parameter Counting Example

- Given:
  - Input dimension: $d = 100$
  - Hidden dimension: $h = 256$
  - Output/vocabulary: $V = 10{,}000$
- Parameter count:

$$W_{hx}: 256 \times 100 = 25{,}600$$
$$W_{hh}: 256 \times 256 = 65{,}536$$
$$b_h: 256$$
$$W_{ho}: 10{,}000 \times 256 = 2{,}560{,}000$$
$$b_o: 10{,}000$$

- Total: 2,661,392 parameters



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

RNN Forward Pass: Simple Example

- Dimensions:
  - Input: $d = 2$
  - Hidden: $h = 3$
  - Output: $V = 2$ (binary classification)
- Parameters (simplified, small values):

$$W_{hx} = \begin{bmatrix} 0.5 & 0.3 \\ 0.2 & 0.4 \\ 0.1 & 0.2 \end{bmatrix}, \quad W_{hh} = \begin{bmatrix} 0.3 & 0.2 & 0.1 \\ 0.1 & 0.4 & 0.2 \\ 0.2 & 0.1 & 0.3 \end{bmatrix}$$

$$b_h = \begin{bmatrix} 0.1 \\ 0.1 \\ 0.1 \end{bmatrix}, \quad W_{ho} = \begin{bmatrix} 0.4 & 0.3 & 0.5 \\ 0.2 & 0.6 & 0.1 \end{bmatrix}, \quad b_o = \begin{bmatrix} 0.1 \\ 0.1 \end{bmatrix}$$

- Input sequence: $x_1 = [1.0, 0.5]^T$, $x_2 = [0.8, 1.2]^T$
- Initial hidden state: $h_0 = [0, 0, 0]^T$
- Activation: $\phi = \tanh$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

RNN Forward Pass: Time Step $t = 1$

Compute h_1:

$$z_1 = W_{hh}h_0 + W_{hx}x_1 + b_h$$

$$= \begin{bmatrix} 0.3 & 0.2 & 0.1 \\ 0.1 & 0.4 & 0.2 \\ 0.2 & 0.1 & 0.3 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0.5 & 0.3 \\ 0.2 & 0.4 \\ 0.1 & 0.2 \end{bmatrix} \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix} + \begin{bmatrix} 0.1 \\ 0.1 \\ 0.1 \end{bmatrix} = \begin{bmatrix} 0.75 \\ 0.50 \\ 0.30 \end{bmatrix}$$

$$h_1 = \tanh(z_1) = \begin{bmatrix} 0.635 \\ 0.462 \\ 0.291 \end{bmatrix}$$

Compute o_1:

$$o_1 = W_{ho}h_1 + b_o = \begin{bmatrix} 0.4 & 0.3 & 0.5 \\ 0.2 & 0.6 & 0.1 \end{bmatrix} \begin{bmatrix} 0.635 \\ 0.462 \\ 0.291 \end{bmatrix} + \begin{bmatrix} 0.1 \\ 0.1 \end{bmatrix} = \begin{bmatrix} 0.538 \\ 0.532 \end{bmatrix}$$

RNN Forward Pass: Time Step $t = 2$

Compute h_2 (now using h_1 from previous step):

$$z_2 = W_{hh}h_1 + W_{hx}x_2 + b_h$$

$$= \begin{bmatrix} 0.3 & 0.2 & 0.1 \\ 0.1 & 0.4 & 0.2 \\ 0.2 & 0.1 & 0.3 \end{bmatrix} \begin{bmatrix} 0.635 \\ 0.462 \\ 0.291 \end{bmatrix} + \begin{bmatrix} 0.5 & 0.3 \\ 0.2 & 0.4 \\ 0.1 & 0.2 \end{bmatrix} \begin{bmatrix} 0.8 \\ 1.2 \end{bmatrix} + \begin{bmatrix} 0.1 \\ 0.1 \\ 0.1 \end{bmatrix} = \begin{bmatrix} 1.172 \\ 1.047 \\ 0.680 \end{bmatrix}$$

$$h_2 = \tanh(z_2) = \begin{bmatrix} 0.826 \\ 0.780 \\ 0.592 \end{bmatrix}$$

Compute o_2:

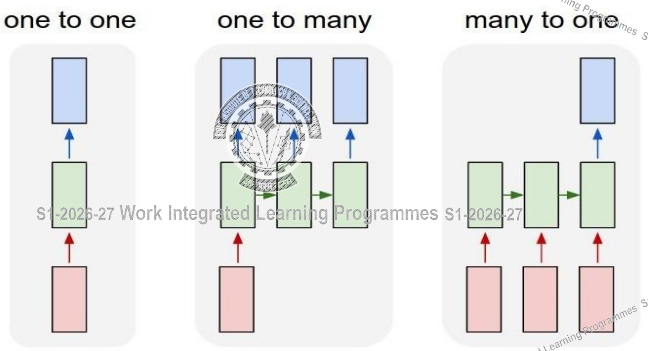
$$o_2 = W_{ho}h_2 + b_o = \begin{bmatrix} 0.4 & 0.3 & 0.5 \\ 0.2 & 0.6 & 0.1 \end{bmatrix} \begin{bmatrix} 0.826 \\ 0.780 \\ 0.592 \end{bmatrix} + \begin{bmatrix} 0.1 \\ 0.1 \end{bmatrix} = \begin{bmatrix} 0.861 \\ 0.793 \end{bmatrix}$$

Activation Functions in RNNs

Common choices for φ:

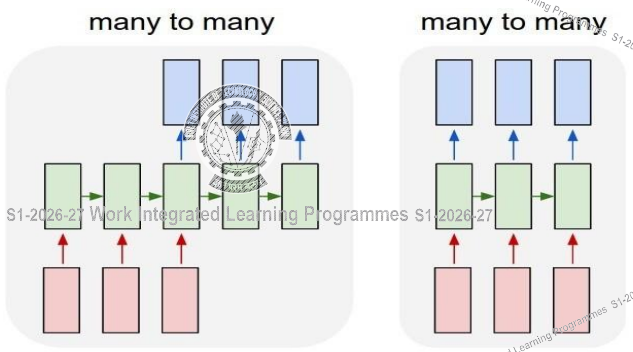
- Hyperbolic tangent (tanh):
  - Range: $[-1, 1]$
  - Centered at zero (helps with gradient flow)
  - Most common for vanilla RNNs
- ReLU:
  - Range: $[0, \infty)$
  - Can help with vanishing gradient
  - May cause exploding activations

Types of RNN



Courtesy: Andrej Karpathy

TYPES OF RNN



Courtesy: Andrej Karpathy

TYPES OF RNN AND APPLICATIONS

- **One to one** – Generic neural network, Image classification
- **One to many** – Music generation, Image Captioning
- **Many to one** – Movie review or Sentiment Analysis
- **Many to many** – Machine translation Synced
- **Many to many** – Video classification

TRAINING OBJECTIVE FOR RNNS

- Goal: Learn parameters $\theta = \{W_{hx}, W_{hh}, W_{ho}, b_h, b_o\}$ to maximize sequence likelihood. Let N be number of training sequences and V is vocabulary size.
- Maximum Likelihood Estimation:

$$\theta^* = \arg\max_\theta \prod_{i=1}^{N} P(x_1^{(i)}, \ldots, x_T^{(i)} | \theta) \tag{10}$$

- Log-likelihood (easier to optimize):

$$\theta^* = \arg\max_\theta \sum_{i=1}^{N} \sum_{t=1}^{T} \log P(x_t^{(i)} | x_1^{(i)}, \ldots, x_{t-1}^{(i)}, \theta) \tag{11}$$

- Equivalently, minimize negative log-likelihood:

$$\mathcal{L}(\theta) = -\sum_{i=1}^{N} \sum_{t=1}^{T} \log P(x_t^{(i)} | x_1^{(i)}, \ldots, x_{t-1}^{(i)}, \theta) \tag{12}$$

LOSS FUNCTION FOR SEQUENCES

- Per-timestep cross-entropy loss:

$$\ell_t = -\sum_{k=1}^{V} 1[x_t = k] \log P(x_t = k | x_1, \ldots, x_{t-1}) \tag{13}$$

- Total sequence loss:

$$\mathcal{L} = \sum_{t=1}^{T} \ell_t = \sum_{t=1}^{T} \log P(x_t | x_1, \ldots, x_{t-1}) \tag{14}$$

- Average per-token loss:

$$\bar{\mathcal{L}} = \frac{1}{T} \sum_{t=1}^{T} \ell_t \tag{15}$$

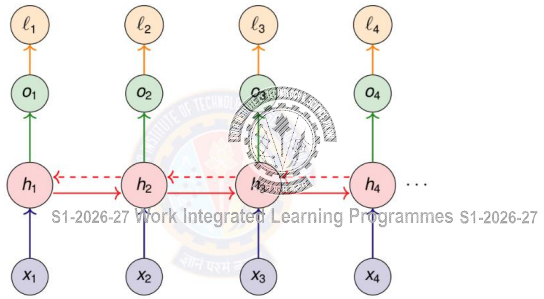
- This average loss is typically reported during training

Backpropagation Through Time (BPTT)

- Challenge: How to compute gradients in RNNs?
  - Parameters shared across all time steps
  - Hidden state depends on entire history
  - Need to backpropagate through time
- Key insight: Unroll RNN into feedforward network
  - Apply standard backpropagation
  - Sum gradients from all time steps (parameter sharing)
- BPTT: Backpropagation Through Time
  - Forward pass: compute outputs for all time steps
  - Backward pass: compute gradients from $t = T$ to $t = 1$
  - Gradients flow backward through time

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

BPTT: Computational Graph



- Orange: Forward pass computes loss at each time step
- Red dashed: Backward pass propagates gradients through time
- Gradients accumulate contributions from all future time steps

BPTT: Key Gradient Equations

- Loss function over sequence:

$$\mathcal{L} = \sum_{t=1}^{T} \ell_t \tag{16}$$

- Gradient at output layer:

$$\frac{\partial \mathcal{L}}{\partial o_t} = \frac{\partial \ell_t}{\partial o_t} \tag{17}$$

- Gradient at hidden state (recursive form):

$$\frac{\partial \mathcal{L}}{\partial h_t} = \frac{\partial \ell_t}{\partial h_t} + \frac{\partial \mathcal{L}}{\partial h_{t+1}} \frac{\partial h_{t+1}}{\partial h_t} \tag{18}$$

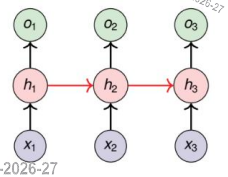
- Parameter gradients (sum over time):

$$\frac{\partial \mathcal{L}}{\partial W} = \sum_{t=1}^{T} \frac{\partial \mathcal{L}}{\partial h_t} \frac{\partial h_t}{\partial W} \tag{19}$$

BPTT: Numerical Example Setup

- Simple 3-timestep example
- Dimensions:
  - Input: $d = 2$
  - Hidden: $h = 2$
  - Output: $V = 2$
- Parameters:

$$W_{hh} = \begin{bmatrix} 0.5 & 0.3 \\ 0.2 & 0.4 \end{bmatrix} \quad W_{hx} = \begin{bmatrix} 0.4 & 0.3 \\ 0.2 & 0.5 \end{bmatrix} \quad W_{ho} = \begin{bmatrix} 0.6 & 0.4 \\ 0.3 & 0.7 \end{bmatrix}$$



- Biases: $b_h = b_o = \mathbf{0}$ (simplified)
- Activation: $\phi = \tanh$

BPTT: Forward Pass

- Input sequence: $x_1 = [1, 0]^T$, $x_2 = [0, 1]^T$, $x_3 = [1, 1]^T$
- Initial: $h_0 = [0, 0]^T$
- Time $t = 1$:

$$h_1 = \tanh(W_{hh}h_0 + W_{hx}x_1) = \tanh\left(\begin{bmatrix}0.4\\0.2\end{bmatrix}\right) = \begin{bmatrix}0.380\\0.197\end{bmatrix}$$

$$o_1 = W_{ho}h_1 = \begin{bmatrix}0.307\\0.252\end{bmatrix}$$

- Time $t = 2$:

$$h_2 = \tanh(W_{hh}h_1 + W_{hx}x_2) = \tanh\left(\begin{bmatrix}0.549\\0.576\end{bmatrix}\right) = \begin{bmatrix}0.499\\0.520\end{bmatrix}$$

$$o_2 = W_{ho}h_2 = \begin{bmatrix}0.507\\0.513\end{bmatrix}$$

- Time $t = 3$:

$$h_3 = \tanh(W_{hh}h_2 + W_{hx}x_3) = \tanh\left(\begin{bmatrix}1.105\\0.979\end{bmatrix}\right) = \begin{bmatrix}0.803\\0.754\end{bmatrix}$$

$$o_3 = W_{ho}h_3 = \begin{bmatrix}0.784\\0.769\end{bmatrix}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

BPTT: Backward Pass

- Assume target outputs: $y_1 = [1, 0]^T$, $y_2 = [0, 1]^T$, $y_3 = [1, 0]^T$
- Softmax probabilities computed from o_t
- Output gradients (softmax + cross-entropy):

$$\frac{\partial \ell_3}{\partial o_3} = [\text{softmax}(o_3) - y_3] = \begin{bmatrix}-0.496\\0.496\end{bmatrix}$$

- Hidden state gradient at $t = 3$:

$$\frac{\partial \mathcal{L}}{\partial h_3} = W_{ho}^T \frac{\partial \ell_3}{\partial o_3} = \begin{bmatrix}-0.149\\0.149\end{bmatrix}$$

- Hidden state gradient at $t = 2$ (recursive):

$$\frac{\partial \mathcal{L}}{\partial h_2} = \frac{\partial \ell_2}{\partial h_2} + \frac{\partial \mathcal{L}}{\partial h_3}\frac{\partial h_3}{\partial h_2}$$

where $\frac{\partial h_3}{\partial h_2} = \text{diag}(1 - h_3^2) \cdot W_{hh}$ (chain rule with tanh derivative)

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

BPTT: Parameter Gradients

- Gradient for W_{ho} (accumulated over time):

$$\frac{\partial \mathcal{L}}{\partial W_{ho}} = \sum_{t=1}^{3} \frac{\partial \ell_t}{\partial o_t} h_t^T \tag{20}$$

- Gradient for W_{hh} (accumulated over time):

$$\frac{\partial \mathcal{L}}{\partial W_{hh}} = \sum_{t=1}^{3} \frac{\partial \mathcal{L}}{\partial h_t} \cdot \text{diag}(1 - h_t^2) \cdot h_{t-1}^T \tag{21}$$

- Gradient for W_{hx}:

$$\frac{\partial \mathcal{L}}{\partial W_{hx}} = \sum_{t=1}^{3} \frac{\partial \mathcal{L}}{\partial h_t} \cdot \text{diag}(1 - h_t^2) \cdot x_t^T \tag{22}$$

- All parameter gradients are sums over time steps
- Once computed, use gradient descent: $W \leftarrow W - \eta \frac{\partial \mathcal{L}}{\partial W}$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Vanishing Gradients: The Problem

- Gradient flow through time:

$$\frac{\partial \mathcal{L}}{\partial h_t} = \frac{\partial \mathcal{L}}{\partial h_{t+1}}\frac{\partial h_{t+1}}{\partial h_t} = \frac{\partial \mathcal{L}}{\partial h_{t+1}} \cdot \text{diag}(\phi'(z_t)) \cdot W_{hh} \tag{23}$$

- From $t = T$ to $t = 1$:

$$\frac{\partial \mathcal{L}}{\partial h_1} = \frac{\partial \mathcal{L}}{\partial h_T} \prod_{t=1}^{T-1} \text{diag}(\phi'(z_t)) \cdot W_{hh} \tag{24}$$

- Problem: Product of many matrices
  - If $\|W_{hh}\| < 1$ and $|\phi'| < 1$ (e.g., tanh): gradient vanishes
  - If $\|W_{hh}\| > 1$: gradient explodes
- Consequence:
  - Difficult to learn long-term dependencies
  - Early time steps receive negligible gradient signal

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Vanishing Gradients: Numerical Example

- Setup: $n = 3$, W_{hh} with $\lambda_{max} = 0.8$
- Assume: $|\phi'(z_t)| \approx 0.5$ at each step
- Gradient magnitude ratio across k time steps:

$$\text{After 5 steps}: \quad (0.8 \times 0.5)^5 = 0.4^5 = 0.01024$$
$$\text{After 10 steps}: \quad (0.8 \times 0.5)^{10} = 0.4^{10} = 0.0001$$
$$\text{After 20 steps}: \quad (0.8 \times 0.5)^{20} = 0.4^{20} \approx 10^{-8}$$

- Interpretation:
  - Gradient at $t = 1$ is 10^{-8} times gradient at $t = 20$
  - Early time steps receive almost no learning signal
  - Long-term dependencies cannot be learned

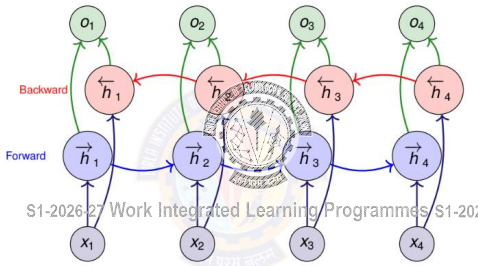
Why Vanishing Gradients are Problematic?

- **Learning difficulty:**
  - Parameters affecting early time steps barely update
  - Model cannot learn long-term patterns
  - Training becomes ineffective for long sequences
- **Example scenarios where this fails:**
  - "I grew up in France... [50 words]... I speak fluent _"
  - Long-range syntactic dependencies in language
  - Time series with seasonal patterns (yearly cycles)
- **Solutions:**
  - Better architectures: LSTM, GRU
  - Careful initialization
  - Skip connections

Bidirectional RNNs: Motivation

- **Limitation of standard RNN: Only uses past context**
  - Predicts based on $x_1, \ldots, x_{t-1}$
  - Cannot use future information
- **Many tasks benefit from full context:**
  - Sentiment analysis: "The movie was not _" (need to see "good" or "bad")
  - Named entity recognition: "I met Paris in Paris" (person vs city - need full context)
  - Speech recognition: Future phonemes help disambiguate current sound
  - Machine translation: Full sentence context improves translation
- **Key idea: Process sequence in both directions**
  - Forward RNN: left-to-right
  - Backward RNN: right-to-left
  - Combine information from both

Bidirectional RNN Architecture



- Blue: Forward RNN processes $x_1 \rightarrow x_T$
- Red: Backward RNN processes $x_T \rightarrow x_1$
- Green: Output uses both forward and backward states

Bidirectional RNN: Mathematical Formula

- Forward hidden states:

$$\overrightarrow{h}_t = \phi(W_{\overrightarrow{hh}} \overrightarrow{h}_{t-1} + W_{\overrightarrow{hx}} x_t + b_{\overrightarrow{h}}) \quad (25)$$

- Backward hidden states:

$$\overleftarrow{h}_t = \phi(W_{\overleftarrow{hh}} \overleftarrow{h}_{t+1} + W_{\overleftarrow{hx}} x_t + b_{\overleftarrow{h}}) \quad (26)$$

- Combine both directions:

$$h_t = [\overrightarrow{h}_t; \overleftarrow{h}_t] \quad (27)$$

where $[;]$ denotes concatenation.

- Output:

$$o_t = W_{ho} h_t + b_o = W_{ho}[\overrightarrow{h}_t; \overleftarrow{h}_t] + b_o \quad (28)$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Bidirectional RNN: Parameter Counting

- Given:
  - Input dimension: d
  - Forward hidden: h_f
  - Backward hidden: h_b
  - Output: V
- Forward RNN:

$W_{\overrightarrow{hx}} : h_f \times d$
$W_{\overrightarrow{hh}} : h_f \times h_f$
$b_{\overrightarrow{h}} : h_f$

- Backward RNN:

$W_{\overleftarrow{hx}} : h_b \times d$
$W_{\overleftarrow{hh}} : h_b \times h_b$
$b_{\overleftarrow{h}} : h_b$

- Output layer:

$W_{ho} : V \times (h_f + h_b)$
$b_o : V$

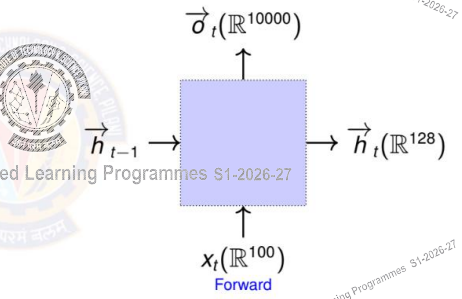
Total Parameters: $h_f^2 + h_b^2 + d(h_f + h_b) + V(h_f + h_b) + h_f + h_b + V$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Example Bidirectional RNN: Parameter Counting

- Given:
  - Input dimension: $d = 100$
  - Forward hidden: $h_f = 128$
  - Backward hidden: $h_b = 128$
  - Output: $V = 10{,}000$
- Forward RNN:

$W_{\overrightarrow{hx}} : 128 \times 100 = 12{,}800$
$W_{\overrightarrow{hh}} : 128 \times 128 = 16{,}384$
$b_{\overrightarrow{h}} : 128$
Subtotal : 29,312

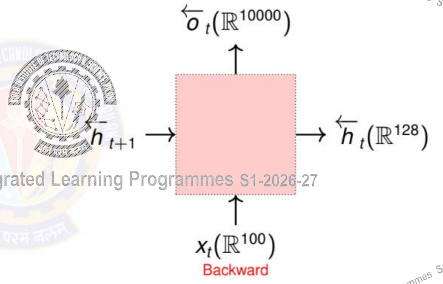


S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Example Bidirectional RNN: Parameter Counting

- Backward RNN:

$W_{\overleftarrow{hx}} : 128 \times 100 = 12{,}800$
$W_{\overleftarrow{hh}} : 128 \times 128 = 16{,}384$
$b_{\overleftarrow{h}} : 128$
Subtotal : 29,312



- Output layer: $W_{ho} : 10{,}000 \times 256 = 2{,}560{,}000$, $b_o : 10{,}000$
- Total: $29{,}312 + 29{,}312 + 2{,}570{,}000 = 2{,}628{,}624$ parameters

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

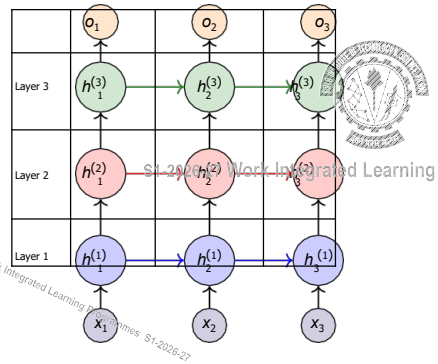
When to Use Bidirectional RNNs

- **Use bidirectional RNNs when:**
  - Entire sequence available before prediction
  - Task benefits from future context
  - Examples: text classification, named entity recognition, machine translation (encoder)
- **Do NOT use when:**
  - Real-time/online prediction required
  - Causal modeling (future cannot influence past)
  - Examples: language modeling, real-time speech recognition, time series forecasting
- **Key limitation: Must process entire sequence**
  - Cannot make predictions until sequence complete
  - Higher computational cost (two RNNs)
  - Not suitable for streaming applications

Deep (Stacked) RNNs: Motivation

- Why stack RNN layers?
  - Single RNN layer: limited representational capacity
  - Deep networks learn hierarchical features
  - Lower layers: simple patterns
  - Higher layers: complex, abstract patterns
- Examples of hierarchical features:
  - Text: characters → words → phrases → sentences
  - Speech: phonemes → syllables → words
  - Time series: short-term → medium-term → long-term trends
- Empirical observation:
  - Deeper RNNs often outperform wider single-layer RNNs
  - Typical depth: 2-4 layers (diminishing returns beyond this)

Deep RNN Architecture



Each layer processes output of previous layer

Horizontal arrows: recurrent connections within layer

Vertical arrows: feed-forward connections between layers

Deep RNN: Mathematical Formulation

- Layer ℓ hidden state update:

$$h_t^{(\ell)} = \phi(W_{hh}^{(\ell)} h_{t-1}^{(\ell)} + W_{hx}^{(\ell)} h_t^{(\ell-1)} + b_h^{(\ell)}) \tag{29}$$

where:
  - $h_t^{(\ell)} \in \mathbb{R}^{h_\ell}$: hidden state at layer ℓ, time t
  - $W_{hh}^{(\ell)} \in \mathbb{R}^{h_\ell \times h_\ell}$: recurrent weights for layer ℓ
  - $W_{hx}^{(\ell)} \in \mathbb{R}^{h_\ell \times h_{\ell-1}}$: input weights from layer $\ell - 1$
  - $h_t^{(0)} = x_t$: input is layer 0
- Output from top layer L:

$$o_t = W_{ho} h_t^{(L)} + b_o \tag{30}$$

- Each layer learns different level of abstraction

Deep RNN: Parameter Counting

- For L layers with hidden dimensions $h_1, h_2, \ldots, h_L$:
- Layer 1:

  $W_{hx}^{(1)} : h_1 \times d, \quad W_{hh}^{(1)} : h_1 \times h_1, \quad b_h^{(1)} : h_1 \qquad \text{Total Layer 1} : h_1 d + h_1^2 + h_1$
- Layer ℓ ($2 \le \ell \le L$):

  $W_{hx}^{(\ell)} : h_\ell \times h_{\ell-1}, \quad W_{hh}^{(\ell)} : h_\ell \times h_\ell, \quad b_h^{(\ell)} : h_\ell \qquad \text{Total / layer} : h_\ell h_{\ell-1} + h_\ell^2 + h_\ell$
- Output layer:

  $W_{ho} : V \times h_L, \quad b_o : V \qquad \text{Total output} : V \times h_L + b_o$
- Total parameters:

$$h_1 d + \sum_{\ell=1}^{L} (h_\ell^2 + h_\ell) + \sum_{\ell=2}^{L} h_\ell h_{\ell-1} + V h_L + V$$

Deep RNN: Numerical Example Setup

- Architecture: 2-layer RNN
  - Input dimension: $d = 2$
  - Layer 1 hidden: $h_1 = 2$
  - Layer 2 hidden: $h_2 = 2$
  - Output: $V = 2$
- Sequence: $x_1 = [1, 0]^T$, $x_2 = [0, 1]^T$
- Parameters (simplified):

$$W_{hx}^{(1)} = \begin{bmatrix} 0.5 & 0.2 \\ 0.3 & 0.4 \end{bmatrix}, \quad W_{hh}^{(1)} = \begin{bmatrix} 0.4 & 0.2 \\ 0.1 & 0.3 \end{bmatrix}$$

$$W_{hx}^{(2)} = \begin{bmatrix} 0.3 & 0.5 \\ 0.4 & 0.2 \end{bmatrix}, \quad W_{hh}^{(2)} = \begin{bmatrix} 0.3 & 0.1 \\ 0.2 & 0.4 \end{bmatrix}$$

- Biases: $\mathbf{0}$ (simplified), Activation: $\phi = \tanh$
- Initial states: $h_0^{(1)} = h_0^{(2)} = [0, 0]^T$

Deep RNN: Forward Pass at $t = 1$

- Layer 1 at $t = 1$:

$$\begin{aligned} h_1^{(1)} &= \tanh(W_{hh}^{(1)} h_0^{(1)} + W_{hx}^{(1)} x_1) \\ &= \tanh\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0.5 & 0.2 \\ 0.3 & 0.4 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \end{bmatrix} \right) = \begin{bmatrix} 0.462 \\ 0.291 \end{bmatrix} \end{aligned}$$

- Layer 2 at $t = 1$ (uses $h_1^{(1)}$ as input):

$$\begin{aligned} h_1^{(2)} &= \tanh(W_{hh}^{(2)} h_0^{(2)} + W_{hx}^{(2)} h_1^{(1)}) \\ &= \tanh\left(\begin{bmatrix} 0 \\ 0 \end{bmatrix} + \begin{bmatrix} 0.3 & 0.5 \\ 0.4 & 0.2 \end{bmatrix} \begin{bmatrix} 0.462 \\ 0.291 \end{bmatrix} \right) = \begin{bmatrix} 0.277 \\ 0.238 \end{bmatrix} \end{aligned}$$

Deep RNN: Forward Pass at $t = 2$

- Layer 1 at $t = 2$:

$$\begin{aligned} h_2^{(1)} &= \tanh(W_{hh}^{(1)} h_1^{(1)} + W_{hx}^{(1)} x_2) \\ &= \tanh\left(\begin{bmatrix} 0.4 & 0.2 \\ 0.1 & 0.3 \end{bmatrix} \begin{bmatrix} 0.462 \\ 0.291 \end{bmatrix} + \begin{bmatrix} 0.5 & 0.2 \\ 0.3 & 0.4 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \end{bmatrix} \right) = \begin{bmatrix} 0.417 \\ 0.451 \end{bmatrix} \end{aligned}$$

- Layer 2 at $t = 2$:

$$\begin{aligned} h_2^{(2)} &= \tanh(W_{hh}^{(2)} h_1^{(2)} + W_{hx}^{(2)} h_2^{(1)}) \\ &= \tanh\left(\begin{bmatrix} 0.3 & 0.1 \\ 0.2 & 0.4 \end{bmatrix} \begin{bmatrix} 0.277 \\ 0.238 \end{bmatrix} + \begin{bmatrix} 0.3 & 0.5 \\ 0.4 & 0.2 \end{bmatrix} \begin{bmatrix} 0.417 \\ 0.451 \end{bmatrix} \right) = \begin{bmatrix} 0.408 \\ 0.408 \end{bmatrix} \end{aligned}$$

Deep RNN: Gradient Flow Considerations

- Gradient must flow through:
  - Time (horizontal connections within each layer)
  - Depth (vertical connections between layers)
- Compound vanishing gradient problem:
  - Gradients vanish in time dimension (as in single-layer RNN)
  - Additional vanishing through depth
  - Problem is more severe than single-layer RNN
- Mitigation strategies:
  - Use LSTM/GRU cells instead of vanilla RNN (upcoming)
  - Skip connections between layers (residual connections)
  - Careful initialization
  - Layer normalization

Practical limit: Typically 2-4 layers

- Deeper networks harder to train
- Diminishing returns beyond 4 layers

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

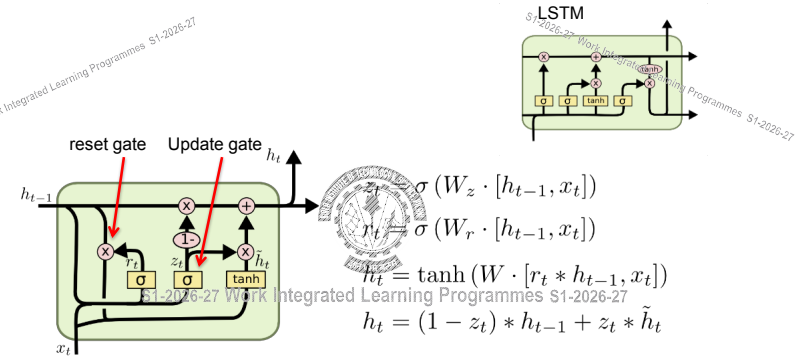
GRU: Motivation

- Vanilla RNN problems:
  - Vanishing gradients prevent learning long-term dependencies
  - All information in hidden state weighted equally
  - No mechanism to selectively remember or forget
- Key insight: Use gates to control information flow
  - Gates: Neural network layers with sigmoid activation $\sigma \in [0, 1]$
  - $\sigma = 0$: Block information completely
  - $\sigma = 1$: Let all information pass through
  - $\sigma \in (0, 1)$: Partially allow information
- GRU advantages:
  - Learn what to remember and what to forget
  - Better gradient flow through time
  - Can capture long-term dependencies
  - Fewer parameters than LSTM

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

GRU – gated recurrent unit

(more compression)



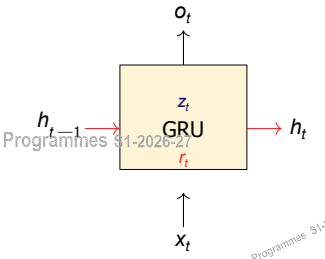
- It combines the forget and input into a single update gate.
- It also merges the cell state and hidden state.
- This is simpler than LSTM. There are many other variants too.

X,*: element-wise multiply

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

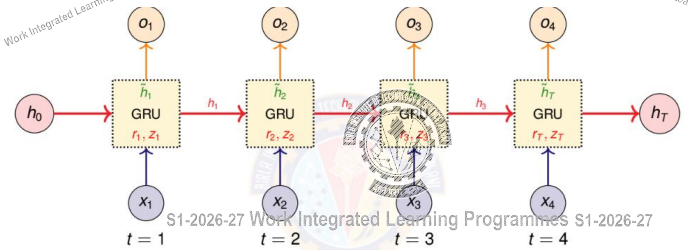
GRU: Architecture Overview

- Two gates:
  - Reset gate r_t: Controls how much past information to forget
  - Update gate z_t: Controls how much to update hidden state
- Candidate hidden state $\tilde{h}_t$: Proposed new content
- Final hidden state h_t: Interpolation between old and new
- All gates learned from data



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

GRU: Unrolled Architecture Across Time



- Each GRU cell contains: reset gate r_t, update gate z_t, candidate $\tilde{h}_t$
- Hidden state h_t flows through time (recurrent connections)
- Input x_t at each time step
- Output o_t computed from hidden state
- Same GRU parameters shared across all time steps

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

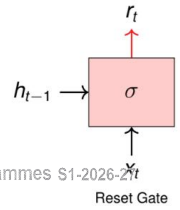
GRU: Reset Gate

- Reset gate equation:

$$r_t = \sigma(W_r x_t + U_r h_{t-1} + b_r) \quad (31)$$

where σ is sigmoid activation.

- Purpose: Decides how much past information to forget
  - $r_t \approx 0$: Ignore previous hidden state (reset memory)
  - $r_t \approx 1$: Keep all previous information
  - Element-wise operation: different dimensions reset differently
- Intuition: When starting new topic/context, reset gate can "forget" irrelevant past



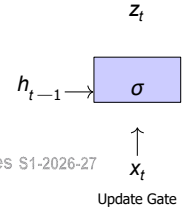
S1-2026-27 Work Integrated Learning Programmes S1-2026-27

GRU: Update Gate

- Update gate equation:

$$z_t = \sigma(W_z x_t + U_z h_{t-1} + b_z) \quad (32)$$

- Purpose: Decides how much to update hidden state
  - $z_t \approx 0$: Keep old hidden state h_{t-1} (don't update)
  - $z_t \approx 1$: Replace with new candidate $\tilde{h}_t$ (full update)
  - Acts like a "memory gate"
- Intuition: For long-term dependencies, keep z_t small to preserve information



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

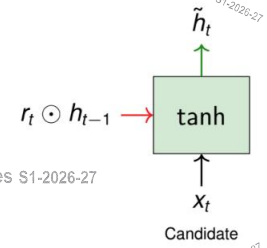
GRU: Candidate Hidden State

- Candidate hidden state equation:

$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1}) + b_h) \quad (33)$$

where $\odot$ is element-wise multiplication.

- Purpose: Compute new candidate content
  - Similar to vanilla RNN hidden state update
  - But uses reset gate: $r_t \odot h_{t-1}$
  - If $r_t \approx 0$: ignore past, compute from x_t only
  - If $r_t \approx 1$: use full past information
- Activation: tanh gives range $[-1, 1]$

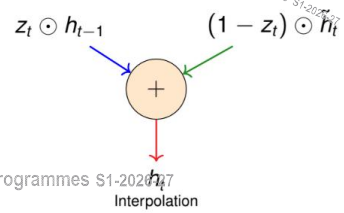


S1-2026-27 Work Integrated Learning Programmes S1-2026-27

GRU: Final Hidden State Update

- Final hidden state equation:

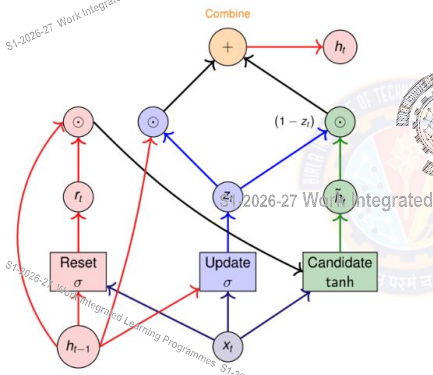
$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t \quad (34)$$



- Interpretation: Linear interpolation
  - If $z_t = 0$: $h_t = \tilde{h}_t$ (use new candidate)
  - If $z_t = 1$: $h_t = h_{t-1}$ (keep old state)
  - If $z_t = 0.5$: $h_t = 0.5h_{t-1} + 0.5\tilde{h}_t$ (blend both)
- Key property: Update gate controls how much information flows from past
- This helps with gradient flow (no vanishing!)

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

GRU: Computational Graph



- Reset: $r_t \odot h_{t-1}$ for candidate
- Update: Controls interpolation
- Candidate: $\tilde{h}_t$ with reset past
- Final:

$$h_t = z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

GRU: Numerical Example - Setup

- Dimensions:
  - Input: $d = 2$
  - Hidden: $h = 2$
- Parameters (simplified, small values):

$$W_r = \begin{bmatrix} 0.3 & 0.2 \\ 0.1 & 0.4 \end{bmatrix}, \quad U_r = \begin{bmatrix} 0.5 & 0.1 \\ 0.2 & 0.3 \end{bmatrix}, \quad b_r = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$W_z = \begin{bmatrix} 0.4 & 0.3 \\ 0.2 & 0.5 \end{bmatrix}, \quad U_z = \begin{bmatrix} 0.3 & 0.2 \\ 0.1 & 0.4 \end{bmatrix}, \quad b_z = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$$W_h = \begin{bmatrix} 0.5 & 0.3 \\ 0.2 & 0.4 \end{bmatrix}, \quad U_h = \begin{bmatrix} 0.4 & 0.2 \\ 0.1 & 0.3 \end{bmatrix}, \quad b_h = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

- Input: $x_t = [1.0, 0.5]^T$, Previous: $h_{t-1} = [0.6, 0.4]^T$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

GRU: Numerical Example - Gates Computation

- Step 1: Reset gate

$$\begin{aligned} r_t &= \sigma(W_r x_t + U_r h_{t-1}) \\ &= \sigma\left(\begin{bmatrix} 0.3 & 0.2 \\ 0.1 & 0.4 \end{bmatrix}\begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix} + \begin{bmatrix} 0.5 & 0.1 \\ 0.2 & 0.3 \end{bmatrix}\begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}\right) \\ &= \sigma\left(\begin{bmatrix} 0.4 \\ 0.3 \end{bmatrix} + \begin{bmatrix} 0.34 \\ 0.24 \end{bmatrix}\right) = \sigma\left(\begin{bmatrix} 0.74 \\ 0.54 \end{bmatrix}\right) = \begin{bmatrix} 0.677 \\ 0.632 \end{bmatrix} \end{aligned}$$

- Step 2: Update gate

$$\begin{aligned} z_t &= \sigma(W_z x_t + U_z h_{t-1}) \\ &= \sigma\left(\begin{bmatrix} 0.4 & 0.3 \\ 0.2 & 0.5 \end{bmatrix}\begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix} + \begin{bmatrix} 0.3 & 0.2 \\ 0.1 & 0.4 \end{bmatrix}\begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix}\right) \\ &= \sigma\left(\begin{bmatrix} 0.55 \\ 0.45 \end{bmatrix} + \begin{bmatrix} 0.26 \\ 0.22 \end{bmatrix}\right) = \sigma\left(\begin{bmatrix} 0.81 \\ 0.67 \end{bmatrix}\right) = \begin{bmatrix} 0.692 \\ 0.662 \end{bmatrix} \end{aligned}$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

- Step 3: Reset previous hidden state

$$r_t \odot h_{t-1} = \begin{bmatrix} 0.677 \\ 0.632 \end{bmatrix} \odot \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix} = \begin{bmatrix} 0.406 \\ 0.253 \end{bmatrix}$$

- Step 4: Candidate hidden state

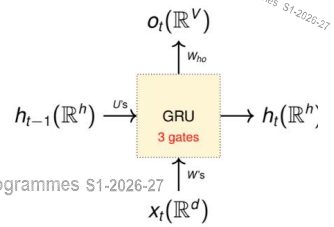
$$\begin{aligned} \tilde{h}_t &= \tanh(W_h x_t + U_h(r_t \odot h_{t-1})) \\ &= \tanh\left(\begin{bmatrix} 0.5 & 0.3 \\ 0.2 & 0.4 \end{bmatrix} \begin{bmatrix} 1.0 \\ 0.5 \end{bmatrix} + \begin{bmatrix} 0.4 & 0.2 \\ 0.1 & 0.3 \end{bmatrix} \begin{bmatrix} 0.406 \\ 0.253 \end{bmatrix} \right) \\ &= \tanh\left(\begin{bmatrix} 0.65 \\ 0.40 \end{bmatrix} + \begin{bmatrix} 0.213 \\ 0.116 \end{bmatrix} \right) = \begin{bmatrix} 0.696 \\ 0.475 \end{bmatrix} \end{aligned}$$

- Step 5: Final hidden state (interpolation)

$$\begin{aligned} h_t &= z_t \odot h_{t-1} + (1 - z_t) \odot \tilde{h}_t \\ &= \begin{bmatrix} 0.692 \\ 0.662 \end{bmatrix} \odot \begin{bmatrix} 0.6 \\ 0.4 \end{bmatrix} + \begin{bmatrix} 0.308 \\ 0.338 \end{bmatrix} \odot \begin{bmatrix} 0.696 \\ 0.475 \end{bmatrix} \\ &= \begin{bmatrix} 0.415 \\ 0.265 \end{bmatrix} + \begin{bmatrix} 0.214 \\ 0.161 \end{bmatrix} = \begin{bmatrix} 0.629 \\ 0.426 \end{bmatrix} \end{aligned}$$

- Interpretation:
  - Update gate $z_t \approx [0.69, 0.66]$: mostly keep old state
  - Reset gate $r_t \approx [0.68, 0.63]$: use most of past for candidate
  - Final state is weighted blend: $\sim 69\%$ old, $\sim 31\%$ new

- Given:
  - Input dimension: d
  - Hidden dimension: h
- Parameters:



$$\begin{aligned} \text{Reset gate} &: W_r : h \times d, \quad U_r : h \times h, \quad b_r : h \\ \text{Update gate} &: W_z : h \times d, \quad U_z : h \times h, \quad b_z : h \\ \text{Candidate} &: W_h : h \times d, \quad U_h : h \times h, \quad b_h : h \\ \text{Output layer} &: W_{ho} : V \times h, \quad b_o : V \\ \text{Total parameters} &: 3hd + 3h^2 + 3h + Vh + V \end{aligned}$$

Numerical example: $d = 100$, $h = 256$

$$\text{Total parameters} = 3 \times (256 \times 100 + 256^2 + 256) = 274{,}176$$

- Key gradient flow equation:

$$\frac{\partial h_t}{\partial h_{t-1}} = z_t + (1 - z_t) \odot \frac{\partial \tilde{h}_t}{\partial h_{t-1}} \tag{35}$$

- Direct path: z_t provides direct connection from h_{t-1} to h_t
  - No multiplication by weight matrix
  - No activation function derivative
  - Gradient flows directly when z_t is large
- Comparison with vanilla RNN:
  - Vanilla RNN: $\frac{\partial h_t}{\partial h_{t-1}} = \text{diag}(\phi'(z_t)) \cdot W_{hh}$
  - GRU: Has additive term z_t that bypasses nonlinearity
  - Similar to residual connections in deep networks
- Learning: Network learns when to open/close gates
  - For long-term dependencies: keep z_t large
  - Preserves gradient signal across many time steps

Long term dependency example

- Example:
- **LM task:** *When she tried to print her tickets, she found that the printer was out of toner. She went to the stationery store to buy more toner. It was very overpriced. After installing the toner into the printer, she finally printed her* ___
- Dependency between *"tickets"* on the 7th step and the target word *"tickets"* at the end.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

LSTM

- Length of a sentence is an issue with RNN
  - Vanishing and exploding gradient are issues
- LSTM is an extension for RNN , which basically extends the memory
- LSTM can read , write and delete information from its memory
- LSTN will have 3 gates
  - i.e. input gate , forget gate and output gate

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Long Short Term Memory - LSTM

- Building block of an LSTM is a memory cell, which replaces the hidden layer of standard RNNs
- LSTM unit consists of
  - Cell state (C_t) → Responsible for carrying long-term memory.
  - Hidden state (h_t) → Represents the output at each time step.
  - Three gates → Forget Gate, Input Gate, and Output Gate
- LSTM has many variants which include different number of gates and also different arrangement of gates
- Another equally popular variant of LSTM is Gated Recurrent Unit(GRU).

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

LSTM: MOTIVATION

GRU limitations:

- Single hidden state combines both long-term and short-term information
- Update gate controls both memory and output together
- Less explicit control over information flow

LSTM innovation: Separate memory from hidden state

- Cell state c_t: Long-term memory (information highway)
- Hidden state h_t: Short-term memory (working memory)
- More fine-grained control with 3 gates instead of 2

Key advantages:

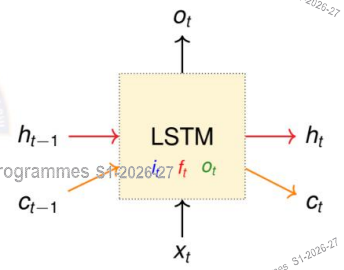
- Cell state provides uninterrupted gradient flow
- Forget gate can selectively remove information
- Output gate separates what to remember from what to output
- Better for very long sequences (100+ time steps)

Most widely used RNN variant in practice

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

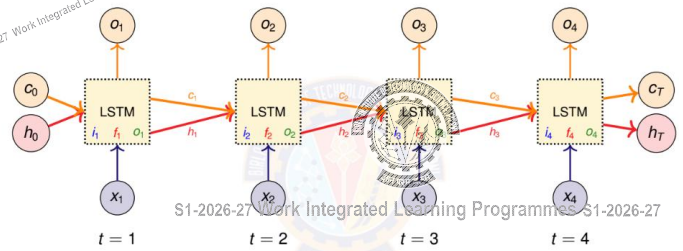
LSTM: Architecture Overview

- Three gates:
  - Forget gate f_t: What to remove from cell state
  - Input gate i_t: What new info to add to cell state
  - Output gate o_t: What to output from cell state
- Two states:
  - Cell state c_t: Long-term memory
  - Hidden state h_t: Short-term output
- Candidate $\tilde{c}_t$: New information to add



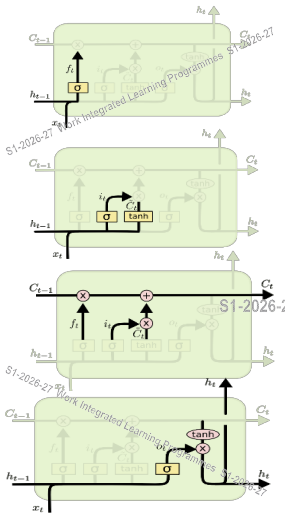
S1-2026-27 Work Integrated Learning Programmes S1-2026-27

LSTM: Unrolled Architecture Across Time



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

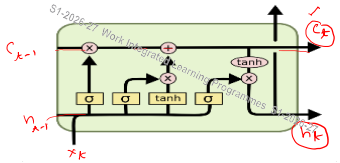
- Cell state c_t: Long-term memory highway (upper path)
- Hidden state h_t: Short-term working memory (lower path)
- Both states flow through time with same parameters



$$f_t = \sigma\left(W_f \cdot [h_{t-1}, x_t] \; + \; b_f\right)$$

$$i_t = \sigma\left(W_i \cdot [h_{t-1}, x_t] \; + \; b_i\right)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] \; + \; b_C)$$



i_t decides what component is to be updated.
C'_t provides change contents

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

Updating the cell state

$$o_t = \sigma\left(W_o \; [h_{t-1}, x_t] \; + \; b_o\right)$$
$$h_t = o_t * \tanh\left(C_t\right)$$

Decide what part of the cell state to output

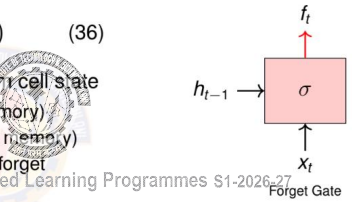
S1-2026-27 Work Integrated Learning Programmes S1-2026-27

LSTM: Forget Gate

- Forget gate equation:

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f) \qquad (36)$$

- Purpose: Decides what to remove from cell state
  - $f_t \approx 0$: Forget everything (erase memory)
  - $f_t \approx 1$: Remember everything (keep memory)
  - Element-wise: different dimensions forget differently



- Applied to cell state:

$$c_t = f_t \odot c_{t-1} + \ldots \qquad (37)$$

- Example: When topic changes, forget gate removes old context

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

LSTM: Input Gate and Candidate

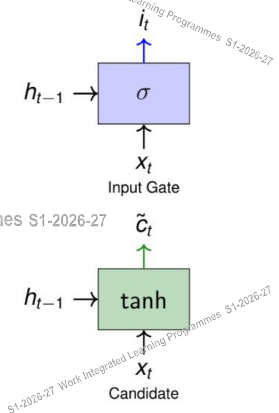
- Input gate equation:

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i) \quad (38)$$

- Candidate cell state:

$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c) \quad (39)$$

- Purpose:
  - $\tilde{c}_t$: Proposes new information to add
  - i_t: Decides how much of $\tilde{c}_t$ to add
  - $i_t \approx 0$: Ignore new information
  - $i_t \approx 1$: Fully accept new information
- Works together: $i_t \odot \tilde{c}_t$ added to cell state



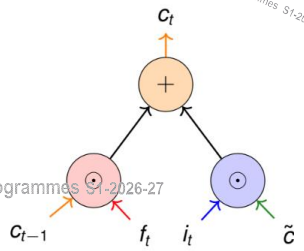
S1-2026-27 Work Integrated Learning Programmes S1-2026-27

LSTM: Cell State Update

- Cell state update equation:

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \quad (40)$$

- Two operations:
  - $f_t \odot c_{t-1}$: Forget selected parts of old memory
  - $i_t \odot \tilde{c}_t$: Add selected new information
- Key insight: Additive update
  - Direct path from c_{t-1} to c_t (when $f_t \approx 1$)
  - Gradient flows easily through addition
  - No multiplication by weight matrix
  - This is why LSTM solves vanishing gradients!



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

LSTM: Output Gate and Hidden State

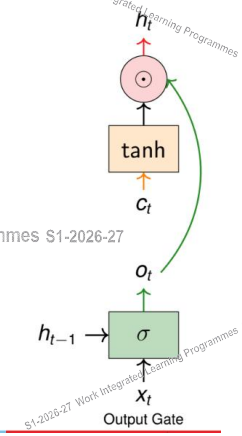
- Output gate equation:

$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o) \quad (41)$$

- Hidden state equation:

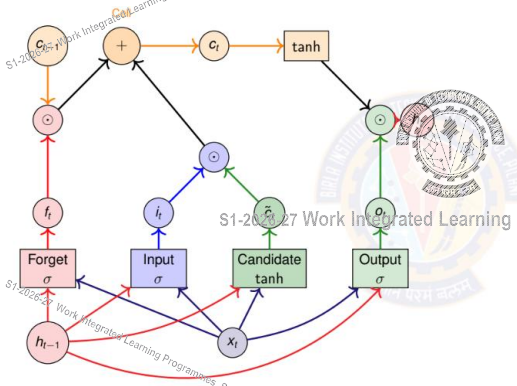
$$h_t = o_t \odot \tanh(c_t) \quad (42)$$

- Purpose:
  - $\tanh(c_t)$: Squash cell state to $[-1, 1]$
  - o_t: Decides what parts of cell state to output
  - Separates what to remember (c_t) from what to show (h_t)
- Key distinction: Can remember without outputting



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

LSTM: Computational Graph



- Forget: $f_t \odot c_{t-1}$
- Input: $i_t \odot \tilde{c}_t$
- Cell: $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$
- Output: $h_t = o_t \odot \tanh(c_t)$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

LSTM: Parameter Counting

- Given:
  - Input dimension: d
  - Hidden dimension: h
- Parameters:

Forget gate : $W_f : h \times d,\ U_f r : h \times h,\ b_f : h$

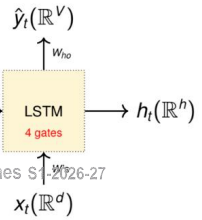
Input gate : $W_i : h \times d,\ U_i : h \times h,\ b_i : h$

Output gate : $W_o : h \times d,\ U_o : h \times h,\ b_o : h$

Candidate : $W_c : h \times d,\ U_c : h \times h,\ b_c : h$

Output layer : $W_{ho} : V \times h,\ b_o : V$

Total : $4hd + 4h^2 + 4h + Vh + V$



Numerical example: $d = 100,\ h = 256$

$$\text{Total} = 4 \times (256 \times 100 + 256^2 + 256) = 365{,}568 \text{ parameters}$$

LSTM: Why It Solves Vanishing Gradients

- Key gradient flow through cell state:

$$\frac{\partial c_t}{\partial c_{t-1}} = f_t + (\text{other terms}) \tag{43}$$

- Direct additive path:
  - Cell state update: $c_t = f_t \odot c_{t-1} + i_t \odot c_t$
  - Gradient: $\frac{\partial c_t}{\partial c_{t-1}} = f_t$ (when $f_t \approx 1$)
  - No weight matrix multiplication
  - No activation function derivative
  - Gradient flows uninterrupted when forget gate is open
- Learning: Network learns when to open/close gates
  - For long-term dependencies: keep f_t large, i_t small
  - Preserves both information and gradients

LSTM vs GRU: Comparison

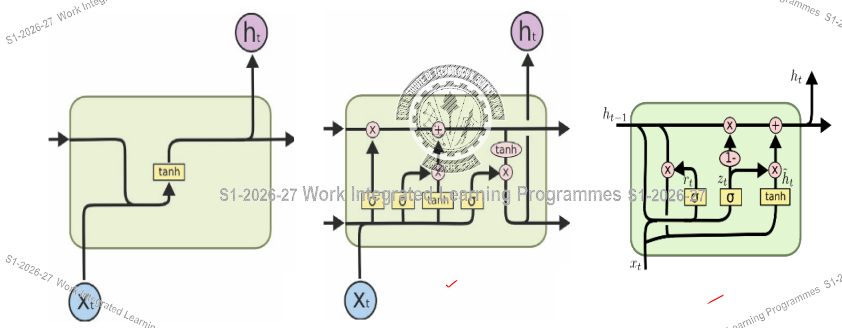
Property	GRU	LSTM
Number of gates	2 (reset, update)	3 (forget, input, output)
Memory types	1 (hidden state)	2 (cell + hidden)
Parameters	$3(hd + h^2 + h)$	$4(hd + h^2 + h)$
Relative params	Fewer (~ 25% less)	More
Training speed	Faster	Slower
Memory usage	Lower	Higher
Gradient flow	Good	Excellent
Long sequences	Good (50-100 steps)	Better (100+ steps)
Popularity	Increasing	Most widely used

When to use GRU: Smaller datasets, shorter sequences, faster training

When to use LSTM: Longer sequences, more data available, maximum performance needed

LSTM Vs GRU

	LSTM	GRU
Gates	3 (Forget, Input, Output)	2 (Reset, Update)
Cell State	Yes (separate from hidden state)	No (merged with hidden state)
Memory Retention	Better for long-term dependencies	Slightly less effective
Training Speed	Slower due to more computations	Faster (fewer parameters)
Best Use Case	Long sequences (e.g., machine translation, time-series)	Short sequences, quick training (e.g., speech recognition)



(a) RNN (b) LSTM (c)GRU

RNN ARCHITECTURES: COMPARISON

Property	Vanilla RNN	GRU	LSTM
Architecture			
Gates	0	2 (reset, update)	3 (forget, input, output)
Memory states	1 (hidden)	1 (hidden)	2 (cell + hidden)
Activation	tanh/ReLU	tanh + sigmoid	tanh + sigmoid
Parameters			
Formula	$2h^2 + 2hd + 2h + Vh$	$3h^2 + 3hd + 3h$	$4h^2 + 4hd + 4h$
Relative size	Smallest	Medium (~33% less)	Largest
Performance			
Gradient flow	Poor (vanishing)	Good	Excellent
Long sequences	Fails (10-20 steps)	Good (50-100)	Best (100+ steps)
Training speed	Fastest	Fast	Slower
Memory usage	Lowest	Medium	Highest
Practical Use			
When to use	Short sequences Small models	General purpose Resource constraints	Long sequences Maximum performance
Popularity	Rare (baseline)	Increasing	Most widely used

SUMMARY

- Use GRU, when dependency is short. Eg: Weather forecasting
- Use LSTM, when dependency is long. Eg: NLP Translation
- Use BRNN, dependency is in both direction. Eg: Stock prediction

Numerical on RNN

Consider the following snippet

Model = Sequential()

Model. add(SimpleRNN(3, input _ shape= (4,5),activation=activation[0]))

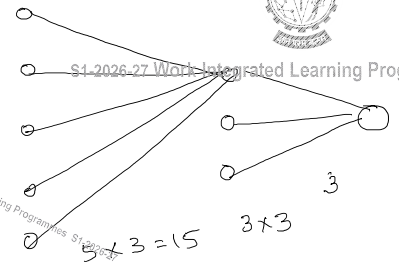
Model . add(Dense(1,activation= "Sigmoid"))

a) Draw the architecture

b) Calculate total number of Parameters in this

Discussion 1 _Continued

```
Model = Sequential()
Model. add(SimpleRNN(3, input _ shape= (4,5),activation=activation[0]))
Model . add(Dense(1,activation= "Sigmoid"))
```



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Numerical on RNN

➢Consider the following code snippet and draw the corresponding architecture. And also calculate number of parameters.

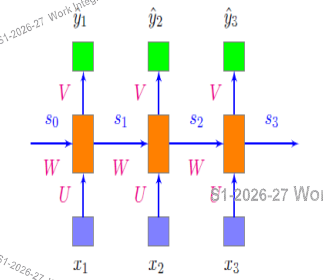
- Model = Sequential()
- Model. add(SimpleRNN(4, input _ shape= (4,6),activation=activation[0]))
- Model . add(Dense(1,activation= "Sigmoid")

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

- Total number of Parameters:
- RNN Layer: Wxh + Whh + bh = 4*6 + 4*4 + 4 = 44
- Output Layer: Wo = 1*4 and bo: 1 1*4 + 1 = 5
- Total number of parameters: 44+5 = 49

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Numerical on RNN



Compute the outputs in each timestep and the state after timestep = 3 for the Vanilla RNN given below. Assume the biases as zeros.

X = [1 1 1]

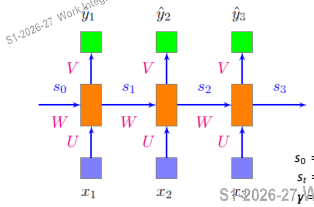
W = [0.2 0.3 0.8]

U = [0.5 0.6 0.2]

V = [0.4 0.2 0.1]

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Discussion 3



$s_0 = 0$

$s_t = \sigma(Ux_t + Ws_{t-1} + b)$

$y = g(Vs_t + c)$

Compute the outputs in each timestep and the state after timestep = 3 for the Vanilla RNN given below. Assume the biases as zeros.

$X = [1\ 1\ 0]^T$ $W = [0.2\ \ 0.3\ \ 0.8]^T$

$U = [0.5\ \ 0.6\ \ 0.2]^T$ $V = [0.4\ \ 0.2\ \ 0.1]^T$

(a) Compute the outputs in each timestep and the state after timestep=3 for the Vanilla RNN given below. Assume the biases as zeros. [5]

$$X = \begin{bmatrix} 1 & 1 & 0 \end{bmatrix}^\top$$

$$W = \begin{bmatrix} 0.2 & 0.3 & 0.8 \end{bmatrix}^\top$$

$$U = \begin{bmatrix} 0.5 & 0.6 & 0.2 \end{bmatrix}^\top$$

$$V = \begin{bmatrix} 0.4 & 0.2 & 0.1 \end{bmatrix}^\top$$

$s_t = \sigma(U x_t + W s_{t-1} + b)$

$\hat{y}_t = Relu(V s_t + c)$ Relu is assumed

$s_0 = 0 \quad b = 0 \quad c = 0$

$s_1 = \sigma(0.5 * 1 + 0.2 * 0 + 0) = 0.6$

$\hat{y}_1 = \max(0, 0.4 * 0.6 + 0) = 0.24$

$s_2 = \sigma(0.6 * 1 + 0.3 * 0.6 + 0) = 0.68$

$\hat{y}_2 = \max(0, 0.2 * 0.68 + 0) = 1.36$

$s_3 = \sigma(0.2 * 0 + 0.8 * 0.68 + 0) = 0.63$

$\hat{y}_3 = \max(0, 0.1 * 0.63 + 0) = 1.26$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Numerical on RNN

A company is trying to automate generating reports from videos of ultrasound scans. The videos of a beating heart of fetus at a particular section or slice. The report to be generated describes the heart condition and its location and any abnormalities detected. The videos are of 36 frames each. Each frame is 360 x 240 grayscale image,

a) Suggest suitable type of RNN to be used for this application and justify your selection.
b) Compute the depth of the RNN suggested
c) Should an LSTM or GRU or Bidirectional RNN should be used for this application. Justify your choice.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Numerical on RNN

A company is trying to automate generating reports from videos of ultrasound scans. The videos of a beating heart of fetus at a particular section or slice. The report to be generated describes the heart condition and its location and any abnormalities detected. The videos are of 36 frames each. Each frame is 360 x 240 grayscale image,

a) Suggest suitable type of RNN to be used for this application and justify your selection.
b) Compute the depth of the RNN suggested
c) Should an LSTM or GRU or Bidirectional RNN should be used for this application. Justify your choice.

a) Many to Many RNN has to be used. For each frame ,the position of the fetus and the condition has to output
b) Depth = 36,the number of frames per video
c) LSTM is better

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

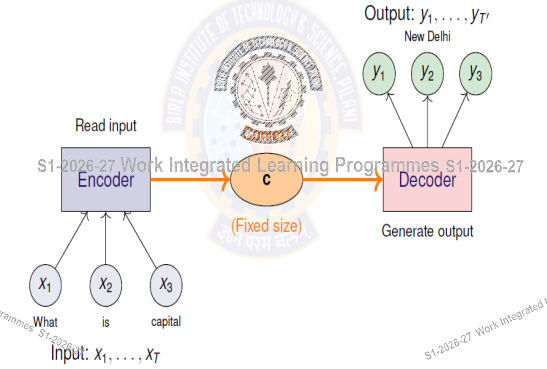
Attention Mechanism

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Encoder-Decoder Architecture

Key Idea: Two-stage process

1. Encoder: Read and compress entire input into context vector $\mathbf{c}$
2. Decoder: Generate output sequence from context, one token at a time



Flexibility: T (input length) can differ from T' (output length)

How Encoder-Decoder Works

- Phase 1: Encoding
  - Process input sequence with RNN:

$$h_t^{\text{enc}} = \phi(W_{hh}^{\text{enc}} h_{t-1}^{\text{enc}} + W_{hx}^{\text{enc}} x_t + b_h^{\text{enc}})$$

  - Extract context from final hidden state:

$$\mathbf{c} = h_T^{\text{enc}}$$

- Phase 2: Decoding
  - Initialize decoder with context:

$$h_0^{\text{dec}} = \mathbf{c}$$

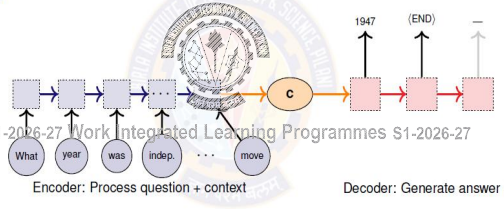
  - Generate output sequence autoregressively:

$$h_t^{\text{dec}} = \phi(W_{hh}^{\text{dec}} h_{t-1}^{\text{dec}} + W_{hy}^{\text{dec}} y_{t-1} + b_h^{\text{dec}})$$

  - Output probability distribution:

$$P(y_t | y_{<t}, \mathbf{c}) = \text{softmax}(W_o h_t^{\text{dec}} + b_o)$$

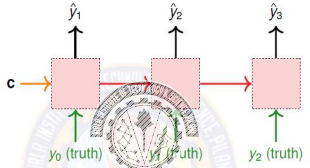
- Task: Answer question from context
  Context: "India gained independence in 1947. Gandhi led the movement."
  Question: "What year was independence?"
  Answer: "1947"



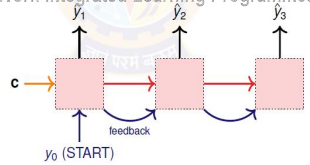
- Process:
  - Encoder reads entire input ($T \approx 15$ tokens)
  - Context $\mathbf{c}$ encodes: "answer is about year and independence"
  - Decoder generates: "1947" then ⟨END⟩ ($T' = 1$ meaningful token)

Training vs Inference

- Training (Teacher Forcing): Use ground truth as decoder input

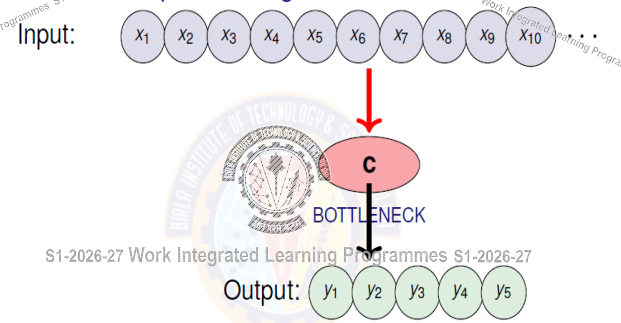


- Inference (Free Running): Use model's own predictions



- Exposure Bias: Decoder trained on ground truth but tested on own predictions!

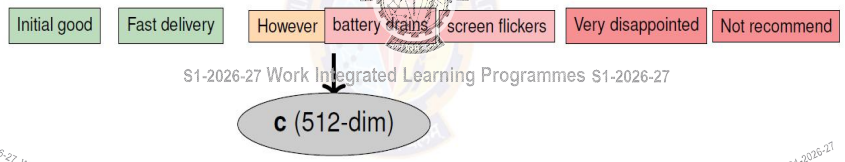
- All input information must pass through fixed-size vector **c**.



- Problems:
  1. Information loss: Long sequences → early tokens forgotten
  2. Fixed capacity: Vector size (e.g., 512-dim) limits information storage
  3. Uniform encoding: All input parts weighted equally, no selectivity

Bottleneck Impact: Concrete Example

- Task: Sentiment analysis of product review
- Input (30 words):
  "Ordered this laptop for work. Initial impressions were good. Fast delivery. However, after 2 weeks, battery drains in 2 hours. Screen flickers constantly. Very disappointed. Would not recommend."

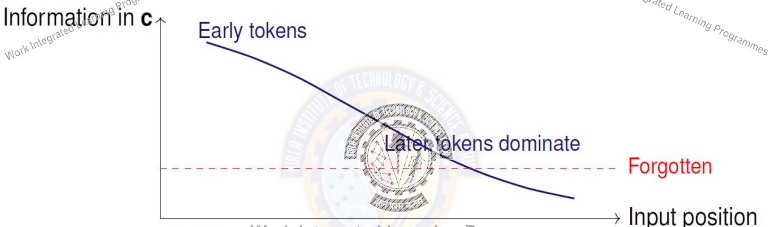


Early positive sentiment lost?
Key phrases "very disappointed" weakened?

- Issue: When generating "Negative", decoder needs to emphasize later parts, but **c** treats all parts equally!

Why Fixed Context Fails for Long Sequen

Visualization of information loss:



Observations from research (Sutskever et al., 2014):

- Performance degrades significantly for sequences $>$ 30 tokens
- First few words often lost by the time decoder generates output
- Model struggles with long-range dependencies

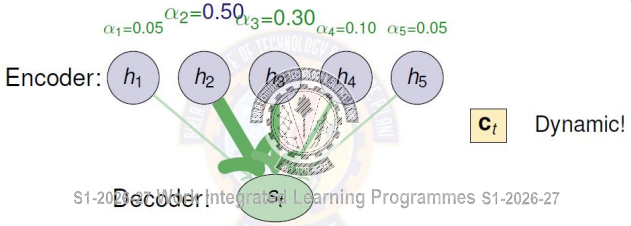
Question: How can we avoid this bottleneck?

Attention Is All You Need

Ashish Vaswani* Google Brain avaswani@google.com
Noam Shazeer* Google Brain noam@google.com
Niki Parmar* Google Research nikip@google.com
Jakob Uszkoreit* Google Research usz@google.com
Llion Jones* Google Research llion@google.com
Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu
Łukasz Kaiser* Google Brain lukaszkaiser@google.com
Illia Polosukhin* ‡ illia.polosukhin@gmail.com

The Solution: Attention Mechanisms

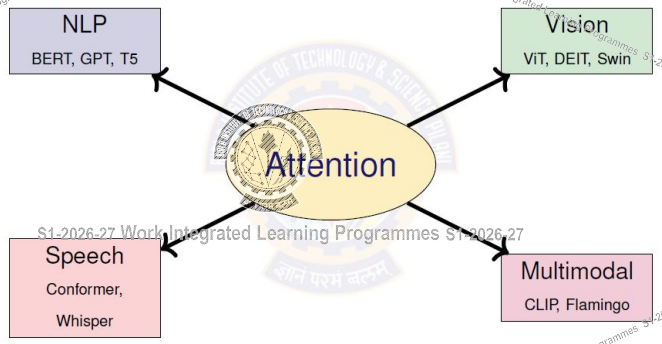
- Core Insight: Instead of single fixed context, let decoder selectively attend to relevant input parts at each step.



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

- Key Differences from Encoder-Decoder:
  - Before: Single context $\mathbf{c}$ for entire decoding
  - After: Different context $\mathbf{c}_t$ at each decoder step t
  - Weights α_i learned automatically from data
  - Focuses on relevant input positions dynamically

Attention has become the dominant mechanism across AI domains.

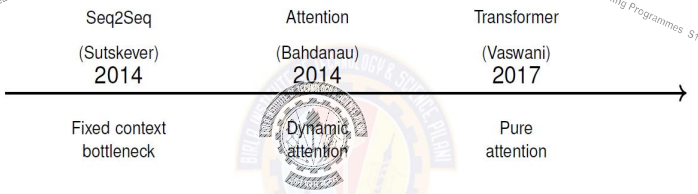


S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Understanding attention is essential for modern deep learning.

Historical Context: The Breakthrough

Timeline:



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Impact of Bahdanau et al. (2014):

- Introduced attention for neural machine translation
- Improved BLEU scores significantly on long sentences
- Decoder could "align" to relevant source words
- Laid foundation for Transformer architecture (2017)

Attention

Definition

Attention is a mechanism that computes a weighted sum of values based on the similarity between a query and keys.

- Components:
  - Query ($\mathbf{q}$): What we're looking for (decoder state)
  - Keys ($\{\mathbf{k}_i\}$): What's available to match (encoder states)
  - Values ($\{\mathbf{v}_i\}$): Actual content to retrieve (encoder states)
- Mathematical Form:

$$\text{Attention}(\mathbf{q}, \{\mathbf{k}_i, \mathbf{v}_i\}_{i=1}^n) = \sum_{i=1}^n \alpha_i \mathbf{v}_i$$

$$\text{Attention weights} \quad \alpha_i = \frac{\exp(\text{score}(\mathbf{q}, \mathbf{k}_i))}{\sum_{j=1}^n \exp(\text{score}(\mathbf{q}, \mathbf{k}_j))} \quad \text{(softmax normalization)}$$

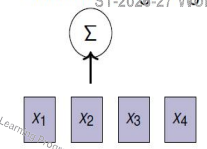
Core Concept: Weighted Combinations

Key Principle: Not all inputs contribute equally – attention learns relevance.

Without Attention:

$$\text{output} = f\left(\sum_{i=1}^{n} x_i\right)$$

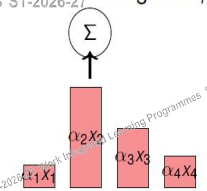
Uniform weighting



With Attention:

$$\text{output} = f\left(\sum_{i=1}^{n} \alpha_i x_i\right)$$

Learned weights α_i

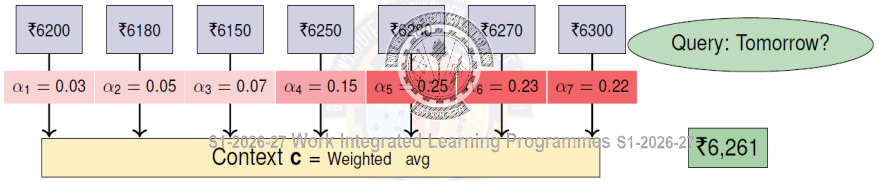


Constraint: Weights sum to 1: $\sum_{i=1}^{n} \alpha_i = 1$ (probability distribution)

Example 1: Stock Price Prediction

Task: Predict tomorrow's gold price (₹/gram) using past 7 days.

Input: Gold prices = [₹6,200, ₹6,180, ₹6,150, ₹6,250, ₹6,280, ₹6,270, ₹6,300]



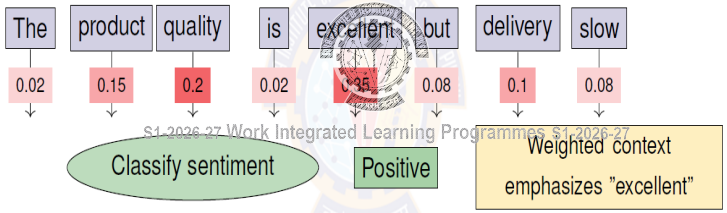
Observation: Recent days (5-7) receive higher attention weights α_i.

$$\begin{aligned}\mathbf{c} &\approx 0.03 \times 6200 + 0.05 \times 6180 + 0.07 \times 6150 + 0.15 \times 6250 + \\ &\quad 0.25 \times 6280 + 0.23 \times 6270 + 0.22 \times 6300 \\ &\approx 6261\end{aligned}$$

Example 2: Sentiment Analysis

Task: Classify review sentiment while identifying key phrases.

Input: "The product quality is excellent but delivery was very slow"

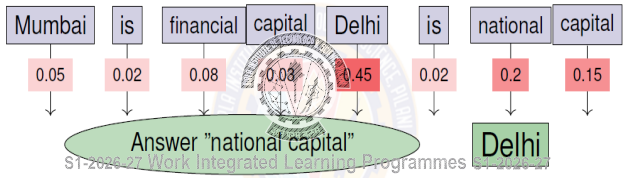


Key Insight: Model learns "excellent" ($\alpha_5 = 0.35$) is most important for sentiment, despite negative word "slow" present.

Example 3: Q& A with Attention

Context: "Mumbai is the financial capital. Delhi is the national capital."

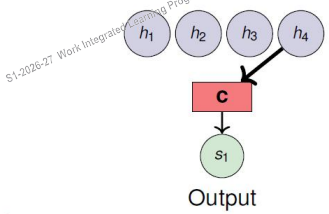
Question: "What is the national capital?"



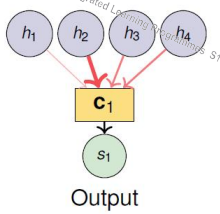
Attention weights show:

- Highest attention on "Delhi" ($\alpha_5 = 0.45$)
- Secondary attention on "national" ($\alpha_7 = 0.20$) and "capital" ($\alpha_4 = 0.15$)
- Model correctly ignores "financial capital"

Without Attention:



With Attention:



Issues:

- Fixed bottleneck $\mathbf{c}$
- Information loss
- Poor long sequences

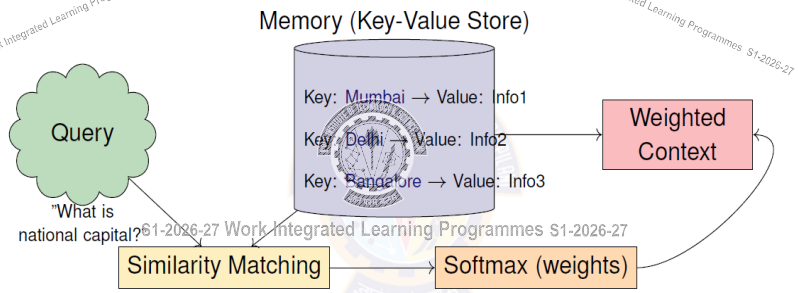
Benefits:

- Dynamic context $\mathbf{c}_t$
- Selective focus
- Handles long sequences

Key Difference: Context adapts to what decoder needs at each step!

Attention as Information Retrieval

Analogy: Attention works like a database lookup system.



Process:

1. Query searches memory via **key similarity**
2. Compute attention weights based on query-key match
3. Retrieve **values** weighted by attention

Query-Key-Value (QKV): Components

Three distinct roles in attention mechanism:

- Query ($\mathbf{q}$):
  - What information are we seeking?
  - Comes from decoder state: "I need to generate next word"
  - Example: Decoder state when generating "Delhi"
- Keys ($\{\mathbf{k}_i\}$):
  - What's available to match against?
  - Comes from encoder states: representations of input tokens
  - Example: Encoded representations of [Mumbai, is, financial, capital, Delhi, ...]
- Values ($\{\mathbf{v}_i\}$):
  - What actual content do we retrieve?
  - Also from encoder states (often same as keys)
  - Example: Contextual embeddings of input words

Intuition: Query searches Keys, retrieves matching Values.

QKV Paradigm: Detailed Mechanics

Step-by-step attention computation:

1. Compute compatibility scores between query and all keys:

$$e_i = \text{score}(\mathbf{q}, \mathbf{k}_i) \quad \text{for } i = 1, \ldots, n$$

2. Normalize with softmax to get attention weights:

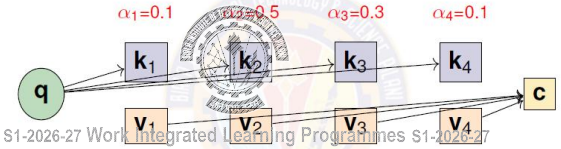
$$\alpha_i = \frac{\exp(e_i)}{\sum_{j=1}^{n} \exp(e_j)} \quad (\text{ensures } \sum_i \alpha_i = 1)$$

3. Compute weighted sum of values:

$$\mathbf{c} = \sum_{i=1}^{n} \alpha_i \mathbf{v}_i$$

QKV Paradigm: Visual

- Query $\mathbf{q}$: What information are we seeking? Decoder state
- Key $\mathbf{k}_i$: What's available to match against? Encoder state
- Value $\mathbf{v}_i$: What actual content do we retrieve? Encoder state



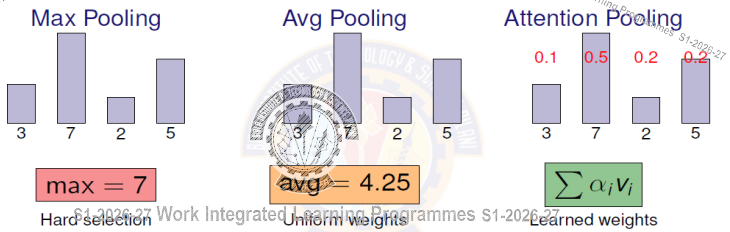
S1-2026-27 Work Integrated Learning Programmes S1-2026-27

$$\alpha_i = \mathit{softmax}(\text{score}(\mathbf{q}, \mathbf{k}_i)) \quad \text{for } i = 1, \ldots, n$$

$$\mathbf{c} = \sum_{i=1}^{n} \alpha_i \mathbf{v}_i$$

Attention as Weighted Pooling

Pooling Perspective: Attention is soft, adaptive pooling over sequence.



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Key Difference:

- Max/Avg pooling: Fixed, uniform aggregation
- Attention: Data-dependent, learned aggregation based on query

Dot-Product Attention

Simplest form: Measure similarity via inner product.

$$\text{score}(\mathbf{q}, \mathbf{k}_i) = \mathbf{q}^\top \mathbf{k}_i = \sum_{j=1}^{d} q_j k_{i,j}$$

$$\text{Attention}_i = \frac{\exp(\mathbf{q}^\top \mathbf{k}_i)}{\sum_{j=1}^{n} \exp(\mathbf{q}^\top \mathbf{k}_j)} = \text{softmax}(\mathbf{q}^\top \mathbf{K}^\top)_i$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

where $\mathbf{K} = [\mathbf{k}_1, \ldots, \mathbf{k}_n]^\top \in \mathbb{R}^{n \times d}$.

Example: Dot-Product Attention

Example: Stock price prediction

- Query: $\mathbf{q} = [0.8, 0.6]^\top$ (tomorrow's expected trend features)
- Keys: $\mathbf{k}_1 = [0.5, 0.3]^\top$, $\mathbf{k}_2 = [0.9, 0.7]^\top$, $\mathbf{k}_3 = [0.4, 0.2]^\top$
- Scores:

$$e_1 = 0.8(0.5) + 0.6(0.3) = 0.58$$

$$e_2 = 0.8(0.9) + 0.6(0.7) = 1.14 \quad \text{(highest!)}$$

$$e_3 = 0.8(0.4) + 0.6(0.2) = 0.44$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

- Weights: $\alpha_1 = 0.21$, $\alpha_2 = 0.62$, $\alpha_3 = 0.17$ (after softmax)
- Interpretation: Day 2 most aligned with query $\rightarrow$ highest attention.

Scaled Dot-Product Attention

Problem: For large dimension d, dot products grow large → softmax saturates.
Solution (Vaswani et al., 2017): Scale by $\sqrt{d_k}$ to normalize variance.

$$\text{score}(\mathbf{q}, \mathbf{k}) = \frac{\mathbf{q}^\top \mathbf{k}}{\sqrt{d_k}}$$

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

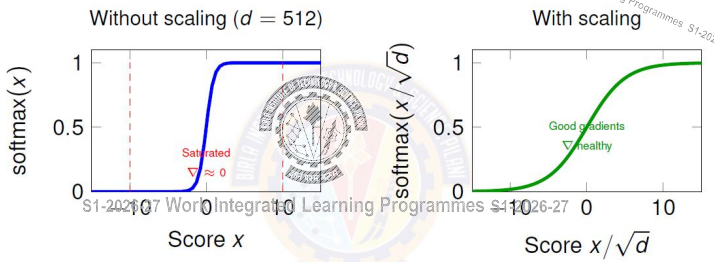
S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Benefits:

- Keeps scores in reasonable range regardless of dimension
  - Prevents vanishing gradients in softmax
  - Standard in all Transformer models

Why Scaling Matters: Gradient Flow

Effect of scaling on softmax gradients:



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Impact:

- Left: Large scores push softmax to extremes (0 or 1)
- Right: Scaled scores maintain gradient flow
- Critical for training deep networks (6-24+ layers)

Additive Attention (Bahdanau, 2014)

- Original attention mechanism for neural machine translation.

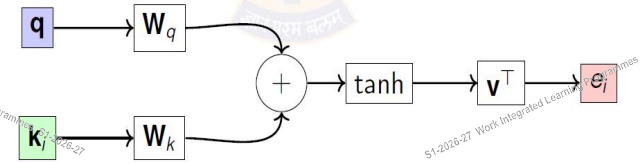
$$\text{score}(\mathbf{q}, \mathbf{k}_i) = \mathbf{v}^\top \tanh(\mathbf{W}_q\mathbf{q} + \mathbf{W}_k\mathbf{k}_i)$$

where:

- $\mathbf{W}_q \in \mathbb{R}^{h \times d_q}$: Query projection matrix
- $\mathbf{W}_k \in \mathbb{R}^{h \times d_k}$: Key projection matrix
- $\mathbf{v} \in \mathbb{R}^h$: Learnable weight vector
- h: Hidden dimension (hyperparameter, e.g., 256)

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

- Computation flow:



- Advantage: Can handle $d_q \neq d_k$ (different query/key dimensions).

Comparison: Dot-Product vs Additive

Aspect	Dot-Product (Scaled)	Additive (Bahdanau)
Formula	$\frac{\mathbf{q}^\top \mathbf{k}}{\sqrt{d_k}}$	$\mathbf{v}^\top \tanh(\mathbf{W}_q\mathbf{q} + \mathbf{W}_k\mathbf{k})$
Parameters	None	$\mathbf{W}_q, \mathbf{W}_k, \mathbf{v}$
Complexity	$O(d)$ per pair	$O(d \cdot h + h)$ per pair
Expressiveness	Limited to linear	Nonlinear (tanh)
Parallelization	Excellent (matrix mult)	Moderate
Dimension req.	Requires $d_q = d_k$	Works with $d_q \neq d_k$
Modern usage	Transformers	Early seq2seq

Historical note: Additive was better for RNN-based models (2014-2016), but scaled dot-product dominates in Transformer era (2017+).

Computational Complexity Analysis

For sequence length n, dimension d, and m queries:

Operation	Scaled Dot-Product	Additive
Score computation	$O(mnd)$	$O(mndh + mnh)$
Softmax	$O(mn)$	$O(mn)$
Weighted sum	$O(mnd)$	$O(mnd)$
Total	$\mathbf{O(mnd)}$	$\mathbf{O(mndh)}$

Matrix formulation (batch processing):

- Scaled dot-product: $\mathbf{QK}^\top \in \mathbb{R}^{m \times n}$ computed in one GPU operation
- Highly parallelizable on modern hardware (GPUs/TPUs)
- Memory efficient: $O(mn)$ for attention matrix

Practical impact: Dot-product is 3-5× faster in practice due to better hardware utilization.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Scoring Functions: Overview

The scoring function $\text{score}(\mathbf{q}, \mathbf{k})$ measures query-key compatibility.

Method	Score Function	Parameters
Dot Product	$\mathbf{q}^\top \mathbf{k}$	None
Scaled Dot Product	$\frac{\mathbf{q}^\top \mathbf{k}}{\sqrt{d_k}}$	None
Additive (Bahdanau)	$\mathbf{v}^\top \tanh(\mathbf{W}_q \mathbf{q} + \mathbf{W}_k \mathbf{k})$	$\mathbf{W}_q, \mathbf{W}_k, \mathbf{v}$
Multiplicative (Luong)	$\mathbf{q}^\top \mathbf{W} \mathbf{k}$	$\mathbf{W}$

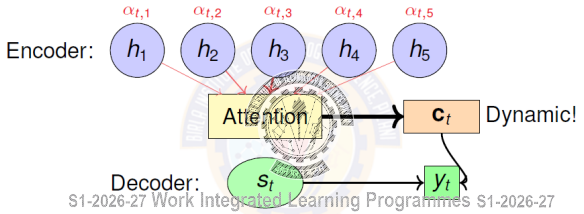
Design Trade-offs:

- Dot product: Fast (matrix ops), no parameters, requires $d_q = d_k$
- Scaled Dot Product : Prevents saturation for large dimensions (Transformer default)
- Additive: More expressive, flexible dimensions, more parameters

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Bahdanau Attention: The Breakthrough

Key Innovation (Bahdanau et al., 2014): Replace fixed context with dynamic attention.



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Context at time t:

$$\mathbf{c}_t = \sum_{i=1}^{T} \alpha_{t,i} \mathbf{h}_i$$

where $\alpha_{t,i}$ depends on current decoder state s_{t-1} and encoder state h_i.

Bahdanau Attention: Math

At each decoder time step t:

- Step 1: Compute alignment scores
- Step 2: Normalize to attention weights
- Step 3: Compute context vector
- Step 4: Update decoder state
- Step 5: Generate output distribution

$$e_{t,i} = \mathbf{v}^\top \tanh(\mathbf{W}_s s_{t-1} + \mathbf{W}_h h_i)$$

$$\alpha_{t,i} = \frac{\exp(e_{t,i})}{\sum_{j=1}^{T} \exp(e_{t,j})}$$

$$\mathbf{c}_t = \sum_{i=1}^{T} \alpha_{t,i} \mathbf{h}_i$$

$$s_t = \text{RNN}([y_{t-1}; \mathbf{c}_t], s_{t-1})$$

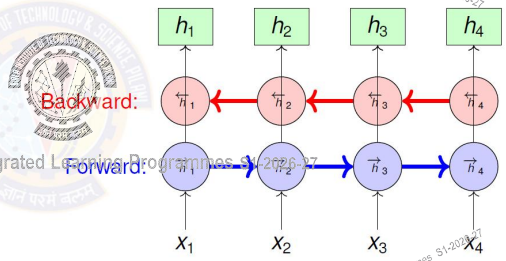
$$P(y_t | y_{<t}, \mathbf{x}) = \text{softmax}(\mathbf{W}_o [s_t; \mathbf{c}_t])$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Encoder with Attention: Bidirectional RN

Bahdanau used bidirectional RNN encoder to capture context from both directions.

$$\overrightarrow{h}_i = \text{RNN}_{\text{fwd}}(x_i, \overrightarrow{h}_{i-1})$$
$$\overleftarrow{h}_i = \text{RNN}_{\text{bwd}}(x_i, \overleftarrow{h}_{i+1})$$
$$h_i = [\overrightarrow{h}_i; \overleftarrow{h}_i]$$

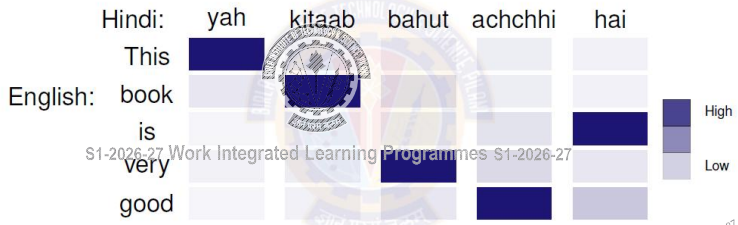


Benefit: h_i encodes both past and future context for position i.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Attention Alignment: Visualization

Example: Hindi to English translation
Source (Hindi): yah kitaab bahut achchhi hai
Target (English): This book is very good

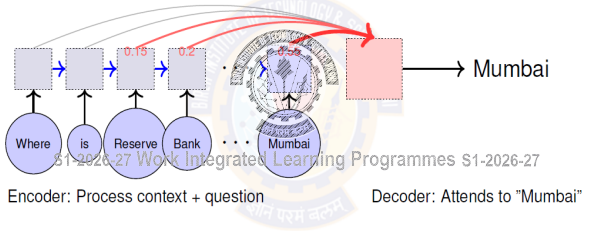


Observations:

- Strong diagonal pattern: word-level alignment
- "yah" ↔ "This": $\alpha = 0.85$
- "kitaab" ↔ "book": $\alpha = 0.82$

Example: Q& A with Attention

Context: "Mumbai is the financial capital. The Reserve Bank is located there."
Question: "Where is Reserve Bank?"
Answer: "Mumbai"



Attention interpretation:

- Decoder focuses on context mentioning "Reserve Bank" and "Mumbai"
- Highest weight ($\alpha = 0.55$) on "Mumbai" in context
- Successfully extracts answer from relevant part of input

Attention Types:

Type	Q Source	K,V Source	Use Case
Self-Attention	Same sequence	Same sequence	Intra-sequence relationships
Cross-Attention	Sequence A	Sequence B	Inter-sequence alignment (seq2seq)
Masked Attention	Same sequence	Same (causal mask)	Autoregressive generation
Multi-Head	Same (parallel)	Same (parallel)	Multiple relationship types

Key Distinctions:

- Source of Q, K, V: Where do query, key, value vectors come from?
- Masking: What positions can attend to what?
- Purpose: What relationships are being modeled?

Self-Attention: Definition and Purpose

- Self-attention: Query, key, and value all come from the *same* sequence.

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V$$

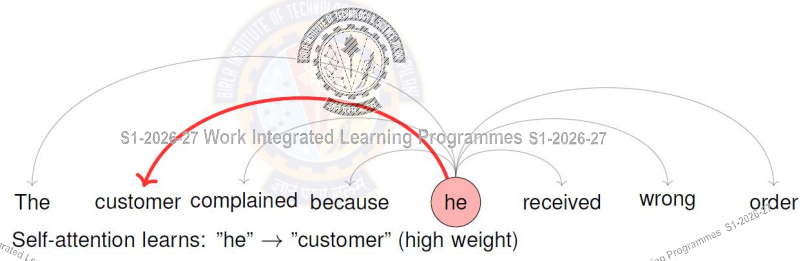
where $\mathbf{X} = [x_1, \ldots, x_n]^\top$ is the input sequence.

- Purpose: Model relationships *within* a sequence.
- Time Series Example: Stock price prediction using past 24 hours
  - Input: Gold prices over 24 hours: [₹6,120, ₹6,200, . . ., ₹6,280]
  - Self-attention allows each hour to attend to all other hours
  - Example: Hour 24 might attend strongly to hours 17, 10, 3 (7-hour periodicity)
  - Captures temporal patterns automatically
- Text Example: "The student didn't pass the exam because he was unprepared"
  - Self-attention helps "he" attend to "student"
  - Resolves coreference within the sentence
  - Captures long-range dependencies ("he" → "student")

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Self-Attention: Visualization

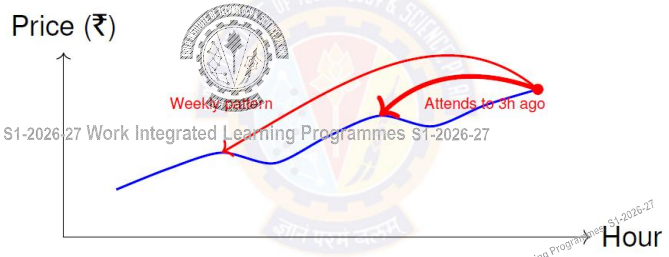
Text Example: Pronoun resolution with self-attention
Sentence: "The customer complained because he received wrong order"



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Self-Attention: Visualization

Time Series: Each time step attends to relevant past time steps



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Cross-Attention: Inter-Sequence Alignment

- Cross-attention: Query from one sequence, keys and values from *another*.

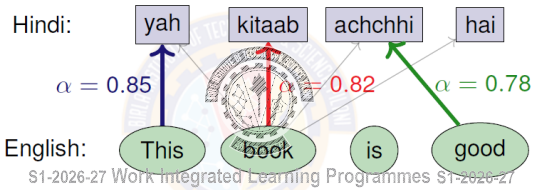
$$\mathbf{Q} = \mathbf{X}_A\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}_B\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}_B\mathbf{W}_V$$

- Time Series Example: Multivariate forecasting
  - Sequence A: Stock prices (query current state)
  - Sequence B: Economic indicators (keys/values: inflation, GDP, interest rates)
  - Cross-attention: Stock prices attend to relevant economic indicators
  - Example: Gold price queries inflation rate and USD exchange rate
- Text Example: Hindi → English translation
  - Sequence A (Decoder): English words being generated
  - Sequence B (Encoder): Hindi source sentence
  - When generating "book", decoder attends to Hindi "kitaab"
  - Example: Generating "yah kitaab achchhi hai" → "This book is good"
    - ★ "book" (EN) attends to "kitaab" (HI) with $\alpha \approx 0.85$
    - ★ "good" (EN) attends to "achchhi" (HI) with $\alpha \approx 0.78$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Cross-Attention: Visualization

- Machine Translation Example: Hindi to English



- Key: Decoder (English) dynamically attends to relevant parts of encoder (Hindi).
- This is exactly Bahdanau attention – the original cross-attention mechanism!

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Masked Attention: Causal/Autoregressive

- Masked attention: Self-attention with *causal constraint* – can only attend to past.

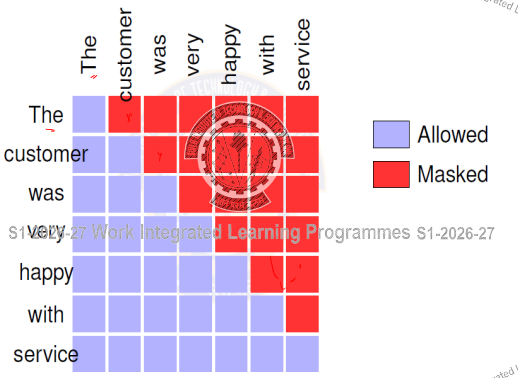
$$\text{Mask}_{ij} = \begin{cases} 0 & \text{if } j \leq i \quad \text{(allowed)} \\ -\infty & \text{if } j > i \quad \text{(masked)} \end{cases}$$

- Time Series Example: Stock price prediction (prevent future leakage)
  - Predicting price at time t using only times $1, \ldots, t-1$
  - Mask prevents attending to future times $t+1, t+2, \ldots$
  - Ensures no "cheating" during training
  - Example: Predict tomorrow's gold price without seeing tomorrow's actual price
- Text Example: GPT-style language modeling
  - Task: Predict next word given previous words. "The customer was very __"
  - Token at position 5 ("very") can only attend to positions 1-4
  - Cannot see future tokens during training or inference
  - Autoregressive: generate one token at a time, left-to-right

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Masked Attention: Causal Mask Visualization

- Attention mask pattern: Query (row) attends to Key (column)



- Interpretation: Lower triangular pattern ensures token i only sees tokens $\leq i$.
- Implementation: Add $-\infty$ to masked positions before softmax $\rightarrow$ weight = 0.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Multi-Head Attention: Concept

- Motivation: Single attention may miss different types of relationships.
- Solution: Use multiple attention "heads" in parallel, each learning different patterns.
- Time Series Example: Gold price with 3 heads
  - Head 1: Attends to short-term trends (last 2-3 days)
  - Head 2: Attends to weekly patterns (7-day cycles)
  - Head 3: Attends to monthly patterns (30-day trends)
  - Combined: Richer representation than single attention
- Text Example: Sentence with 3 heads
  - Syntax head: Subject-verb-object relationships
    - ★ "The customer complained" – verb attends to subject
  - Semantic head: Meaning and coreference
    - ★ "he" attends to "customer" (resolves pronoun)
  - Positional head: Adjacent word dependencies
    - ★ "very happy" – modifier-adjective pair

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Multi-Head Attention: Architecture

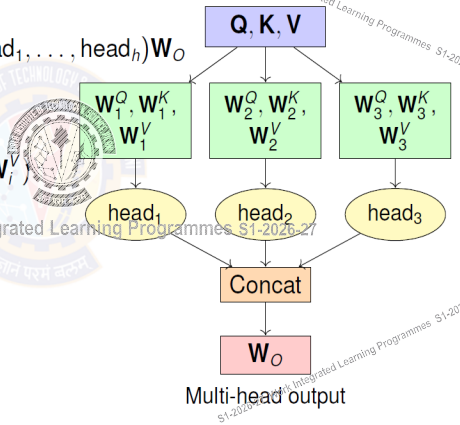
- Formula (Vaswani et al., 2017):

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\mathbf{W}_O$$

where each head i computes:

$$\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V)$$

- Typical hyperparameters:
  - $h = 8$ heads
  - $d_{\text{model}} = 512$
  - $d_k = d_v = 64$ per head



When to Use Which Attention Type?

Attention Type	Use When...
Self-Attention	Modeling relationships within a single sequence Building contextual representations (e.g., BERT encoder) Capturing long-range dependencies in text or time series
Cross-Attention	Aligning two different sequences (seq2seq) Translation, summarization, question answering Transformer decoder attending to encoder outputs
Masked	Autoregressive generation (language modeling) Preventing information leakage from future GPT-style models, time series forecasting
Multi-Head	Capturing diverse relationship types simultaneously All modern Transformers (BERT, GPT, T5, ViT) When you need multiple attention patterns

Solved example (without masking the future)

Playing
q1 = [0.212 0.04 0.63 0.36]
k1 = [0.31 0.84 0.963 0.57]
v1 = [0.36 0.83 0.1 0.38]

Outside
q2 = [0.1 0.14 0.86 0.77]
k2 = [0.45 0.94 0.73 0.58]
v2 = [0.31 0.36 0.19 0.72]

Playing	Outside
Dot Product q1.k1 = 0.91121 q1.k2 = 0.8017 Scaled Dot Product, w'=4 $e_{11} = 0.455605$ $e_{12} = 0.40085$	Dot Product q2.k1 = 1.41568 q2.k2 = 1.251 Scaled Dot Product, w'=4 $e_{21} = 0.70784$ $e_{22} = 0.6255$

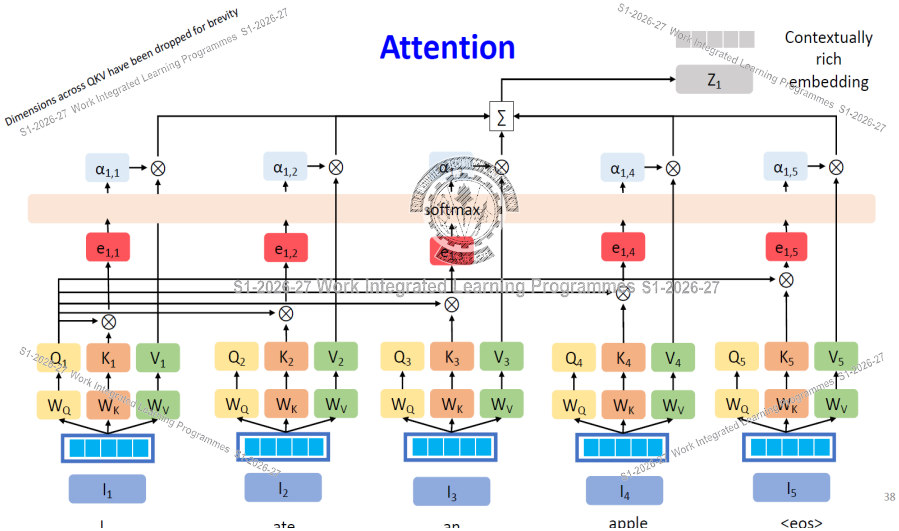
Playing	Outside
$\alpha_{11} = \frac{\exp(e_{11})}{\exp(e_{11})+\exp(e_{12})} = 0.513685331$	$\alpha_{21} = \frac{\exp(e_{21})}{\exp(e_{21})+\exp(e_{22})} = 0.520573378$
$\alpha_{12} = \frac{\exp(e_{12})}{\exp(e_{11})+\exp(e_{12})} = 0.486314669$	$\alpha_{22} = \frac{\exp(e_{22})}{\exp(e_{21})+\exp(e_{22})} = 0.479426622$

y_1 $= \alpha_{11}.v_1 + \alpha_{12}.v_2 =$ [0.335684267 0.601432106 0.14376832 0.545346987]

y_2 $= \alpha_{21}.v_1 + \alpha_{22}.v_2 =$ [0.336028669 0.604669487 0.143148396 0.543005052]

Self-Attention: Formal Definition

- Self-attention allows each position to attend to all positions in the *same* sequence.
- Given input $\mathbf{X} = [\mathbf{x}_1, \ldots, \mathbf{x}_n] \in \mathbb{R}^{n \times d}$:
  1. Linear projections:
$$\mathbf{Q} = \mathbf{X}\mathbf{W}_Q, \quad \mathbf{K} = \mathbf{X}\mathbf{W}_K, \quad \mathbf{V} = \mathbf{X}\mathbf{W}_V$$
  2. Scaled dot-product attention:
$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$
  3. Output:
$$\mathbf{Z} = \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) \in \mathbb{R}^{n \times d_v}$$
- Key property: Every output $\mathbf{z}_i$ is a weighted combination of *all* inputs $\{\mathbf{x}_j\}_{j=1}^n$.
- Difference from RNN: Direct connections between all positions (no sequential bottleneck).



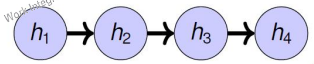
CNNs vs RNNs vs Self-Attention

Aspect	CNN	RNN	Self-Attn
Receptive field	Local (kernel k)	Sequential (history)	Global (all)
Path length	$O(\log n)$	$O(n)$	$O(1)$
Parallelization	High	Sequential	High
Complexity	$O(k \cdot n \cdot d^2)$	$O(n \cdot d^2)$	$O(n^2 \cdot d)$
Long-range	Many layers	Vanishing grad	Direct
Position info	Implicit	Built-in	Need encoding

- Key insight: Self-attention provides $O(1)$ path between any two positions!
- Trade-off: Better long-range modeling but $O(n^2)$ memory and computation.

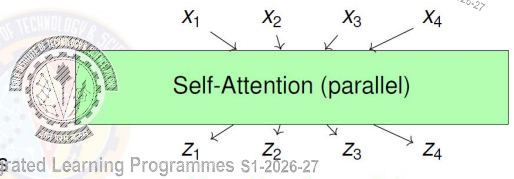
Parallelization: RNN vs Self-Attention

RNN (Sequential):



Must compute sequentially

Self-Attention (Parallel):



All positions in parallel

Bottleneck: h_4 depends on h_3 depends on h_2 depends on h_1

Benefit: All outputs computed simultaneously on GPU!

Result: Transformers train 10-100× faster than RNNs for long sequences.

Position-Invariance Problem

- Critical issue: Self-attention is *permutation-invariant*!

$$\text{Attention}(\mathbf{X}) = \text{Attention}(\mathbf{PX})$$

  for any permutation matrix $\mathbf{P}$.
- Example: These sentences are treated identically:
  - Original: "The customer was happy with service"
  - Shuffled: "service with happy was customer The"
- Why? Attention weights depend only on *content*, not *position*.
- Problem for NLP: Word order matters!
  - "Dog bites man" $\neq$ "Man bites dog"
  - Need position information to distinguish
- Solution: Inject positional information into input embeddings.

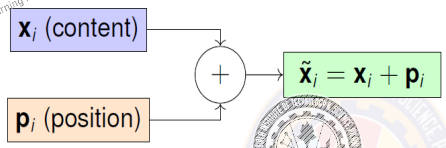
Positional Embeddings

- **positional encoding** indicates the absolute and relative position of the tokens in the sequence.
- For example, in "*The cat watches the dog run fast.*", if the position of "cat" and "dog" were interchanged, it would alter subject and object and thus the meaning of the sentence.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Positional Encoding: The Solution

Add position information to input embeddings:



Two approaches:

1. Learned positional embeddings (BERT, GPT-2):
   - $\mathbf{p}_i$ is a learned parameter for each position i
   - Simple, but doesn't generalize beyond training length
2. Fixed sinusoidal encoding (Original Transformer):
   - $\mathbf{p}_i$ computed using sine/cosine functions
   - Deterministic, generalizes to any sequence length

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

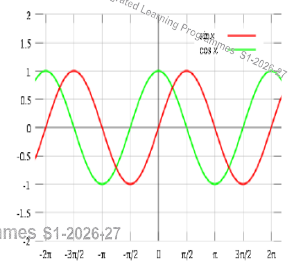
Absolute Positional Encoding: Sinusoidal

Vaswani et al. (2017): Use fixed sinusoidal functions.

$$\mathrm{PE}(\mathrm{pos}, 2i) = \sin\left(\frac{\mathrm{pos}}{10000^{2i/d_{\mathrm{model}}}}\right)$$

$$\mathrm{PE}(\mathrm{pos}, 2i+1) = \cos\left(\frac{\mathrm{pos}}{10000^{2i/d_{\mathrm{model}}}}\right)$$

- pos: Position in sequence $(0, 1, 2, \ldots)$
- i: Dimension index $(0, 1, \ldots, d_{\mathrm{model}}/2 - 1)$
- d_{model}: Model dimension (e.g., 512)



Intuition:

- Different dimensions use different frequencies
- Lower dimensions = lower frequency (long-range patterns)
- Higher dimensions = higher frequency (fine-grained positions)
- Each position gets unique encoding vector

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Why Sinusoidal Encoding Works

- Key mathematical property: For any fixed offset k, $\mathrm{PE}(\mathrm{pos}+k)$ is a *linear function* of $\mathrm{PE}(\mathrm{pos})$.

$$\mathrm{PE}(\mathrm{pos}+k, 2i) = \mathbf{M}_k \cdot \mathrm{PE}(\mathrm{pos}, 2i) \quad \text{for some transformation matrix } \mathbf{M}_k$$

- Trigonometric identity:

$$\sin(\alpha+\beta) = \sin\alpha\cos\beta + \cos\alpha\sin\beta$$
$$\cos(\alpha+\beta) = \cos\alpha\cos\beta - \sin\alpha\sin\beta$$

- Example: For dimension i, let $\omega_i = 1/10000^{2i/d}$

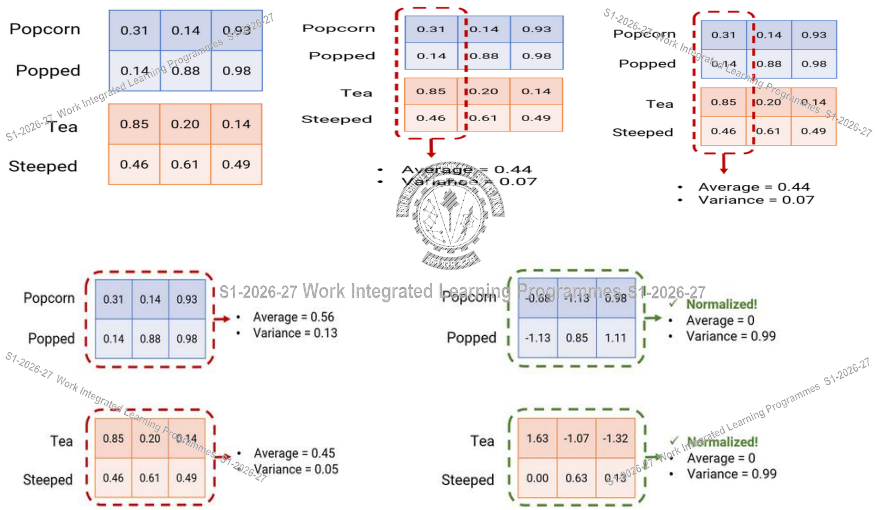
$$\mathrm{PE}(\mathrm{pos}+k, 2i) = \sin((\mathrm{pos}+k)\omega_i) = \sin(\mathrm{pos}\omega_i)\cos(k\omega_i) + \cos(\mathrm{pos}\omega_i)\sin(k\omega_i)$$

This is a linear combination of $\mathrm{PE}(\mathrm{pos}, 2i)$ and $\mathrm{PE}(\mathrm{pos}, 2i+1)$.

- Benefit: Model can learn to attend by *relative position* (e.g., "3 positions back").

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Normalization(Batch and Layer)



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Summary: Attention Mechanisms

1. **Attention Mechanism**
   - Dynamic, weighted aggregation based on relevance
   - Query-Key-Value framework for information retrieval
   - Solves bottleneck by attending selectively
2. **Scoring Functions**
   - Dot-product: simple, fast
   - Scaled dot-product: prevents saturation (Transformer default)
   - Additive: more expressive, slower
3. **Attention Types**
   - Self: intra-sequence, Cross: inter-sequence
   - Masked: causal, Multi-head: diverse patterns
4. **Positional Encoding**
   - Required for position-aware self-attention

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Key Formulas to Remember

- 1. Attention (general):

$$\text{Attention}(\mathbf{q}, \{\mathbf{k}_i, \mathbf{v}_i\}) = \sum_{i=1}^{n} \alpha_i \mathbf{v}_i \quad \text{where} \quad \alpha_i = \text{softmax}\left(\text{score}(\mathbf{q}, \mathbf{k}_i)\right)$$

- 2. Scaled Dot-Product Attention:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}$$

- 3. Multi-Head Attention:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \ldots, \text{head}_h)\mathbf{W}_O$$

- 4. Positional Encoding:

$$\text{PE}(\text{pos}, 2i) = \sin\left(\frac{\text{pos}}{10000^{2i/d}}\right), \quad \text{PE}(\text{pos}, 2i+1) = \cos\left(\frac{\text{pos}}{10000^{2i/d}}\right)$$

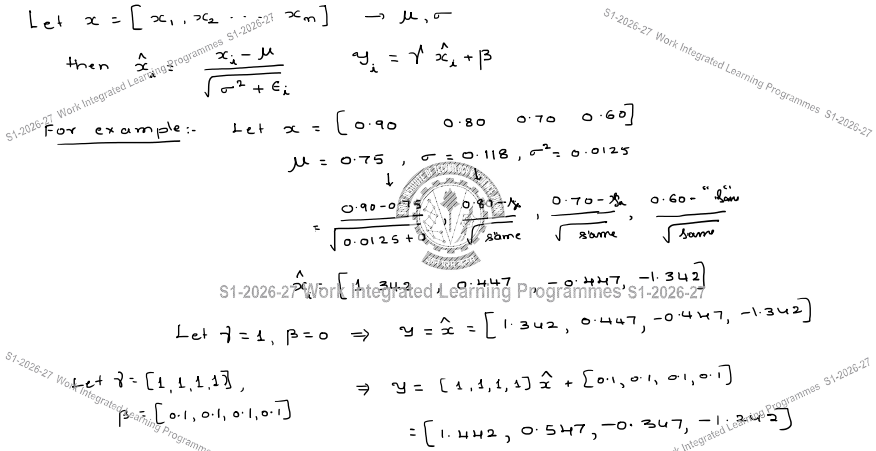
S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Normalization(Batch and Layer)

- Transformers use **Layer Normalization (LayerNorm)** — not Batch Normalization like CNNs typically do. Let's go through why, how it works, and a worked example.
- **Why LayerNorm and not BatchNorm?**
- **BatchNorm** normalizes across the *batch* dimension (i.e., across different examples, for a fixed feature). This works well for images but is problematic for sequences: sentences have variable length, batch statistics get noisy with small batches, and it creates a dependency between different sequences in a batch — undesirable at inference time.
- **LayerNorm** instead normalizes across the *feature* dimension, independently for **each token, in each example**. This makes it stable regardless of batch size or sequence length, and each token's normalization is self-contained.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Layer Normalization



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

TRANSFORMER

The Transformer Solution (2017)

- Key Insight: "Attention is All You Need"
  - Replace recurrence entirely with self-attention
  - Each token directly attends to all other tokens
  - No sequential dependency → full parallelization
- How Transformers Solve RNN Problems:

Problem	RNN	Transformer
Parallelization	Sequential	Parallel
Long-range deps	Vanishing gradients	Direct connections
Memory	Fixed hidden state	Attention to all tokens

- The Impact:
  - 2017: Machine translation breakthrough
  - 2018: BERT revolutionizes NLP (encoder-only)
  - 2018-2020: GPT series (decoder-only)
  - 2020-present: Foundation of all large language models
- Result: Transformers became universal architecture for sequences

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

From Transformers to Modern AI

- Transformers Enable Scaling:
  - Parallel processing → train on massive datasets
  - Direct connections → learn complex patterns
  - Flexible architecture → adapt to any domain
- This Scaling Led To:
  1. **Large Language Models (LLMs)**
     - ★ 100B+ parameters: GPT-4, Claude, Gemini
     - ★ Trained on trillions of tokens
     - ★ General intelligence: reasoning, coding, writing
  2. **Small Language Models (SLMs)**
     - ★ 1B-7B parameters: Phi-3, Gemma, Mistral-7B
     - ★ On-device deployment: phones, laptops
     - ★ Privacy-preserving, low-latency
  3. **Specialized Models**
     - ★ Vision: ViT, CLIP (images without convolutions)
     - ★ Speech: Whisper, Wav2Vec2 (audio understanding)

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

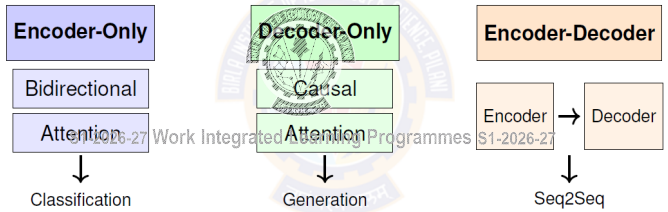
Real-world Applications

1. Conversational AI
   - ChatGPT, Claude, Gemini: Natural dialogue with context
   - Customer support automation: 24/7 intelligent responses
   - Virtual assistants: Multi-turn conversations with memory
2. Code Generation & Software Development
   - GitHub Copilot: Real-time code completion
   - CodeLlama, StarCoder: Function generation from descriptions
3. Content Creation
   - Writing assistance: Articles, emails, reports
   - Summarization: Long documents → key points
   - Translation: 100+ languages with context
4. Scientific Research
   - AlphaFold 3: Protein structure prediction
   - Drug discovery: Molecular design and screening
5. Next: How do these models differ? Three transformer variants

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

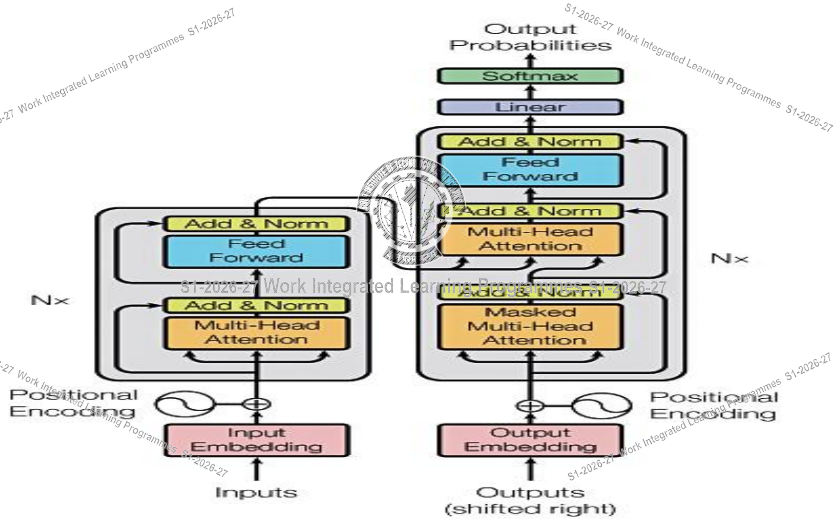
Three Transformer Architectures

- Why Three Variants?
  - Different tasks need different information flow
  - Trade-off between understanding vs generation
  - Computational efficiency for specific use cases



- Selection Criteria:
  - Need to classify or extract? → Encoder-only
  - Need to generate text? → Decoder-only
  - Input and output very different? → Encoder-decoder

S1-2026-27 Work Integrated Learning Programmes S1-2026-27



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Transformer – Encoder Only (Numerical)

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Setup

- Dimension of the model = 4
- Dimension of K = Dimension of V = 3
- single attention head,
- Input: I am
- 2 tokens: x1 = "I", x2 = "am"

Input embeddings (X)

	d1	d2	d3	d4
x1	1	0	1	0
x2	0	1	0	1

Weight matrices (learned parameters, 1×3 each)

Wq

1	0	1
1	0	0
0	0	1
0	1	1

Wk

0	0	1
1	1	0
0	1	0
1	1	0

Wv

0	2	0
0	3	0
1	0	3
1	1	0

Encoder-only means:

- No masking — every token sees every other token
- No decoder , no cross-attention
- Used for understanding tasks (BERT, RoBERTa): classification, NER, embeddings

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Step 1 — Project into Q, K, V

Q = X·Wq K = X·Wk V = X·Wv

Worked example — Q1:

x1·Wq = [1·1+0·1+1·0+0·0, 1·0+0·0+1·0+0·1, 1·1+0·0+1·1+0·1] = [1, 0, 2]

Q

	q1	q2	q3
x1	1	0	2
x2	1	1	1

K

	k1	k2	k3
x1	0	1	1
x2	2	2	0

V

	v1	v2	v3
x1	1	2	3
x2	1	4	0

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

- Each token now has its own Query, Key, and Value vector — three different views of the same embedding, each shaped by a separate learned projection.

Steps 2–3 — Score & Scale

scores = Q·K^T scaled = scores / √d (d = 3, √3 ≈ 1.732)

Raw scores (Q·K^T)

	K1	K2
Q1	2	2
Q2	2	4

Scaled scores (÷ 1.732)

	K1	K2
Q1	1.155	1.155
Q2	1.155	2.309

Why scale by √d?

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

- As d grows, dot products grow in magnitude, pushing softmax into regions with extremely small gradients.
- Dividing by √d keeps the scores — and the softmax — well-behaved regardless of dimension.

- Token 1 ("I") scores 1.155 against both tokens — a tie.
- Token 2 ("am") scores higher against itself (2.309) than against "I" (1.155) — already

Steps 4–5 — Softmax & Weighted Output

- **Softmax (row-wise)**
  - **Row 1 (token "I"):** [0.500, 0.500]
  - **Row 2 (token "am"):** [0.240, 0.760]
    - e^1.155 = 3.17, e^2.309 = 10.07, sum = 13.24
    - 0.240 = 3.17/13.24 , 0.760 = 10.07/13.24
- **Attention output Z = softmax · V**
  - Z1 = 0.5·[1,2,3] + 0.5·[1,4,0] = **[1.000, 3.000, 1.500]**
  - Z2 = 0.24·[1,2,3] + 0.76·[1,4,0] = **[1.000, 3.521, 0.719]**
- **Final self-attention output (Z)**

	z1	z2	z3
x1	1.000	3.000	1.500
x2	1.000	3.521	0.719

- **Reading the result**
  - "am" (x2) ends up attending mostly to itself (76%) — its output Z2 is pulled close to V2.
  - "I" (x1) split evenly (50/50), so Z1 blends V1 and V2.
  - Attention weights are learned and data-dependent, not fixed.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Completing the Encoder Block

What happens after attention, on the way to the next layer

1	**Add & Norm**	Residual connection adds the block's input back to Z, then LayerNorm stabilizes the distribution: LayerNorm(X + Z).
2	**Feed-Forward**	A position-wise network applied identically to every token: FFN(x) = ReLU(x·W1 + b1)·W2 + b2.
3	**Add & Norm (again)**	A second residual + LayerNorm wraps the feed-forward sublayer before passing output to the next stacked block.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

This entire block stacks N times

(N = 12 in BERT-base). Because it is encoder-only, every token attends bidirectionally to every other token — no masking, no decoder, no cross-attention — which is why encoder-only models excel at understanding tasks rather than text generation.

Key Takeaway

- **Self-attention is a learned, data-dependent weighting scheme.**
- The same architecture, given different Wq/Wk/Wv weights, produces entirely different attention patterns. Here, "I" split its attention evenly across both tokens, while "am" attended mostly to itself (76%) — an asymmetry that falls directly out of training, not out of any hand-coded rule.

- **Bi directional:** Every token attends to every other token in both directions — no causal mask.
- **Stacked N times:** One block repeats (e.g. N=12 in BERT-base) to build deep contextual representations.
- **Used for understanding :** Classification, NER, embeddings — not autoregressive generation.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Transformer – Encoder Only (Numerical 2)

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Example

- **Sequence of 3 tokens: "I" "love" "NLP"**
- **Embedding dimension: d(model) = 4** *(and here, $d_k = d_v = 4$ for simplicity)*
- **INPUT EMBEDDING MATRIX X (3 × 4)**

I	1	0	1	0
love	0	2	0	2
NLP	1	1	1	1

EACH ROW = ONE TOKEN'S EMBEDDING

```
Row 1  → "I"     = [1, 0, 1, 0]
Row 2  → "love"  = [0, 2, 0, 2]
Row 3  → "NLP"   = [1, 1, 1, 1]
```

Goal: turn each row into a new vector that also reflects the other tokens it should "attend to."

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Query, Key & Value Weight Matrices

621

- **LEARNED PARAMETERS**
- Three learned 4×4 projection matrices turn each token embedding into a Query, a Key, and a Value vector. (Simplified integer values here, purely for illustration.)

Q W^Q

1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1

K W^K

0	1	0	0
1	0	0	0
0	0	0	1
0	0	1	0

V W^V

1	0	1	0
0	1	0	1
1	0	0	1
0	1	1	0

$Q = X \cdot W^Q$ $K = X \cdot W^K$ $V = X \cdot W^V$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Project Embeddings into Q, K, V

622

$Q = X\ W^Q$ $K = X\ W^K$ $V = X\ W^V$

I	1	0	1	0
love	0	2	0	2
NLP	1	1	1	1

I	0	1	0	1
love	2	0	2	0
NLP	1	1	1	1

I	2	0	1	0
love	2	2	2	2
NLP	2	2	2	1

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

- *Each row of Q, K, and V still corresponds to one token — but now in three different learned "roles": what it's looking for (Q), what it offers (K), and what it carries (V).*

Raw Attention Scores — Q K^T for "I"

623

- Take the query vector for "I" and dot it against every key vector to see how strongly "I" relates to each token (including itself).
  - **Q ("Ia") = [1, 0, 1, 0]**

K_1 (I)	[0, 1, 0, 1]	1(0)+0(1)+1(0)+0(1)	=	0
K_2 (love)	[2, 0, 2, 0]	1(2)+0(0)+1(2)+0(0)	=	4
K_3 (NLP)	[1, 1, 1, 1]	1(1)+0(1)+1(1)+0(1)	=	2

Row 1 of QK^T = **[0, 4, 2]**

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Scale, then Softmax

624

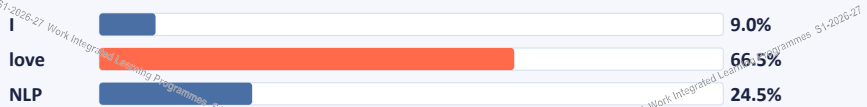
[0, 4, 2] / 2 = [0, 2, 1]

- Scaling prevents large dot products from pushing softmax into tiny gradients as d_k grows.

softmax([0, 2, 1]) = $[e^0, e^2, e^1] / (e^0 + e^2 + e^1)$ = [1, 7.39, 2.72] / 11.11

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

ATTENTION WEIGHTS FOR "I"



Weighted Sum of V — Output for "I"

- Multiply each value vector by its attention weight and sum them — producing a new, contextualized representation of "I."

0.09 × [2, 0, 1, 0]

0.665 × [2, 2, 2, 2]

0.245 × [2, 2, 2, 1]

Output₁ ≈ [2.00, 1.82, 1.91, 1.575]

- *This 4-dim vector replaces the original embedding of "I" — now blended mostly with "love" (66.5%) and "NLP" (24.5%), the tokens most relevant to it.*
- *Repeating Steps for rows 2 and 3 of Q yields contextualized vectors for "love" and "NLP," completing the 3×4 output matrix.*

Transformer _ Encoder Only(with multi head attention)
(Numerical 3)

Setup

- **Two tokens, projected through the encoder's first attention block**

The example

INPUT EMBEDDINGS

		DIMENSIONS	
x_1	[1, 0, 1]	3	**model** size of each token embedding
x_2	[0, 1, 1]	2	**k and v** size of Q/K/V after projection
		2	**heads (h)** parallel attention computations
		2	**seq length** tokens in this example

- **Think of x_1 and x_2 as two words' embeddings sitting side by side in a sequence of length 2.**

Head 1 — Self-Attention

Project to Q, K, V → score → scale → softmax → weighted sum

Query / Key / Value projections

Token	Q	K	V
x_1	[2, 1]	[1, 2]	[2, 1]
x_2	[1, 2]	[2, 1]	[1, 1]

Scaled scores use ÷√d = ÷1.414

Attention weights & output

Token	Softmax weights (t_1, t_2)	Head-1 Output
x_1	[0.330, 0.670]	[1.33, 1.00]
x_2	[0.670, 0.330]	[1.67, 1.00]

WORKED EXAMPLE — TOKEN x_1

Q·K scores: $Q_1 \cdot K_1 = 4$ $Q_1 \cdot K_2 = 5$

Scaled: 4/1.414 = 2.828
5/1.414 = 3.536

Softmax: e^2.828 = 16.92
e^3.536 = 34.34
weights = [0.330, 0.670]

Output: $0.330 \cdot V_1 + 0.670 \cdot V_2$
= 0.330[2,1] + 0.670[1,1]
= [1.33, 1.00]

Head 2 — Self-Attention

Different learned weights → a different attention pattern

Query / Key / Value projections

Token	Q	K	V
x_1	[1, 2]	[2, 0]	[1, 1]
x_2	[1, 1]	[1, 1]	[2, 1]

Attention weights & output

Token	Softmax weights (t_1, t_2)	Head-2 Output
x_1	[0.330, 0.670]	[1.67, 1.00]
x_2	[0.500, 0.500]	[1.50, 1.00]

- *x_2's raw scores tie exactly (2 = 2) → softmax splits attention 50/50 between the two tokens.*

- Head 1 gave x_2 an asymmetric split — [0.670, 0.330] — favoring its own token. Head 2, with different W_Q/W_K/W_V, gives x_2 an even [0.500, 0.500] split instead.
- Same input tokens, same position — two heads extracted two different relationships.
- That's the entire point of running several attention heads in parallel: each is free to specialize in a different kind of dependency.

Concatenate Heads + Output Projection

- Merge the two heads back into one d_model-sized vector

Concatenation

Token	Head 1	Head 2	Concatenated [h_1, h_2]
x_1	[1.33, 1.00]	[1.67, 1.00]	[1.33, 1.00, 1.67, 1.00]
x_2	[1.67, 1.00]	[1.50, 1.00]	[1.67, 1.00, 1.50, 1.00]

Output projection × W_o (4×3 → back to d_model = 3)

Token	Concatenated × W_o	Multi-Head Attention Output
x_1	[1.33, 1.00, 1.67, 1.00] × W_o	[3.00, 2.67, 2.33]
x_2	[1.67, 1.00, 1.50, 1.00] × W_o	[3.17, 2.50, 2.67]

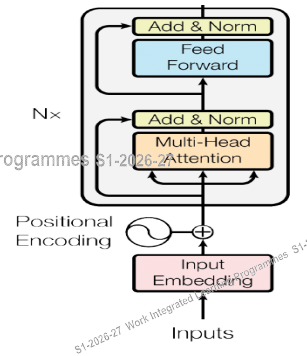
- *W_o learns how to best recombine information gathered by every head into a single vector — this is the last step inside the multi-head attention sublayer, before the residual connection.*

Residual Connection + Layer Norm

Add the block's input back in, then normalize

Add & Norm (shown for token x_1)

Residual add	$x_1 + MHA_1 = [1,0,1] + [3.00, 2.67, 2.33]$	**= [4.00, 2.67, 3.33]**
Mean	(4.00 + 2.67 + 3.33) / 3 = 3.333	
Variance	mean of squared deviations = 0.294	
Std dev	√0.294 = 0.543	
Normalize	(each value − mean) / std	**≈ [1.228, −1.221, −0.006]**



(γ = 1, β = 0 for simplicity — in practice these are learned scale/shift parameters. Token x_2 follows the identical procedure.)

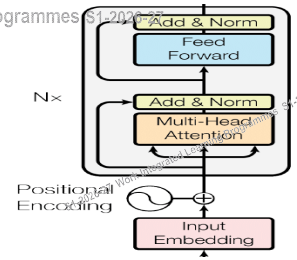
Position-Wise Feed-Forward Network

Same two linear layers, applied identically to every token

Expand → ReLU → project back (shown for token x_1)

1	**Input**	*LN(Residual$_1$)*	[1.228, −1.221, −0.006]
2	**Layer 1 (× W_1, d_ff = 4)**	*pre-activation*	[1.222, −2.449, −1.215, 2.443]
3	**ReLU**	*negatives zeroed*	[1.222, 0, 0, 2.443]
4	**Layer 2 (× W_2, back to d_model = 3)**	*FFN output*	[3.665, 0, 1.222]

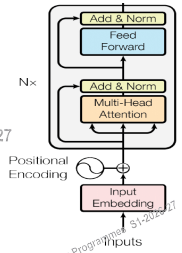
The FFN's weights (W_1, W_2) are shared across every position in the sequence — this is what "position-wise" means. Only the attention sublayer lets tokens exchange information; the FFN processes each one independently.



Summary

❖ Token x_1 traced start to finish — then the output feeds into the next stacked layer

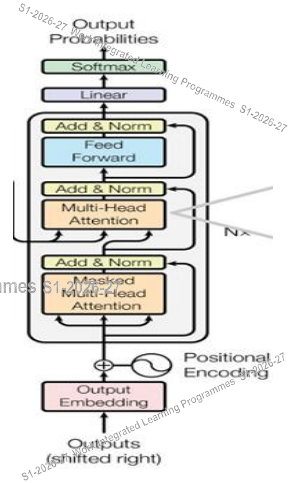
- **Embeddings + positional encoding** ✓ x_1 = [1, 0, 1]
- **Multi-head self-attention** ✓ 2 heads → concat → $\times W_o$ = [3.00, 2.67, 2.33]
- **Residual + LayerNorm** ✓ ≈ [1.228, −1.221, −0.006]
- **Position-wise feed-forward** ✓ [3.665, 0, 1.222]
- **Residual + LayerNorm** ✓ [4.893, −1.221, 1.216] → normalize
- **Pass to next encoder layer** ✓ repeats 6–12× in real models



- *The decoder adds one more sublayer — cross-attention to the encoder's output — and masks future positions in self-attention.*

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Transformer _ Decoder Only (Numerical)



Setup

- Same weight matrices as the encoder example — the mask is the only structural change
- **Dimensions : model = 4 ,Key = Value = 3**
- **Task: Next word prediction; The cat sat -------**
- single head, 3 tokens: x1="The", x2="cat", x3="sat"
- **Input embeddings (X)**

	d1	d2	d3	d4
x1	1	0	1	0
x2	0	1	0	1
x3	1	1	0	0

- **Weight matrices (learned parameters, 4×3 each)**

Wq

1	0	1
1	0	0
0	0	1
0	1	1

Wk

0	0	1
1	1	0
0	1	0
1	1	0

Wv

0	2	0
0	3	0
1	0	3
1	1	0

Decoder-only means:

- Causal mask — token i sees only tokens 1..i
- No encoder, no cross-attention
- Used for generation (GPT): text completion, chat, one token at a time

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Step 1 — Project into Q, K, V

Q = X·Wq K = X·Wk V = X·Wv

Q

	q1	q2	q3
x1	1	0	2
x2	1	1	1
x3	2	0	1

K

	k1	k2	k3
x1	0	1	1
x2	2	2	0
x3	1	1	1

V

	v1	v2	v3
x1	1	2	3
x2	1	4	0
x3	0	5	0

- **New this time — Q3, K3, V3 for "sat":**
- x3·Wq = [2,0,1] x3·Wk = [1,1,1] x3·Wv = [0,5,0]

- Every token still gets its own Query, Key, and Value vector — identical mechanics to the encoder.
- The difference comes next.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Step 2 — Score, then Apply the Causal Mask

- This mask is the entire mechanical difference from an encoder block

Raw scores (Q·K^T)

	K1	K2	K3
Q1	2	2	3
Q2	2	4	3
Q3	1	4	3

After causal mask (future = −∞)

	K1	K2	K3
Q1	2	−∞	−∞
Q2	2	4	−∞
Q3	1	4	3

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

What the mask enforces

- Row Q1 ("The") keeps only its score against K1 — it cannot see "cat" or "sat" yet.
- Row Q2 ("cat") keeps scores against K1, K2 — it can see "The" and itself, not "sat".
- Row Q3 ("sat") keeps all three — by the last position, everything before it is visible.

Steps 3–4 — Scale & Softmax

- **scaled = masked_scores / √d (√3 ≈ 1.732), then softmax row-wise**

Scaled scores

	K1	K2	K3
Q1	1.155	−∞	−∞
Q2	1.155	2.309	−∞
Q3	0.577	2.309	1.732

Softmax weights (row-wise)

	K1	K2	K3
Q1	1.000	0.000	0.000
Q2	0.240	0.760	0.000
Q3	0.102	0.575	0.323

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

- **Row 3 worked out — token "sat" attending to all prior tokens**

  e^0.577 = 1.781, e^2.309 = 10.07, e^1.732 = 5.652, sum = 17.50

  [1.781, 10.07, 5.652] / 17.50 = [0.102, 0.575, 0.323]

- Masked positions get exactly 0 weight — softmax turns −∞ into 0 probability, so no information leaks from the future.

Step 5 — Weighted Output Z = softmax · V

- Z1 = 1.0·[1,2,3] = **[1.000, 2.000, 3.000]**
- Z2 = 0.24·[1,2,3] + 0.76·[1,4,0] = **[1.000, 3.520, 0.720]**
- Z3 = 0.102·[1,2,3] + 0.575·[1,4,0] + 0.323·[0,5,0] = **[0.677, 4.119, 0.305]**

Final causal self-attention output (Z)

	z1	z2	z3
x1	1.000	2.000	3.000
x2	1.000	3.520	0.720
x3	0.677	4.119	0.305

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Reading the result

Z1 and Z2 exactly match the encoder-only example — at those early positions the causal mask hides nothing new. Z3 is unique to the decoder: it blends all three tokens, leaning most on x2 "cat" (57.5%).

Step 6 — Predict the Next Token

- Decoder-only's real job: only the last position's vector feeds the output layer

❖ W_out (3×4, one column per vocab word)

1	0	2	1
0	1	1	0
1	1	0	2

vocab = {on, quickly, down, up}

❖ logits = Z3 · W_out

- [0.677, 4.119, 0.305] · W_out = [0.982, 4.424, 5.473, 1.287]
- Softmax over vocab:

e^0.982=2.67, e^4.424=83.35, e^5.473=238.0, e^1.287=3.62,

sum=327.6

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Next-token probability distribution

word	on	quickly	down	up
probability	0.008	0.254	0.726	0.011

Autoregressive generation

"down" wins at 72.6% — e.g. completing "The cat sat down." That word is appended to the sequence, and the whole causal-attention process repeats to predict the token after it.

Encoder-Only vs. Decoder-Only

	Encoder-only (BERT-style)	Decoder-only (GPT-style)
Attention direction	Bidirectional — every token sees every token	Causal — token i sees only tokens ≤ i
Extra step	None	Mask applied before softmax
Cross-attention	None	None (only in encoder–decoder models)
Typical output	Contextual embeddings, all tokens in parallel	One next-token distribution per position
Typical use	Classification, NER, embeddings	Text generation, completion, chat

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Transformer(complete) (Numerical)

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Setup

Input : English – I am a man

Output : Hindi – मैं एक आदमी हूं

English token	Embedding
I	$x_1 = [1, 0, 1]$
am	$x_2 = [0, 1, 1]$
a	$x_3 = [1, 1, 0]$
man	$x_4 = [0, 0, 1]$

मैं एक आदमी हूं

main ek aadmī hū ँ

Decoder sequence (teacher forcing):

<start> मैं, एक, आदमी, हूं → predicts → मैं, एक, आदमी, हूं, <end>

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

- *Dimension(model) = 3 · single head · W_Q = W_K = W_V = I (so Q = K = V = embedding)*
- **NOTE** : Residual-add, LayerNorm, and the position-wise FFN are not re-derived here — that exact arithmetic (mean/variance/normalize, expand→ReLU→project) was already worked through earlier step by step in the self-attention . This example focuses on what's new: cross-attention and autoregressive decoding.

Encoder Self-Attention — Raw Scores

Every source token compared against every other source token

Q · K (= $x_i \cdot x_j$ since Q = K = X)

	I	am	a	man
I	2	1	1	1
am	1	2	1	1
a	1	1	2	0
man	1	1	0	1

Scale every score by ÷√d = ÷1.732 before softmax — same scaling rule as the single-sentence example.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Notice the diagonal is always highest (a token is most similar to itself) and “a”/“man” score 0 — with real trained embeddings, semantically related words would score higher, not just identical ones.

Encoder Self-Attention — Output

Softmax weights blend the embeddings into context-aware vectors

Softmax weights & resulting context vectors

Token	→ I	→ am	→ a	→ man	Context vector
I	0.373	0.209	0.209	0.209	[0.582, 0.418, 0.791]
am	0.209	0.373	0.209	0.209	[0.418, 0.582, 0.791]
a	0.230	0.230	0.410	0.129	[0.641, 0.641, 0.590]
man	0.281	0.281	0.158	0.281	[0.438, 0.438, 0.842]

These become $Enc_1..Enc_4$ (the encoder's final output)

After add & norm and the position-wise FFN (same mechanics as before, not re-derived here), these context vectors become $Enc_1..Enc_4$. They are cached once and reused by every decoder layer, at every decoding step — the encoder runs exactly once per sentence.

Decoder Step 1 — Masked Self-Attention

Generating the very first Hindi word

Only <start> is available

y_0 = <start> = [1, 1, 1]

With only one token in the decoder sequence, the causal mask has nothing to hide — <start> can only attend to itself.

Attention weight = 1.0

$SelfAttn_0 = y_0 = [1, 1, 1]$ (unchanged)

WHAT "MASKED" MEANS

Once मैं is generated and fed back in at Step 2, it can attend to <start> and itself — never to words generated after it.

The mask sets those future-position scores to −∞ before softmax, so their weight becomes exactly 0. This lets the whole target sequence be trained in parallel while still matching true one-word-at-a-time generation.

Cross-Attention

- Decoder Query meets Encoder Key/Value — this is how source meaning reaches the target
- **Q from decoder, K & V from the encoder's output**

Encoder token	Raw score (Q·Enc)	Scaled	Softmax weight
I	1.791	1.034	0.250
am	1.791	1.034	0.250
a	1.872	1.081	0.262
man	1.718	0.992	0.239

- $Q_dec = SelfAttn_0$ = [1, 1, 1]
- scaled by ÷√3 = ÷1.732

➤ **CROSS-ATTENTION OUTPUT**

- **$0.250 \cdot Enc_1 + 0.250 \cdot Enc_2 + 0.262 \cdot Enc_3 + 0.239 \cdot Enc_4$ = [0.522, 0.522, 0.750]**
- *Weights are fairly even here because the query [1,1,1] doesn't discriminate strongly — with real trained weights, the query for "I" would sharply prefer the encoder position for "I."*

Final Linear Layer + Softmax

- Project to vocabulary size, then pick the most likely Hindi word
- d_1 = [1,1,1] + [0.522, 0.522, 0.750] = [1.522, 1.522, 1.750] (post add & norm + FFN)

Candidate word	Weight vector	Logit	Probability
मैं (main)	[1, 1, 1]	**4.794**	**0.884**
हूं (hūँ)	[0, 0, 1]	1.750	0.042
एक (ek)	[1, 0, 0]	1.522	0.033
आदमी (aadmī)	[0, 1, 0]	1.522	0.033
<end>	[0, 0, 0]	0.000	0.007

- **Predicted word: मैं (main) — argmax, 88.4% confidence ✓**

Autoregressive Continuation

- *The predicted word feeds back in; Steps 3–6 repeat until <end>*
- **One word at a time**

Step	Decoder input so far	Masked self-attn sees	Predicted word
1	<start>	<start> only	मैं
2	<start>, मैं	<start>, मैं	एक
3	<start>, मैं, एक	<start>, मैं, एक	आदमी
4	<start>, मैं, एक, आदमी	all four	हूं
5	<start>, मैं, एक, आदमी, हूं	all five	<end> — stop

Final translation: **मैं एक आदमी हूं** *(main ek aadmī hūṁ)*

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Encoder runs once; the decoder loop repeats once per output word

1	**Encoder: self-attention over source**	*"I am a man" → $Enc_1..Enc_4$*
2	**Encoder: add & norm, then FFN**	*cached once per sentence*
3	**Decoder: masked self-attention**	*sees only prior target words*
4	**Decoder: cross-attention to $Enc_1..Enc_4$**	*pulls in source meaning*
5	**Decoder: add & norm, FFN, add & norm**	*same mechanics as the encoder*
6	**Final linear + softmax → next word**	*repeat until <end>*

"I am a man" → मैं एक आदमी हूं

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Transformer(complete) (Numerical)

Task: Translate –"My name is Ravi" into French *"Mon, nom, est, Ravi"*

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

The Full Pipeline, End to End

- Nine steps: four build the encoder, five build the decoder.

ENCODER (reads English)	DECODER (writes French)
1 **Input Embedding** Token → dense vector	5 **Decoder Embedding** French tokens (shifted right) + PE
2 **Positional Encoding** Inject word order (sin/cos)	6 **Masked Self-Attention** Attend only to earlier French words
3 **Self-Attention** Each English word gathers context	7 **Cross-Attention** French queries read the English encoder output
4 **Add & Norm + FFN** Per-position transform → encoder output	8 **Feed-Forward** Add & Norm → final decoder representation
	9 **Linear + Softmax** Probability over French vocabulary

- Step 4's output (the encoded English sentence) is read by Step 7 — that's where translation alignment happens.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

STEP 1 · ENCODER _Input Embeddings

➢ Each token is looked up in an embedding table — here, a toy 4-dimensional vector.

My	0.10	0.20	0.30	0.40
name	0.50	0.10	0.20	0.30
is	0.20	0.40	0.10	0.30
Ravi	0.90	0.80	0.70	0.60

X = the 4×4 matrix stacking all four rows — this is the encoder's raw input before any position information is added.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

STEP 2 · ENCODER _ Positional Encoding

- Sine/cosine signals inject word-order information, since attention itself is order-blind.

$PE(pos, 2i) = \sin(pos / 10000^{2i/d})$

$PE(pos, 2i+1) = \cos(pos / 10000^{2i/d})$

Positional Encoding (PE) — rows = position 0..3

0.0000	1.0000	0.0000	1.0000
0.8415	0.5403	0.0100	1.0000
0.9093	-0.4161	0.0200	0.9998
0.1411	-0.9900	0.0300	0.9996

Encoder input = X + PE

0.10	1.20	0.30	1.40
1.34	0.64	0.21	1.30
1.11	-0.02	0.12	1.30
1.04	-0.19	0.73	1.60

- Row order = My, name, is, Ravi. This X_in matrix is what actually enters the self-attention layer next.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Why Position Information Is Needed

- *Self-attention on its own has no idea what order the words came in.*
- Self-attention treats a sentence as a SET, not a sequence
- Q, K, V are computed independently per token and combined only through weighted averages — nothing in that math depends on where a token sits.

Original order

My | name | is | Ravi

Scrambled order

Ravi | is | name | My

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

- Same multiset of embeddings in → the exact same attention weights and outputs come out, regardless of which order the four words were fed in.
- That's clearly wrong for language — "Ravi is name My" is not the same sentence. Positional encoding fixes this by adding a position-specific fingerprint to each embedding before attention ever runs, so word order becomes part of the numbers themselves.

Positional Encoding

- *Every symbol in PE(pos, i) maps to something concrete.*

pos	Token's position in the sequence — 0 for "My", 1 for "name", 2 for "is", 3 for "Ravi".
i	Indexes pairs of dimensions. With d_model = 4: i=0 → dims 0–1, i=1 → dims 2–3.
sin / cos	Even dimension → sine, odd dimension → cosine. Each (i) pair forms one wave oscillating at its own frequency.
10000^(2i/d)	Controls wavelength. Low i = fast-changing wave; high i = slow-changing wave — like a clock's second hand and hour hand together pinning down a specific time.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Worked example — pos = 1 ("name")

i=0: angle = $1 / 10000^{0} = 1.0$ → PE(1,0)=sin(1.0)=0.8415, PE(1,1)=cos(1.0)=0.5403

i=1: angle = $1 / 10000^{0.5} = 0.01$ → PE(1,2)=sin(0.01)=0.0100, PE(1,3)=cos(0.01)=1.0000

Why Sine and Cosine, Specifically

657

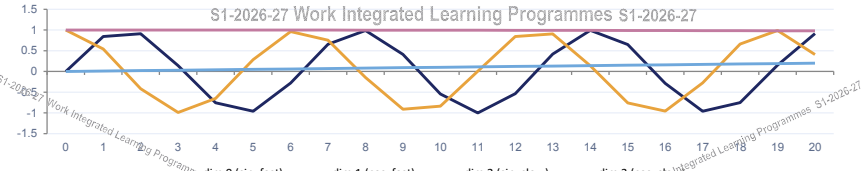
- *Not just any position signal works — this one has three properties that matter.*

Bounded values
Raw position counts (0,1,2,3…) grow without limit and would swamp the embedding. sin/cos always stay in [-1, 1].

Relative position = linear
PE(pos+k) is a linear function (a rotation) of PE(pos), via trig addition formulas — helping the model learn by relative offset, not just absolute position.

Deterministic, not learned
Computed once from a formula — no extra parameters, and it generalizes to sequence lengths never seen in training.

PE values across positions 0–20 (all 4 dimensions of this toy model)



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

ENCODER(Self-Attention)

658

- Every English word compares itself against every other English word.

$Q = X_in \cdot WQ \quad K = X_in \cdot WK \quad V = X_in \cdot WV$

$scores = Q \cdot K^T / \sqrt{d} \qquad weights = softmax(scores)$

$output = weights \cdot V$

- W_Q, W_K, W_V are learned 4×4 matrices (shown here at their random-initialization values).

Softmax attention weights (rows sum to 1)

	My	name	is	Ravi
My	0.208	0.262	0.248	0.282
name	0.211	0.271	0.222	0.296
is	0.239	0.269	0.214	0.277
Ravi	0.252	0.271	0.208	0.270

- **Attention output (weights · V) → each word's representation now blends in context from the others.**

0.112	0.165	0.505	-0.272
0.116	0.170	0.500	-0.268
0.108	0.179	0.478	-0.255
0.105	0.184	0.467	-0.249

- Add & Norm (residual + LayerNorm) closes out this sub-layer before the feed-forward network.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

ENCODER _ Feed-Forward → Final Encoder Output

659

- A position-wise FFN, then Add & Norm, produces the representation the decoder will read.
  - $FFN(x) = ReLU(x \cdot W_1) \cdot W_2$ (dims 4 → 8 → 4, applied independently to each position)

FINAL ENCODER OUTPUT

-1.451	1.205	-0.331	0.577
1.644	-0.593	-0.170	-0.081
1.131	-1.466	-0.844	0.675
0.389	-1.716	0.553	0.773

My name is Ravi

This matrix is the encoder's final output.

- It's a contextualized representation of "My name is Ravi" — each row still corresponds to one English word, but now carries information gathered from the whole sentence.
- The decoder never sees the raw English words again. From this point on, it only sees this matrix — via cross-attention in Step 7.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

DECODER - Masked Self-Attention

660

- The decoder reads its own (shifted) output so far — but never the future.

Decoder input (teacher-forced, shifted right):

<sos>	Mon	nom	est

target to predict: Mon, nom, est, Ravi

- **Causal mask — future positions forced to −∞ before softmax**

	<sos>	Mon	nom	est
<sos>	1.00	—	—	—
Mon	0.51	0.49	—	—
nom	0.33	0.32	0.34	—
est	0.25	0.23	0.25	0.27

Why mask?

- At training and inference time, the decoder must predict each word using only the words already generated. "est" (row 4) attends across <sos>, Mon, nom, and itself — but 0% to "Ravi", which doesn't exist yet at that point.
- This is what makes the decoder autoregressive.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

DECODER: Cross-Attention — Where Translation Happens

- French queries; English keys and values. This is the alignment step.

Q ← decoder (French)
K, V ← encoder output (English)

- **Cross-attention weights — each French position's focus over English**

	My	name	is	Ravi
<sos>	0.257	0.171	0.236	**0.336**
Mon	**0.409**	0.149	0.194	0.248
nom	**0.447**	0.206	0.189	0.158
est	0.317	**0.344**	0.222	0.118

➢ **Highlighted = strongest link**

- Mon" and "nom" both attend most to "My" — a sensible pattern even in this untrained toy model.
- In a fully trained transformer, this table is what learns real alignments: Ravi ↔ Ravi, name ↔ nom, is ↔ est.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

DECODER: Feed-Forward → Linear → Softmax

- The final projection turns the decoder state into word probabilities.

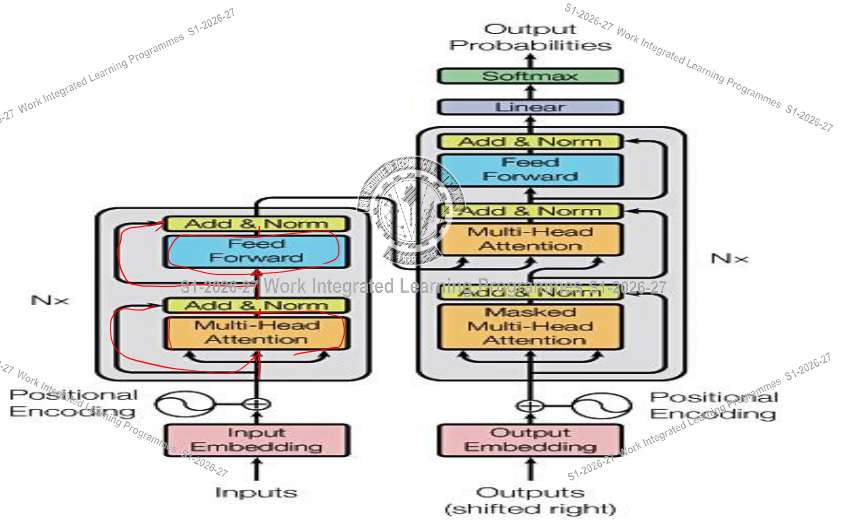
Decoder saw	Mon	nom	est	Ravi	<eos>	Predicted
<sos>	0.195	0.190	0.155	0.187	0.273	<eos>
Mon	0.301	0.140	0.305	0.106	0.148	est
nom	0.365	0.122	0.319	0.108	0.086	Mon
est	0.311	0.167	0.157	0.221	0.144	Mon

➢ **Why the predictions look "wrong"**

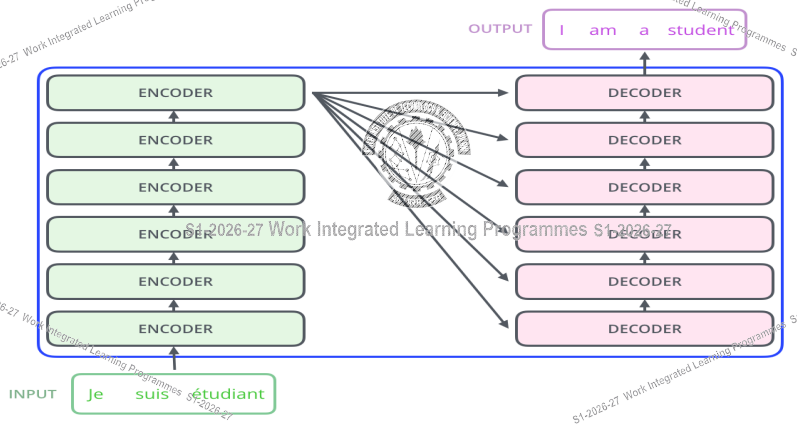
- Every weight matrix used in this walkthrough (W_Q, W_K, W_V, W_1, W_2, W_out) was randomly initialized, not trained. The goal was to make the mechanics traceable by hand — embedding → positional encoding → self-attention → cross-attention → feed-forward → softmax — not to reproduce a correct translation.
- During real training, backpropagation adjusts every matrix over millions of sentence pairs until cross-attention reliably aligns "Ravi"↔"Ravi", "name"↔"nom", "is"↔"est", "My"↔"Mon"/"m'appelle", and softmax sharply peaks on the correct next word at each step.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

TRANSFORMERS



S1-2026-27 Work Integrated Learning Programmes S1-2026-27



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

BERT

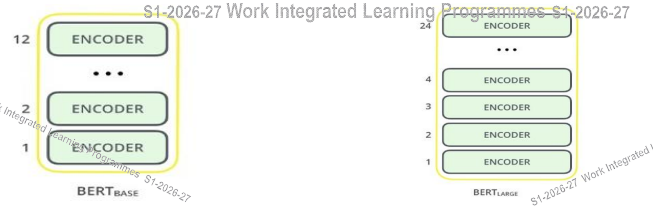
- BERT, which stands for Bidirectional Encoder Representations from Transformers.
- In 2018, Google introduced and open-sourced BERT. In its research stages, the framework achieved groundbreaking results in 11 natural language understanding tasks, including sentiment analysis, semantic role labeling, sentence classification ,words with multiple meanings etc.
- BERT is a “deeply bidirectional” model. Bidirectional means that BERT learns information from both the left and the right side of a token’s context during the training phase

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

BERT

- The BERT architecture builds on top of Transformer. We currently have two variants available:–
  - BERT Base: 12 layers (transformer blocks), 12 attention heads, and 110 million parameters–
  - BERT Large: 24 layers (transformer blocks), 16 attention heads and, 340 million parameter

S1-2026-27 Work Integrated Learning Programmes S1-2026-27



Pre-trained models

- patentBERT - a BERT model fine-tuned to perform patent classification.
- docBERT - a BERT model fine-tuned for document classification.
- bioBERT - a pre-trained biomedical language representation model for biomedical text mining.
- VideoBERT - a joint visual-linguistic model for process unsupervised learning of an abundance of unlabeled data on Youtube.
- SciBERT - a pretrained BERT model for scientific text
- G-BERT - a BERT model pretrained using medical codes with hierarchical representations using graph neural networks (GNN) and then fine-tuned for making medical recommendations.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Pre-trained models

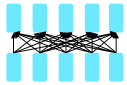
- TinyBERT by Huawei - a smaller, "student" BERT that learns from the original "teacher" BERT, performing transformer distillation to improve efficiency. TinyBERT produced promising results in comparison to BERT-base while being 7.5 times smaller and 9.4 times faster at inference.
- DistilBERT by HuggingFace - a supposedly smaller, faster, cheaper version of BERT that is trained from BERT, and then certain architectural aspects are removed for the sake of efficiency
- AstraZeneca and NVIDIA developed MegaMolBART, a transformer tailored for drug discovery

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Pre-training for three types of architectures

- **Encoder-only models**
  - ✓ Gets bidirectional context – can condition on future!
  - ✓ How do we train them to build strong representations?
  - ✓ Also known as *auto-encoding models*
- For tasks that require understanding of the input, e.g sentence classification and named entity recognition
- Representatives of this family of models include:

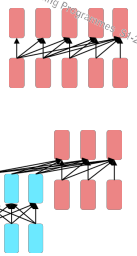
ALBERT, BERT, DistilBERT, , ELECTRA, RoBERTa



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Pre-training for three types of architectures

- **Decoder-only models**
  - ✓ Language models
  - ✓ Nice to generate from; can't condition on future words
  - ➢ These models are also known as *auto-regressive models*.
  - ➢ Representatives of this family of models include: CTRL, GPT series, Transformer XL
- **Encoder-decoder models or sequence-to-sequence models**
  - ✓ For generative tasks that require an input, such as translation or summarization . BART/T5-like



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

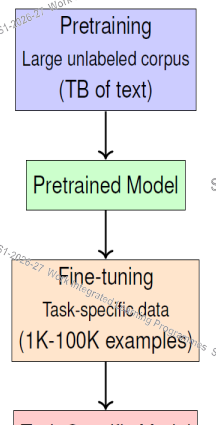
HuggingFace Hub

- What is HuggingFace?
  - Open platform for ML models, datasets, and applications
  - 500,000+ pre-trained models available
  - Easy-to-use `transformers` library (Python)
  - Community-driven: researchers, companies, hobbyists
- Popular Models on HuggingFace:
  - **Encoder-only**: `bert-base-uncased`, `roberta-large`, `distilbert-base`
  - **Decoder-only**: `gpt2`, `meta-llama/Llama-3-8B`, `mistralai/Mistral-7B`
  - **Encoder-decoder**: `t5-base`, `facebook/bart-large`, `google/flan-t5-large`
- Benefits: Instant access, no training needed, proven performance

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Transfer Learning & Fine-tuning

Transfer Learning Paradigm:



Self-supervised: MLM, autoregressive

Supervised: Classification, NER, etc.

Why Not Train From Scratch?

- Requires massive data (billions of tokens)
- Expensive compute (thousands of GPU-hours)
- Pretrained models already learned general language patterns

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Fine-tuning Example: Sentiment Analysis

Task: Classify movie reviews as positive/negative
Dataset: IMDb (25K train, 25K test)
Base Model: BERT or RoBERTa

```python
from transformers import (AutoTokenizer, Trainer,
    AutoModelForSequenceClassification,  TrainingArguments)

# Load model and tokenizer
tokenizer = AutoTokenizer.from_pretrained('bert-base-uncased')
model = AutoModelForSequenceClassification.from_pretrained(
    'bert-base-uncased', num_labels=2
)

# Tokenize data
def tokenize_function(examples):
    return tokenizer(examples['text'], padding='max_length',
                     truncation=True, max_length=512)

tokenized_train = train_dataset.map(tokenize_function, batched=True)
```



```python
# Training arguments
training_args = TrainingArguments(
    output_dir='./results', num_train_epochs=3,
    per_device_train_batch_size=16, per_device_eval_batch_size=64,
    learning_rate=2e-5, weight_decay=0.01,
    evaluation_strategy='epoch', save_strategy='epoch',
    load_best_model_at_end=True,
)

# Trainer
trainer = Trainer(model=model, args=training_args,
    train_dataset=tokenized_train, eval_dataset=tokenized_val,
)

# Train
trainer.train()

# Evaluate
results = trainer.evaluate()
print(f"Accuracy: {results['eval_accuracy']:.4f}")
```

Summary

- What We Learned:
  - Transformers revolutionized AI through pure attention
  - Three architectures: Encoder-only, Decoder-only, Encoder-decoder
  - Foundation of all modern LLMs and multimodal models
  - HuggingFace makes pretrained models easily accessible
  - Fine-tuning adapts models to specific tasks efficiently
- Emerging Trends:
  - Multimodal Transformers: Text + Image + Audio (GPT-4V, Gemini)
  - Efficiency Improvements: Sparse attention, linear attention, LoRA
  - Long Context: 100K-1M token context windows (Gemini, Claude)
  - Mixture of Experts (MoE): Sparse activation for efficiency
- Practical Takeaway:
  - Start with pretrained models from HuggingFace
  - Fine-tune on your task-specific data

Optimizers

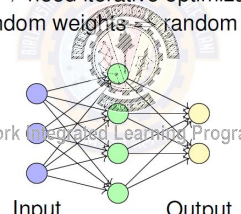
S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Why Optimization Matters

- Machine Learning Goal: Find parameters θ^* that minimize loss

$$\theta^* = \arg\min_{\theta} \mathcal{L}(\theta)$$

- Deep neural networks have millions to billions of parameters
- No closed-form solution $\Rightarrow$ need iterative optimization algorithms
- Without optimization: random weights $\rightarrow$ random predictions $\Rightarrow$ no learning



$\theta = \{W_1, b_1, W_2, b_2, ...\}$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Gradient Descent: One Iteration Example

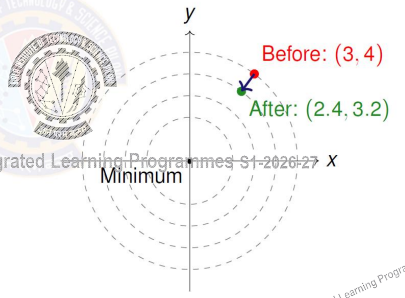
- Simple 2D function: $f(x, y) = x^2 + y^2$
- Gradient: $\nabla f = [2x, 2y]$
- Starting point: $(x_0, y_0) = (3, 4)$, Learning rate $\eta = 0.1$

Before Update:

- Position: $(3, 4)$
- Loss: $f(3, 4) = 9 + 16 = 25$
- Gradient: $\nabla f = [6, 8]$

After Update:

- $x_1 = 3 - 0.1(6) = 2.4$
- $y_1 = 4 - 0.1(8) = 3.2$
- Loss: $f(2.4, 3.2) = 16$ (36% reduction!)



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Loss Functions Review

Loss Function	Formula	Gradient	Use Case
Mean Squared Error	$\frac{1}{2}(\hat{y} - y)^2$	$(\hat{y} - y)$	Regression tasks
Mean Absolute Error	$\lvert\hat{y} - y\rvert$	$\text{sign}(\hat{y} - y)$	Regression with outliers
Binary Cross-Entropy	$-[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$	$(\hat{y} - y)$	Binary classification
Categorical Cross-Entropy	$-\sum_k y_k \log \hat{y}_k$	$(\hat{\mathbf{y}} - \mathbf{y})$	Multi-class classification

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

- Key observation: All gradients have form (prediction $-$ target) or similar
- Simple gradients enable efficient backpropagation
- Choice of loss function depends on problem type and data characteristics

Optimization Challenges - Concepts

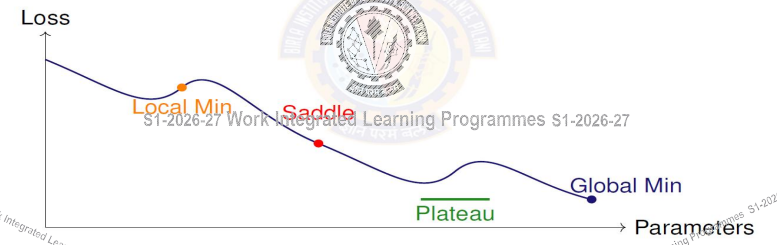
- Saddle Points:
  - Gradient $\nabla \mathcal{L} = 0$ but not a minimum
  - Point where some directions go up, others go down
  - Common in high-dimensional spaces
- Plateaus:
  - Flat regions with very small gradients
  - Learning becomes very slow
  - Hard to determine if stuck or just slow progress
- Non-Convex Landscapes:
  - Multiple local minima
  - No guarantee of finding global minimum
  - Different initializations lead to different solutions

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Optimization Challenges - Visualization

How to Identify:

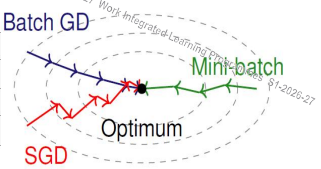
- Loss curve flattens but validation error still high
- Gradient norms approach zero: $\|\nabla \mathcal{L}\| \to 0$
- Training progress stagnates for many epochs



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Gradient Descent Variants: Comparison

Method	Gradient	Epoch	Memory	Convergence
Batch GD	$\frac{1}{N}\sum_{i=1}^{N}\nabla\mathcal{L}_i$	1	High	Smooth, slow
SGD	$\nabla\mathcal{L}_i$	N	Low	Noisy, fast
Mini-batch	$\frac{1}{B}\sum_{i=1}^{B}\nabla\mathcal{L}_i$	N/B	Medium	Balanced



- Example: Dataset with $N = 10,000$ samples, batch size $B = 32$
  - Batch GD: 1 update per epoch (all data at once)
  - SGD: 10,000 updates per epoch (one sample at a time)
  - Mini-batch: $10,000/32 \approx 313$ updates per epoch
- Practical choice: Mini-batch with $B \in \{32, 64, 128, 256\}$
  - Balances computation speed (GPU efficiency) with convergence stability
  - More updates than Batch GD, less noise than SGD

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Algorithm: Mini-Batch Gradient Descent

Algorithm 1: Mini-Batch Gradient Descent

Data: Learning rate η, batch size B, initial parameters θ

while *stopping criterion not met* **do**
  Shuffle training data
  for *each mini-batch* $\mathcal{B}$ *of size* B **do**
    Compute loss on mini-batch: $\mathcal{L}(\theta) = \frac{1}{B}\sum_{i\in\mathcal{B}}\mathcal{L}_i(\theta)$
    Compute gradient: $\mathbf{g} = \nabla_\theta \mathcal{L}(\theta) = \frac{1}{B}\sum_{i\in\mathcal{B}}\nabla\mathcal{L}_i(\theta)$
    Update parameters: $\theta \leftarrow \theta - \eta\mathbf{g}$

- We'll use this as our baseline for numerical examples

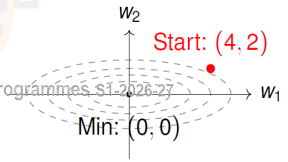
S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Toy Problem for Numerical Examples

- Loss function: $\mathcal{L}(w_1, w_2) = w_1^2 + 4w_2^2$ (elongated bowl)
- Gradient: $\nabla\mathcal{L} = [2w_1, 8w_2]$
- Initial point: $(w_1, w_2) = (4, 2)$, Loss $= 16 + 16 = 32$
- Learning rate: $\eta = 0.1$

Key Characteristics:

- w_2 direction 4× steeper than w_1
- Fixed LR causes oscillation in w_2
- Adaptive methods should help



We'll track this problem through Mini-batch GD, Momentum, and Adam

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

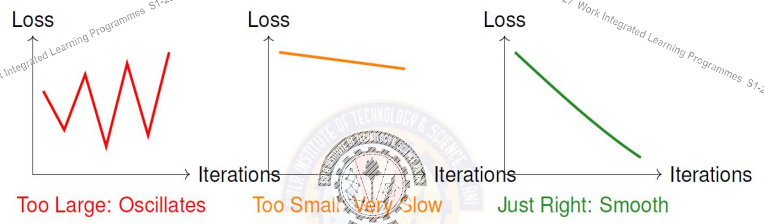
Mini-Batch GD: Numerical Example

- Problem: $\mathcal{L}(w_1, w_2) = w_1^2 + 4w_2^2$, start at $(4, 2)$, $\eta = 0.1$

Iter	Loss	∇_{w_1}	∇_{w_2}	$w_1 \leftarrow w_1 - \eta\nabla_{w_1}$	$w_2 \leftarrow w_2 - \eta\nabla_{w_2}$
0	32.00	8.0	16.0	$4 - 0.1(8) = 3.20$	$2 - 0.1(16) = 0.40$
1	10.88	6.4	3.2	$3.2 - 0.1(6.4) = 2.56$	$0.4 - 0.1(3.2) = 0.08$
2	6.58	5.1	0.64	$2.56 - 0.1(5.1) = 2.05$	$0.08 - 0.1(0.64) = 0.016$
3	4.21	4.1	0.13	$2.05 - 0.1(4.1) = 1.64$	$0.016 - 0.1(0.13) = 0.003$

- Observation: Loss decreases: $32 \rightarrow 10.88 \rightarrow 6.58 \rightarrow 4.21$
- Issue: w_2 takes large steps ($\Delta w_2 = -1.6$) compared to w_1 ($\Delta w_1 = -0.8$) due to steeper gradient (coefficient 8 vs 2). This creates unbalanced progress: w_2 moves too aggressively.
- This motivates: Momentum (smooth updates) and adaptive methods (per-parameter rates)

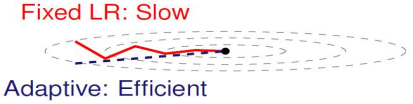
Learning Rate Effects



- Too large η: Oscillation, potential divergence, loss jumps around
  - Eg: $\eta = 1.0$: Loss oscillates wildly: $15.3 \rightarrow 8.2 \rightarrow 19.7$ - diverging!
- Too small η: Very slow progress, may not converge in reasonable time
  - Eg: $\eta = 0.0001$: Barely improves: $22.5 \rightarrow 21.8 \rightarrow 20.1$ - would need 1000s of iterations
- Optimal η: Fast, smooth convergence to minimum
  - Eg: $\eta = 0.01$: Smooth descent: $22.5 \rightarrow 15.2 \rightarrow 3.1$ - reaches low loss quickly

Motivation for Adaptive Techniques

- Problem with Fixed Learning Rate:
  - All parameters updated with same learning rate
  - Different parameters may need different rates
  - Some directions in loss landscape are steeper than others
- Sparse vs Dense Features:
  - Sparse features: appear rarely, need larger updates when they do
  - Dense features: appear frequently, need smaller, careful updates
- Elongated Contours:
  - Fixed LR causes slow zigzag progress
  - Need different step sizes for different directions



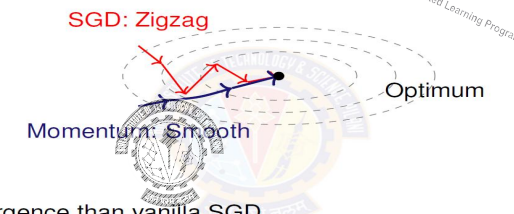
Momentum: Physical Intuition

- Analogy: Rolling ball gathering velocity down a hill
  - Ball accumulates momentum in consistent directions
  - Dampens oscillations in perpendicular directions
  - Can roll through small bumps (escape shallow local minima)
- Mathematical Formulation:

$$\mathbf{v}_t = \beta\mathbf{v}_{t-1} + \nabla_\theta \mathcal{L}(\theta_t)$$
$$\theta_{t+1} = \theta_t - \eta\mathbf{v}_t$$

- Key Components:
  - $\mathbf{v}_t$: velocity (accumulated gradient)
  - β: momentum coefficient, typically 0.9
  - Gradient adds to velocity, not directly to parameters

Momentum: Visualization



Benefits:

- Faster convergence than vanilla SGD
- Smooths out oscillations in ravines
- Better escape from local minima

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Algorithm: SGD with Momentum

Algorithm 2: SGD with Momentum

Data: Learning rate η, momentum $\beta = 0.9$, initial parameters θ

Initialize velocity $\mathbf{v} = 0$

while *stopping criterion not met* **do**

  for *each mini-batch* $\mathcal{B}$ **do**

    Compute gradient: $\mathbf{g} = \frac{1}{B}\sum_{i\in\mathcal{B}} \nabla_\theta \mathcal{L}_i(\theta)$

    Update velocity: $\mathbf{v} \leftarrow \beta\mathbf{v} + \mathbf{g}$

    Update parameters: $\theta \leftarrow \theta - \eta\mathbf{v}$

- Hyperparameter: $\beta = 0.9$ means 90% of previous velocity retained
- Additional memory: store velocity vector same size as parameters
- When to use: General deep learning, production systems

Momentum: Numerical Example

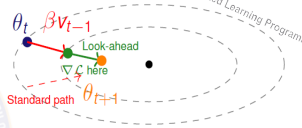
- Same problem, reduced LR: $\mathcal{L}(w_1, w_2) = w_1^2 + 4w_2^2$, start at $(4, 2)$, $\eta = 0.05$, $\beta = 0.9$

Iter	Loss	$[\nabla w_1, \nabla w_2]$	$v_1 \leftarrow \beta v_1 + w_1$	$v_2 \leftarrow \beta v_2 + w_2$	w_1	w_2
0	32.00	$[8, 16]$	8	16	3.6	1.2
1	18.72	$[7.2, 9.6]$	$7.2 + 7.2 = 14.4$	$14.4 + 9.6 = 24$	2.88	0
2	8.29	$[5.76, 0]$	$13.0 + 5.76 = 18.7$	$21.6 + 0 = 21.6$	1.94	-1.08
3	8.43	$[3.88, -8.64]$	$16.9 + 3.88 = 20.7$	$19.4 - 8.64 = 10.8$	0.91	-1.62

- Still oscillates! $w_2 : 2 \to 1.2 \to 0 \to -1.08 \to -1.62$
- Key lesson: Momentum amplifies steps - needs smaller η than vanilla GD
- For remaining examples: We'll use $\eta = 0.05$ for fair comparison

Nesterov Accelerated Gradient

- Standard Momentum:
  1. Compute gradient at current θ_t
  2. Add to velocity
  3. Take step
- Nesterov Momentum:
  1. "Look ahead": jump to $\theta_t - \beta v_{t-1}$
  2. Compute gradient there
  3. Add to velocity, take step
- Benefit: More responsive to gradient changes, slightly faster convergence



Update:

$$v_t = \beta v_{t-1} + \nabla \mathcal{L}(\theta_t - \beta v_{t-1})$$

$$\theta_{t+1} = \theta_t - \eta v_t$$

From Momentum to Adaptive Methods

- Momentum solved one problem:
  - Smooths oscillations by accumulating velocity
  - Faster convergence in consistent directions
- But still has a limitation:
  - All parameters still share the same learning rate η
  - In our toy example: w_1 has small gradients, w_2 has large gradients
  - Ideally: w_1 needs larger rate, w_2 needs smaller rate
- Next: Adaptive methods
  - AdaGrad: Accumulates squared gradients → per-parameter rates
  - RMSprop: Fixes AdaGrad's shrinking learning rate
  - Adam: Combines RMSprop + Momentum = best of both worlds

Why Different Parameters Need Different Rates

- Example from our toy problem: $\mathcal{L}(w_1, w_2) = w_1^2 + 4w_2^2$

Parameter	Actual Gradients	Problem	Needs
w_1	[8.0, 6.4, 5.1, 4.1]	Moderate, decreasing	Moderate LR
w_2	[16.0, 3.2, 0.64, 0.13]	Large initially, drops fast	Adaptive LR

- Observation from Mini-batch GD iterations:
  - ∇_{w_2} starts 2× larger than ∇_{w_1} (16 vs 8)
  - ∇_{w_2} decreases faster: $16 \to 3.2 \to 0.64$ (80% reduction)
  - ∇_{w_1} decreases slower: $8 \to 6.4 \to 5.1$ (36% reduction)
- Problem with fixed LR:
  - Early iterations: w_2 takes huge steps (needs smaller LR)
  - Later iterations: w_1 still needs progress (needs maintained LR)
- Adaptive solution: Scale LR inversely with gradient history

AdaGrad: Per-Parameter Learning Rates

- Key Idea: Adapt learning rate for each parameter based on historical gradients
  - Parameters with large gradients ⇒ smaller effective learning rate
  - Parameters with small gradients ⇒ larger effective learning rate
- Update Rules:

$$\mathbf{s}_t = \mathbf{s}_t + \mathbf{g}_t \odot \mathbf{g}_t$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\mathbf{s}_t + \epsilon}} \odot \mathbf{g}_t$$

  where $\mathbf{g}_t = \nabla_\theta \mathcal{L}(\theta_t)$ and $\odot$ is element-wise multiplication
- Hyperparameter: $\epsilon = 10^{-8}$ for numerical stability
- Problem: Learning rate decreases monotonically
  - $\mathbf{s}_t$ accumulates squared gradients without bound
  - Eventually $\eta/\sqrt{\mathbf{s}_t} \to 0$ and learning stops
- Use Case: Sparse features, NLP with large vocabularies

RMSprop: Fixing AdaGrad

- Problem with AdaGrad: Accumulated sum $\mathbf{s}_t$ grows forever
- RMSprop Solution: Use exponential moving average instead

$$\mathbf{s}_t = \gamma \mathbf{s}_{t-1} + (1-\gamma)\mathbf{g}_t^2$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\mathbf{s}_t + \epsilon}} \mathbf{g}_t$$

- Key Differences from AdaGrad:
  - $\mathbf{s}_t$ doesn't grow indefinitely (exponential decay)
  - Recent gradients weighted more heavily than old ones
  - Learning rate doesn't vanish over time
- Hyperparameters:
  - $\gamma = 0.9$ (decay rate for moving average)
  - $\eta = 0.001$ (learning rate)
  - $\epsilon = 10^{-8}$ (numerical stability)
- Use Case: RNNs, non-stationary problems, online learning

S1-2026-27 Work Integrated Learning Programmes

Algorithm: RMSprop

Algorithm 3: RMSprop

Data: Learning rate $\eta = 0.001$, decay rate $\gamma = 0.9$, $\epsilon = 10^{-8}$

Data: Initial parameters θ

Initialize accumulated gradient $\mathbf{s} = 0$

while *stopping criterion not met* **do**

  for *each mini-batch* $\mathcal{B}$ **do**

    Compute gradient: $\mathbf{g} = \frac{1}{B} \sum_{i \in \mathcal{B}} \nabla_\theta \mathcal{L}_i(\theta)$

    Accumulate squared gradient: $\mathbf{s} \leftarrow \gamma \mathbf{s} + (1 - \gamma)\mathbf{g} \odot \mathbf{g}$

    Update parameters: $\theta \leftarrow \theta - \frac{\eta}{\sqrt{\mathbf{s}+\epsilon}} \odot \mathbf{g}$

- Good empirical performance across many problems
- Still missing: no momentum term

RMSprop: Numerical Example

Same problem: $\mathcal{L}(w_1, w_2) = w_1^2 + 4w_2^2$, start at $(4, 2)$, $\eta = 0.05$, $\gamma = 0.9$, $\epsilon = 10^{-8}$

Iteration 0: $s_1 = s_2 = 0$

$$\text{Loss} = w_1^2 + 4w_2^2 = 4^2 + 4(2^2) = 16 + 16 = 32$$

$$g_1 = \nabla w_1 = 2w_1 = 2(4) = 8$$

$$g_2 = \nabla w_2 = 8w_2 = 8(2) = 16$$

$$s_1 \leftarrow \gamma s_1 + (1 - \gamma) g_1^2 = 0.9(0) + 0.1(8^2) = 0 + 6.4 = 6.4$$

$$s_2 \leftarrow \gamma s_2 + (1 - \gamma) g_2^2 = 0.9(0) + 0.1(16^2) = 0 + 25.6 = 25.6$$

$$\text{eff}_\eta_1 = \frac{\eta}{\sqrt{s_1 + \epsilon}} = \frac{0.05}{\sqrt{6.4}} = \frac{0.05}{2.53} = 0.020$$

$$\text{eff}_\eta_2 = \frac{\eta}{\sqrt{s_2 + \epsilon}} = \frac{0.05}{\sqrt{25.6}} = \frac{0.05}{5.06} = 0.010$$

$$w_1 \leftarrow w_1 - \text{eff}_\eta_1 \cdot g_1 = 4 - 0.02(8) = 4 - 0.16 = 3.84$$

$$w_2 \leftarrow w_2 - \text{eff}_\eta_2 \cdot g_2 = 2 - 0.01(16) = 2 - 0.16 = 1.84$$

RMSprop: Numerical Example

- Same problem: $\mathcal{L}(w_1, w_2) = w_1^2 + 4w_2^2$, start at $(4, 2)$, $\eta = 0.05$, $\gamma = 0.9$, $\epsilon = 10^{-8}$

Iter	Loss	$\nabla_{w_1}, \nabla_{w_2}$	$s_1 \leftarrow$	$s_2 \leftarrow$	eff η_1	eff η_2	$w_1 \leftarrow$	$w_2 \leftarrow$
0	32.00	[8, 16]	6.4	25.6	0.020	0.010	3.84	1.84
1	28.30	[7.68, 14.72]	11.7	44.7	0.015	0.0075	3.72	1.73
2	25.80	[7.44, 13.84]	16.0	59.3	0.0125	0.0065	3.63	1.64
3	23.93	[7.26, 13.12]	19.7	70.6	0.011	0.0060	3.55	1.56

- Key observation:
  - Larger gradients (w_2) $\rightarrow$ larger s_2 $\rightarrow$ smaller effective LR (automatic adaptation!)
  - Loss decreases steadily: $32 \rightarrow 28.30 \rightarrow 25.80 \rightarrow 23.93$
- Issue: Slow convergence compared to momentum (no velocity accumulation)
- Iter 3 comparison: Mini-batch GD: 4.21, Momentum: 8.43, RMSprop: 23.93

RMSprop vs Momentum: What's Missing?

Feature	Momentum	RMSprop
Accumulates velocity?	✓	×
Smooths oscillations?	✓	×
Per-parameter learning rates?	×	✓
Adapts to gradient history?	×	✓

- Momentum: Great for smoothing, but fixed LR for all parameters
- RMSprop: Great for adaptation, but no velocity accumulation
- Question: Can we get the best of both?
- Answer: **Yes! Adam combines both ideas**
  - First moment m (like momentum): accumulates gradients
  - Second moment v (like RMSprop): accumulates squared gradients
  - Result: smooth updates with per-parameter adaptation

Adam: Adaptive Moment Estimation

- Key Idea: Combine momentum (first moment) + RMSprop (second moment)
- Update Rules :

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t \quad \text{(first moment: momentum)}$$
$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\mathbf{g}_t^2 \quad \text{(second moment: adaptive LR)}$$
$$\theta_{t+1} = \theta_t - \frac{\eta \mathbf{m}_t}{\sqrt{\mathbf{v}_t} + \epsilon}$$

- Default Hyperparameters:
  - $\beta_1 = 0.9$ (momentum decay)
  - $\beta_2 = 0.999$ (RMSprop decay)
  - $\eta = 0.001$ or 0.0003 (learning rate)
  - $\epsilon = 10^{-8}$ (numerical stability)
- Note: Full Adam includes bias correction (shown in next slide)

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Algorithm: Adam

Algorithm 4: Adam

Data: $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$

Data: Initial parameters θ

Initialize first moment $\mathbf{m} = 0$, second moment $\mathbf{v} = 0$, timestep $t = 0$

while *stopping criterion not met* **do**

  for *each mini-batch* $\mathcal{B}$ **do**

    $t \leftarrow t + 1$

    Compute gradient: $\mathbf{g} = \frac{1}{B}\sum_{i \in \mathcal{B}} \nabla_\theta \mathcal{L}_i(\theta)$

    Update first moment: $\mathbf{m} \leftarrow \beta_1 \mathbf{m} + (1-\beta_1)\mathbf{g}$

    Update second moment: $\mathbf{v} \leftarrow \beta_2 \mathbf{v} + (1-\beta_2)\mathbf{g}^2$

    Bias correction: $\hat{\mathbf{m}} = \mathbf{m}/(1-\beta_1^t)$, $\hat{\mathbf{v}} = \mathbf{v}/(1-\beta_2^t)$

    Update parameters: $\theta \leftarrow \theta - \eta \hat{\mathbf{m}}/(\sqrt{\hat{\mathbf{v}}} + \epsilon)$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Adam: Numerical Example (Iteration 0)

Same problem: $\mathcal{L}(w_1, w_2) = w_1^2 + 4w_2^2$, start at $(4, 2)$, $\eta = 0.05$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$

Iteration 0:

$m_1 = m_2 = 0, \quad v_1 = v_2 = 0, \quad t = 1$

$\text{Loss} = w_1^2 + 4w_2^2 = 4^2 + 4(2^2) = 32$

$g_1 = \nabla_{w_1} = 2w_1 = 2(4) = 8$

$g_2 = \nabla_{w_2} = 8w_2 = 8(2) = 16$

$m_1 \leftarrow \beta_1 m_1 + (1-\beta_1) g_1$
$= 0.9(0) + 0.1(8) = 0.8$

$m_2 \leftarrow \beta_1 m_2 + (1-\beta_1) g_2$
$= 0.9(0) + 0.1(16) = 1.6$

$v_1 \leftarrow \beta_2 v_1 + (1-\beta_2) g_1^2$
$= 0.999(0) + 0.001(64) = 0.064$

$v_2 \leftarrow \beta_2 v_2 + (1-\beta_2) g_2^2$
$= 0.999(0) + 0.001(256) = 0.256$

$\hat{m}_1 = \frac{m_1}{1-\beta_1^t} = \frac{0.8}{1-0.9^1} = \frac{0.8}{0.1} = 8$

$\hat{m}_2 = \frac{m_2}{1-\beta_1^t} = \frac{1.6}{0.1} = 16$

$\hat{v}_1 = \frac{v_1}{1-\beta_2^t} = \frac{0.064}{1-0.999^1} = \frac{0.064}{0.001} = 64$

$\hat{v}_2 = \frac{v_2}{1-\beta_2^t} = \frac{0.256}{0.001} = 256$

$w_1 \leftarrow w_1 - \eta \frac{\hat{m}_1}{\sqrt{\hat{v}_1} + \epsilon} = 4 - 0.05 \times \frac{8}{\sqrt{64}} = 3.95$

$w_2 \leftarrow w_2 - \eta \frac{\hat{m}_2}{\sqrt{\hat{v}_2} + \epsilon} = 2 - 0.05 \times \frac{16}{\sqrt{256}} = 1.95$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Adam: Numerical Example

- Problem: $\mathcal{L}(w_1, w_2) = w_1^2 + 4w_2^2$, start at $(4, 2)$
- Hyperparameters: $\eta = 0.05$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$

Iter	Loss	∇	m_1, m_2	v_1, v_2	$\hat{m}_1, \hat{m}_2$	$\hat{v}_1, \hat{v}_2$	$w_1 \leftarrow$	$w_2 \leftarrow$
0	32.00	[8, 16]	[0.8, 1.6]	[0.064, 0.256]	[8, 16]	[64, 256]	$4 - 0.05 = 3.95$	$2 - 0.05 = 1.95$
1	30.81	[7.9, 15.6]	[1.51, 3.0]	[0.126, 0.499]	[7.95, 15.79]	[63.2, 249.6]	$3.95 - 0.05 = 3.90$	$1.95 - 0.05 = 1.90$
2	29.65	[7.8, 15.2]	[2.14, 4.22]	[0.187, 0.730]	[7.89, 15.57]	[62.4, 243.5]	$3.90 - 0.05 = 3.85$	$1.90 - 0.05 = 1.85$
3	28.51	[7.7, 14.8]	[2.70, 5.28]	[0.246, 0.948]	[7.84, 15.38]	[61.6, 237.3]	$3.85 - 0.05 = 3.80$	$1.85 - 0.05 = 1.80$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

- Observation: Steady decrease: $32 \to 30.81 \to 29.65 \to 28.51$
- Notice: Updates are nearly constant (0.05 per iteration) due to bias correction balancing momentum and adaptive rates
- Iter 3 comparison ($\eta = 0.05$): Mini-batch: 4.21, Momentum: 8.43, RMSprop: 23.93, Adam: 28.51

Adam: Why So Popular?

- Bias Correction:
  - Ensures unbiased estimates, especially in early iterations
- What Adam Provides:
  - Momentum for faster convergence and smoothing
  - Individual adaptive learning rates per parameter
- Why It's the Default Choice:
  - Works well out-of-the-box with minimal tuning
  - Robust across different problem types
  - Combines multiple successful ideas
  - Good performance on wide range of architectures
- When to Prefer SGD+Momentum:
  - Production systems where you can afford careful tuning
  - Computer vision tasks (sometimes better final performance)
  - When you need best possible generalization

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

AdamW: Decoupled Weight Decay

- Difference from Adam: Separates weight decay from gradient-based update
- Standard Adam with L2 regularization:
  - L2 penalty added to loss: $\mathcal{L}_{total} = \mathcal{L} + \frac{\lambda}{2}\|\theta\|^2$
  - Gradient includes regularization term
  - Interacts with adaptive learning rates in unintended ways
- AdamW Update:

$$\theta_{t+1} = \theta_t - \eta \left(\frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} + \lambda \theta_t \right)$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

  - Weight decay applied directly to parameters
  - Decoupled from gradient-based adaptive update
- Hyperparameters:
  - Same as Adam: $\beta_1 = 0.9, \beta_2 = 0.999, \eta = 0.001, \lambda = 0.01$ (weight decay)
- When to use: When combining optimization with regularization, often better than Adam with L2

Adam vs AdamW: Side-by-Side Comparison

Aspect	Adam	AdamW
L2 Regularization	Added to loss: $\mathcal{L} + \frac{\lambda}{2}\|\theta\|^2$	Applied directly to weights
Gradient	Includes reg. term: $\nabla\mathcal{L} + \lambda\theta$	Only from loss: $\nabla\mathcal{L}$
Update Rule	$\theta \leftarrow \theta - \frac{\eta\hat{m}}{\sqrt{\hat{v}}+\epsilon}$	$\theta \leftarrow \theta - \eta\left(\frac{\hat{m}}{\sqrt{\hat{v}}+\epsilon} + \lambda\theta\right)$
Adaptive Rates	Applied to gradient+reg	Applied only to gradient
Weight Decay	Depends on adaptive rates	Constant, decoupled
When to Use	General optimization	With L2 regularization
Performance	Good	Often better generalization

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

- Key insight: AdamW separates optimization from regularization
- Practical advice: Use AdamW when you need weight decay/L2 regularization

Example

Gradient Descent

$g_t = \nabla f(x_t) = 2x_t$

$x_{t+1} = x_t - \eta \cdot g_t$

t	x_t	$g_t = 2x_t$	$x_{t+1} = x_t - \eta g_t$
0	4.000	8.000	3.200
1	3.200	6.400	2.560
2	2.560	5.120	2.048
			x_3 = 2.048

SGD with Momentum

$g_t = \nabla f(x_t)$

$v_t = \beta \cdot v_{t-1} + \eta \cdot g_t$

$x_{t+1} = x_t - v_t$

(β = 0.9, v_0 = 0)

t	x_t	$g_t = 2x_t$	v_t	x_{t+1}
0	4.000	8.000	0.800	3.200
1	3.200	6.400	1.360	1.840
2	1.840	3.680	1.592	0.248
				x_3 = 0.248

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Nesterov Accelerated Gradient (NAG)

$u_t = x_t - \beta \cdot v_{t-1}$ (look-ahead point)

$g_t = \nabla f(u_t)$

$v_t = \beta \cdot v_{t-1} + \eta \cdot g_t$

t	x_t	u_t	$g_t = 2u_t$	v_t	x_{t+1}
0	4.000	4.000	8.000	0.800	3.200
1	3.200	2.480	4.960	1.216	1.984
2	1.984	0.890	1.780	1.272	0.712

Example

Adagrad

$g_t = \nabla f(x_t)$

$s_t = s_{t-1} + g_t^2 \quad (s_0 = 0)$

$x_{t+1} = x_t - \eta/\sqrt{(s_t+\varepsilon)}\ g_t$

t	x_t	g_t	s_t	$\eta/\sqrt{(s_t+\varepsilon)}$	x_{t+1}
0	4.000	8.000	64.00	0.01250	3.900
1	3.900	7.800	124.84	0.00895	3.830
2	3.830	7.660	183.52	0.00738	3.774
					$x_3 = 3.774$

RMS Prop

$g_t = \nabla f(x_t)$

$r_t = \beta \cdot r_{t-1} + (1-\beta) \cdot g_t^2$

$(r_0=0,\ \beta=0.9)$

$x_{t+1} = x_t - \eta/\sqrt{(r_t+\varepsilon)} \cdot g_t$

t	x_t	g_t	r_t	$\eta/\sqrt{(r_t+\varepsilon)}$	x_{t+1}
0	4.000	8.000	6.400	0.03953	3.684
1	3.684	7.368	11.188	0.02990	3.464
2	3.464	6.927	14.868	0.02594	3.284
					$x_3 = 3.284$

Adam (Adaptive Moment Estimation)

$g_t = \nabla f(x_t)$

$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t \quad$ (1st moment)

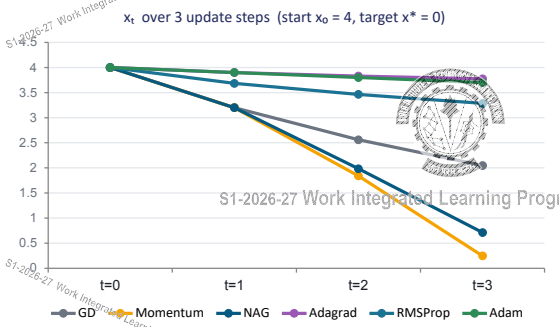
$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2 \quad$ (2nd moment)

$m_t' = m_t/(1-\beta_1^t),\ v_t' = v_t/(1-\beta_2^t)$ (bias-correct)

$x_{t+1} = x_t - \eta \cdot m_t'/(\sqrt{v_t'}+\varepsilon)$

$(\beta_1=0.9,\ \beta_2=0.999)$

t	x_t	g_t	m_t'	v_t'	x_{t+1}
0	4.000	4.000	8.000	64.000	3.900
1	3.200	7.800	7.895	62.42	3.800
2	3.800	7.600	7.786	60.87	3.700



- Momentum and NAG converge fastest here — the accumulated velocity compounds each step.
- Adagrad and Adam start with a step size close to η itself: for a single parameter, $g/\sqrt{g^2} \approx \pm 1$, so the adaptive term normalizes step size rather than scaling it.
- That's an artifact of this 1-D toy problem. Adaptive methods earn their keep in high dimensions, where each parameter has its own gradient scale.
- RMSProp's decaying average lets it move faster than Adagrad, which never forgets old gradients.

Optimizer	Adapts LR?	Uses momentum?	Typical use
GD / SGD	No	No	Simple convex problems, teaching baseline
Momentum	No	Yes	Escaping shallow local minima, ravines
NAG	No	Yes (look-ahead)	Faster, more stable convergence than plain momentum
Adagrad	Yes (per-param)	No	Sparse features (e.g. NLP with rare words)
RMSProp	Yes (EMA)	No	Non-stationary objectives, RNNs
Adam	Yes (EMA)	Yes	Default choice for most deep learning today

Which one should you actually use?

Start here

Adam with η ≈ 1e-3 to 3e-4. It's the default in almost every modern paper and framework for good reason.

Tight compute / simple problem

Plain SGD (+ momentum) with a tuned learning-rate schedule can match or beat Adam and generalizes slightly better in some vision tasks.

Sparse gradients (NLP, embeddings)

Adagrad's per-parameter decay was originally designed for exactly this; RMSProp/Adam usually still work fine.

Training is unstable / diverging

Lower η, add gradient clipping, or switch from Adam to RMSProp — check β_2 in Adam isn't too aggressive.

Rule of thumb: the numerical examples used a single, shared η = 0.1 to keep the arithmetic comparable — in practice, each optimizer has its own natural learning-rate range, and that choice matters more than the optimizer choice itself.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

GD	$x_{t+1} = x_t - \eta \cdot g_t$
Momentum	$v_t = \beta v_{t-1} + \eta g_t \rightarrow x_{t+1} = x_t - v_t$
NAG	$v_t = \beta v_{t-1} + \eta \nabla f(x_t - \beta v_{t-1}) \rightarrow x_{t+1} = x_t - v_t$
Adagrad	$s_t = s_{t-1} + g_t^2 \rightarrow x_{t+1} = x_t - \eta / \sqrt{(s_t + \varepsilon)} \cdot g_t$
RMSProp	$r_t = \beta r_{t-1} + (1-\beta) g_t^2 \rightarrow x_{t+1} = x_t - \eta / \sqrt{(r_t + \varepsilon)} \cdot g_t$
Adam	m_t', v_t' bias-corrected $\rightarrow x_{t+1} = x_t - \eta \cdot m_t' / (\sqrt{v_t'} + \varepsilon)$

$g_t = \nabla f(x_t)$ · β, β_1, β_2 are decay rates · ε is a tiny constant (e.g. 1e-8) preventing division by zero

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Why Schedule Learning Rate?

- Problem with Fixed Learning Rate:
  - Early training: large gradients, need to explore broadly
  - Late training: near optimum, need fine-grained adjustments
  - Fixed LR: either too aggressive later or too conservative early
- Solution: Decrease learning rate over time
  - Start with larger LR for faster initial progress
  - Gradually reduce LR for stable convergence
- Common Strategies:
  - Step decay: drop by fixed factor at intervals
  - Exponential decay: smooth continuous reduction
  - Cosine annealing: smooth with possible restarts
  - Warmup: gradually increase then decay

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

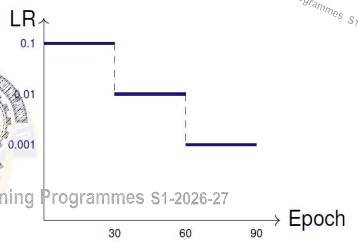
Step Decay Schedule

- Formula:

$$\eta_t = \eta_0 \times \gamma^{\lfloor t/\text{step_size} \rfloor}$$

where t is the current epoch number

- Hyperparameters:
  - η_0: initial learning rate
  - γ: drop factor (typical: 0.1)
  - step_size: epochs between drops (typical: 30)



- When to use: Computer vision, simple and interpretable
- Example: $\eta_0 = 0.1$, $\gamma = 0.1$, step = 30
  - Epoch 10: $\eta = 0.1 \times 0.1^{\lfloor 10/30 \rfloor} = 0.1 \times 0.1^0 = 0.1$
  - Epoch 40: $\eta = 0.1 \times 0.1^{\lfloor 40/30 \rfloor} = 0.1 \times 0.1^1 = 0.01$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

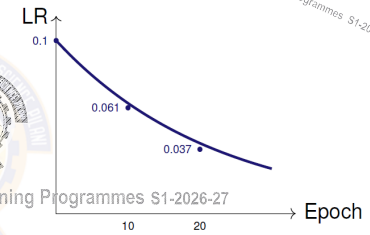
Exponential Decay Schedule

- Formula:

$$\eta_t = \eta_0 \times e^{-\lambda t}$$

where t is the current epoch number

- Hyperparameters:
  - η_0: initial learning rate
  - λ: decay rate (typical: 0.01-0.1)
- Advantages:
  - Smooth, continuous decay
  - No sudden drops
  - Easy to tune with single λ



- Example: $\eta_0 = 0.1$, $\lambda = 0.05$
  - Epoch 10: $\eta = 0.1 \times e^{-0.05(10)} = 0.1 \times e^{-0.5} = 0.1 \times 0.606 = 0.061$
  - Epoch 20: $\eta = 0.1 \times e^{-0.05(20)} = 0.1 \times e^{-1.0} = 0.1 \times 0.368 = 0.037$

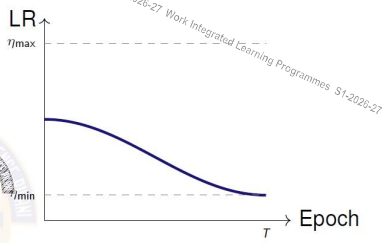
S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Cosine Annealing Schedule

- Formula:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min})\left(1 + \cos\left(\frac{\pi t}{T}\right)\right)$$

where t is current epoch, T is total epochs



- Hyperparameters:
  - $\eta_{\max}$: initial/max LR
  - $\eta_{\min}$: minimum LR
  - T: total training epochs
- Advantages:
  - Very smooth decay
  - No decay rate tuning needed
  - Popular in modern deep learning

- Example: $\eta_{\max} = 0.1$, $\eta_{\min} = 0.001$, $T = 100$
  - Epoch 0: $\eta = 0.001 + \frac{0.099}{2}(1+\cos(0)) = 0.1$
  - Epoch 50: $\eta = 0.001 + 0.0495(1 + \cos(\pi)) = 0.001$

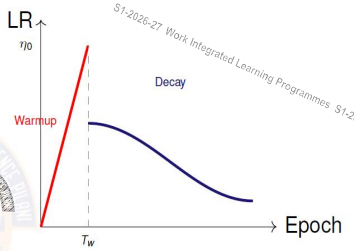
S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Warmup Strategy

- Concept: Gradually increase LR from 0 to target over initial epochs
- Linear Warmup:

$$\eta_t = \eta_0 \times \frac{t}{T_{\text{warmup}}}$$

for $t \leq T_{\text{warmup}}$



- Why Needed?
  - Large initial LR causes instability
  - Important for large batches/models
- Typical: 5-10 epochs warmup, then apply decay schedule

- Example: $\eta_0 = 0.1$, $T_{\text{warmup}} = 5$, then cosine to epoch 100
  - Epoch 2: $\eta = 0.1 \times \frac{2}{5} = 0.04$ (warmup)
  - Epoch 5: $\eta = 0.1 \times \frac{5}{5} = 0.1$ (warmup complete)
  - Epoch 6-100: Apply cosine annealing from 0.1 to 0.001

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Gradient Clipping

- Problem: Exploding gradients cause training instability
  - Gradients become very large (NaN, Inf values)
  - Weight updates are too large
  - Loss diverges or oscillates wildly
- Solution: Gradient Clipping
  - Norm-based Clipping
  - Value-based Clipping
- When to use: RNNs, transformers, unstable training, very deep networks

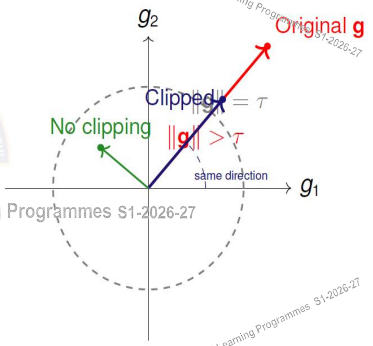
S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Gradient Clipping

- Norm-based Clipping:

$$\mathbf{g} \leftarrow \begin{cases} \mathbf{g} & \text{if } \|\mathbf{g}\| \leq \tau \\ \tau \frac{\mathbf{g}}{\|\mathbf{g}\|} & \text{if } \|\mathbf{g}\| > \tau \end{cases}$$

  - Hyperparameter: τ (typical: 1.0-5.0)
  - Preserves gradient direction
  - Most commonly used



- Value-based Clipping:

$$g_i \leftarrow \text{clip}(g_i, -\tau, \tau)$$

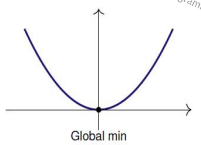
  - Clips each gradient component
  - Hyperparameter: τ (typical: 0.5-1.0)
  - May change gradient direction

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

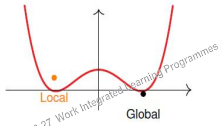
Loss Landscape Visualization

- **Convex Loss:**
  - Single bowl shape
  - Any local minimum = global minimum
  - Example: Linear regression
  - Easy to optimize
- **Non-Convex Loss:**
  - Multiple valleys, peaks, saddles
  - Many local minima
  - Example: Deep neural networks
  - Requires sophisticated optimizers
- **Why Visualize?**
  - Understand optimizer behavior
  - Identify problematic regions
  - Guide hyperparameter choices

Convex:



Non-Convex:



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

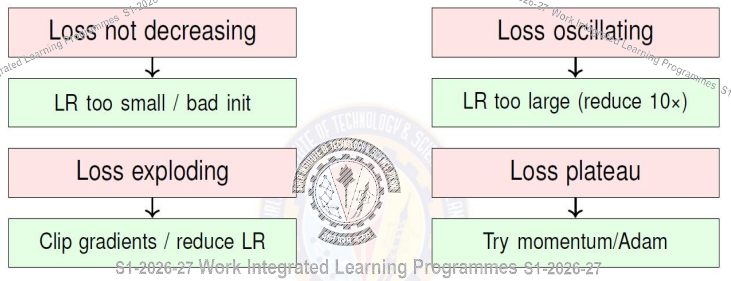
Batch Size Effects on Optimization

Aspect	Small Batch (16-32)	Large Batch (256-1024)
Gradient noise	High	Low
Updates/epoch	Many (N/B)	Few (N/B)
Convergence	Noisy but explores well	Smooth but may get stuck
Generalization	Often better	May need LR scaling
Training time	Slower (more updates)	Faster (GPU efficient)
Memory usage	Low	High
Learning rate	Can use larger	May need warmup

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

- **Linear Scaling Rule:** When doubling batch size, double learning rate
  - Maintains similar convergence speed
- **Practical Recommendation:**
  - Start with batch size 64-128
  - Increase if GPU memory allows and training is slow
  - Use warmup when increasing batch size

Debugging Training Issues



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

- **General Debugging Checklist:**
  - Plot training and validation loss curves
  - Monitor gradient norms: check for NaN/Inf
  - Verify data loading and preprocessing
  - Try overfitting small batch (sanity check)

Stopping Criteria

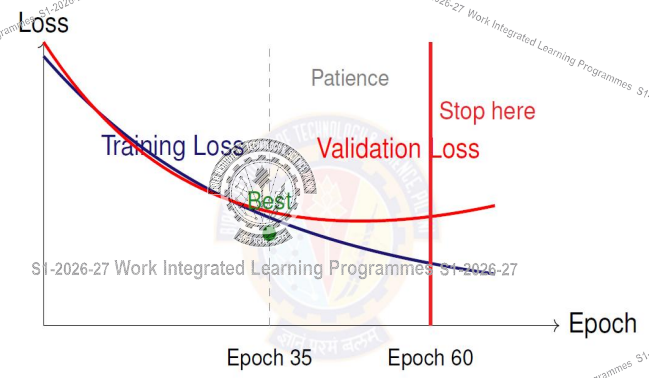
- **Common Stopping Criteria:**

Criterion	Description	Typical Value
Max epochs	Predetermined training limit	100-200 epochs
Early stopping	Validation loss stops improving	Patience: 5-10 epochs
Gradient threshold	Gradient norm below threshold	$\|\nabla \mathcal{L}\| < 10^{-6}$
Loss threshold	Loss reaches target value	Problem-specific

- **Early Stopping (Most Common):**
  - Track best validation loss
  - If no improvement for p epochs (patience), stop
  - Restore model to best validation checkpoint
  - Prevents overfitting
- **Practical Strategy:** Combine max epochs + early stopping
  - Set max epochs as upper bound
  - Use early stopping to terminate earlier if converged

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Early Stopping: Visualization



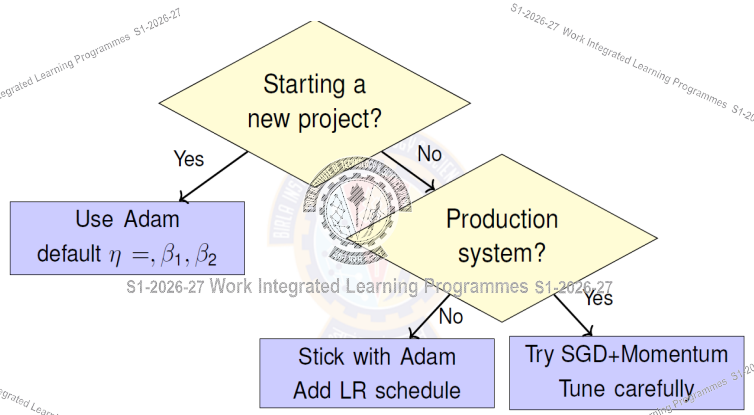
- Best model: Epoch 35 (lowest validation loss)
- Patience: Wait 25 more epochs with no improvement
- Stop: At epoch 60, restore model from epoch 35

Optimizer Comparison Summary

Method	Convergence	Memory	Tuning	Stability
SGD	Slow	Low	Hard	High
SGD+Momentum	Medium	Low	Medium	High
AdaGrad	Good	Medium	Easy	Medium
RMSprop	Good	Medium	Easy	Medium
Adam	Fast	Medium	Very Easy	High
AdamW	Fast	Medium	Very Easy	High

Optimizer	Best Use Cases
Adam/AdamW	Default choice, quick experiments, NLP, most problems
SGD+Momentum	Production CV, when you can afford tuning, best final performance
RMSprop	RNNs, online learning, non-stationary problems
AdaGrad	Sparse features, NLP with large vocabularies

When to Use Which Optimizer



- For best results: Try SGD+Momentum with careful tuning
- Special cases: RMSprop for RNNs, AdaGrad for sparse data

Hyperparameter Summary Table

Optimizer	Key Hyperparameters	Typical Values	Sensitivity
SGD	η	0.01-0.1	High
SGD+Momentum	η, β	0.01-0.1, 0.9	Medium
AdaGrad	η, ϵ	0.01, 10^{-8}	Medium
RMSprop	η, γ, ϵ	0.001, 0.9, 10^{-8}	Medium
Adam	$\eta, \beta_1, \beta_2, \epsilon$	0.001, 0.9, 0.999, 10^{-8}	Low
AdamW	$\eta, \beta_1, \beta_2, \epsilon, \lambda$	0.001, 0.9, 0.999, 10^{-8}, 0.01	Low

Other Hyperparameters	Typical Values
Batch size	32, 64, 128, 256 (powers of 2)
Warmup epochs	5-10 (or 5-10% of total)
Gradient clipping threshold	1.0-5.0 (norm) or 0.5-1.0 (value)
Early stopping patience	5-10 epochs

How to Tune Hyperparameters

- Priority Order:

1. Learning Rate	Grid search: [1e-5, 1e-4, 1e-3, 1e-2]
2. Batch Size	Try: 32 → 64 → 128
3. LR Schedule	Add warmup + cosine
4. Optimizer-specific	Usually keep defaults

- Learning Rate Tuning:
  - Start with optimizer default (0.001 for Adam, 0.01 for SGD)
  - If loss explodes: reduce by 10×
  - If too slow: increase by 3-5×
  - Use LR range test for systematic search

Hyperparameter Interactions

- Learning Rate × Batch Size:
  - Linear scaling rule: Double batch → Double LR
  - Example: Batch 32 ($\eta = 0.01$) → Batch 256 ($\eta = 0.08$)
  - Large batches need warmup to stabilize
- Learning Rate × Optimizer:
  - Adam: Less sensitive, $\eta = 0.001$ works broadly
  - SGD: Very sensitive, needs careful tuning
  - Momentum amplifies steps → needs smaller LR than vanilla SGD
- Batch Size × Generalization:
  - Small batches (32-64): Noisy gradients → regularization effect → better generalization
  - Large batches (256+): Smooth gradients → may overfit → needs LR warmup
- LR Schedule × Optimizer:
  - Adam + aggressive schedule: Can hurt performance
  - SGD benefits more from schedules than Adam

Implementation Best Practices

- Workflow:
  1. Start with Adam ($\eta = 0.001$), default hyperparameters
  2. Monitor training curves (train loss, val loss, gradient norms)
  3. Add cosine annealing schedule
  4. Apply gradient clipping if training unstable
  5. Use warmup for large models/batches
  6. Implement early stopping (patience=10)
  7. Save checkpoints regularly
- Sanity Checks:
  - Overfit small batch first (10-100 examples)
  - Verify loss decreases on single batch
  - Check gradient norms are reasonable (not NaN/Inf)
  - Visualize first few batches of data
- Common Pitfalls:
  - Not monitoring training curves
  - Using wrong default LR (0.01 for Adam instead of 0.001)
  - Batch size too large without LR scaling
  - Training too long without early stopping

Key Takeaways

1. Adam is the default choice
   - Works well out-of-the-box: $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$
   - Robust to hyperparameter choices
   - Combines momentum + adaptive learning rates
2. SGD+Momentum for production
   - Needs careful tuning but often better final performance
   - Popular in computer vision
   - Use when you have time and resources to tune
3. Learning rate is most important
   - Tune this first before anything else
   - Too large: divergence; Too small: slow training
   - Use schedules (cosine annealing) for better convergence
4. Batch size matters
   - Start with 64-128
   - Small batches: better generalization
   - Large batches: need LR scaling + warmup

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Key Takeaways (Continued)

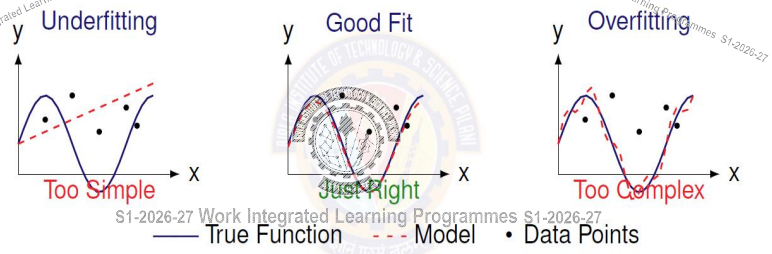
5. Use LR schedules + warmup
   - Cosine annealing: smooth, no tuning needed
   - Warmup (5-10 epochs): prevents early instability
   - Especially important for large models/batches
6. Gradient clipping for stability
   - Essential for RNNs, transformers
   - Norm clipping (threshold: 1-5) most common
   - Prevents exploding gradients
7. Monitor everything
   - Plot train/validation loss curves
   - Track gradient norms
   - Use early stopping (patience: 10 epochs)
   - Save checkpoints regularly
8. No universal solution
   - Different problems need different approaches
   - Understand trade-offs and experiment
   - Start simple (Adam), add complexity if needed

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Regularization of Deep Models

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

The Learning Problem: A Visual Journey



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Key Insight: Regularization helps us find the "just right" model complexity

What is Regularization?

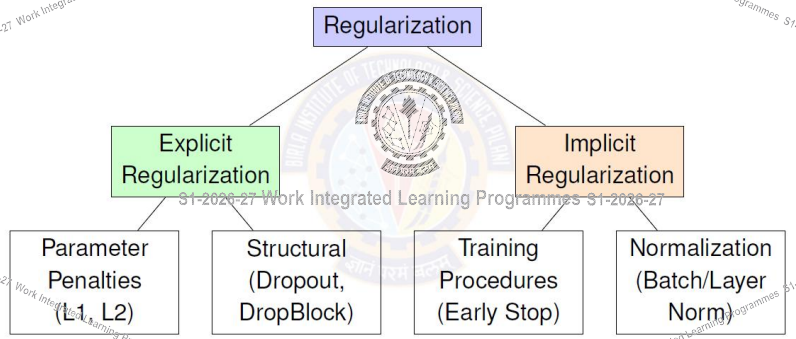
Regularization

Adding constraints to prevent models from becoming too complex and memorizing training data

- Real-world Analogy:
  - Like speed limits on roads - prevent "overly aggressive" driving
  - Like grammar rules in language - guide but don't completely restrict
  - Like budget constraints - force prioritization of important features
- In Machine Learning Terms:
  - Prevents overfitting by constraining model complexity
  - Improves generalization to unseen data
  - Adds inductive bias about what makes a good model
  - May reduce training performance but improves test performance

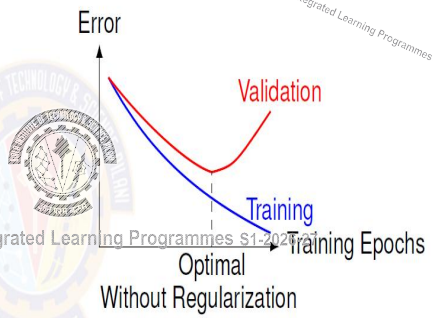
S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Regularization Taxonomy



Why Do We Need Regularization?

- Modern Neural Networks:
  - Millions to billions of parameters
  - Can memorize entire datasets
  - High representational capacity
  - Complex optimization landscape
- Without Regularization:
  - Perfect training accuracy
  - Poor test performance
  - Unstable training
  - Sensitive to data variations

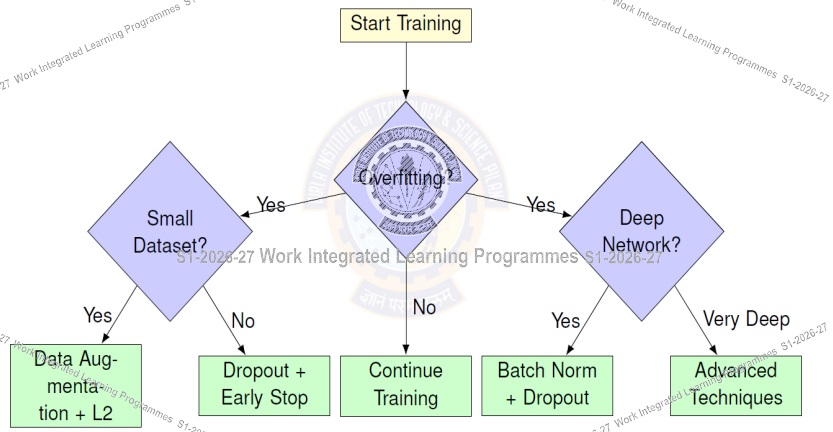


The Paradox: More powerful models need stronger constraints to generalize well!

Application Domains and Regularization

Domain	Primary Challenges	Key Techniques	Considerations
Computer Vision	Large images Spatial correlations Translation invariance	Data Augmentation Batch Normalization DropBlock	Memory intensive GPU constraints
NLP	Sequential dependencies Variable length inputs Context modeling	Dropout Layer Normalization Attention Dropout	Sequence length Vocabulary size
Time Series	Temporal patterns Non-stationarity Long dependencies	Recurrent Dropout Early Stopping Weight Decay	Data scarcity Distribution shift
Tabular Data	Mixed data types Feature interactions Small datasets	L1/L2 Regularization Dropout Feature Selection	Interpretability Feature engineering

When to Apply Regularization?



Vanishing Gradient Problem

- Problem:
  - Gradients become exponentially small in deeper layers
  - Early layers learn very slowly
  - Network fails to capture long-range dependencies
- Mathematical Cause:

$$\frac{\partial \mathcal{L}}{\partial W^{(1)}} = \frac{\partial \mathcal{L}}{\partial h^{(L)}} \prod_{l=2}^{L} \frac{\partial h^{(l)}}{\partial h^{(l-1)}} \frac{\partial h^{(1)}}{\partial W^{(1)}}$$

If $\left| \frac{\partial h^{(l)}}{\partial h^{(l-1)}} \right| < 1$ then gradients vanish exponentially.

- Common in:
  - Deep networks with sigmoid/tanh activations
  - RNNs with long sequences

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Exploding Gradient Problem

- Problem:
  - Gradients become exponentially large
  - Causes unstable training
  - Weights can overflow to infinity
- Mathematical Cause: If $\left| \frac{\partial h^{(l)}}{\partial h^{(l-1)}} \right| > 1$, then gradients explode exponentially.
- Symptoms:
  - Loss becomes NaN or infinity
  - Very large weight updates
  - Unstable training curves
- Solutions:
  - Gradient clipping
  - Better weight initialization
  - Batch normalization
  - Residual connections

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Covariate Shift

- Internal Covariate Shift: Change in distribution of network activations during training
- Problem:
  - Each layer's input distribution changes as previous layers learn
  - Slows down training
  - Requires careful initialization and learning rates
- Example:
  - As weights in layer 1 change, the input distribution to layer 2 changes, making it harder for layer 2 to adapt.
- Solution:
  - Batch Normalization addresses this by normalizing layer inputs

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Hyperparameter Sensitivity

- The Problem:
  - Regularization strength significantly affects performance
  - Wrong values can hurt more than help
  - Interactions between multiple regularization techniques
- Solutions:
  - Grid/Random Search: Systematic exploration
  - Bayesian Optimization: Efficient search
  - Learning Rate Scheduling: Adaptive approaches
  - Early Stopping: Automatic selection
- **Best Practice:** Start conservative, monitor both training and validation metrics, tune gradually.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Computational Overhead

Technique	Memory	Training Time	Inference	Implementation
L1/L2 Regularization	+0%	+1-2%	0%	Easy
Dropout	+5%	+10%	0%	Easy
Batch Normalization	+15%	+15%	+5%	Medium
Data Augmentation	+50%	+50%	0%	Medium
Advanced Dropout	+20%	+20%	0%	Hard

Mitigation Strategies:

- Mixed Precision: Reduce memory and computation
- Gradient Accumulation: Handle large effective batch sizes
- Progressive Training: Start simple, add complexity gradually

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Training Instability

- Common Instability Issues:
  1. Gradient Explosion/Vanishing
     - ★ Batch norm helps with vanishing gradients
     - ★ Gradient clipping for explosion
     - ★ Proper initialization crucial
  2. Learning Rate Sensitivity
     - ★ Regularization changes optimal learning rate
     - ★ May need to reduce LR when adding regularization
     - ★ Adaptive optimizers help
  3. Batch Size Dependencies
     - ★ Batch norm sensitive to batch size
     - ★ Dropout effectiveness varies with batch size
     - ★ Layer norm more stable alternative
- **Debugging Strategy:** Monitor gradient norms, activation statistics, and loss landscapes.

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Importance of Weight Initialization

- Why Important?
  - Poor initialization can cause vanishing/exploding gradients
  - Affects convergence speed and final performance
  - Can break symmetry between neurons
  - Critical for deep networks
- Goals:
  - Maintain reasonable activation magnitudes
  - Preserve gradient flow
  - Break symmetry
  - Enable stable training

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Common Initialization Methods

1. Zero Initialization: $W = 0$
   Problem: All neurons compute same function (symmetry)
2. Random Small Values: $W \sim \mathcal{N}(0, 0.01^2)$
   Problem: May cause vanishing gradients
3. Random Large Values: $W \sim \mathcal{N}(0, 1^2)$
   Problem: May cause exploding gradients and saturation

Solution: Use Parameter Initialization

Guideline:

- ReLU/Leaky ReLU: He initialization
- Sigmoid/Tanh: Xavier initialization preservation

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Xavier/Glorot Initialization

- Principle: Keep variance of activations and gradients constant across layers
- Formula:

For Linear Activations: $W \sim \mathcal{N}\left(0, \frac{1}{n_{in}}\right)$ or $W \sim \mathcal{N}\left(0, \frac{2}{n_{in} + n_{out}}\right)$

Uniform Distribution: $W \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{in} + n_{out}}}, \sqrt{\frac{6}{n_{in} + n_{out}}}\right)$

where n_{in} = number of input units, n_{out} = number of output units

- Best for: Sigmoid and tanh activations

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

He Initialization

- Motivation: Designed for ReLU activations
- Formula:

He normal: $W \sim \mathcal{N}\left(0, \frac{2}{n_{in}}\right)$

He uniform: $W \sim \mathcal{U}\left(-\sqrt{\frac{6}{n_{in}}}, \sqrt{\frac{6}{n_{in}}}\right)$

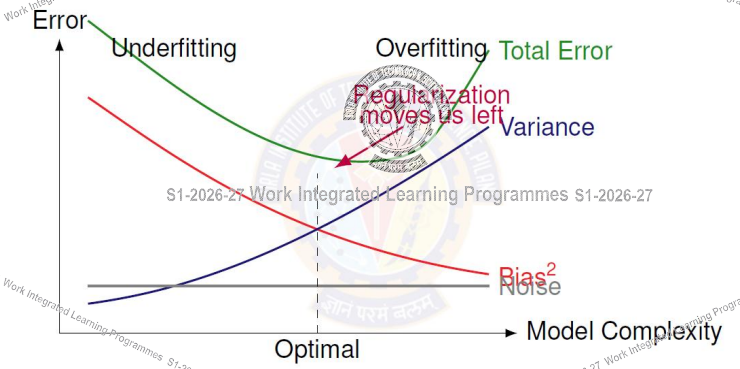
S1-2026-27 Work Integrated Learning Programmes S1-2026-27

where n_{in} = number of input units

- Reasoning:
  - ReLU kills half the neurons on average
  - Need to compensate with larger variance
  - Factor of 2 accounts for ReLU's effect

Classical Regularization Techniques

Understanding the Bias-Variance Tradeoff

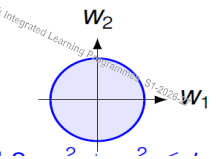


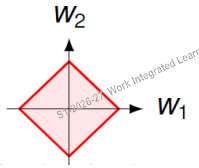
S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Key Insight: Regularization reduces variance at the cost of slightly increased bias

Parameter Norm Penalties: Review

Property	L2 (Ridge)	L1 (Lasso)
Formula	$\lambda \sum_i w_i^2$	$\lambda \sum_i \lvert w_i \rvert$
Gradient	λw_t	$\lambda \cdot \text{sign}(w_t)$
Effect on Weights	Shrinks proportionally	Drives to exact zero
Sparsity	No	Yes
Feature Selection	No	Yes (automatic)
Geometry	Circle/Sphere	Diamond/Octahedron
Differentiable	Yes	No (at zero)
Use Case	General smoothness	Feature selection
Typical λ	10^{-5} to 10^{-2}	10^{-4} to 10^{-1}

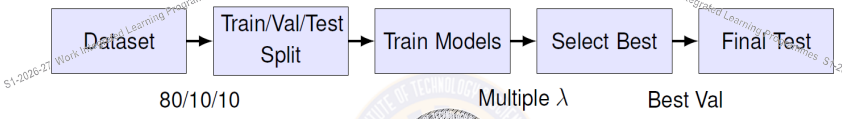




S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Model Selection and Cross-Validation

The Model Selection Pipeline:



K-Fold Cross-Validation:

1. Divide training data into k folds
2. For each fold i: train on other $k-1$ folds, validate on fold i
3. Average validation performance across all folds
4. Select hyperparameters with best average performance

Benefits:

- More reliable performance estimates
- Better use of limited data
- Reduces variance in hyperparameter selection

Modern Deep Learning Regularization

Dropout: From Concept to Implementation

- The Core Idea: Prevent complex co-adaptations on training data by randomly omitting feature detectors
- Training Phase Algorithm:

Algorithm 1: Dropout Training Phase

Input: Training samples, dropout probability p

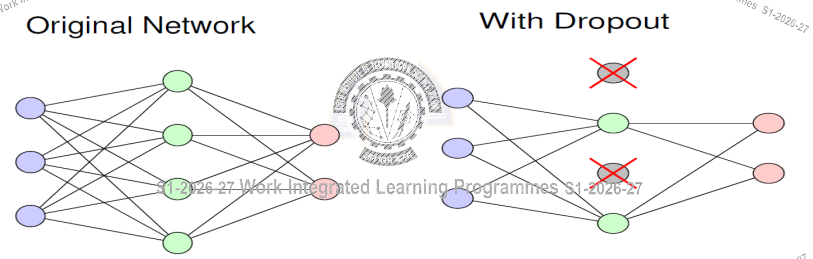
Output: Forward pass with dropout

foreach *training sample* **do**
  foreach *layer l with dropout* **do**
    $m^{(l)} \sim \text{Bernoulli}(1-p)$ // Generate mask
    $\tilde{h}^{(l)} = h^{(l)} \odot m^{(l)}$ // Apply mask
    $h^{(l+1)} = f(W^{(l+1)}\tilde{h}^{(l)} + b^{(l+1)})$ // Forward pass

- Inference Phase:

$$h^{(l+1)} = f(W^{(l+1)}(1-p)h^{(l)} + b^{(l+1)})$$

Dropout: Visualization



Dropout Variants for Different Architect

1. Standard Dropout
   - Random neurons
   - Best for: Fully connected layers
   - Rate: 0.5-0.8 for hidden layers, 0.1-0.2 for input
2. DropConnect
   - Random connections
3. DropBlock
   - For CNNs – Random spatial blocks
   - More effective for convolutional layers
   - Block size typically 7×7 or similar
4. Attention Dropout
   - For Transformers
   - Apply dropout to attention weights
   - Prevents over-reliance on specific attention patterns
   - Usually lower rates (0.1-0.3)

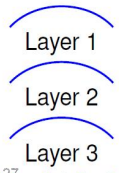
Batch Normalization: The Game Changer

- The Problem it Solves:

Without Batch Norm

Distributions shift!

With Batch Norm



Stable distributions!

- Benefits:
  - Allows higher learning rates (10-100× faster training)
  - Less sensitive to initialization
  - Acts as regularizer (reduces need for dropout)
  - Enables very deep networks

Batch Normalization: Complete Algorithm

Algorithm 2: BatchNorm Training Phase

Input: Mini-batch $\mathcal{B} = \{x_1, x_2, \ldots, x_m\}$

Input: Parameters: γ, β (learnable)

1 $\mu_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m} x_i$ // Batch mean

2 $\sigma^2_{\mathcal{B}} \leftarrow \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_{\mathcal{B}})^2$ // Batch variance

3 $\hat{x}_i \leftarrow \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}}$ // Normalize

4 $y_i \leftarrow \gamma \hat{x}_i + \beta$ // Scale and shift

5 $\mu_{running} \leftarrow (1-\alpha)\mu_{running} + \alpha\mu_{\mathcal{B}}$ // Update running stats

6 $\sigma^2_{running} \leftarrow (1-\alpha)\sigma^2_{running} + \alpha\sigma^2_{\mathcal{B}}$

Inference Phase:

$$\hat{x} = \gamma \frac{x - \mu_{running}}{\sqrt{\sigma^2_{running} + \epsilon}} + \beta$$

Batch Normalization: Numerical Example

- Setup: Mini-batch of 4 samples, single feature
- Input: $x = [10, 12, 8, 14]$
- Step 1: Batch Statistics

$$\mu_{\mathcal{B}} = \frac{10+12+8+14}{4} = 11$$

$$\sigma^2_{\mathcal{B}} = \frac{1+1+9+9}{4} = 5$$

- Step 2: Normalize ($\epsilon = 0.01$)

$$\hat{x}_1 = \frac{10-11}{\sqrt{5+0.01}} = \frac{-1}{2.24} = -0.45$$

$$\hat{x}_2 = (12-11)/2.24) = 0.45$$

$$\hat{x}_3 = (8-11)/2.24) = -1.34$$

$$\hat{x}_4 = (14-11)/2.24) = 1.34$$

- Step 3: Scale & Shift
  Let $\gamma = 2, \beta = 3$

$$y_1 = 2(-0.45) + 3 = 2.1$$

$$y_2 = 2(0.45) + 3 = 3.9$$

$$y_3 = 2(-1.34) + 3 = 0.32$$

$$y_4 = 2(1.34) + 3 = 5.68$$

- Verification:
  - Normalized: $\mu = 0, \sigma^2 = 1$
  - After γ, β: Distribution restored
  - Network learns optimal γ, β
- **Key:** γ, β allow network to undo normalization if needed

Layer Normalization: Motivation

- Why Not Always Batch Norm?
  - ✗ Requires large batch sizes
  - ✗ Unstable with batch size = 1
  - ✗ Different behavior: train vs test
  - ✗ Poor for RNNs (variable lengths)
- Layer Norm Solution
  - ✓ Works with any batch size
  - ✓ Same computation: train & test
  - ✓ Perfect for RNNs & Transformers
  - ✓ Normalizes across features

Layer Normalization: Definition & Formul

- **Layer Normalization:** Normalize across all features for each sample independently
- Mathematical Formulation: For input $x \in \mathbb{R}^d$ (d features), for each sample:

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i \quad \text{(mean across features)}$$

$$\sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2 \quad \text{(variance across features)}$$

$$\hat{x}_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} \quad \text{(normalize)}$$

$$y_i = \gamma \hat{x}_i + \beta \quad \text{(scale \& shift)}$$

where $\gamma, \beta \in \mathbb{R}^d$ are learnable parameters

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Layer Normalization: Algorithm

Algorithm 3: Layer Normalization

Input: Sample $x = [x_1, x_2, \ldots, x_d] \in \mathbb{R}^d$
Input: Parameters: $\gamma, \beta \in \mathbb{R}^d$ (learnable)
Input: Hyperparameter: $\epsilon = 10^{-5}$ (stability)
Output: Normalized output $y = [y_1, y_2, \ldots, y_d]$

$\mu \leftarrow \frac{1}{d}\sum_{i=1}^{d} x_i$ // Mean across features
$\sigma^2 \leftarrow \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2$ // Variance across features
for $i = 1$ **to** d **do**
  $\hat{x}_i \leftarrow \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$ // Normalize
  $y_i \leftarrow \gamma_i \hat{x}_i + \beta_i$ // Scale & shift
return $y = [y_1, y_2, \ldots, y_d]$

Note: No running statistics needed! Same computation for training and inference

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Layer Normalization: Numerical Example

- Setup: Single sample with 4 features
- Input: $x = [2, 8, 4, 10]$ (one sample, 4 features)
- Step 1: Compute Statistics

$$\mu = \frac{2+8+4+10}{4} = \frac{24}{4} = 6$$

$$\sigma^2 = \frac{16+4+4+16}{4} = 10$$

- Step 2: Normalize ($\epsilon = 0.01$)

$$\hat{x}_1 = \frac{2-6}{\sqrt{10+0.01}} = \frac{-4}{3.16} = -1.27$$

$$\hat{x}_2 = (8-6)/3.16) = 0.63$$

$$\hat{x}_3 = (4-6)/3.16) = -0.63$$

$$\hat{x}_4 = (10-6)/3.16) = 1.27$$

- Step 3: Scale & Shift
  Let $\gamma = [1, 2, 1, 2]$, $\beta = [0, 1, 0, 1]$

$$y_1 = 1(-1.27) + 0 = -1.27$$

$$y_2 = 2(0.63) + 1 = 2.26$$

$$y_3 = 1(-0.63) + 0 = -0.63$$

$$y_4 = 2(1.27) + 1 = 3.54$$

- Key Observations:
  - Normalized features: $\mu = 0, \sigma^2 = 1$
  - Each feature can have different γ_i, β_i
  - Independent of batch size
  - Same computation always

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Batch Norm vs Layer Norm: Comparison

Property	Batch Norm	Layer Norm
Normalization Axis	Across batch	Across features
Depends on Batch Size	Yes	No
Running Statistics	Yes (train)	No
Train vs Test	Different	Identical
Batch Size = 1	Unstable	Works fine
Variable Seq Length	Problematic	Perfect
Best For	CNNs	RNNs, Transformers
Parameters	$2\times$ features	$2\times$ features
Computational Cost	Low	Low

When to Use:

- Batch Norm: CNNs, large batches, stable batch sizes
- Layer Norm: RNNs, Transformers, small/variable batches

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

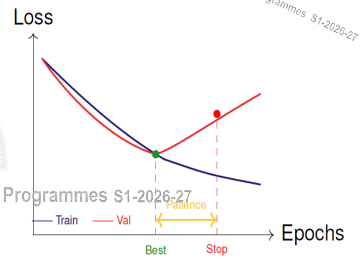
Early Stopping: Concept and Visualization

Concept:

- Monitor validation error during training
- Stop training when validation error starts increasing
- Implicit regularization technique
- Prevents overfitting automatically

Key Parameters:

- Patience: Number of epochs to wait = 5-20 epochs
- Min Delta: Minimum improvement threshold = 0.001 to 0.01
- Restore Best: Use best weights, not final
- Monitor: val_loss or val_accuracy



Key Insight: Early stopping finds the sweet spot between underfitting and overfitting

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Early Stopping: Complete Algorithm

Algorithm 4: Early Stopping

Initialize: best_val_error $= \infty$, patience_counter $= 0$, best_model $=$ None

```
for epoch = 1 to max_epochs do
    Train model for one epoch
    val_error = compute_validation_error()
    if val_error < best_val_error − min_delta then
        best_val_error = val_error
        patience_counter = 0
        best_model = save_model() // New best model saved
    else
        patience_counter += 1
        if patience_counter ≥ max_patience then
            Early stopping triggered
            break
restore_best_model()
```

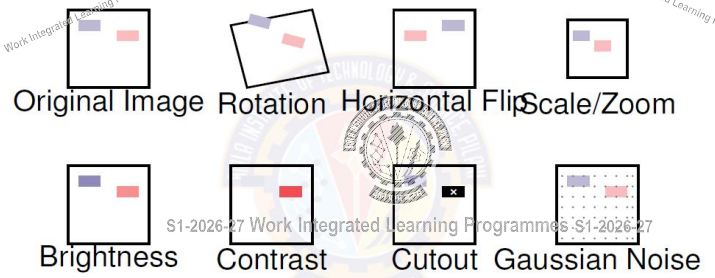
S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Data Augmentation: Beyond Basic

- Concept: Artificially increase training data by applying transformations
- For Images:
  - Rotation, flipping, scaling, cropping, translation, Color jittering, brightness adjustment, Mixup, CutMix
- For Text:
  - Synonym replacement, Back-translation, Random insertion/deletion
- Benefits:
  - Improves generalization
  - Reduces overfitting
  - Makes model more robust

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Data Augmentation: Visual Examples



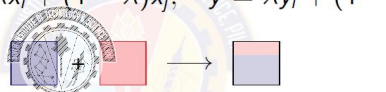
Implementation: Apply 2-3 augmentations per training sample, with 50% probability each

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

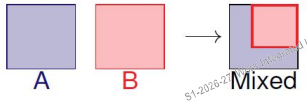
Data Augmentation: Beyond Basic

- Modern Mixing Techniques:
  1. Mixup: Linear interpolation of samples

$$\tilde{x} = \lambda x_i + (1-\lambda)x_j, \quad \tilde{y} = \lambda y_i + (1-\lambda)y_j$$



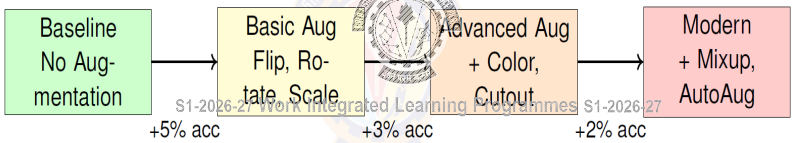
  2. CutMix: Combines spatial mixing with label mixing
     - ★ Cut and paste patches between images
     - ★ Mix labels proportionally to patch areas



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Data Augmentation: Implementation Best

Progressive Augmentation Strategy:

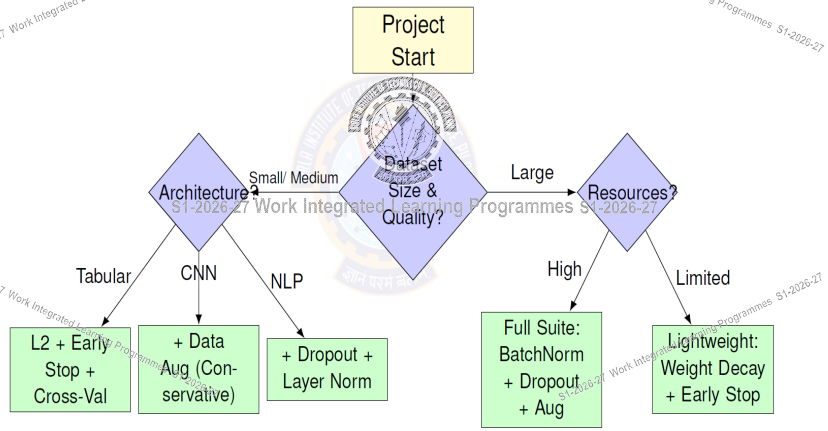


Performance Impact: Well-designed augmentation can improve generalization by 5-15% with minimal computational overhead

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Practical Implementation Workflow

Regularization Selection Decision Tree



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Hyperparameter Tuning Strategy

Priority Order for Tuning:

1. Learning Rate (most important!) $[10^{-4}, 10^{-1}]$
2. Regularization Strength L2 regularization: $[10^{-5}, 10^{-1}]$
3. Dropout Rates (different for each layer) $[0.1, 0.8]$ depending on layer
4. Batch Size (affects batch norm) $[16, 512]$ (powers of 2)
5. Data Augmentation intensity

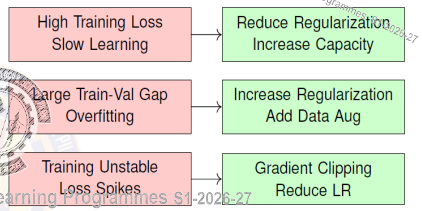
S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Performance Monitoring & Troubleshootin

Real-time Monitoring:

Metric	Alert Threshold
Train-Val Gap	$> 15\%$
Gradient Norm	> 5 or < 0.1
Val Loss Trend	$\uparrow$ 3+ epochs
Weight Magnitude	> 1 or < 0.01

Problem Diagnosis:



S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Monitoring Frequency:

- Every Epoch: Loss, accuracy
- Every 5 Epochs: Gradients, activations
- Mid-training: Hyperparameter check

Emergency Protocol:

1. Check data pipeline
2. Verify architecture
3. Monitor gradients/weights
4. Test minimal regularization

Architecture-Specific Regularization

CNN-Specific Regularization

1. **Spatial Regularization:**
   - DropBlock: Drop spatial blocks instead of individual pixels
   - Cutout: Zero out random patches
   - SpatialDropout: Drop entire feature maps
2. **Data Augmentation for Images:**
   - Geometric: Rotation, scaling, flipping, cropping
   - Photometric: Color jittering, brightness, contrast
   - Advanced: Mixup, CutMix, AugMix
3. **Architecture Choices:**
   - Batch Normalization: After conv, before activation
   - Global Average Pooling: Reduces overfitting vs FC layers
   - Residual Connections: Implicit regularization

RNN-Specific Regularization

1. **Temporal Challenges:**
   - Vanishing/exploding gradients over time
   - Overfitting to sequence patterns
   - Variable sequence lengths
2. **Dropout Variants:**
   - Input Dropout: On input sequences
   - Output Dropout: On final hidden states
   - Recurrent Dropout: On hidden-to-hidden connections (careful!)
   - Variational Dropout: Same mask across time steps
3. **Gradient Clipping**
4. **Layer Normalization:**
   - More stable than batch norm for RNNs
   - Applied to hidden states
   - Helps with internal covariate shift

Transformer-Specific Regularization

1. **Attention Mechanisms:**
   - Attention Dropout: On attention weights
   - Multi-head Attention: Implicit regularization through diversity
   - Attention Temperature: Scale attention logits
2. **Position and Structure:**
   - Positional Dropout: On positional encodings
   - Layer Dropout: Skip entire transformer layers
   - Path Dropout: Random attention/feedforward paths
3. **Modern Techniques:**
   - Label Smoothing: For classification heads
   - Gradient Accumulation: Larger effective batch sizes
   - Warmup + Decay: Learning rate scheduling

Regularization Interactions and Conflicts

Positive Synergies:

- ✓ Batch Norm + Weight Decay: Complementary mechanisms
- ✓ Data Augmentation + Dropout: Different noise types
- ✓ L2 + Early Stopping: Double protection against overfitting

Potential Conflicts:

- ✏ Batch Norm + Dropout: May interfere with normalization
- ✏ High L1 + High Dropout: Excessive sparsity
- ✏ Strong Data Aug + Small Dataset: Information loss

Resolution Strategies:

- Apply dropout after batch norm, not before
- Use lower dropout rates with strong L1 regularization
- Reduce data augmentation intensity for small datasets
- Monitor multiple metrics to detect conflicts

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Common Implementation Mistakes

Critical Errors:

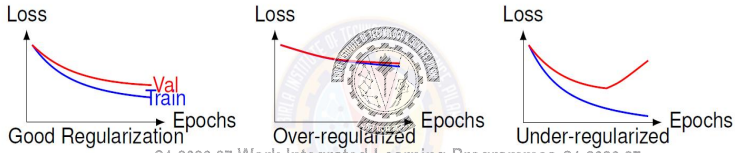
1. Dropout During Inference
   - Wrong: Keep dropout active during testing
   - Right: Disable dropout, scale weights appropriately
2. Batch Norm Placement
   - Wrong: BN → Conv → Activation
   - Right: Conv → BN → Activation
3. Learning Rate with Regularization
   - Wrong: Same LR with and without regularization
   - Right: Reduce LR when adding strong regularization
4. Data Leakage in Augmentation
   - Wrong: Augment before train/val split
   - Right: Split first, then augment training set only

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

Debugging Regularized Models

Diagnostic Tools and Techniques:

1. Loss Curve Analysis

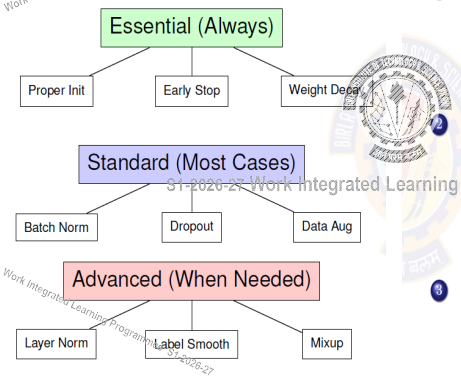


S1-2026-27 Work Integrated Learning Programmes S1-2026-27

2. Gradient and Weight Monitoring
   - Plot gradient norms per layer
   - Monitor weight distributions
   - Track activation statistics
   - Check for dead neurons (all zero activations)

Regularization Strategy Summary

Regularization Hierarchy:



Three Key Principles:

1. Start Simple, Add Gradually
   - Begin with essential techniques
   - Validate each addition
   - Monitor train-val gap
2. Match to Problem Domain
   - CNNs → Batch Norm + Data Aug
   - NLP/RNNs → Layer Norm + Dropout
   - Small data → Conservative augmentation
3. Monitor & Adapt
   - Track multiple metrics
   - Detect technique conflicts
   - Tune hyperparameters iteratively

Key Takeaways

Golden Rule: Regularization should improve test performance, not just training stability

1. **Trade-offs are worth it:** 20% slower training for 10% better generalization is a good investment
2. **Regularization is insurance:** Prevents catastrophic overfitting, especially important for production models
3. **Combine complementary techniques:** Modern best practices use multiple regularization methods together (e.g., Batch Norm + Dropout + Data Aug)
4. **Over-regularization hurts:** Too much constraint can be worse than none - model won't learn effectively
5. **Implementation details matter:** Dropout placement, BN order, augmentation timing - small mistakes cause big problems